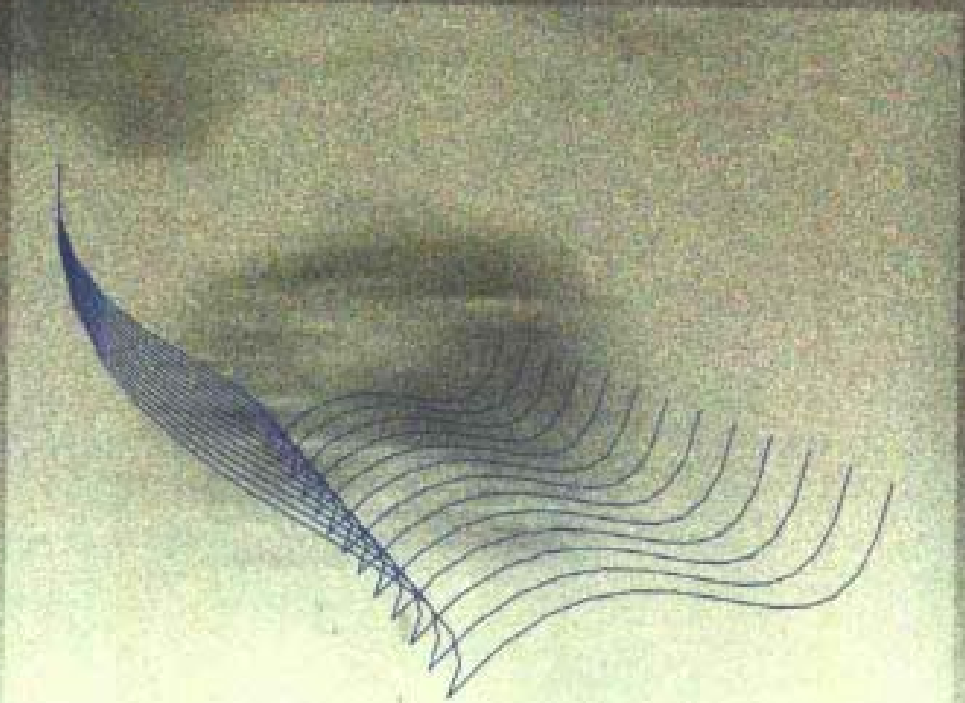
新世纪高等院校精品教材



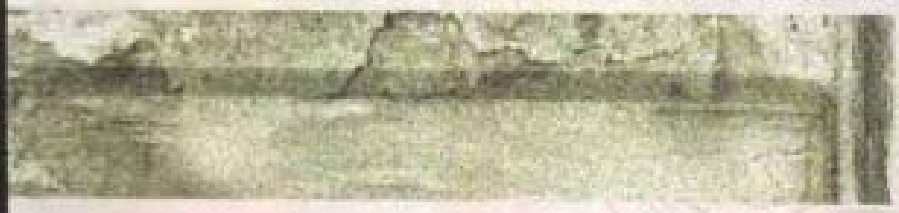
操作系统

学习指导和考试指导

李善平　主编

浙江大学出版社

操作系统学习指导和考试指导



ISBN 7-308-03581-6

9 787308 035811 >

ISBN 7-308-03581-6/TP · 253 定价：29.00元

新世纪高等院校精品教材

操作系统学习指导和考试指导

李善平　主编

浙江大学出版社

内容简介

这是一本关于操作系统原理的学习参考书，是大学操作系统课程考试，以及研究生入学考试的指导书，面向大学本科和研究生读者。书的作者全部是大学在岗教师，各自主讲所供职大学的操作系统课程。书中的习题由作者认真挑选，覆盖了操作系统原理的全部内容，如进程管理、存储管理、文件系统管理、设备管理和综合类题目等。绝大多数习题都附有参考答案，均由本书作者作答。

图书在版编目（CIP）数据

操作系统学习指导和考试指导 / 李善平主编. —杭州：浙江大学出版社，2004.1
ISBN 7-308-03581-6

Ⅰ.操… Ⅱ.李… Ⅲ.操作系统－高等学校－教学参考资料 Ⅳ.TP316

中国版本图书馆 CIP 数据核字（2003）第 122493 号

责任编辑 杜希武
封面设计 俞亚彤
出版发行 浙江大学出版社
（杭州浙大路 38 号 邮政编码 310027）
（E-mail：zupress@mail.hz.zj.cn）
（网址：http://www.zjupress.com）
排　　版 浙江大学出版社电脑排版中心
印　　刷 浙江良渚印刷厂印刷
开　　本 787mm×1092mm 1/16
印　　张 19.25
字　　数 492 千
版 印 次 2004 年 1 月第 1 版 2004 年 1 月第 1 次印刷
印　　数 0001—3000
书　　号 ISBN 7-308-03581-6/TP·253
定　　价 29.00 元

本书编写人员

（以姓氏笔画为序）

任午令（杭州商学院　教授）
宋广华（浙江大学　副教授）
陆忆红（浙江工业大学　副教授）
李善平（浙江大学　教授）
张泉方（浙江大学　副教授）
季江民（浙江大学　副教授）
郑扣根（浙江大学　教授）
梁红兵（杭州电子工学院　副教授）

前言

由于工作需要，我在浙江大学计算机学院主讲操作系统课程已有十年。对计算机操作系统的理解也由浅入深，以至于目前的研究方向（嵌入式系统、应用服务器等）与操作系统紧密相关。在学习、研究操作系统过程中，深感动手之重要性。

与众多学子一样，我也经历了中考、高考、考研、考博、考托（不过现在已经脱离“苦海”，反过来出试卷考察新来的浙江大学计算机专业本科生、研究生了。对不住！）。甜酸苦辣中，知道练兵对于实战的重要性。

所以，当浙江大学出版社提议编写一本操作系统学习指导时，我欣然接受。随后联系了几位在高校主讲操作系统课程的老师，大家都乐意为苦读中的学子们做些事。

我们这儿奉献给大家的是一本关于操作系统原理的学习参考书，是大学操作系统课程考试，以及研究生入学考试的指导书。

书中的习题由作者广泛搜集，认真挑选，覆盖了操作系统原理的全部内容，如进程管理、存储管理、文件系统管理、设备管理、综合类问题等。绝大多数习题都附有参考答案，均由本书作者作答。当然，有些习题可以用多种方法解答，或者有多种答案。

全书由李善平主编。大家共同搜集习题后，由郑扣根负责总论，张泉方负责概述部分，梁红兵、宋广华负责进程管理部分，陆亿红、李善平负责存储管理部分，季江民负责文件系统部分，任午令负责 I/O 设备管理部分。最后由李善平校核并统稿。尽管如此，书中的疏漏还是难免的，敬请读者批评、指针，以便再版时改正。

李善平

2004 年元旦于浙大求是园

目 录

第 1 章 操作系统简介 …… 1

1.1 概述 …… 1
1.2 计算机系统结构 …… 2
1.3 操作系统结构 …… 2
1.4 习题与解答 …… 3

第 2 章 进程管理 …… 18

2.1 概述 …… 18
2.2 进程概念 …… 18
2.3 线程 …… 19
2.4 CPU 调度 …… 19
2.5 进程同步 …… 20
2.6 死锁 …… 20
2.7 习题与解答 …… 21
2.8 OSC* 习题与解答 …… 133

第 3 章 存储管理 …… 149

3.1 概述 …… 149
3.2 内存管理 …… 149
3.3 虚拟内存 …… 150
3.4 习题与解答 …… 151
3.5 OSC 习题与解答 …… 206

第 4 章 文件系统 …… 218

4.1 概述 …… 218
4.2 文件系统接口 …… 218
4.3 文件系统实现 …… 219

* OSC 即《Operating System Concepts (6th Edition)》, Silberschatz A 等著。

4.4 习题与解答 …… 220
4.5 OSC 习题与解答 …… 242

第 5 章 I/O 管理 …… 248

5.1 概述 …… 248
5.2 I/O 系统 …… 248
5.3 大容量存储器的结构 …… 249
5.4 习题与解答 …… 250
5.5 OSC 习题与解答 …… 280

附录 研究生入学考试操作系统试卷选编 …… 285

参考文献 …… 298

第1章

操作系统简介

1.1 概述

操作系统已经发展了45年，它始终围绕着两个主要目的。第一，操作系统试图调度计算活动以确保计算机系统的高性能。第二，操作系统提供一个便于开发和运行程序的环境。最初，计算机只能通过控制台来使用。汇编程序、装载程序、连接程序和编译程序改善了系统编程的方便性，但也增加了大量的设置时间。为了减少设置时间，采用了操作员和批处理的方式。

批处理系统通过使用驻留操作系统以允许自动切换作业，进而大大地提高了计算机的整体使用率。计算机不再需要等待人工操作。但是，CPU使用率仍然很低，这是因为I/O设备的速度要比CPU慢。当然，可以对慢设备采取离线操作的方法，即在一个CPU上使用多个总带读入和磁带打印系统。

为了改善计算机系统的整体性能，开发人员引入了多道程序设计的概念，这样多个作业可以同时位于内存中。CPU在这些作业之间来回切换而提高了使用率，也降低了执行作业所需要的总时间。

多道程序设计也允许分时。分时操作系统允许多个(甚至数百个)用户同时交互地使用一个计算机系统。

PC是微机。它们与大型机相比，相对比较小并且便宜。这些计算机的操作系统在许多方面都得益于大型机操作系统的发展成果。不过，由于单个用户可以独用计算机，因而CPU利用率不再是主要问题。因此，有的大型操作系统的设计决策不再适用于这些小系统。其他设计决策，如安全性等，因为PC可以通过网络或WWW与其他计算机和用户相连，对于微型机和大型机都同样适用。

并行系统有多个紧密通信的CPU。这些CPU共享总线，有时也共享内存和外设。这些系统提供了高计算量和高可靠性。分布式系统允许对分布在各地的主机资源进行共享。集群系统允许多个系统对位于共享存储中的数据进行操作，即使一部分集群成员失败也能正常工作。

硬实时系统常常用于控制专用应用设备。硬实时操作系统具有明确的、固定的时间约束，处理必须在规定的约束内完成，否则系统失败。软实时系统没有严格的时间约束，不支持最终时限调度。

近来，由于Internet和WWW(World Wide Web，万维网)的影响，现代操作系统也集成了

WWW 浏览器、网络和通信软件。

1.2 计算机系统结构

通过将一台计算机上的 CPU 操作和 I/O 操作交替执行，多道程序设计和分时系统改善了性能。这种交替的工作方式要求 CPU 与 I/O 设备之间的数据传输或者采用轮询方式，或者采用对 I/O 端口的中断驱动访问方式，或者是 DMA 方式来处理。

为了让计算机执行程序，程序必须位于内存中。内存是处理器能直接访问的惟一大容量存储区域。内存是字节或字的数组，容量为数百 KB 到数十 GB。每个字都有其地址。内存是易失性存储器，当没有电源时会失去其内容。绝大多数计算机系统都提供了外存以扩充内存。对外存的主要要求是，它要能长久地存储大量数据。最常用的外存是磁盘，它提供对数据和程序的存储。磁盘是非易失性的，且能随机访问。磁带主要用于备份，用于存储不常使用的信息，也作为系统之间的信息传递媒介。

根据速度和价格，可以将计算机系统的不同存储系统按层次来组织。最高层最为昂贵但也最快。随着向层次结构下面移动，单位的价格通常降低，而访问时间通常增加。

操作系统必须确保计算机系统的正确操作。为了防止用户干预系统的正常操作，硬件有两种模式：用户模式和监督程序模式。许多指令（如 I/O 指令和停机指令）都有特权，但只能在监督程序模式下执行。操作系统所驻留的内存也必须加以保护以防止用户程序修改。定时器防止无穷循环。这些工具（如双模式、特权指令、内存保护、定时器中断）都是操作系统用以实现正确操作的基本单元。

1.3 操作系统结构

操作系统提供若干服务。在最底层，系统调用允许运行程序直接向操作系统发出请求。在高层，命令解释程序或操作系统外壳使得用户不必编写程序就能发出请求。命令可以来自文件（批处理模式），或者直接来自键盘输入（交互模式或分时模式）。系统程序用来满足一些常用的用户操作。

请求类型随请求级别的变化而变化。系统调用级别提供基本功能，如进程控制，文件和设备管理。由命令解释程序或系统程序来完成的高级别请求需要转换成一系列的系统请求。系统服务可分成许多类型：程序控制、状态请求和 I/O 请求。程序出错可作为对服务的一种隐式请求。

在定义了系统服务之后，就可开发操作系统的结构。需要用各种表记录一些信息，这些信息定义了计算机的系统状态和系统的作业状态。

设计一个新操作系统是个重大任务。在设计开始之前，要定义好系统目标。它们是选择各种必要算法和策略的基础。

由于操作系统非常庞大，所以模块化设计很重要。按一系列层或采用微内核来设计系统是比较好的技术。虚拟机概念采用了分层方法，并将操作系统内核和硬件都作为硬件来考虑。其他操作系统可以建立在这一虚拟机之上。例如，实现 JVM（Java Virtual Machine，Java 虚拟机）的操作系统能运行所有 Java 程序，因为 JVM 为 Java 程序抽象化了底层系统，提供了与平台无关接口。

在整个操作系统设计周期中，我们必须仔细区分策略和实现细节(机制)。在后面需要修改策略时，这种区分允许最大限度的灵活性。

现在的操作系统几乎都是用系统实现语言或高级语言来编写的。这一特征改善了操作系统的实现、维护和可移植性。为特定机器配置创建操作系统，我们必须执行系统生成工作。

1.4　习题与解答

一、单项选择题

1. 操作系统是对_____进行管理的软件。

A. 软件　　B. 硬件　　C. 计算机资源　　D. 应用程序

答：C

2. 从用户的观点看，操作系统是_____。

A. 用户与计算机之间的接口　　B. 控制和管理计算机资源的软件

C. 合理地组织计算机工作流程的软件　　D. 由若干层次的程序按一定的结构组成的有机体

答：A

3. 操作系统的功能是进行处理机管理、_____管理、设备管理及信息管理。

A. 进程　　B. 存储器　　C. 硬件　　D. 软件

答：B

4. 操作系统中采用多道程序设计技术提高 CPU 和外部设备的_____。

A. 利用率　　B. 可靠性　　C. 稳定性　　D. 兼容性

答：A

5. 操作系统是现代计算机系统不可缺少的组成部分，是为了提高计算机的_____和方便用户使用计算机而配备的一种系统软件。

A. 速度　　B. 利用率　　C. 灵活性　　D. 兼容性

答：B

6. 操作系统的基本类型主要有_____。

A. 批处理系统、分时系统及多任务系统

B. 实时操作系统、批处理操作系统及分时操作系统

C. 单用户系统、多用户系统及批处理系统

D. 实时系统、分时系统和多用户系统

答：B

7. 操作系统是一种_____。

A. 通用软件　　B. 系统软件　　C. 应用软件　　D. 软件包

答：B

8. 操作系统的_____管理部分负责对进程进行调度。

A. 主存储器　　B. 控制器　　C. 运算器　　D. 处理机

答：D

9. 下列选择中，_____不是操作系统关心的主要问题。(浙江大学 2003 年研究生试题)

A. 管理计算机裸机　　B. 设计、提供用户程序与计算机硬件系统的界面

C. 管理计算机系统资源　　D. 高级程序设计语言的编译器

答:D

10. 按照计算机系统层次结构的一般原则，从用户的角度将依次(①、②、③)看到_____。(浙江大学 2003 年研究生试题)

A. ① C 语言编译程序，② 用 C 语言写的某单位人事管理程序，③ 操作系统

B. ① 用 C 语言写的某单位人事管理程序，② C 语言编译程序，③ 操作系统

C. ① 操作系统，② 用 C 语言写的某单位人事管理程序，③ C 语言编译程序

D. ① 用 C 语言写的某单位人事管理程序，② 操作系统，③ C 语言编译程序

答:B

11. 系统功能调用是_____。(浙江大学 2003 年研究生试题)

A. 用户编写的一个子程序　　B. 高级语言中的库程序

C. 操作系统中的一条命令　　D. 操作系统向用户程序提供的接口

答:D

12. _____是帮助管理计算机资源的一整套程序。(浙江大学 2002 年研究生试题)

A. 应用程序　　B. 备份程序　　C. 诊断程序　　D. 操作系统程序

答:D

13. _____不是一个操作系统环境。(浙江大学 2002 年研究生试题)

A. 赛扬(Celeron)　　B. Windows CE　　C. LINUX　　D. Solaris

答:A

14. _____系统具有同时管理和运行多个应用程序的能力。(浙江大学 2002 年研究生试题)

A. GUI　　B. Windows Applications

C. Multitasking　　D. Networking

答:C

15. _____是一个 Microsoft 公司竭力反对的操作系统。(浙江大学 2002 年研究生试题)

A. OS/2　　B. Windows NT　　C. LINUX　　D. Macintosh

答:C

16. 所谓_____是指将一个以上的作业放入主存，并且同时处于运行状态，这些作业共享处理机的时间和外围设备等其他资源。

A. 多重处理　　B. 多道程序设计　　C. 实时处理　　D. 共行执行

答:B

17. 多道批处理系统的硬件支持是 20 世纪 60 年代初发展起来的_____。

A. ASIC 技术　　B. 通道和中断机构　　C. 集成电路　　D. 高速内存

答:B

18. 下面关于操作系统的叙述中正确的是_____。

A. 批处理作业必须具有作业控制信息。

B. 分时系统不一定都具有人机交互功能。

C. 从响应时间的角度看，实时系统与分时系统差不多。

D. 由于采用了分时技术，用户可以独占计算机的资源。

答:A

19. _____操作系统允许在一台主机上同时连接多台终端，多个用户可以通过各自的终端同

时交互地使用计算机。

A.网络 B.分布式 C.分时 D.实时

答:C

20.如果分时操作系统的时间片一定,那么_____,则响应时间越长。

A.用户数越少 B.用户数越多 C.内存越少 D.内存越多

答:B

21.分时操作系统通常采用_____策略为用户服务。

A.可靠性和灵活性 B.时间片轮转 C.时间片加权分配 D.短作业优先

答:B

22._____操作系统允许用户把若干个作业提交给计算机系统。

A.单用户 B.分布式 C.批处理 D.监督

答:C

23.在_____操作系统控制下,计算机系统能及时处理由过程控制反馈的数据并作出响应。

A.实时 B.分时 C.分布式 D.单用户

答:A

24.下面6个系统中,必须是实时操作系统的有_____个。

计算机辅助设计系统 航空订票系统 过程控制系统 机器翻译系统 办公自动化系统 计算机激光照排系统

A.1 B.2 C.3 D.4

答:C

25.设计实时操作系统时,首先应考虑系统的_____。

A.可靠性和灵活性 B.实时性和可靠性 C.灵活性和可靠性 D.优良性和分配性

答:B

26.若把操作系统看作计算机系统资源的管理者,下列不属于操作系统所管理的资源是_____。

A.程序 B.内存 C.CPU D.中断

答:D

27.在下列操作系统的各个功能组成部分中,_____不需要硬件的支持。

A.进程调度 B.时钟管理 C.地址映射 D.中断系统

答:A

28.实时操作系统必须在_____内响应来自外部的事件。(西安电子科技大学2000年研究生试题)

A.响应时间 B.周围时间 C.规定时间 D.调度时间

答:C

29.多道程序设计是指_____。(西安电子科技大学2002年研究生试题)

A.在实时系统中并发运行多个程序

B.在分布系统中同一时刻运行多个程序

C.在一台处理机上同一时刻运行多个程序

D.在一台处理机上并发运行多个程序

答:D

30. 操作系统提供给程序员的接口是_____。

A. 进程　　B. 系统调用　　C. 库函数　　D. B 和 C

答:B

31. 配置了操作系统的计算机是一台比原来的物理计算机功能更强的计算机,这样的计算机只是一台逻辑上的计算机,称为_____计算机。

A. 并行　　B. 真实　　C. 虚拟　　D. 共享

答:C

32. 批处理系统的主要缺点是_____。(清华大学 1996 年研究生试题)

A. CPU 利用率低　　B. 不能并发执行　　C. 缺少交互性　　D. 以上都不是

答:C

33. 操作系统_____采用了以空间换时间的技术。

A. SPOOLing 技术　　B. 覆盖技术　　C. 通道技术　　D. 虚拟存储技术

答:A

34. 分时系统中为了使多个用户能够同时与系统交互,最关键的问题是_____。

A. 计算机具有足够快的运算速度　　B. 能快速进行内外存之间的信息交换

C. 系统能够及时接收多个用户的输入　　D. 短时间内所有用户程序都能运行

答:C

35. 实时操作系统对可靠性和安全性的要求极高,它_____。

A. 十分注意系统资源的利用率　　B. 不强调响应速度

C. 不强求系统资源的利用率　　D. 不必向用户反馈信息

答:C

36. 操作系统是一种__A__,在操作系统中采用多道程序设计方式能提高 CPU 和外部设备的__B__。一般来说,为了实现多道程序设计,计算机需要有__C__。操作系统__D__已经成为工作站上的主流操作系统,对于使用__D__操作系统的计算机系统来说__E__语言是主要的高级语言。

A:(1)通用软件;(2)系统软件;(3)应用软件;(4)软件包。

B:(1)利用效率;(2)可靠性;(3)稳定性;(4)兼容性。

C:(1)更大的内存;(2)更快的外部设备;(3)更快的 CPU;(4)更先进的终端;

D:(1)VMS;(2)DOS;(3)MVS;(4)UNIX。

E:(1)Pascal;(2)Ada;(3)C;(4)Fortran。

答:A—(2)B—(1)C—(1)D—(4)E—(3)

37. 操作系统是一种系统软件,它有许多种类,DOS 属于__A__操作系统。而__B__已成为工作站上主流操作系统。

中断一般可分成__C__和__D__。__D__又可分成为两类,一类是程序中固定地址越界,数据溢出等错误所引起的中断,另一类可称为__E__。

供选择的答案:

A. ① 分时　② 分布式　③ 单用户　④ 网络

B. ① VMS　② MVS　③ DOS　④ UNIX

C,D,E. ① 硬件中断　② 缺页中断　③ 软件中断　④ 自愿中断

⑤ 强迫中断 ⑥ 停电中断　⑦ I/O 中断　⑧ 时钟中断

答：A—③　B—④　C—③　D—①　E—⑦

38. 在计算机系统中配置操作系统的主要目的是__A__，操作系统的主要功能是管理计算机系统中的__B__，其中包括__C__管理和__D__管理，以及设备管理和文件管理。这里的__C__管理主要是对进程进行管理。

A：(1)增强计算机系统的功能；(2)提高系统资源的利用率；(3)提高系统的运行速度；(4)合理地组织系统的工作流程，以提高系统吞吐量。

B：(1)程序和数据；(2)进程；(3)资源；(4)作业；(5)任务。

C，D：(1)存储器；(2)虚拟存储器；(3)运算器；(4)处理机；(5)控制器。

答：A—(2)　B—(3)　C—(4)　D—(1)

39. 操作系统有多种类型：(1)允许多个用户以交互方式使用计算机的操作系统，称为__A__；(2)允许多用户将若干个作业提交给计算机系统集中处理的操作系统称为__B__；(3)在__C__的控制下，计算机系统能及时处理由过程控制反馈的数据，并做出响应；(4)在 IBM-PC 中，操作系统称为__D__。

A，B，C，D：(1)批处理操作系统；(2)分时操作系统；(3)实时操作系统；(4)微机操作系统；(5)多处理操作系统。

答：A—(2)　B—(1)　C—(3)　D—(4)

40. 分时系统的响应时间(及时性)主要是根据__A__确定的，而实时系统的响应时间则是由__B__确定的。

A，B：(1)时间片大小；(2)用户数目；(3)计算机运行速度；(4)用户所能接受的等待时间；(5)控制对象所能接受的时延；(6)实时调度。

答：A—(4)　B—(5)

41. 分时系统中，为使多个用户能够同时与系统交互，最关键的问题是__A__，当用户数目为 100 时，为保证响应不超过 2 秒；此时的时间片最大应为__B__。

A：(1)计算机具有足够的运行速度；(2)内存容量应足够大；(3)系统能及时地接收多个用户输入；(4)能在短时间内，让所有用户程序都能运行；(5)能快速进行内外存对换。

B：(1)10ms；(2)20ms；(3)50ms；(4)100ms；(5)200ms。

答：A—(4)　B—(2)

42. 在设计分时操作系统时，首先要考虑的是__A__；在设计实时操作系统时，首先要考虑的是__B__；在设计批处理系统时，首先要考虑的是__C__。

A，B，C：(1)灵活性和可适应性；(2)交互性和响应时间；(3)周转时间和系统吞吐量；(4)实时性和可靠性。

答：A—(2)　B—(4)　C—(3)

43. 从总体上说，多道程序设计技术可__A__单位时间的算题量，对每一个算题开始到全部完成所需要的时间可能__B__。

A，B：(1)增加 (2)减少 (3)延长 (4)缩短

答：A—(1)　B—(3)

44. 批处理系统的主要缺点是__A__。

A：(1)CPU 的利用率不高　(2)失去了交互性　(3)不具备并行性　(4)以上都不是

答：A—(2)

45. 在只有一个 CPU 的情况下，多道程序环境下的各道程序在宏观上是_____运行，在微观

上则是_____执行。

A.并行 B.串行 C.资源 D.作业

答:A, B

46.能对外部输入的信息在规定时限内处理完毕并作出反应的操作系统称为_____。

A.分时操作系统 B.批处理操作系统

C.实时操作系统 D.多处理机操作系统

答:C

47. _A_以操作系统为支撑环境,也就是说,操作系统为__A__提供服务。操作系统紧贴__B__并把__B__改造成功能更强大,使用更方便的__C__,操作系统本身的活动部分并发,部分顺序地执行,并发部分称为__D__,顺序部分称为__E__。

供选择的答案:

A:(1)系统软件 (2)应用软件 (3)非中断驱动软件 (4)中断驱动软件

B,C,D,E:(1)CPU (2)裸机 (3)微核 (4)外核 (5)个人计算机 (6)内核 (7)虚拟计算机 (8)网络计算机

答:A—(2) B—(2) C—(7) D—(6) E—(4)

二、填空题

1.计算机系统是由_____系统和_____系统两部分组成。

答:硬件,软件

2.采用多道程序设计技术能充分发挥_____与_____并行工作的能力。

答:CPU,外设

3.操作系统是计算机系统的一种系统软件,它以尽量合理、有效的方式组织和管理计算机的_____,并控制程序的运行,使整个计算机系统能高效地运行。

答:资源

4.多道程序设计的特点是多道、_____和_____。(西安电子科技大学 2001 年研究生试题)

答:宏观上并行、微观上串行

5.操作系统向用户提供了两类接口,一类是_____,另一类是_____。(西安电子科技大学 2001 年研究生试题)

答:命令级接口,程序级接口

6.多道程序的设计是利用了_____和_____的并行工作能力来提高系统效率的。(陕西省 1995 年自考题)

答:CPU, 外设

7.为了实现 CPU 与外部设备的并行工作,系统引入了_____硬件机制。(北京大学 1997 年研究生试题)

答:中断

8.操作系统是计算机系统中的一个_____,它管理和控制计算机系统中的_____。(清华大学 1999 年研究生试题)

答:系统软件,软件和硬件资源

9.允许多个用户以交互式使用计算机的操作系统称为_____。

答:分时操作系统

10. 允许多个用户将多个作业提交给计算机集中处理的操作系统称为_____。

答：批处理操作系统

11. 计算机系统能及时处理过程控制数据并做出响应的操作系统称为_____。

答：实时操作系统

12. 对操作系统的总体设计目标来说，批处理操作系统应注重提高计算机的效率，尽量增加系统的_____，分时操作系统应保证用户_____；而实时操作系统则应在保证及时响应和处理有关事件的前提下，再考虑_____。

答：平均吞吐量，所能忍受的响应时间，系统资源的利用率

13. 在一个具有分时兼批处理的计算机操作系统中，如果有终端作业和批处理作业混合同时执行，则_____作业应优先占用处理机。

答：终端型

14. 为了实现多道程序设计，计算机系统在硬件方面必须提供两种支持，它们是_____和_____。

答：中断，通道

15. 批处理系统的基本特征是“批量”，它把提高作业的_____作为主要设计目标，同时也兼顾作业的_____。

答：吞吐量，周转时间

16. 单道批处理系统是在解决_____和_____的矛盾中发展起来的。

答：人机矛盾，CPU 和 I/O 设备之间速度不匹配

17. 操作系统的基本功能包括__①__管理、__②__管理、__③__管理、__④__管理。除此之外还为用户使用操作系统提供了用户接口。

答：①处理机　②存储器　③设备　④文件

18. 如果一个操作系统兼有批处理、分时处理和实时处理操作系统三者或其中两者的功能，这样的操作系统称为_____。

答：通用操作系统

19. 在分时和批处理系统结合的操作系统中引入了“前台”和“后台”作业的概念，其目的是_____。

答：为了提高 CPU 的利用率

20. 分时操作系统的主要特征有三个，即__①__、__②__和__③__。

答：①多路性　②交互性　③独占性

21. 实时操作系统与分时操作系统的主要区别是_____。

答：及时性和高可靠性

22. 如果操作系统具有很强的交互性，可同时供多个用户使用，但时间响应不太及时，则属于__①__类型；如果操作系统可靠，时间响应及时但仅有简单的交互能力，则属于__②__类型；如果操作系统在用户提交作业后，不提供交互能力，它所追求的是计算机资源的高利用率，大吞吐量和作业流程的自动化，则属于__③__类型。

答：①分时操作系统　②实时操作系统　③批处理操作系统

23. 在主机控制下进行的输入/输出操作称为_____操作。

答：联机输入/输出

24. 按内存中同时运行程序的数目可以将批处理系统分为两类：__①__和__②__。

答:①单道批处理系统 ②多道批处理系统

25. 并发和_____是操作系统的两个最基本的特征,两者之间互为存在条件。

答:共享

26. _____系统不允许用户随时干预自己程序的运行。

答:批处理

27. 操作系统的主要性能参数有__①__和__②__等。__①__指的是单位时间内系统处理的作业量。__②__指的是在一个给定时间内,系统的一个指定成分被使用的时间比例。

答:①吞吐率 ②利用率

三、判断题

1. 采用多道程序设计的系统中,系统的程序道数越多,系统的效率越高。(西安电子科技大学2001年研究生试题) ( )

答:错误

2. 操作系统的设计必须要保证进程具有可再现性。(电子科技大学1996年研究生试题) ( )

答:正确

3. 应用软件是加在裸机上的第一层软件。 ( )

答:错误

4. 多道程序设计是利用了CPU和通道并行工作来提高系统利用率的。(陕西省1998年自考题) ( )

答:正确

5. 操作系统特征之一的"不确定性"是指程序运行的结果是不确定的。 ( )

答:错误

6. 多道程序设计可以缩短系统中作业的执行时间。(陕西省1998年自考题) ( )

答:错误

7. 分时系统和多道程序设计技术的出现,标志着现代操作系统的正式形成。(陕西省1998年自考题) ( )

答:正确

8. 从响应的角度看,分时系统与实时系统的要求相似。 ( )

答:错误

9. 使计算机系统能够被方便地使用和高效地工作是操作系统的两个主要设计目标。 ( )

答:正确

10. 分时操作系统首先要考虑的问题是交互性和响应时间。 ( )

答:正确

11. 时间片轮转法一般用于分时系统(西安理工大学2001年研究生考试试题) ( )

答:正确

12. 实时系统的交互性允许用户访问专用的服务程序 ( )

答:错误

13. 把进行资源管理和控制程序执行的功能集中组成一种软件称之为操作系统 ( )

答:正确

14. 分时操作系统中,时间片越小越好 ( )

答：错误

15. 程序中断是不可屏蔽中断。（　　）

答：错误

16. 联机用户接口是指用户与操作系统之间的接口，它不是命令接口。（　　）

答：错误

17. 批处理系统不允许用户随时干预自己程序的运行。（　　）

答：正确

18. 实时系统中的实时控制系统只能用于生产过程的自动控制，不能用于监测制导性控制。（　　）

答：错误

19. 操作系统的功能要点是系统资源管理和提供用户界面。（　　）

答：正确

20. 在实时系统中，用户进程的优先级可以高于系统进程的优先级。（　　）

答：正确

21. 对批处理作业，运行时不须提供相应的作业控制信息。（　　）

答：错误

22. 用零作除数将产生异常。（　　）

答：正确

23. 断点与恢复点是一致的。（　　）

答：错误

24. 管态下能执行所有指令。（　　）

答：正确

25. 资源共享是现代操作系统的一个基本特征（　　）

答：正确

26. 用户在程序中可以直接使用系统调用。（　　）

答：正确

四、简答题

1. 什么是操作系统？它有什么基本特征？

答：操作系统是指控制和管理计算机的软、硬件资源，合理组织计算机的工作流程，方便用户使用的程序集合。

操作系统具有 4 个基本特征。

(1)并发性。指宏观上在一段时间内有多道程序在同时运行，而微观上这些程序是在交替执行。

(2)共享性。因程序的并发无规律，使系统中的软、硬件资源不再为某个程序独占，而是由多个程序共同使用。

(3)虚拟性。多道程序设计技术把一台物理计算机虚拟为多台逻辑上的计算机，使得每个用户都感觉是“独占”计算机。

(4)不确定性。多道程序系统中，各程序之间存在着直接或间接的联系，程序的推进速度受到其他程序的影响。这样，程序运行的顺序，程序完成的时间以及程序运行的结果都是不确定的。

2. 什么是多道程序设计技术？多道程序设计的优点是什么？为什么说直到出现中断和通道技术后，多道程序概念才变为有用的？

答：多道程序设计技术是指同时把多个作业(程序)放入内存并允许它们交替执行和共享系统中的各类资源；当一道程序因某种原因(如 I/O 请求)而暂停执行时，CPU 立即转去执行另一道程序。操作系统在引入多道程序设计技术后，使得系统具有了多道，宏观上并行，微观上串行的特点。

采用多道程序设计减少了 CPU 时间的浪费，增加了系统吞吐量，提高了系统的效率。

多道程序并发执行是指有的程序正在 CPU 上执行，而另一些程序正在 I/O 设备上进行传输，即通过 CPU 操作与外设传输在时间上的重叠减少 CPU 时间的浪费，并提高了系统的效率。实现 CPU 操作与外设传输在时间上的重叠必须有中断和通道技术支持，其原因如下：

(1)通道是一种控制一台或多台外部设备的硬件机构，它一旦被启动就独立于 CPU 运行，从而做到了输入输出操作与 CPU 并行工作。但早期 CPU 与通道的联络方法是由 CPU 向通道发出询问指令来了解通道工作是否完成。若未完成，则主机就循环询问直到通道工作结束为止。因此，这种询问方式是无法真正做到 CPU 与 I/O 设备并行工作的。

(2)在硬件上引入了中断技术。所谓中断，就是在输入输出结束时，或硬件发生某种故障时，由相应硬件(即中断机构)向 CPU 发出信号。这时 CPU 立即停下手头的工作而转向处理中断请求，待处理完中断后再继续原来手头的工作。

因此，通道技术和中断技术结合起来就可实现 CPU 与 I/O 设备并行工作，即 CPU 启动通道传输数据后便去执行其他程序的计算工作，而通道则进行输入输出操作；当通道工作结束时，再通过中断机构向 CPU 发出中断请求，CPU 则暂停正在执行的操作，对出现的中断进行处理，处理完后则继续原来的工作。这样，就真正做到了 CPU 与 I/O 设备并行工作。此时，多道程序的概念才变为现实。

3. 操作系统是随着多道程序设计技术的出现逐步发展起来的，要保证多道程序的正常运行，在技术上需要解决哪些基本问题？(北方交通大学 1999 年研究生试题)

答：要保证正常运行，在技术上需要解决的问题如下：

(1)在多道程序之间应如何分配被它们共享的处理机，使得 CPU 既能满足各程序运行的需要，又有较高的利用率。此外，一旦将 CPU 分配给某程序后，应何时收回。

(2)如何为每道程序分配必要的内存空间，使它们各得其所，但又不会因相互重叠而丢失信息。此外，应如何防止因某道程序出现异常情况而破坏其他程序。

(3)系统中可能有多种类型的 I/O 设备供多道程序共享，应如何分配这些 I/O 设备，如何做到既方便用户对设备的使用，又能提高设备的利用率。

(4)在现代计算机系统中通常都存放着大量的程序和数据，应如何组织它们才便于用户使用。此外，还有信息保存的安全性和一致性问题。

(5)对于系统中的各种应用程序，它们有的属于计算型，有的属于 I/O 型，有些既重要又紧迫，有些又要求系统能及时响应，这时系统应如何组织和安排这些作业(程序)的工作流程。

4. 批处理、分时和实时操作系统各有什么特点？(武汉大学 1997 年研究生试题)

答：批处理、分时和实时操作系统的各自特点如下。

(1)批处理系统。操作人员将作业成批装入计算机并由计算机管理运行，在程序的运行期间用户不能干预，因此批处理系统的特点是：用户脱机使用计算机，作业成批处理，系统内多道程序并发执行以及交互能力差。

(2)分时系统。不同用户通过各自的终端以交互方式共用一台计算机，计算机以"分时"的方法轮流为每个用户服务。分时系统的主要特点是：多个用户同时使用计算机的同时性，人机问答的交互性，每个用户独立使用计算机的独占性，以及系统响应的及时性。

(3)实时系统。实时监控控制对象并能做出及时反应。实时系统的特点为：可靠性高、响应及时但资源利用率低。

5.分时系统和实时系统有什么区别？设计适用实时环境的操作系统的主要困难是什么？(西北工业大学2000年研究生试题)

答：分时系统和实时系统的主要区别如下：

(1)系统的设计目标不同。分时系统的设计目标是提供一种随时可供多个用户使用的通用性很强的系统，而实时系统则大多数都具有某种特殊用途，是一种专用系统。

(2)交互性强弱不同。分时系统的交互性强而实时系统的交互性弱。

(3)响应时间长短不同。分时系统的响应时间以人能够接受的时间为准，通常为秒级；而实时系统的响应时间则因受控对象而异，变化范围较大。

设计适用于实时环境的操作系统的主要困难是：在实时环境规定的时间限额内对用户作出相应的反应。如果系统不能在规定的时间限额内完成指定的任务，那么就可能导致整个实时系统的崩溃。因此，在设计这类操作系统时，设计者必须保证所采用的调度策略及相关技术不会使响应时间超过实时环境所规定的时间限额。

6.分时操作系统形成和发展的主要动力是什么？

答：如果说批处理系统形成和发展的主要推动力是不断提高系统资源利用率和提高系统吞吐量，那么，分时系统形成和发展的主要动力则是为了更好地满足用户需要，表现为如下几个方面。

(1)缩短了作业周转时间。在批处理系统中，用户将作业提交给系统，需要等一批作业都运行完成后才能得到结果，这对那些执行时间很短的小作业用户来说是不愿接受的。分时系统则是分时为每个终端用户服务，当一个程序运行结束时，用户立即可获得结果而无须等到所有用户程序都运行结束。

(2)提供了人机交互能力。在批处理系统中，用户一旦把作业提交给系统就放弃了对作业的控制能力。并且为了使系统能够运行作业，用户还需编写作业说明书，并在作业说明书中给出作业运行中可能出现的各种情况的处理方法，这无疑增加了用户的难度。因此，用户迫切需要能够做到边运行、边调试，以使可以随时针对所出现的问题采取适当的解决办法，亦即用户希望系统提供良好的人机交互界面。

(3)独立地使用计算机。各用户都希望自己能够独立地使用计算机而不受其他用户的干扰。

7.如何理解操作系统的不确定性？

答：在单道程序环境下，由于程序独占全机软、硬件资源，且完全在一个封闭环境下运行，所以，如果使用相同的输入，则多次运行的结果必然相同。

引入多道程序技术后，由多道程序共享全机软、硬件资源，即各程序之间存在着直接或间接的联系。此时程序的运行就不是在一个封闭的环境下，要受到其他程序和系统资源的影响。这会引起程序运行的结果、程序运行的顺序和完成的时间不确定。此外，在多道程序环境下，外部设备的中断、输入输出的请求等出现的时间都不确定。这些不确定因素给操作系统设计带来了很大的复杂性。对于程序执行结果的不确定是绝不允许的，这是操作系统必须解决的主要问题；而程序运行顺序及完成时间的不确定是多道程序环境所决定的。因此，

操作系统应能对运行顺序进行协调和管理。至于外部设备的中断、输入输出请求甚至程序运行故障等不确定性，操作系统应在出现的时刻及时响应并予以解决。如果操作系统的设计考虑不周，也可能造成系统的混乱和崩溃。

8. 在用户与操作系统之间存在哪几种类型的接口？它们的主要功能是什么？（哈尔滨工业大学 2000 年研究生试题）

答：用户与操作系统之间的接口主要有系统调用、键盘命令和作业控制命令。系统调用可直接写在用户程序中，它是为了方便地使用系统资源而提供的系统调用命令。键盘命令用于联机交互环境，用户从键盘上调入命令后操作系统解释并执行这个命令。作业控制命令则用于脱机批处理环境下的作业说明书中，用户用这些命令编写作业说明书，以告诉系统对作业的控制意图和处理方式 。

9. 下列哪些指令必须是特权指令（也即只能在核心模式下运行）？

A. 改变存储器管理的寄存器　　B. 写程序指针

C. 读取日期时钟　　D. 设置日期时钟

E. 改变处理器的优先级

答：A. 是的　B. 是　C. 不是　D. 是的　E. 是的

10. 下列哪些指令必须是特权指令（也即只能在核心模式下运行）？

A. 设置核心模式　　B. 系统重启动

C. 读取程序状态字　　D. 关闭中断

E. 写指令寄存器

答：A. 是的　B. 是的　C. 是　D. 是的　E. 是

11. 操作系统的三个主要用途是什么？

答：（1）以方便和有效的方式，为计算机用户在计算机硬件上执行程序提供环境。

（2）为解决特定的任务，分配不同的计算机资源。分配进程应该尽可能公正和有效。

（3）作为控制程序，提供两个主要功能：①为防止出错和误操作，监控用户程序的执行；②管理和控制各种 I/O 设备。

12. 系统调用的作用是什么？

答：通过系统调用，使用户进程可以访问操作系统提供的服务。

13. 一个分层结构操作系统由裸机，用户，CPU 调度和进程同步操作，文件管理，作业管理，内存管理，设备管理，命令管理等部分组成。试按层次结构的原则从内到外将各部分重新排列。（中科院计算所 1997 年研究生试题）

答：按层次结构的原则从内到外依次为：裸机，CPU 调度，进程同步操作，内存管理，作业管理，设备管理，文件管理，命令管理和用户。

14. 对于一个正确运转的计算机系统，保护操作系统是很重要的。但为了向用户提供更大的灵活性，应尽可能少地对用户加以限制。下面列出的各操作通常是加以保护的。试问至少有哪几条指令需加以保护？（北京理工大学 1996 年研究生试题）

（1）改变成用户方式；

（2）改变成系统方式；

（3）从存放操作系统的存储区上读取数据；

（4）将数据写到存放操作系统的存储区上；

（5）从存储操作系统的存储区取指令；

(6)打开计时器

(7)关闭计时器。

答:至少第(2)(4)操作应加以保护。因为这两条操作是对操作系统本身的内容进行修改,如果交由用户任意操作就会对操作系统造成破坏并最终导致系统运行出错或系统崩溃,所以应在任何情况下都加以保护。其他几种操作在一般情况下也应加以保护,但不会出现像上述两种操作那样的破坏性,而本题是针对最少保护而言的,故不做选择。

15. 试从独立性、同时性、交互性和及时性等 4 个方面来比较批处理系统、分时系统和实时系统的性能。

答:分析比较见表 1.1 所示。

表 1.1　批处理、分时与实时三种系统的比较

特征 \ 类型	批处理系统		分时系统	实时系统
	单道	多道		
独立性	无	无	有	有(最强)
同时性	无	有	有(最强)	有(较弱)
交互性	无	无	有(最强)	有(较弱)
及时性	无	无	有	有(最强)

16. 从宏观结构上看,操作系统有哪几种结构设计方法?你认为哪几种方法较好?为什么?(武汉大学 1999 年研究生试题)

答:从操作系统结构设计观点来看,早期的操作系统主要是单(处理)机操作系统,在结构设计方法上主要采用了以下两种模式。

(1)整体结构设计模式。把操作系统组织成一个过程模块集合,其中的任何一个过程都可以调用其他过程。这种整体式结构的特点不强调信息的隐蔽,即每个过程对其他过程都是可见的,过程之间调用不受任何拘束。但整体结构的系统扩充起来较为困难,因为修改一个过程可能导致系统中其他若干过程的修改,而且一个过程中的错误可能会株连到其他过程的正确运行,即操作系统的正确性难以保证。

(2)层次式结构设计模式。层次式方法把操作系统划分为若干层,每一层有若干模块,每个模块提供一组可被其他模块调用的功能;并且,在任一特定层次上的代码只能调用较低层次上的代码。按此种模式构造的第一个操作系统是 Dijkstra 开发的 THE 系统(1968 年)。层次式结构设计的优点是把模块间的复杂依赖关系转化为单向依赖关系,即高层软件依赖低层软件,而低层软件不依赖高层软件。这样,在设计低层软件时,可不考虑高层软件的实现方法,且高层软件的错误也不会影响到低层软件。这种单向方法给维护和调试操作系统的工作带来了方便。

传统的整体式、层次式结构设计法对单处理机的操作系统结构设计是卓有成效的,但对多计算机系统(如网络系统、分布式系统及多处理机系统)而言则有较大缺陷,不能满足需要。多计算机系统中操作系统结构设计的主要模式有以下几种。

(3)客户/服务器模式。把操作系统划分为若干个进程,且每个进程实现单独的一套服务功能。采用客户/服务器模式构造的操作系统,其组成部件小而自成一个独立的子系统。由于每个服务器是以独立的用户态进程方式进行的,因而单个服务器的故障不会引起操作系

统其他部分的崩溃。这种模式的优点是简化了基本操作系统，提高了可靠性并适合于分布式计算环境。

(4)对象模式。面向对象的程序设计方法就是把系统中所有的资源，如进程、文件、内存，将此封装看成是一个实体。从程序设计者来看，对象是一个程序模块；从用户来看，对象为他们提供了所希望的操作。采用面向对象的方法，程序员可以依照自己的意图去构造自己的对象并将问题映射到对象上。这种方法直观而又自然，程序员可以把主要精力放在系统级上，而对实现的细节不必过多关心。如 Windows NT 操作系统广泛使用了对象来表示共享的系统，但 Windows NT 只能算是一个基于对象的系统。

(5)对称多处理模式。如果一个操作系统在系统中的所有处理机上运行且共享同一内存，这样的系统就是一个对称多处理系统。采用对称多处理模式构造的操作系统最适合于共享存储器结构的多处理机系统，即紧耦合的多处理机系统。

综上所述，各种模式都是针对不同操作系统的，因此，只有在确定了操作系统的类型后才能决定不同方法的好坏。

17. 什么是分布式操作系统？它与网络操作系统有何不同？试说明分布式操作系统或网络操作系统在传统的操作系统管理模式上需要哪些改进。(西北大学 2000 年研究生试题)

答：分布式操作系统是以实现并行任务分配，并行进程通信，分布控制机构，以及实现分散资源管理等功能为目的的系统程序。网络操作系统是以资源共享和信息交换为目的的操作系统。

分布式系统和网络操作系统都是多机系统的支撑软件，都基于 I/O 或网络互联，但在很多情况下，网络操作系统是在本机局部操作系统之上建立的，形成了两个层次；而分布式操作系统则是以计算机网络为基础所构成的一个独立的整体，它对用户来说是透明的。

分布式操作系统或网络操作系统除了具有传统操作系统所应有的处理机管理、存储管理、设备管理和文件管理功能外，还应提供以下功能：

(1)实现网络中各节点机之间的通信；

(2)实现网络中硬、软件资源的共享；

(3)提供多种网络服务软件；

(4)提供网络用户的应用程序接口。

对分布式系统来说，除了具有网络操作系统的功能外，还因各节点机不存在主从或层次关系，因而增加了控制机构的复杂性。首先，由于各节点的自治性，它们之间发生冲突的概率要高得多，使同步问题变得复杂，死锁的问题也难以处理。其次，由于系统透明性的要求，使得系统故障的检测和用户操作的检查都变得更加困难。

18. 试对分时操作系统和实时操作系统进行比较。

答：我们可以从以下几个方面对这两种操作系统进行比较：

A. 实时操作系统与分时操作系统一样都能为多个用户服务。系统按分时原则为多个终端用户服务；而对实时控制系统(属于实时操作系统范畴)，则表现为经常对多路现场信息进行采集以及对多个对象或多个执行机构进行控制。

B. 实时操作系统与分时操作系统一样，每个用户各占一个终端，彼此独立操作，互不干扰。因此用户感觉就像他一人独占计算机；而在实时控制系统中信息的采集和对象的控制也都是彼此互不干扰的。

C. 实时操作系统对响应时间的要求与分时操作系统类似，都是以人所能接受的等待时间

来确定的；而实时控制系统的响应时间则是以控制对象所能接受的延时来确定的。

D. 分时操作系统是一种通用系统，主要用于运行终端用户程序，因此它具有较强的交互能力。而实时操作系统虽然也有交互能力，但其交互能力不及前者。

E. 分时操作系统要求系统可靠，相比之下，实时操作系统则要求系统高度可靠。

第2章

进程管理

2.1 概述

进程可以被简单地理解为一个正在执行中的程序。为了完成任务，进程需要一些资源，比如使用CPU时间、存储器、文件以及I/O设备。这些资源可以在进程创建时或者运行的时候分配给它。

在大部分系统中，进程是工作的单元。这种系统由一组进程组成：执行系统代码的操作系统进程，执行用户代码的用户进程。所有这些进程都会同时执行。

尽管传统上一个进程在运行时只有一个控制线程，但大部分的现代操作系统都支持多线程的进程。

操作系统在管理进程与线程的过程中需要负责以下工作：用户和系统进程的创建和删除；进程的调度；为进程的同步、通信、死锁处理提供机制。

2.2 进程概念

进程是执行中的程序。随着进程的执行，它改变状态。进程状态由进程当前活动所定义。每个进程可处于如下状态：创建、就绪、运行、等待或终止。每个进程在操作系统内通过自己的PCB(Process Control Block，进程控制块)来表示。

当前不在执行的进程会放在某个等待队列中。操作系统的两个主要队列是I/O请求队列和就绪队列。就绪队列包括所有可执行的并等待CPU的进程。每个进程都有PCB，PCB链接起来就形成了就绪队列。长期调度策略选择进程来竞争CPU。通常，长期调度会受资源分配考虑尤其是内存管理的影响。短期调度从就绪队列中选择进程。

系统内的进程能并发执行。有许多理由允许并发执行：信息共享，加快计算，模块化和方便性。并发执行需要有创建和删除进程的机制。

操作系统的执行进程可以是独立进程或协作进程。协作进程要有互相通信的机制。主要有两种形式的通信：共享内存和消息系统。共享内存方法要求通信进程共享一些变量。进程通过使用这些共享变量来交换信息。对于共享内存系统，提供通信的责任主要在于应用程序；操作系统只需要提供共享内存。消息系统方法允许进程交换信息。提供通信的主要责任在于操作系统本身。这两种方法并不互相排斥，能在同一操作系统同时实现。

套接字(Socket)定义为通信的端点。两个应用程序间的连接由一对套接字组成,通信频道每端各有一个。RPC(Remote Process Call,远程过程调用)是另一种形式的分布式通信。当一个进程(或线程)调用一个远程应用的方法时,就出现了RPC。RMI(Remote Method Invocation,远程方法调用)是RPC的Java版。RMI允许线程如同调用本地对象的方法一样来调用远程对象的方法。RPC和RMI的主要区别是RPC传递给远程过程的数据是按普通数据结构形式的,而RMI允许把对象传递给远程方法。

2.3 线程

线程是进程内的控制流。多线程进程在同一地址空间内包括多个不同的控制流。多线程的优点包括对用户响应的改进、进程内的资源共享和利用多处理器体系结构的能力。

用户线程对程序员来说是可见的,而对内核来说是未知的。用户空间的线程库通常用以管理用户线程。操作系统支持和管理内核级的线程。通常,用户线程跟内核线程相比,创建和管理要更快。有三种不同模型将用户和内核线程关联起来:多对一模型将许多用户线程映射到一个内核线程;一对一模型将每个用户线程映射到一个相应的内核线程;多对多模型将多个用户线程在同样(或更少)数量的内核线程之间切换。

多线程程序为程序员带来了许多挑战,包括系统调用fork和exec的语义。其他事项包括线程撤销、信号处理和特定线程数据。许多现代操作系统提供了对线程的内核支持,如Windows NT, Windows 2000, Solaris 2。Pthread API提供了一组函数用于在用户级创建和管理线程。Java提供了相似的API以支持线程。不过,由于Java线程是由JVM,而不是由用户级线程库或内核来管理的,因此它并不属于用户级线程或内核级线程。

2.4 CPU调度

CPU调度的任务是从就绪队列中选择一个等待进程,并为其分配CPU。CPU由调度程序分配给所选中的进程。

FCFS(First-Come-First-Served,先来先服务)调度是最简单的调度算法,但是它会让短进程等待非常长的进程。SJF(Shortest Job First,最短作业优先)调度可被证明是最佳的方法,它提供了最短平均等待时间。但实现SJF调度比较困难,因为预测下一个CPU使用时间的长度有难度。SJF算法是通用优先权调度算法(将CPU简单地分配给具有最高优先权的进程)的特例。优先权调度和SJF调度会产生饥饿。老化(Aging)技术可阻止饥饿。

RR(Round-Robin,时间片轮转)调度对于分时(交互)系统更为合适。RR调度让就绪队列的第一个进程使用CPU的q个时间单元,这里q是时间片。在q时间单元之后,如果该进程还没有释放CPU,那么它被抢占并放到就绪队列的尾部。该算法的主要问题是选择时间片。如果时间片太大,那么RR调度就成了FCFS调度;如果时间片太小,那么因上下文切换而引起的调度开销就过大。

FCFS算法是非抢占的;RR算法是抢占的。SJF和优先级算法既可以是抢占的也可以是非抢占的。

多级队列算法允许多个不同算法用于各种类型的进程。最为常用的是前台交互队列(使用RR调度),和后台批处理队列(使用FCFS调度)。多级反馈队列允许进程在队列之间迁

移。

因为有多种不同的调度算法可用，所以我们需要某种方法来选择它们。分析方法使用数学分析的方法以确定算法性能。模拟方法通过对具有代表性的进程采用调度算法模拟并计算其性能来确定优劣。

如果操作系统在内核级支持线程，那么必须调度线程而不是进程来执行。Solaris 2 和 Windows 2000 就是这样的系统，它们采用抢占的、基于优先级的调度算法，并支持实时线程。Linux 进程调度也使用基于优先级算法，并提供实时支持。这三种操作系统通常都偏爱交互进程而不是批处理进程或 CPU 约束进程。

2.5 进程同步

必须为共享数据的一组相互协作的顺序进程提供互斥。一种解决方法是确保在某个时候只能有一个进程或线程才能使用代码的临界区域。当然有许多不同方法来解决临界区问题。

这些用户级解决方案的主要缺点是它们都需要忙等待。信号量可克服这个困难。信号量可用于解决各种同步问题，且可高效地加以实现(尤其是在硬件支持原子操作时)。

各种不同的同步问题(如有限缓存区问题、读写问题、哲学家进餐问题)均很重要，这是因为这些问题是大量并发控制实际问题的抽象。这些问题常用于测试几乎所有新提出的同步方案。

操作系统必须提供机制以防止定时出错问题。人们提出了多个不同结构以处理这些问题。临界区域(Critical Region)可安全高效地解决互斥和任意同步问题。管程(Monitor)为共享抽象数据类型提供了同步机制。条件变量(Conditional Variable)提供了一个方法供管程程序以阻塞其执行，直到其被通知可继续为止。

事务(Transaction)是个程序单元，必须原子执行；即，要么与其相关的所有操作执行完毕，要么什么操作都不做。为了确保原子性(即使在系统出错时)，我们可以使用写前记录日志的方法。所有的更新都记录在日志上，而日志保存在稳定存储上。如果系统出现死机，日志信息可用于恢复更新数据项的状态，这由操作 undo 和 redo 所组成。为了降低在系统出错时搜索日志的额外开销，我们可使用检测点(Check Point)方案。

当多个事务重叠执行时，这样的执行可能不同于这些操作的原子执行。为了确保正确执行，我们必须使用并发控制方案以保证串行化。有各种不同并发控制方案以确保串行化：延迟操作或撤销发布这个操作的事务。最为常用方法为加锁协议和基于时间戳的协议。

2.6 死锁

如果两个或更多的进程永久等待某个事件而该事件只能由这些等待进程的某一个引起，那么会出现死锁状态。从原理上来说，有三种方法可以处理死锁：

- 使用一些协议来预防或避免死锁，确保系统永远都不会进到死锁状态。
- 允许系统进入死锁状态，检测死锁，并恢复。
- 忽略所有的问题，并假设系统中永远都不会出现死锁。这种方法被绝大多数的系统所采用，包括 UNIX。

当且仅当系统内的四个必要条件同时成立时(互斥、占有并等待、非抢占、循环等待)，才会

发生死锁。为了预防死锁，我们要确保这四个必要条件中的一个不成立。

死锁避免算法要比预防算法要求低些，只要事先了解进程使用资源的情况即可。例如，银行家算法需要知道每个进程所请求的每种资源的最大数量。采用这种信息，我们可采用死锁避免算法。

如果是不采用协议以确保死锁不会发生，那么就必须使用检测并恢复方案。必须调用检测算法以确定是否出现了死锁。如果检测到死锁，那么系统必须通过终止某些死锁进程，或通过抢占某些死锁进程的资源从死锁中恢复。

如果系统主要根据代价以选择牺牲进程进行回滚，那么可能会出现饥饿，导致所选择进程永远不能完成其指定任务。

2.7　习题与解答

2.7.1　进程的概念

1. 已知一个求值公式$(A^2+3B)/(B+5A)$，若 A、B 已赋值，试画出该公式求值过程的前趋图。

答：上述公式的求值过程可分解为以下分量计算：

S1：X1 = A * A;

S2：X2 = 3 * B;

S3：X1 = 5 * A;

S4：X4 = X1 + X2;

S5：X5 = B + X3;

S6：X6 = X4/X5;

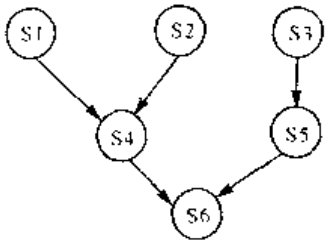


图 2.1　求值过程的前趋图

其中的 Xi 是中间变量。上述分量计算语句中，部分语句是可以并发执行的，而部分语句之间则存在着一定的顺序关系。求值过程的前趋图如图 2.1 所示。

2. 操作系统中为什么要引入进程的概念？为了实现并发进程间的合作和协调工作，以及保证系统的安全，操作系统在进程管理方面应做哪些工作？（南京大学 1997 年研究生试题）

答：为了提高资源的利用率和系统的吞吐量，操作系统中广泛采用多道程序设计技术，它允许多个程序同时进入内存、并发地运行。程序在并发执行时，失去了封闭性，出现了间断性和不可再现性的特征，这说明了通常的程序是不能参与并发执行的。为了使程序能够正确地并发执行，并能够对并发执行的程序加以描述和控制，操作系统中引入了进程的概念。

操作系统在进程管理方面应做如下几方面的工作：

(1)进程控制。其主要工作是创建和撤消进程，以及控制运行中进程的状态转换。

(2)进程调度。其主要工作是按一定的算法，选择一新的就绪进程，把处理机分配给它，并为它设置运行的现场，使其投入执行。

(3)进程同步。其主要工作是通过同步机制对并发执行的进程的执行次序进行协调(具体的协调方式有互斥和同步两种)，以保证各个进程均能有条不紊地运行。

(4)进程通信。其主要工作是实现进程之间的信息交换，以使进程之间能更好地相互合作。

(5)死锁的预防、避免、检测，以及从死锁状态中恢复。

3. 什么是进程控制块？试从进程管理、进程通信、中断处理、文件管理、存储管理、设备管理的角度设计进程控制块应包含的项目。(北京大学 1999 年研究生试题)

答:进程控制块(PCB)是为了使在多道程序环境下不能独立运行的程序成为能独立运行的进程,而为每个程序所配置的一个数据结构,其中存放了用于描述该进程情况和控制进程运行所需的全部信息。系统根据 PCB 而感知相应进程的存在,即 PCB 是进程存在的惟一标志。

PCB 中应该包含以下信息:

(1)从进程管理的角度考虑,PCB 中应该包含进程标识符、进程状态、CPU 状态信息(包括程序计数器、程序状态字、栈指针、通用寄存器等)、进程调度信息(如进程的优先数等)、链接指针(用于将 PCB 链入各种队列)等项信息。

(2)从进程通信的角度考虑,PCB 中应该包含消息队列指针、实现消息队列互斥访问的互斥信号量、描述消息队列中消息个数的资源信号量。

(3)从中断处理的角度考虑,PCB 中应包含 CPU 状态信息(参见(1))。

(4)从文件管理的角度考虑,PCB 中应包含用户文件描述符表,用来登记用户打开的各个文件,并可以通过它找到在内存的相应文件的 FCB(如 UNIX 中的内存索引结点)。

(5)从存储管理的角度考虑,应保存该进程的程序、数据、堆栈在内存和外存的地址和各部分的长度等信息。

(6)从设备管理角度考虑,应有该进程所需资源和已分配到的资源清单。

4. 叙述进程与程序的主要区别。(哈尔滨工业大学 2000 年研究生试题)

答:进程是可并发执行的程序在一个数据集合上的一次执行过程,而程序是指令的有序集合,它们是紧密相关而又完全不同的两个概念。

(1)每个进程实体中均包含了程序段,因此说进程与程序是紧密相关的。但从结构上看,进程实体中除了程序段和数据段外,还必须包含一个数据结构,即进程控制块 PCB。

(2)进程是程序的一次执行过程,因此是动态的;动态性还表现在进程由创建而产生、由调度而执行、由撤消而消亡,即它具有一定的生命周期。而程序则只是一组指令的有序集合,并可永久地存放在某种介质上,其本身不具有运动的含义,因此是静态的。

(3)多个进程实体可同时存放在内存中并发地执行,其实这正是引入进程的目的。而程序(在没为它创建进程时)的并发执行具有不可再现性,因此程序不能正确地并发执行。

(4)进程是一个能够独立运行、独立分配资源和独立接受调度的基本单位。而程序(在没为它创建进程时),因其不具有 PCB,是不可能在多道程序环境下独立运行的。

(5)进程与程序不一一对应。同一个程序的多次运行,将形成多个不同的进程;同一个程序的一次执行也可以产生多个进程(如 UNIX 中通过 fork 调用);而一个进程也可以执行多个程序(如 UNIX 中通过 exec 调用)。

5. 试述线程与进程的主要异同点。

答:进程的引入是为了使程序能够并发执行,而线程的引入则是为了减少并发执行的开销,使 OS 具有更好的并发性。因此,进程和线程一样都可以并发执行,都可以在执行、就绪、阻塞这三种基本状态之间转化;它们都有自己的生命周期,并因创建而产生,由调度而执行,因得不到资源而阻塞,因撤消而消亡;它们之间都存在着相互制约关系,因此都需要同步。进程和线程的主要区别如下:

(1)调度性。在传统的操作系统中,拥有资源的基本单位和独立调度、分派的基本单位都是进程。而在引入线程的 OS 中,则把线程作为调度和分派的基本单位,而把进程作为拥有资源的基本单位。

(2)并发性。在引入线程的 OS 中,不仅进程间可以并发执行,而且在一个进程的多个线程间,也可以并发执行,因而比传统的 OS 具有更好的并发性。

(3)拥有资源。在这两种 OS 中,拥有资源的基本单位都是进程。线程除了一点在运行中必不可少的资源(如线程控制块、程序计数器、一组寄存器值和堆栈)外,本身基本不拥有系统资源,但它可访问其隶属进程的资源。

(4)开销。由于创建或撤消进程时,系统都要为之分配和回收资源,如内存空间和 I/O 设备等;进程切换时所要保存和设置的现场信息也要明显地多于线程,因此,OS 在创建、撤消和切换进程时所付出的开销将显著地大于线程。另外,由于隶属于同一进程的多个线程共享同一地址空间和已打开文件,从而使它们之间的同步和通信的实现也变得更为容易。

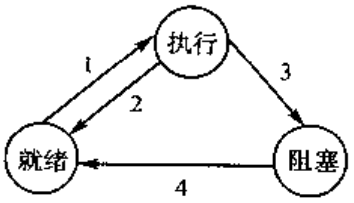


图 2.2　进程状态转换图

6. 某系统的进程状态转换图如图 2.2 所示:

(1)说明引起各种状态转换的典型事件。

(2)分析下述状态转换是否可立即引起其他的状态转换:1,2,3,4。

答:(1)引起各种状态转换的典型事件如表 2-1 所示。

表 2-1　引起进程状态转换的典型事件

状态转换	引起转换的典型事件
转换 1	CPU 调度算法将 CPU 分配给此进程
转换 2	执行进程的时间片用完,或被其他更重要的进程抢占 CPU
转换 3	等待某种事件(如 I/O 的完成,或被他人占用的临界资源变为可用状态)
转换 4	进程所等的事件发生(如 I/O 完成,或所等待的临界资源变为可用状态)

(2)状态转换 1 不会立即引起其他状态转换。

状态转换 2 必然立即引发状态转换 1:状态转换 2 发生后,进程调度程序必然要选出一个新的就绪进程投入运行,该新进程可能是其他进程,也可能是刚从执行状态转换成就绪状态的那个进程。

状态转换 3 可能立即引发状态转换 1:状态转换 3 发生后,若就绪队列空,则不会引起其他任何状态转换;否则,进程调度程序必然要选出一个就绪进程投入运行。

状态转换 4 可能引发状态转换 1:状态转换 4 发生后,若当时无其他进程处于就绪状态和执行状态,则该进程将被立即被调度。另外,状态转换 4 还可能同时引发状态转换 1 和 2:若系统采用抢占调度方式,而新就绪的进程具备抢占 CPU 的条件(如其优先权很高),则它可立即得到 CPU 转换成执行状态,而原来正在执行的进程则转换成就绪状态。

7. 某系统的进程状态图如图 2.3 所示。(武汉大学 1997 年研究生试题)

(1)说明一个进程发生变迁 3,4,6 的原因。

(2)下述因果变迁是否会发生?若会,在什么情况下发生?

A. 3→5　　B. 6→4　　C. 6→7

(3)根据此进程状态图,说明该系统的 CPU 调度策略和调度效果。

答:(1)发生变迁 3 的原因是运行进程提出 I/O 请求。发生变迁 4 的原因是运行进程的时间片到。发生变迁 6 的原因是阻塞进程因所等待的 I/O 操作完成而被 I/O 中断服务程序唤醒。

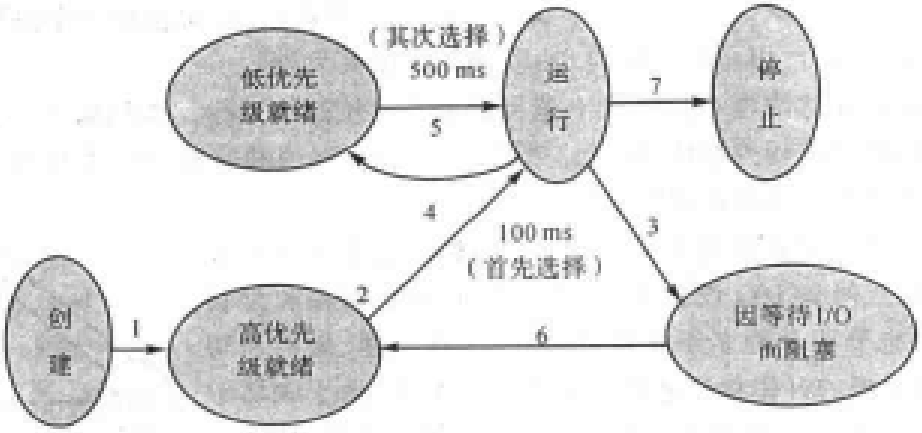


图 2.3 进程状态图

(2)A. 因果变迁 3→5 会发生。当某个运行进程由于 I/O 请求而阻塞时，如果此时系统中不存在高优先级就绪进程而只存在低优先级就绪进程，则系统将选择一低优先级就绪进程投入运行。

B. 因果变迁 6→4 不会发生。发生 4 的原因只可能是时间片用完。

C. 因果变迁 6→7 不会发生。发生 7 的原因是进程在时间片用完前运行结束。

(3)该系统的进程调度策略是多级队列调度策略，它将系统中的所有进程按照优先级的高低组织成高优先级和低优先级两个队列，两个队列均采用时间片轮转调度算法，但高优先级队列的时间片较短(为 100ms)，低优先级队列的时间片较长(为 500ms)，系统总是先调度高优先级队列上的进程，仅当该队列为空时，才调度低优先级队列上的进程。

这种调度策略不仅体现了短作业优先的思想，同时也不会使长作业长期得不到处理，而且它还能较好地满足 I/O 型作业的需要。对每个作业来说，它总是先进入高优先级就绪队列，如果作业较短，则它便能以较小的周转时间完成；如果作业较长，则在运行 100ms 后它将进入低优先级就绪队列，而且当再次调度到该作业时，它的时间片将增大到 500ms；如果是 I/O 型的作业，由于每次完成输入/输出工作后，它将进入高优先级的就绪队列，因此将被优先调度到，这将有效地提高系统中 I/O 设备的利用率和系统的吞吐量。

8. 有两个程序，A 程序按顺序使用 CPU 10s，使用设备甲 5s，使用 CPU 5s，使用设备乙 10s，最后使用 CPU 10s。B 程序按顺序使用设备甲 10s，使用 CPU 10s，使用设备乙 5s，使用 CPU 5s，使用设备乙 10s。在顺序环境下先执行 A 程序再执行 B 程序，CPU 的利用率是多少？

答：由题目所给条件可知，在顺序环境下，先执行 A 程序，再执行 B 程序时：A 程序的运行总时间为：10s + 5s + 5s + 10s + 10s = 40s

其中使用 CPU 时间为：10s + 5s + 10s = 25s

B 程序的运行总时间为：10s + 10s + 5s + 5s + 10s = 40s

其中使用 CPU 时间为：10s + 5s = 15s

因此 CPU 的利用率为：(25s + 15s)/(40s + 40s) = 50%

9. 设某计算机系统有一台输入机、一台打印机。现有两道程序同时投入运行，且程序 A 先开始运行，程序 B 后运行。程序 A 的运行轨迹为：计算 50ms，打印信息 100ms，再计算 50ms，打印信息 100ms，结束。程序 B 的运行轨迹为：计算 50ms，输入数据 80ms，再计算 100ms，结束。试说明：

(1)两道程序运行时，CPU 有无空闲等待？若有，在哪段时间内等待？为什么会空闲等待？

(2)程序 A,B 运行时有无等待现象？若有,在什么时候发生等待现象？

答:两道程序并发运行时的工作情况如图 2.4 所示。

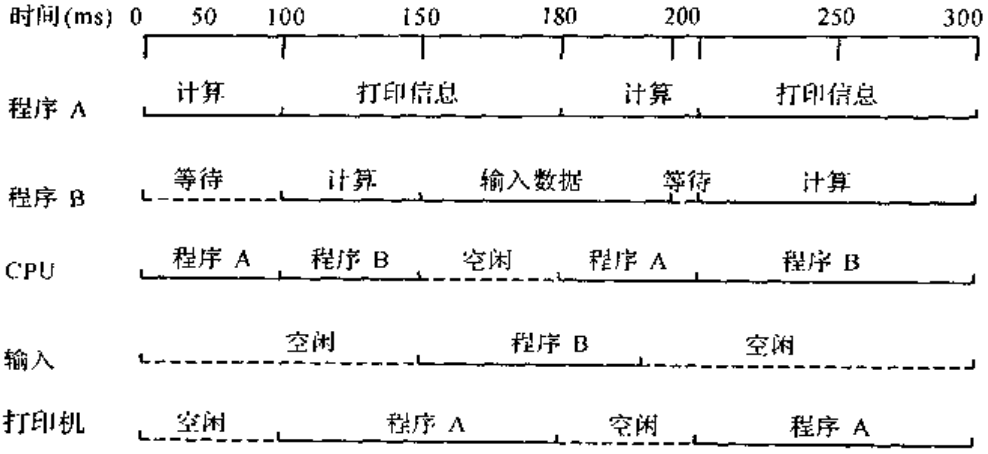


图 2.4　程序 A、B 并发执行时的情况

(1)从图 2.4 可以看出,两道程序运行时,CPU 存在空闲等待的情况。等待现象发生在程序 A 开始运行后 100ms 至 150ms 的时间段内,此时,由于程序 A 正在打印信息,而程序 B 正在输入数据,两者都进入阻塞状态,故 CPU 空闲。

(2)程序 A 运行时不存在等待现象,而程序 B 运行时则存在等待现象。程序 B 的首次等待现象发生在程序 A 开始运行后 0ms 至 50ms 的时间段内,第二次等待现象发生在程序 A 开始运行后 180ms 至 200ms 的时间段内。

10. 在一个单处理机的系统中有 A、B 两个程序,若两个程序单独执行,则需要占用 CPU、设备 IO1、设备 IO2 的时间及顺序如下:

程序 A:CPU(25ms);IO1(30ms);CPU(20ms);IO2(20ms);CPU(20ms);IO1(30ms);CPU(20ms)。

程序 B:CPU(20ms);IO1(30ms);CPU(20ms);IO2(20ms);CPU(10ms);IO2(20ms);CPU(45ms)。

若该系统中:

(a)IO1 和 IO2 为不同步的输入/输出装置,它们能够同时工作。

(b)设程序的优先级 B 高于 A。但是,当程序 A 正在占用 CPU 时,即使程序 B 需要占用 CPU,也不能打断程序 A 的执行而必须等待。

(c)当使用 CPU 之后控制转向 IO1、IO2 时,或者使用 IO1、IO2 之后控制转向 CPU 时,由控制程序执行中断处理,但这段处理时间忽略不计。

(d)程序 A、B 同时进入计算机中。

试问:

(1)哪个程序先结束?

(2)程序全部执行结束需要多少时间?

(3)到程序全部执行完毕时的 CPU 利用率为多少?

(4)程序 A 等待 CPU 的累计时间为多少?

(5)程序 B 等待 CPU 的累计时间为多少?

答:程序 A 与程序 B 的运行过程如图 2.4a 所示。因此:

(1)程序 B 先结束。

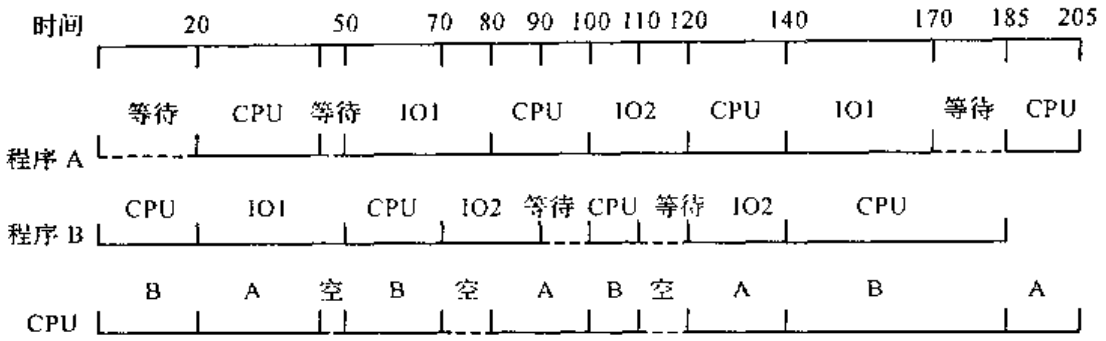


图 2.4a 程序 A、B 并发执行的情况

(2)程序全部结束需要 205ms。

(3)CPU 的利用率为(205－25)/205,即 87.8%。

(4)程序 A 等待 CPU 的时间累计为 35ms。

(5)程序 B 等待 CPU 的时间累计为 10ms。

11.程序运行时 CPU 的空闲时间,即等待 I/O 的时间占运行时间的百分比称 CPU 空转百分比,且 CPU 空转百分比因并发执行的作业道数不同而变化。在每道作业的 I/O 时间占运行时间 25%情况下的 CPU 空转百分比如表 2-2 所示。

表 2-2 CPU 空转百分比(I/O 占运行的 25%)

作业道数	1	2	3	4
CPU 空转比(%)	25	4	0.4	0

为了确定一个作业在多道程序系统中何时完成而引入了"CPU 进度"概念。所谓"CPU 进度"是指在给定的时间间隔内 CPU 为一道作业服务的累计时间。例如,两道作业同时投入运行,每道作业的 I/O 时间占运行时间的 25%且都需 1.5 小时的 CPU 时间,则完成两道作业所需时间为(查表 2-2)得到 CPU 空转百分比为 4%,也就是说在 1 小时内两作业的 CPU 进度为0.96小时,平分给每个作业其 CPU 进度为 0.48 小时。由于每个作业占用 CPU 时间是1.5小时,所以完成两道作业所需的全部时间为 1.5/0.48＝3.13(小时)。

现有一组作业,它们的提交时间和运行时间如表 2-3 所示。假定所有作业的运行时间包含 25%的 I/O 等待时间,且内存的作业平分 CPU 时间,那么在多道程序系统中,计算其作业平均周转时间和作业平均带权周转时间。

表 2-3 作业提交时间和运行时间

作业	提交时间	运行时间
1	10.0	0.3
2	10.2	0.5
3	10.4	0.1
4	10.5	0.4
5	10.8	0.1

答:首先根据作业的运行时间计算出如表 2-4 所示的作业需要 CPU 的时间:

需 CPU 时间＝运行时间×(1－I/O 占运行的百分比)

上述 5 个作业的运行情况如表 2-5 所示。因此,作业 1、2、3、4、5 的周转时间分别为0.356、

0.851、0.181、0.659、0.226，作业的平均周转时间为 0.455。它们的带权周转时间分别为 1.19、1.70、1.81、1.65、2.26，平均带权周转时间为 1.72。

表 2-4　作业需 CPU 的时间

作业	运行时间	需 CPU 时间
1	0.3	0.225
2	0.5	0.375
3	0.1	0.075
4	0.4	0.300
5	0.1	0.075

表 2-5　作业的运行情况

时间	事件	并行作业数	CPU 空转百分比	经过时间	作业	每道作业的 CPU 进度	还需 CPU 时间
10.0	J1 到达	1			J1	0	0.225
10.2	J2 到达	1	25%	0.2	J1	0.15	0.075
					J2	0	0.375
10.356	J1 结束	2	4%	0.156	J1	0.075	0
					J2	0.075	0.3
10.4	J3 到达	1	25%	0.044	J2	0.033	0.267
					J3	0	0.075
10.5	J4 到达	2	4%	0.1	J2	0.048	0.219
					J3	0.048	0.027
					J4	0	0.3
10.581	J3 结束	3	0.4%	0.081	J2	0.027	0.192
					J3	0.027	0
					J4	0.027	0.273
10.8	J5 到达	2	4%	0.219	J2	0.105	0.087
					J4	0.105	0.168
					J5	0	0.075
11.026	J5 结束	3	0.4%	0.226	J2	0.075	0.012
					J4	0.075	0.093
					J5	0.075	0
11.051	J2 结束	2	4%	0.025	J2	0.012	0
					J4	0.012	0.081
11.159	J4 结束	1	25%	0.108	J4	0.081	0

2.7.2　进程的同步和通信

1. 设有两个优先级相同的进程 P1 和 P2 如下。信号量 S1 和 S2 的初值均为 0，试问 P1、P2 并发执行结束后，x、y、z 的值各为多少？

进程 P1

```
y=1;
y=y+2;
```

进程 P2

```
x=1;
x=x+1;
```

```
V(S1);                    P(S1);
z=y+1;                    x=x+y;
P(S2);                    V(S2);
y=z+y;                    z=x+z;
```

答:可以将上述进程分解成以下 6 个程序段:

```
PS1:y=1;          PS2:z=y+1;        PS3:y=z+y;
    y=y+2;
PS4:x=1;          PS5:x=x+y;        PS6:z=x+z;
    x=x+1;
```

并将它们的并发执行关系用前趋图描述出来(如图 2.5 所示)。根据 Bernstein 条件,程序段 PS1 和 PS4 的确是能并发执行的,程序段 PS2 与 PS5 也能并发执行,而程序段 PS3 和 PS6 则不能并发执行,或者说它们的并发执行具有不可再现性。若先执行 PS3,再执行 PS6,则最后 x,y,z 的值分别为 5,7,9;若先执行 PS6,再执行 PS3,则最后 x,y,z 的值分别为 5,12,9。

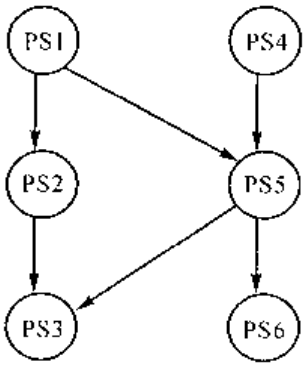


图 2.5 前趋图

2. 举例说明 P、V 操作为什么要求设计成原语(即对同一信号量上的操作必须互斥)。(北京大学 1993 年研究生试题)

答:P、V 操作是用来实现进程同步的原语,它们可以被描述为:

```
P(S)操作:
    S.value--;
    if(S.value<0){
        add this process to S.L;
        block();
    }
V(S)操作:
    S.value++;
    if(S.value<=0){
        remove a process P from S.L;
        wakeup(P);
```

例如,用 P,V 操作来实现进程对临界资源的互斥使用。此时,只需定义一个信号量 S,其初值为{1,NULL},并在临界区前执行 P(S)操作,而在临界区后执行 V(S)操作。如果 P、V 操作不设计成原语,那么在执行 P、V 操作时进程可以被中断。由于在初始状态下,临界资源空闲,故应允许第一个申请临界资源的进程进入临界区使用临界资源,但如果该进程在执行到 P 操作的语句 S.value--后(此时 S.value 的值为 0)便被另一个进程中断,而那个进程也企图通过执行 P(S)操作进入临界区,则第二个进程也必须执行语句 S.value--,从而将 S.value 的值变为 -1,并由于 S.value<0 而被阻塞,而第一个进程再次获得 CPU 后,也同样由于 S.value<0 而被阻塞,这就造成了临界资源虽然空闲但进程却申请不到临界资源的情况,也就是说,此时的 P、V 操作已无法满足同步机制的要求。同样,一个执行 P(S)操作的进程被中断后,另一进程去执行 V(S)操作;或者,一个执行 V(S)操作的进程被中断

后，而另一进程去执行 P(S)或 V(S)操作，都将发生混乱，而难以实现进程同步。因此，P、V 操作必须设计成原语的方式。

3. 试从物理观念上来说明记录型信号量 wait 与 signal 操作？（南京航空航天大学 2000 年研究生试题）

答：从物理观念上来看，一个记录型信号量 S 可以用来表示一类临界资源的使用情况，S.value的初值表示系统中该类资源的总数。在使用过程中，若 S.value>0，则表示系统中还有 S.value 个该类资源处于空闲状态。对它的每次 wait 操作，意味着进程请求一个单位的该类资源，因此描述为 S.value--；当 S.value<0 时，表示该类资源已分配完毕，因此进程应将自己插入 S.L 队列中，并调用 block 原语将自己阻塞，此时 S.value 的绝对值表示因申请该种资源而被阻塞的进程个数。对信号量的每次 signal 操作，表示执行进程释放一个单位的资源，因此要执行 S.value++操作将资源数目加 1；如果加 1 后仍是 S.vlue<=0，则表示仍有等待该资源的进程处于阻塞状态，故还应调用 wakeup 原语唤醒 S.L 队列中的一个进程。

4. 有一操作系统用如下方法定义 P、V 操作：（北京大学 1992 年研究生试题）

```
P(S):
    S:=S-1;
    if S<0 then 本进程进入相应队列尾等待;
V(S):
    S:=S+1;
    if S=0 then 唤醒相应队列中的队尾进程，本进程进入就绪队列;
```

这与传统的 P，V 操作的定义相比有什么缺点？

答：与传统的 P，V 操作的定义相比，上述 P、V 操作有以下缺点：

(1)V 操作中执行 S:=S+1 后，只有当相应队列中存在一个阻塞进程（即 S=0）时，才将该阻塞进程唤醒；如果阻塞的进程数多于一个（即 S<0），则不唤醒任何阻塞进程，这显然是错误的。

(2)V 操作中，每次唤醒一个阻塞进程时，现行进程（即正在执行 V 操作的进程）都将转变为就绪状态从而引起进程调度，在不考虑优先级的情况下，这种做法会增加进程调度的次数，从而增加系统的开销。

(3)假设错误(1)已得以改正，由于该 V 操作中每次唤醒的是相应队列的队尾进程，而 P 操作中又是将新阻塞的进程插入相应队列的队尾，因此将出现先阻塞的进程需要在阻塞队列中长期等待，甚至根本得不到唤醒的现象，这显然是不公平的。

5. 信号量如果只能取 0 或 1，就变成了二元信号量。二元信号量更容易实现，而且，信号量可以由二元信号量替换。以下所列函数试图用二元信号量操作 waitB()和 signalB()替换信号量 wait()和 signal()：

```
wait(semaphore S)
{
    waitB(mutex);
    s=s-1;
    if (s<0)
    {
```

```
            signalB(mutex);
            waitB(delay);
        }
        else
            signalB(mutex);
    }
    signal(semaphore S)
    {
        waitB(mutex);
        s=s+1;
        if (s<=0)
            signalB(delay);
        signalB(mutex);
    }
```

其中,用于互斥的二元信号量 mutex 初始化为 1,用于进程挂起的二元信号量 delay 初始化为 0,请指出该替换算法的错误所在。(浙江大学 2000 年研究生试题)

答:考虑信号量的初值为 2。①进程 P1 和 P2 先后执行 wait(S)后,s 为 0。②进程 P3 正在调用 wait(S),并已经执行完其中的 signalB(mutex),因 CPU 调度而回到就绪队列,此时 s 为 -1。③进程 P4 正在调用 wait(S),并已经执行完其中的 signalB(mutex),因 CPU 调度而回到就绪队列,此时 s 为 -2。④进程 P1 执行 signal(S),使 s 为 -1,使二元信号量 delay 为 1。⑤进程 P2 执行 signal(S),使 s 为 0,使二元信号量 delay 仍保持为 1! ⑥进程 P3 得到 CPU 后,执行 wait(delay)后,从 wait(S)返回,此时 delay 为 0。⑦进程 P4 得到 CPU 后,执行 wait(delay),不得不在原地等待! 按照原意,应该允许两个进程都从 wait(S)返回。

6. 某操作系统将消息缓冲通信作为进程之间的基本通信手段。Send 和 Receive 分别为发送消息和接收消息原语。请设计一种方案,用 Send 和 Receive 原语来实现基于信号量的 P、V 操作。(中科院软件研究所 1993 研究生试题)

答:要用 Send 和 Receive 原语来实现基于信号量的 P、V 操作,可在系统中设置一个专门用于管理信号量的同步进程。此时,对信号量的 P、V 操作均可描述为一个 Send 原语的调用,再加一个 Receive 原语的调用,其中的 Send 原语调用中,调用者将调用进程的指针、指向信号量的指针以及对信号量操作的类型等参数发送给同步进程,而 Receive 调用则等待来自同步进程的应答消息。而同步进程则不断地等待接收消息,在接收到一个消息后,如果相应的操作是 P 操作,则对相应信号量进行 S.value-1 的操作,若操作完成后 S.value 的值小于 0,则将消息的发送者插入 S.L 的队尾,并不对该消息进行应答;否则,便向本次 P 操作的调用者发送应答消息。如果同步进程接收到的是一个要求 V 操作的消息,则对相应信号量进行 S.value+1 的操作,若操作完成后 S.value 的值大于 0,则只需向消息的发送者发送一应答消息;否则,则需从 S.L 的队首摘下一个进程,并分别向该进程和本次 V 操作的调用者发送一应答消息。

7. 使用 P、V 原语和加锁法都可以实现并发进程间的互斥,问:

(1)P、V 原语和加锁法实现互斥有何异同?

(2)使用加锁法实现互斥时,有可能在进程使用临界区时造成不公平现象(即一个进程一直占有临界区,其他进程永远无法使用)。试找出一个不公平现象的例子,并分析产生不公平现象的原因。(清华大学 1995 年研究生试题)

答:加锁法实现互斥是这样的:当某个进程进入临界区之后,它将锁上临界区,直到它退出临界区为止。并发进程在申请进入临界区时,首先测试该临界区是否是上锁的,如果该临界区已被锁住,则该进程要等待该临界区开锁之后才有可能获得临界区。

而用 P、V 原语实现互斥是这样的:为临界资源设置一初值为 1 的信号量,该信号量只能由 P、V 原语操作改变。并发进程在申请进入临界区时,首先执行 P 操作,如果临界区已被占用,则进程阻塞等待被一个 V 操作唤醒;否则允许进程进入临界区。在退出临界区时,则执行 V 操作,若有其他进程等待使用该临界资源,则唤醒队首的进程。

(1)加锁操作与 P、V 原语实现互斥的不同主要有:①加锁操作在临界资源已被占用的情况下,仍要循环测试锁的状态,这将耗费大量 CPU 时间;②加锁操作不将等待临界资源的进程按申请次序排队,故有可能造成不公平现象。

(2)例子如下:如果有 P1 进程已通过加锁操作进入临界区内执行,则当 P2 进程申请进入对应的临界区时,将因临界区已上锁而处于忙等的状态,即 P2 并不会阻塞排队。当 P1 退出临界区时,它只需将临界区解锁,但并不像 V 操作那样按次序唤醒等待进程;而且,系统也不一定要立即进行进程调度。如果 P1 进程继续运行并再次申请进入临界区,则因临界区已解锁,它又可顺利进入临界区。这将使 P1 进程一直占用临界区,而 P2 进程则长期等待,从而造成不公平的现象。

8. 我们为某临界区设置一把锁 W,当 W=1 时表示关锁,W=0 时表示锁已打开。试写出开锁原语和关锁原语,并利用它们去实现互斥。

答:相应的关锁原语 lock(W)和开锁原语 unlock(W)可描述为:

```
lock(W): while(W==1);
             W=1;
unlock(W):W=0
```

在利用关锁原语和开锁原语实现进程互斥时,可将临界区 CS 放在其间,即:

```
lock(W);
  CS;
unlock(W);
```

9. 遵循同步机制的 4 条规则,写出以锁机制实现的解决读者—写者问题的同步算法。(中科院计算所 1998 年研究生试题)

答:为了遵循同步机制的 4 条规则,每把锁 W 除了锁的状态 state(1 表示关锁,0 表示锁已打开)外,还必须设置一指针 L 用来将等待开锁的进程排队。相应的关锁原语 lock(W)和开锁原语 unlock(W)可描述为:

```
lock(W): if(W.state==1)
             block(W.L); /* 阻塞调用进程并将其插入 W.L 队列末尾 */
           else W.state=1;
unlock(W):if (W.L!=NULL)
             wakeup(W.L); /* 唤醒 W.L 队列的队首进程 */
           else W.state=0;
```

用锁机制来解决读者—写者问题的同步算法如下：

```
reader(){
    while(1){
        lock(Wr);
        if (readcount==0) lock(Ww);
        readcount++;
        unlock(Wr);
        ⋮
        perform read operation;
        ⋮
        lock(Wr);
        readcount--;
        if (readcount==0) unlock(Ww);
        unlock(Wr);
    }
}
writer(){
    while(1){
        lock(Ww);
        perform write operation;
        unlock(Ww);
    }
}
```

其中，变量 readcount 表示正在进行读的读者数目，由于每个读者都要对它进行访问并作修改，readcount 是一个临界资源，必须被所有读者互斥使用；互斥锁 Wr 用来实现对 readcount 变量的互斥访问，其初始状态为开状态；互斥锁 Ww，用来实现写者之间互斥，或写者与读者之间的互斥，其初值状态也是开状态。

10. 设有 n 个进程共享一个资源，对于如下两种情况：

(1)如果每次只允许一个进程使用该资源；

(2)如果每次允许 m 个进程(m<n)同时使用该资源。

试问：所采用的互斥信号量的初值是否相同？信号量的变化范围如何？

答：两种情况下所采用的互斥信号量的初值不相同。前一种情况下，信号量的初值应设置为 1，而信号量的变化范围为[1-n,1]；后一种情况下，信号量的初值应设置为 m，而信号量的变化范围为[m-n,m]。

11. 下列程序是解决两个进程互斥访问临界区问题的一种方法，试从“互斥”、“有空即进”、“有限等待”等三个方面讨论它的正确性，如果它是正确的，则证明之；如果它不正确，请说明理由。(复旦大学 1999 年研究生试题)

```
program sample;
  var c1,c2:integer;
procedure p1;          /* 第一个进程 p1 */
```

```
    begin
      repeat
        Other section 1;
        repeat
          c1:=1-c2
        until c2<>0;
        Critical section;      /*临界区*/
        C1:=1
      until false
    end
  procedure p2;          /*第二个进程 p2*/
    begin
      repeat
        Other section 2;
        repeat
          c2:=1-c1
        until c1<>0;
        Critical section;      /*临界区*/
        c2:=1
      until false
    end
  begin              /*主程序*/
    c1:=1;
    c2:=1;
      cobegin
          p1;
          p2;      /*进程 p1、p2 开始并行执行*/
      coend
  end
```

答:本题中,p1、p2 通过共享变量 c1 和 c2 来达到资源互斥使用的目的,其中,c1 = 1 表示 p1 不在临界区内,c2 = 1 表示 p2 不在临界区内。通过检查可知:

(1)该算法不能保证"互斥"。请考虑图 2.6 的进程执行次序:

①由于 c1、c2 的初值都为 1,若 p1 先获得 CPU 并申请进入临界区,则它可以进入临界区。在 p1 进入临界区后,c1 = 0,c2 = 1;②此时若进行进程调度并由 p2 获得 CPU,而 p2 也试图进入临界区,执行完 c2:=1-c1 后,c2 = 1,c1 = 0;③此时若又进行 CPU 调度,并且 p1 重新获得 CPU,并在退出临界区后执行 c1:=1,则 c2 = 1,c1 = 1;④若再进行 CPU 调度,并且由 p2 获得 CPU,因 c1 = 1,p2 进入临界区,而此时 c2 = 1,c1 = 1;⑤若再进行 CPU 调度并由 p1 获得 CPU,当 p1 再申请进入临界区时,由于 c2 = 1,p1 将顺利进入临界区。可见,在这种情况下,p1 和 p2 将同时进入临界区。

(2)该算法能保证"有空即进"。因为,c1 和 c2 的初值均为 1;而 c1 只有在 p1 申请进入自己

的临界区时才可能变为0，一旦p1退出临界区，它的值又将被置为1；同样，c2只有在p2申请进入自己的临界区时才可能变为0，一旦p2退出临界区，它的值又将被置为1。所以，当临界资源空闲时，不管它是否曾经被使用过，c1和c2的值均为1；此时，若p1进程申请进入自己的临界区，则先执行c1：=1－c2将c1置为0，并因c2＜＞0条件成立而结束循环测试，随后进入自己的临界区；对p2进程，情况也是如此。

(3)该算法不能保证"有限等待"。请考虑如下的进程执行次序：

①假设p1先获得CPU并申请进入临界区，c1＝0，c2＝1；②此时若进行进程调度并由p2获得CPU，而p2也试图进入临界区，执行完c2：＝1－c1后，c2＝1，c1＝0，无法进入；③此时若又进行CPU调度，并且p1重新获得CPU，并在退出临界区后执行c1：＝1；p1再次试图进入临界区，执行完c1：＝1－c2后，c2＝1，c1＝0，成功进入临界区；④若再进行CPU调度，并且由p2获得CPU，因c1＝0，c2＝1，p2继续等待。若调度算法导致重复③和④，p2将无限等待。

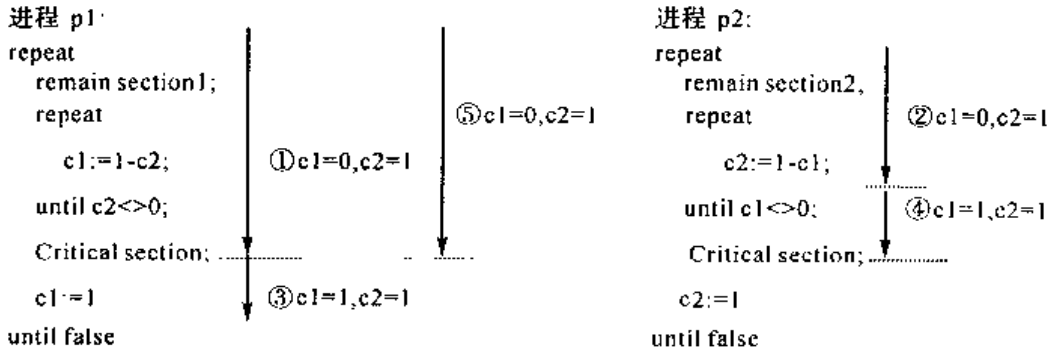


图2.6 两进程的执行顺序

12.何谓临界区？下面给出的实现两个进程互斥的算法是安全的吗？为什么？（中国科技大学1998年研究生试题）

```
#define TRUE 1
#define FALSE 0

int flag[2];
flag[0]=flag[1]=FALSE;

enter_crtsec(i)
int i;
{
  while(flag[1-i]);
  flag[i]=TRUE;
}
leave_crtsec(i)
int i;
{
  flag[i]=FALSE;
}
```

```
process i; /* i=0 or i=1 */
  ...
  enter_crtsec(i);      /* 进入临界区 */
  critical section;
  leave_ctrsec(i);      /* 离开临界区 */
```

答:(1)临界区是进程中访问临界资源的那段代码。而临界资源是指要求进程互斥共享的资源,如打印机、共享变量等。为保证各进程对临界资源的互斥访问,应保证它们互斥地进入自己的临界区。

(2)上述算法不安全,因为,enter_crtsec()不是一个原语,而可以在执行过程中被中断,故在某些情况下,该算法不能保证进程互斥地进入临界区。例如:①初始状态下,flag[0] = flag[1]=0,无任何进程在临界区内执行;②此时,若 process 0 申请进入临界区,它将执行 enter_crtsec(0),由于 flag[1]=0, while 语句结束,它将执行下一个语句 flag[0]=1;③若此时,process 0 进程被中断而将 CPU 调度给 process 1,并且 process 1 也恰好申请进入临界区,由于 process 0 还没来得及执行 flag[0]=1,即 flag[0]仍为 0,故 process 1 将 flag[1]置为 1,并顺利地进入临界区;④此时,若再次进行进程调度并由 process 0 获得 CPU,它将 flag[0]置成 1 后,也将进入自己的临界区。可见,这种情况下,两个进程可同时在各自的临界区内执行。

13. 下面是两个并发执行的进程。它们能正确运行吗? 若不能请举例说明,并改正之。(北京航空航天大学 2002 研究生试题)

```
parbegin
  var x:integer;
  process P1                    process P2
  var y, z:integer;               var t, u:integer;
  begin                           begin
    x:=1;                             x:=0;
    y:=0;                             t:=0;
    if x≥1 then y:=y+1;               if x≤1 then t:=t+2;
    z:=y;                             u:=t;
  end                             end
parend
```

答:上述两个并发执行的进程共享整型变量 x,且共享时没能做到互斥,因此它们的执行结果具有不确定性。如:若先执行 P1,并在它执行完语句 x:=1 后,进行进程调度,执行 P2,并在 P2 结束后再调度执行 P1,则最后结果是 x=y=z=0,t=u=2;若先执行 P1,并在它执行完 if 语句后,再调度 P2 执行,则最后结果是 x=0,y=z=1,t=u=2。

可将上述程序改正为:

```
parbegin
  var x:integer;
    s:semaphore:=1;
  process P1                    process P2
```

```
    var y, z:integer;          var t, u:integer;
    begin                      begin
      p(s);                      p(s);
      x:=1;                      x:=0;
      y:=0;                      t:=0;
      if x≥1 then y:=y+1;        if x≤1 then t:=t+2;
      v(s);                      v(s);
      z:=y;                      u:=t;
    end                        end
  parend
```

14. 下述关于双进程临界区问题的算法(对编号为 id 的进程)是否正确:(浙江大学 2000 年研究生试题)(国防科技大学 1999 年研究生试题)

```
do{
    blocked[id]=true;
    while(turn ! =id)
  {
    while(blocked[1-id]);
    turn=id;
  }

  编号为 id 的进程的临界区
  blocked[id]=false;
  编号为 id 的进程的非临界区
} while (ture);
```

其中,布尔型数组 blocked[2]初始值为为{false, false},整型 turn 初始值为 0,id 代表进程编号(0 或 1)。请说明它的正确性,或指出错误所在。

答:①假设 p1 先获得 CPU 并进入临界区;②此时若进行进程调度并由 p0 获得 CPU,而 p0 也试图进入临界区,它只能在"while(blocked[1]);"等待,无法进入;③此时若又进行 CPU 调度,并且 p1 重新获得 CPU,并在退出临界区后执行 blocked[1]=false;④若再进行 CPU 调度,并且由 p0 获得 CPU;p0 执行完 while,但还没有执行 turn=0;⑤CPU 调度,使 p1 执行;它执行 while(turn ! = 1),发现条件为假,于是进入临界区;⑥CPU 调度,使 p0 执行;p0 执行完 turn=0 后,也进入临界区!

此时,临界区互斥条件无法满足。

15. 如图 2.7 所示,三个合作进程 P1、P2、P3,它们都需通过同一输入设备输入各自的数据 a、b、c,该输入设备必须互斥地使用,而且其第一个数据必须由 P1 进程读取,第二个数据必须由 P2 进程读取,第三个数据则必须由 P3 进程读取。然后,三个进程分别对输入数据进行下列计算:

```
P1:x=a+b
P2:y=a*b
```

P3：z＝y＋c－a

最后，P1 进程通过所连的打印机将计算结果 x、y、z 的值打印出来。请用信号量实现它们的同步。

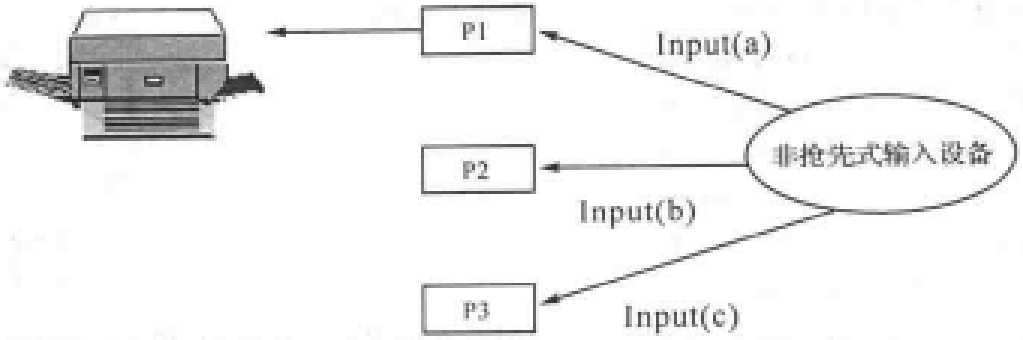


图 2.7　P1、P2、P3 的工作示意图

答：为了控制三个进程依次使用输入设备进行输入，需分别设置三个信号量 S1、S2、S3，其中 S1 的初值为 1，S2 和 S3 的初值为 0。使用上述信号量后，三个进程不会同时使用输入设备，故不必再为输入设备设置互斥信号量。另外，还需设置信号量 Sb、Sy、Sz 来分别表示数据 b 是否已经输入，以及 y，z 是否已计算完成，它们的初值均为 0。三个进程的动作可描述为：

```
P1：
    P(S1);
    从输入设备输入数据 a;
    V(S2);
    P(Sb);
    x＝a＋b;
    P(Sy);
    P(Sz);
    使用打印机打印出 x、y、z 的结果;
    P2:
        P(S2);
        从输入设备输入数据 b;
        V(S3);
        V(Sb);
        y＝a＊b;
        V(Sy);
        V(Sy);
    P3:
        P(S3);
        从输入设备输入数据 c;
        P(Sy);
        Z＝y＋c－a;
        V(Sz);
```

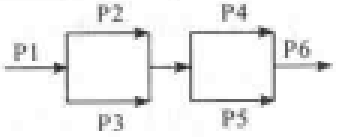


图 2.8　各进程的执行顺序

16. 一组合作进程，执行顺序如图 2.8 所示。请用 P、V 操作实现进程间的同步操作。（国防科

技大学 2000 年研究生试题)

答:图 2.9 示出了上述并发进程之间的前趋关系,为了使上述进程同步,可设置 8 个信号量 a,b,c,d,e,f,g,h,它们的初值均为 0,而相应的进程可描述为(其中"…"表示进程原来的代码):

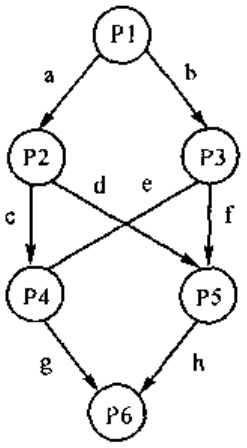


图 2.9 合作进程的前趋图

```
P1() | …; V(a); V(b); |
P2() | P(a); …; V(c); V(d); |
P3() | P(b); …; V(e); V(f); |
P4() | P(c); P(e); …; V(g); |
P5() | P(d); P(f); …; V(h); |
P6() | P(g); P(h); …; |
```

17. 设有 8 个程序 prog1,prog2,…,prog8。它们在并发系统中执行时有如图 2.10 所示的制约关系,试用 P,V 操作实现这些程序间的同步。(北京大学 1991 年研究生试题)

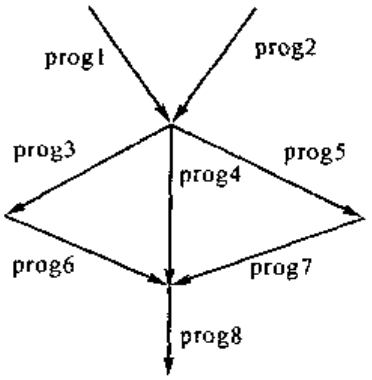


图 2.10 进程并发执行的制约关系

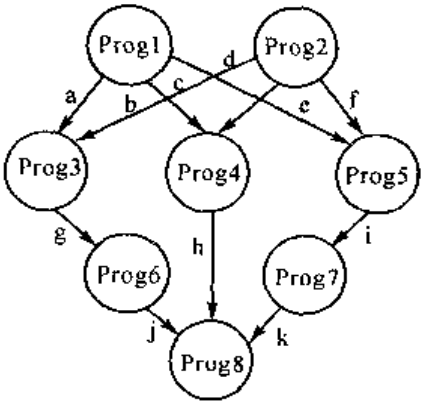


图 2.11 并发进程间的前趋关系

答:上述 8 个程序的前趋关系如图 2.11 所示。为了用 P、V 操作实现这些程序间的同步,须设置 11 个信号量 a,b,c,d,e,f,g,h,i,j,k,它们的初值均为 0。而相应的程序可描述为:

```
prog1() | …; V(a); V(b); V(c); | /* … 表示程序原来的代码,以下同 */
prog2() | …; V(d); V(e); V(f); |
prog3() | P(a); P(d); …; V(g); |
prog4() | P(b); P(e); …; V(h); |
prog5() | P(c); P(f); …; V(i); |
prog6() | P(g); …; V(j); |
prog7() | P(i); …; V(k); |
prog8() | P(j); P(h); P(k); …; |
```

18. 在测量控制系统中,数据采集任务把所采集的数据送入一单缓冲区;计算任务从该缓冲区中取出数据进行计算。试写出利用信号量机制实现两者共享单缓冲区的同步算法。(南京航空航天大学 2000 年研究生试题)

答:要用 P、V 操作实现上述进程的同步,可设置两个信号量,其中 empty 对应空闲的缓冲区,初值为 1;full 对应满缓冲区(或缓冲区中可供计算的数据),初值为 0。

数据采集任务可描述为:

```
collector()|
```

```
        while(1){
            collect data;
            P(empty);
            add data to buffer;
            V(full);
        }
    }
```

计算任务可描述为：

```
    compute(){
        while(1){
            P(full);
            remove data from buffer;
            V(empty);
            compute;
        }
    }
```

19. 有3个并发进程R、M、P，它们共享同一缓冲区。进程R负责从输入设备读信息，每读入一个记录后，就把它放进缓冲区中；进程M在缓冲区中加工读入的记录；进程P把加工后的记录打印输出。读入的记录经加工输出后，缓冲区又可以存放下一个记录。试写出他们能够正确执行的并发程序。

答：在本题中，应设置3个信号量，其中empty对应空闲的缓冲区，初值为1；full1对应缓冲区中待加工的记录，初值为0；full2对应缓冲区中已加工的记录（即待打印输出的记录），初值为0。相应的进程可描述为：

进程R：

```
    while(1){
        P(empty);
        从输入设备读记录到缓冲区中；
        V(full1);
    }
```

进程M：

```
    while(1){
        P(full1);
        在缓冲区中加工记录；
        V(full2);
    }
```

进程P：

```
    while(1){
        P(full2);
        将加工后的记录打印输出；
```

```
        V(empty);
    }
```

20. 现有三个进程 in 、outA 、outB 共享缓冲区 buf(容量为 1)。约定:仅当空时,in 才可以把读入的数据放入(put)buf 中。仅当 buf 有数据且为奇数时,outA 才可以从 buf 中取出(GET)数据使 buf 为空并打印,若 buf 中有数据且为偶数时,则由 outB 从 buf 中取出(GET)数据使 buf 为空并打印。请给出用 P、V 操作实现三个进程并发执行的程序。

答:本题中可设置 3 个信号量,其中 empty 对应空闲缓冲区,初值为 1;full1 对应缓冲区中的奇数,初值为 0;full2 对应缓冲区中的偶数,初值为 0。相应的进程可描述为:

```
进程 in:
    while(1){
        读入数据 data;
        P(empty);
        将读入的数据 data 放入缓冲区中;
        if(data%2==0) V(full2);
            else V(full1);
    }
进程 outA:
    while(1){
        P(full1);
        从缓冲区中取出数据;
        V(empty);
        打印刚取出的数据;
    }
进程 outB:
    while(1){
        P(full2);
        从缓冲区中取出数据;
        V(empty);
        打印刚取出的数据;
    }
```

21. 今有 3 个并发进程 R、M 和 P,它们共享一个可循环使用的缓冲区 B,缓冲区 B 共有 n 个单元。进程 R 负责从输入设备读信息,每读一个字符后,把它们存放在缓冲区 B 的一个单元中;进程 M 负责处理读入字符,若发现读入的字符中有空格符,则把它改变成“;”;进程 P 负责把处理后的字符取出并打印输出。当缓冲区单元中的字符被进程 P 取出后,则又可用来存放下一次读入的字符。请用 P、V 操作为同步机制写出它们能正确并发的程序。(南京大学 1997 年研究生试题)

答:本题中的缓冲区 B 可描述为:

```
char bufferB[n];
```

为解决上述进程的同步问题，需设置 3 个信号量，其中 empty 对应空闲的缓冲单元，初值为 n；full1 对应缓冲区中待 M 处理的字符，初值为 0；full2 对应缓冲中已经处理过而待打印输出的字符，初值为 0。另外，还需定义 3 个整型变量 in、out1、out2，分别用来指示下一个可存放字符的缓冲单元、下一个待 M 处理的缓冲单元及下一个待打印输出的缓冲单元，它们的初值均为 0。相应的进程可描述为：

```
R(){
    char ch;
    while(1){
        从输入设备读入一个字符到 ch 中；
        P(empty);
        bufferB[in]=ch;
        V(full1);
        in=(in+1)%n;
    }
}
M(){
    while(1){
        P(full1);
        if (bufferB[out1] ==' ') bufferB[out1]=';';
        V(full2);
        out1=(out1+1)%n;
    }
}
P(){
    char ch;
    while(1){
        P(full2);
        ch=bufferB[out2];
        V(empty);
        out1=(out1+1)%n;
        将字符 ch 打印输出；
    }
}
```

22.(1)写出 P，V 操作的定义；

(2)有三个进程 PA、PB 和 PC 合作解决文件打印问题：PA 将文件记录从磁盘读入主存的缓冲区 1，每执行一次读一个记录；PB 将缓冲区 1 的内容复制到缓冲区 2，每执行一次复制一个记录；PC 将缓冲区 2 的内容打印出来，每执行一次打印一个记录。缓冲区的大小等于一个记录大小。请用 P、V 操作来保证文件的正确打印。(北京大学 1990 年试题)

答：(1)P、V 操作是针对信号量的两种操作，而且这两种操作都具有原子性。以记录型信

号量为例,如果 S 是一记录型信号量,则 P 操作(记为 P(S))可描述为:

```
S.value--;
if(S.value<0){/* 阻塞现行进程 */
    add this process to S.L;
    block();
}
```

V 操作(记为 V(S))可描述为:

```
S.value++;
if(S.value<=0){
    remove a process P from S.L;
    wakeup(P);
}
```

(2)本题中需设置 4 个信号量,其中 empty1 对应空闲的缓冲区 1,其初值为 1;full1 对应缓冲区 1 中的记录,其初值为 0;empty2 对应空闲的缓冲区 2,其初值为 1;full2 对应缓冲区 2 中的记录,其初值为 0。相应的进程可描述为:

```
PA(){
    while(1){
        从磁盘读入一条记录;
        P(empty1);
        将记录存入缓冲区 1;
        V(full1);
    }
}
PB(){
    while(1){
        P(full1);
        从缓冲区 1 中取出一条记录;
        V(empty1);
        P(empty2);
        将取出的记录存入缓冲区 2;
        V(full2);
    }
}
PC(){
    while(1){
        P(full2);
        从缓冲区 2 中取出一条记录;
        V(empty2);
        将取出的记录打印出来;
    }
```

}

23. 如图 2.12 所示，有三个进程 input，output 和 trans 以及两对供存取数据的单缓冲 IN1、IN2 和 OUT1、OUT2，input 进程把数据交替输入 IN1 和 IN2 中，output 交替输出 OUT1 和 OUT2 中的数据。trans 交替地把 IN1 中的数据变换后送入 OUT1，把 IN2 中的数据变换后送入 OUT2，请用 P、V 操作实现并发执行进程。

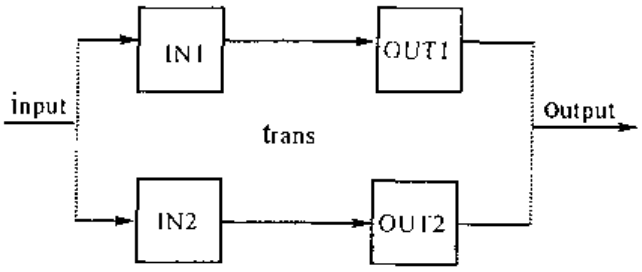


图 2.12　三个进程并发工作的示意图

答：本题中可设置 4 对信号量 in1_empty 及 in1_full，in2_empty 及 in2_full，out1_empty 及 out1_full，out2_empty 及 out2_full，分别对应于相应的空闲缓冲区和缓冲区中的数据，它们的初值均为 1、0。相应的进程可描述为：

```
input(){
    while(1){
        输入下一个数据;
        P(in1_empty);
        将数据存放到缓冲 IN1 中;
        V(in1_full);
        输入下一个数据;
        P(in2_empty);
        将数据存放到缓冲 IN2 中;
        V(in2_full);
    }
}
trans(){
    while(1){
        P(in1_full);
        取出缓冲 IN1 中的数据;
        V(in1_empty);
        将取出的数据进行转换;
        P(out1_empty);
        将数据输入缓冲 OUT1 中;
        V(out1_full);
        P(in2_full);
        取出缓冲 IN2 中的数据;
```

```
            V(in2_empty);
            将取出的数据进行转换;
            P(out2_empty);
            将数据输入缓冲 OUT2 中;
            V(out2_full);
        }
    }
    output(){
        while(1){
            P(out1_full);
            取出缓冲 OUT1 中的数据;
            V(out1_empty);
            将取出的数据打印出来;
            P(out2_full);
            取出缓冲 OUT2 中的数据;
            V(out2_empty);
            将取出的数据打印出来;
        }
    }
```

24. 现有 4 个进程 R1, R2, W1 和 W2, 它们共享可以存放一个数据的缓冲器 B。进程 R1 每次把从磁盘上读入的数据存放到缓冲器 B 中, 供进程 W1 打印输出, 进程 R2 每次从键盘上读一个数据存放到缓冲器 B 中, 供进程 W2 打印输出。当一个进程已把缓冲器中的数打印输出后, 在缓冲器中还没有存入一个新数之前不准任何进程再从缓冲器中取数打印, 用 wait、signal 操作实现 4 个进程并发执行协调地工作。

答: 本题中需设置 3 个信号量, 其中 empty 对应空闲的缓冲器 B, 其初值为 1; full1 对应缓冲器中可供 W1 打印输出的数据, 其初值为 0; full2 对应缓冲器中可供 W2 打印输出的数据, 其初值为 0。相应的进程可描述为:

```
R1(){
    while(1){
        从磁盘读入数据;
        wait(empty);
        将数据存入缓冲器 B 中;
        signal(full1);
    }
}
R2(){
    while(1){
        从键盘读入数据;
        wait(empty);
```

```
            将数据存入缓冲器 B 中;
            signal(full2);
        }
    }
    W1(){
        while(1){
            wait(full1);
            从缓冲器 B 中取数;
            signal(empty);
            打印取出的数;
        }
    }
    W2(){
        while(1){
            wait(full2);
            从缓冲器 B 中取数;
            signal(empty);
            打印取出的数;
        }
    }
```

25. 有 n 个进程(P1, P2, …, Pn)向容量为 M 的缓冲区写数据，每个进程一次写 1 个数据，当缓冲区写满时，另一个读进程 Pr 一次将 M 个数据全部读完，如此反复。请用信号量解决这些进程的同步互斥问题。

答：本题中需要定义下述变量和信号量：

```
data_type buffer[M]; /* data_type 对应于所需要的数据类型，如 int、float 等 */
int in=0; /* 用来指示下一个可存放数据的缓冲区 */
semaphore empty=M; /* 对应空闲的缓冲区 */
semaphore full=0; /* 对应缓冲区中的数据 */
semaphore mutex=1; /* 用来实现 Pi 进程对变量 in 的互斥访问 */
```

进程 Pi 可描述为：

```
Pi(){
    while(1){
        P(empty);
        P(mutex);
        向缓冲区 buffer[in]中写数据;
        in=(in+1)%M;
        V(mutex);
        V(full);
```

```
        }
    }
进程 Pr 可描述为:
    Pr(){
        int i;
        while(1){
            for(i=0;i<M;i++) P(full);
            取出缓冲 buffer[0]到 buffer[M-1]中的数据;
            for(i=0;i<M;i++) V(empty);
        }
    }
```

26. 在生产者和消费者问题中,信号量 mutex, empty, full 的作用是什么? 如果对调生产者进程中的两个 wait 操作和两个 signal 操作,则可能发生什么情况?

答:信号量 mutex 的作用是保证各生产者进程和消费者进程对缓冲池的互斥访问。信号量 empty 和 full 均是资源信号量,它们分别对应于缓冲池中的空闲缓冲区和缓冲池中的产品,生产者需要通过 wait(empty)来申请使用空闲缓冲区,而消费者需要通过 wait(full)才能取得缓冲中的产品,可见,这两个信号量起着同步生产者和消费者的作用,它们保证生产者不会将产品存放到满缓冲区中,而消费者不会从空缓冲区中取产品。

在生产者—消费者问题中,如果将两个 wait 操作,即 wait(full)和 wait(mutex)互换位置,或者 wait(empty)和 wait(mutex)互换位置,都可能引起死锁。考虑系统中缓冲区全满时,若一生产者进程先执行了 wait(mutex)操作并获得成功,当再执行 wait(empty)操作时,它将因失败而进入阻塞状态,它期待消费者执行 signal(empty)来唤醒自己,在此之前,它不可能执行 signal(mutex)操作,从而使企图通过 wait(mutex)进入自己的临界区的其他生产者和所有的消费者进程全部进入阻塞状态,系统进入死锁状态。类似地,消费者进程若先执行 wait(mutex),后执行 wait(full)同样可能造成死锁。

signal(full)和 signal(mutex)互换位置,或者 signal(empty)和 signal(mutex)互换位置,则不会引起死锁,其影响只是使某个临界资源的释放略为推迟一些。

27. 设有一个具有 N 个信息元素的环形缓冲区,A 进程顺序地把信息写入缓冲区,B 进程依次地从缓冲区读出信息。回答下列问题:(中科院软件所 1996 年研究生试题)

(1)叙述 A、B 两进程的相互制约关系;

(2)判别下列用 P、V 操作表示的同步算法是否正确? 如不正确,试说明理由,并修改成正确算法。

```
VAR buffer:ARRAY[0…N-1] OF T;
    in,out:0..N-1;
VAR s1,s2:semaphore;

s1:=0;s2:=N;
in:=0;out:=0;
```

```
PROCEDURE A:
  BEGIN
    REPEAT
      生产数据 m;
      P(s2);
      Buffer[in]:=m;
      in:=(in+1) MOD N;
      V(s1);
    forever
  END
PROCEDURE B:
  BEGIN
    REPEAT
      V(s2);
      m:=Buffer[out];
      消费数据 m;
      out:=(out+1) MOD N;
      P(s1);
    forever
  END
```

答:(1)A,B 进程间的制约关系为:当缓冲区为空时,在 A 进程将信息写入缓冲区以前,B 进程必须等待;当缓冲区满时,A 进程必须等待 B 进程取走信息后才能将信息写入缓冲区。

(2)上述算法是错误的。首先,B 进程在取信息以前没执行 P(S1)操作,可能会使 B 进程到空缓冲区中取信息;其次,B 进程在取信息以前便执行 V(S2)操作,这有可能使 A 进程向满缓冲区中写入信息。故必须将 B 进程修改为:

```
PROCEDURE B:
  BEGIN
    REPEAT
      P(s1);
      m:=Buffer[out];
      V(s2);
      out:=(out+1) MOD N;
      消费数据 m;
    forever
  END
```

与多个生产者和多个消费者的问题不同,本题中由于只有一个生产者和一个消费者,因此,不会发生几个生产者和消费者同时存取同一个缓冲单元的情况,故无须设置互斥信号量 mutex。

28. 以下是生产者—消费者问题的一种解法。(浙江大学 1999 年研究生试题)

(1)写出 PRODUCER 进程和 CONSUMER 进程各自的临界区。

(2)给出解决临界区互斥问题的一种方法。

```
VAR counter:0..n;
    buffer:arry[0..n-1]of item;
PRODUCER:
    repeat
      procduce an item in nextp;
while counter=n do no_op;
      buffer[in]:=nextp;
      in:=in+1 mod n;
      counter:=counter+1;
until false;
CONSUMER:
      repeat
      while counter=0 do no_op;
      nextc:=buffer[out];
      out:=out+1 mod n;
      counter:=counter+1;
      consume the item in nextc
    until false;
```

答:PRODUCER 进程的临界区是 counter := counter - 1;

CONSUMER 进程的临界区是 counter := counter + 1;

可以定义信号量 mutex,初值为 1;两个进程各自在进入临界区前执行 wait(mutex),离开临界区后执行 signal(mutex)。

29. 表 2-6 是两个同步进程的模拟执行,生产者将物品放入共享缓冲区以供消费者使用,缓冲区可以放 2 件物品,使用两个信号量,并置初值为 S1 = 2, S2 = 0。现已知操作情况,请填写信号量值并说明操作结果(正常、阻塞或唤醒)。(提示:缓冲区满,不许放物品;缓冲区空,不许取物品)。

表 2-6 两个同步进程的模拟执行过程

事件	操作情况		信号量值		操作结果说明
	P1(生产者)	P2(消费者)	S1	S2	
0	…	…	2	0	初始化
1	–	P(S2)			
2	P(S1)	–			
3	V(S2)	–			
4	P(S1)	–			
5	–	V(S1)			
6	V(S2)	–			
7	P(S1)	–			
8	V(S2)	–			
9	P(S1)				

答：信号量的值和操作结果填写如表 2-7 所示：

表 2-7　信号量的值和操作结果

操作情况			信号量值		操作结果说明
事件	P1(生产者)	P2(消费者)	S1	S2	
0	…	…	2	0	初始化
1	–	P(S2)	2	−1	P2 阻塞
2	P(S1)	–	1	−1	P1 正常，P2 仍阻塞
3	V(S2)	–	1	0	P1 正常，并唤醒 P2
4	P(S1)	–	0	0	P1 正常
5	–	V(S1)	1	0	P1，P2 均正常
6	V(S2)	–	1	1	P1，P2 均正常
7	P(S1)	–	0	1	P1，P2 均正常
8	V(S2)	–	0	2	P1，P2 均正常
9	P(S1)	–	−1	2	P1 阻塞，P2 正常

30. 试用管程来实现生产者—消费者问题，其中有界缓冲区的大小为 n。

答：利用管程来解决生产者—消费者问题，首先必须为它们建立一个管程：

```
monitor p_c
{
    int in,out,count;
    item_type buffer[n]; /* item_type是产品的类型 */
    condition notfull,notempty;
    void put(item_type product){
        if (count>=n) notfull.wait();/* 等待缓冲池不全满条件成立 */
        buffer[in]=product;
        in=(in+1)% n;
        count++;
        notempty.signal();/* 缓冲池不全空条件成立 */
    }
    void get(item_type product){
        if (count<=0) notempty.wait(); /* 等待缓冲池不全空条件成立 */
        product=buffer[out];
        out=(out+1)%n;
        count--;
        notfull.signal(); /* 缓冲池不全满条件成立 */
    }
    void init(){
        in=out=count=0;
    }
}
```

上述管程 p_c 中包括两个局部过程：过程 put 负责将产品投放到缓冲池中；过程 get

负责从缓冲池中取出产品。另外，整型变量 count 表示缓冲池中已存放有的产品数目，条件变量 notfull、notempty 分别对应于缓冲池不全满、缓冲池不全空两个条件。

而相应的生产者和消费者可描述为：

```
producer(){
    while(1){
        produce an item in nextp;
        p_c.put(nextp);
    }
}
consumer(){
    while(1){
        p_c.get(nextc);
        consume the item in nextc;
    }
}
```

31. 桌上有一只盘子，最多可容纳两个水果，每次只能放入或取出一个水果。爸爸专向盘中放苹果，妈妈专向盘中放橘子；两个儿子专等吃盘子中的橘子，两个女儿专等吃盘子中的苹果。请用 P，V 操作来实现爸爸、妈妈、儿子、女儿之间的同步与互斥关系。（南京大学 2000 年研究生试题）

答：本题中需设置 4 个信号量，其中 empty 表示还可以向盘中放几个水果，其初值为 2；apple 对应已放入盘中的苹果，orange 对应已放入盘中的橘子，它们的初值均为 0；mutex 用来实现对盘子的互斥访问（包括放和取），其初值为 1。相应的进程可描述为：

```
father(){
    while(1){
        P(empty);
        P(mutex);
        向盘中放苹果;
        V(mutex);
        V(apple);
    }
}
mother(){
    while(1){
        P(empty);
        P(mutex);
        向盘中放橘子;
        V(mutex);
        V(orange);
    }
```

```
}
son (){ /* 两个儿子对应同一段代码 */
    while(1){
        P(orange);
        P(mutex);
        从盘中取橘子;
        V(mutex);
        V(empty);
        吃橘子;
    }
}
daughter(){ /* 两个女儿对应同一段代码 */
    while(1){
          P(apple);
        P(mutex);
        从盘中取苹果;
        V(mutex);
        V(empty);
        吃苹果;
    }
}
```

32. 分析下面用信号量解决哲学家进餐问题的同步算法是否满足同步机制的准则。若不满足，说明为什么，并给出满足同步机制准则的同步算法。（中科院软件所 1998 年研究生试题）

```
VAR fork: ARRAY[0..4] OF semaphore;
fork[0]:= fork[1]:= fork[2]:= fork[3]:= fork[4]:=1;
PARBEGIN
Pi:REPEAT
    Think for while;
    P(fork[i]);
    P(fork[(i+1)MOD 5]);
    Eat for while;
    V(fork[i]);
    V(fork[(i+1)MOD 5]);
    UNTIL false
PAREND
```

答：本题中临界资源是 5 支筷子，因此，分别为它们设置了 5 个信号量 fork[0]～fork[4]，它们的初值均为 1。在上述算法中，哲学家对第 i 支筷子的申请通过 P(fork[i])、释放通过 V(fork[i])来进行，若使用的是记录型信号量，则算法能满足“空闲让进”、“互斥”、“让权等待”准则，但不能满足“有限等待”，因为，如果所有的哲学家均通过前一个 P(fork[i])拿到

了一支筷子，则他们因为相互等待第二支筷子而进入死锁状态。

要避免上述算法中的死锁问题，可以有多种方法。首先，如果只允许 4 个哲学家同时进餐，则可以保证至少有一个哲学家可以进餐，最终由他释放出其所用过的两只筷子，从而使更多的哲学家也可以进餐。为此，需设置一个信号量 Sm 来限制同时进餐的哲学家数目，使它不超过 4，故 Sm 的初值也应置成 4。这样，第 i 个哲学家的活动可描述为：

```
Pi:REPEAT
    Think for while;
    P(Sm);
    P(fork[i]);
    P(fork[(i+1)MOD 5]);
    Eat for while;
    V(fork[i]);
    V(fork[(i+1)MOD 5]);
    V(Sm);
UNTIL false
```

其次，规定奇数号哲学家与偶数号哲学家以不同的顺序申请两支筷子，如前者先申请他左边的筷子，而后者先申请他右边的筷子，或者相反，则也能避免死锁的发生。相应的算法可描述为：

```
Pi:REPEAT
    Think for while;
    IF (i MOD 2)=1 THEN BEGIN
        P(fork[i]);
        P(fork[(i+1)MOD 5]);
    END
    ELSE BEGIN
        P(fork[(i+1)MOD 5]);
        P(fork[i]);
    END
    Eat for while;
    V(fork[i]);
    V(fork[(i+1)MOD 5]);
UNTIL false
```

最后，如果用 AND 信号量保证哲学家一次同时申请到左右两支筷子，则也能避免死锁现象：

```
Pi:REPEAT
   Think for while;
SP(fork[i],fork[(i+1)MOD 5]);
   Eat for while;
   SV(fork[i],fork[(i+1)MOD 5]);
 UNTIL false
```

33. 哲学家甲请哲学家乙、丙、丁到某处讨论问题，约定全体到齐后开始讨论；在讨论的间隙四位哲学家进餐，每人进餐时都需使用刀、叉各一把，餐桌上的布置如图 2.13 所示。请用信号量及 P，V 操作说明这四位哲学家的同步、互斥过程。（上海交通大学 1996 年研究生试题）

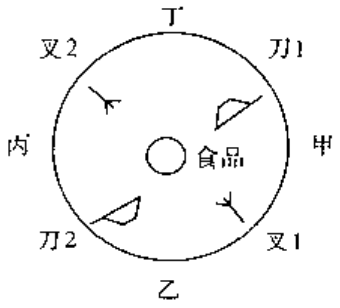


图 2.13　哲学家进餐问题

答：本题中刀和叉均是临界资源，需为它们设置对应的信号量 knife1、knife2、fork1、fork2，初值都为 1。另外，只有当哲学家到齐后，才可以开始讨论问题，故还需设置 4 个初值为 0 的信号量 a, b, c, d 以指出相应的哲学家甲、乙、丙、丁是否已到达。相应的哲学家进程可描述为：

```
Pa(){ /* 哲学家甲 */
    到达;
    V(a);V(a);V(a);
    P(b);P(c);P(d);
    while(1){
        讨论问题;
        P(knife1);
        P(fork1);
        进餐;
        V(knife1);
        V(fork1);
    }
}

Pb(){ /* 哲学家乙 */
    到达;
    V(b);V(b);V(b);
    P(a);P(c);P(d);
    while(1){
        讨论问题;
        P(knife2);
        P(fork1);
        进餐;
        V(knife2);
        V(fork1);
    }
}

Pc(){ /* 哲学家丙 */
    到达;
    V(c);V(c);V(c);
    P(a);P(b);P(d);
    while(1){
        讨论问题;
        P(knife2);
        P(fork2);
        进餐;
        V(knife2);
        V(fork2);
    }
}

Pd(){ /* 哲学家丁 */
    到达;
    V(d);V(d);V(d);
    P(a);P(b);P(c);
    while(1){
        讨论问题;
        P(knife1);
        P(fork2);
        进餐;
        V(knife1);
        V(fork2);
    }
}
```

34. 某系统有一个共享文件 F 可供多个进程同时读，系统为每个进程确定了一个编号，规定同时读文件的进程其编号之和必须小于 n（当有进程要求读文件时，只要它的编号与那些正

在读文件的进程编号之和小于 n,则该请求进程可与那些进程同时读),请用管程实现对文件 F 的正确管理。(南京大学 1999 年研究生试题)

答:本题中需建立以下的管程:

```
monitor readfile
{
    int count,waiting;
    condition less_than_n;
    void read(num:integer){ /* num 为调用进程对应的编号 */
        while(count+num>=n) {
            waiting++;
            less_than_n.wait();
        }
        count=count+num;
    }
    void over(int num){ /* num 为调用进程对应的编号 */
    count=count-num;
        while(waiting>0) { /* 唤醒所有的等待进程 */
            waiting--;
            less_than_n.signal();
        }
    }
    void init(){
        count=waiting=0;
    }
}
```

其中局部变量 count 表示正在读文件 F 的进程的编号之和;局部变量 waiting 用来表示在条件变量 less_than_n 上等待的进程数;局部过程 read(num)用来管理进程读文件;局部过程 over(num)用来结束读文件的操作。条件变量 less_than_n 表示条件"编号之和小于 n"。由于对条件变量的 signal 操作只能唤醒一个进程,故在 over()中要使用 while 语句才能唤醒所有等待的进程。

编号为 num 的读进程 P 可描述为:

```
P(){
    while(1){
        readfile.read(num);
        读文件 F;
        readfile.over(num);
    }
}
```

35. 多个进程共享一个文件，其中只读文件的称为读者，需写文件的称为写者。读者可以同时读，但写者只能独立写。要求：

(1)说明进程间的相互制约关系，应设置哪些信号量？

(2)用 P、V 操作写出其同步算法。

(3)修改上述的同步算法，使得它对写者优先，即一旦有写者到达，后续的读者必须等待，而无论是否有读者在读文件。(中科院软件所 1995 年研究生试题)

答：(1)进程之间有两种相互制约关系：①读者和写者之间的相互制约，即读者和写者之间必须互斥；②写者和写者之间的相互制约，即写者之间必须互斥。

为了解决上述同步问题，首先必须设置一初值为 0 的整型变量 readcount，用来对正在读的读者进行计数，由于该变量由多个读者进程互斥共享，故必须为它设置一个初值为 1 的互斥信号量 rmutex。其次，为了保证一写者和其他写者或读者互斥，还必须设置一初值为 1 的互斥信号量 mutex。

(2)读者和写者的算法描述如下：

```
reader(){
    while(1){
        P(rmutex);
        if(readcount==0) P(mutex); /* 第一个读者必须与写者互斥 */
        readcount++;
        V(rmutex);
        读文件;
        P(rmutex);
        readcount--;
        if(readcount==0) V(mutex);
        V(rmutex);
    }
}
writer(){
    while(1){
        P(mutex);
        写文件;
        V(mutex);
    }
}
```

(3)为了使写者优先，可在原来的读优先算法基础上增加一个初值为 1 的信号量 S，使得当至少有一个写者准备访问共享对象时，它可使后续的读者进程等待写完成；初值为 0 的整型变量 writecount，用来对写者进行计数；初值为 1 的互斥信号量 wmutex，用来实现多个写者对 writecount 的互斥访问。下面的算法不仅保证一写者比其后续的读者优先；在有多个写者的情况下，还保证，后续的写者比先它而到的读者(这些读者是第一个写者的后续读者)优先。

```
reader(){
```

```
    while(1){
        P(S);
        P(rmutex);
        if (readcount = =0) P(mutex);
        readcount + +;
        V(rmutex);
        V(S);
        读文件;
        P(rmutex);
        readcount - -;
        if (readcount = =0) V(mutex);
        V(rmutex);
    }
}
writer(){
    while(1){
        P(wmutex);
        if (writecount = =0) P(S);
        writecount + +;
        V(wmutex);
        P(mutex);
        写文件;
        V(mutex);
        P(wmutex);
        writecount - -;
        If (writecount = =0) V(S);
        V(wmutex);
    }
}
```

36. 设有 P1, P2, P3 3 个进程共享某一资源 F, P1 对 F 只读不写, P2 对 F 只写不读, P3 对 F 先读后写。当一个进程写 F 时, 其他进程对 F 不能进行读写, 但多个进程同时读 F 是允许的。试用 P、V 操作正确实现 P1, P2, P3 的同步互斥。要求: (1)正常运行时不产生死锁; (2)使用 F 的并发度要高。(北京大学 1996 年研究生试题)

答: 本题实际就是一个读者—写者问题, P1 是一个读者; P2 是一个写者; 为了使 F 的并发度较高, 将 P3 先看成读者, 当其完成读操作后, 再将其看成写者。算法中需用到如下的变量定义:

```
int readcount = 0;
semaphore rmutex = 1, mutex = 1;
```

相应的进程可描述为:

```
P1(){
    while(1){
        P(rmutex);
        if(readcount = =0) P(mutex);
        readcount + + ;
        V(rmutex);
        读 F;
        P(rmutex);
        readcount - - ;
        if(readcount = =0) V(mutex);
        V(rmutex);
    }
}
P2(){
    while(1){
        P(mutex);
        写 F;
        V(mutex);
    }
}
P3(){
    while(1){
        P(rmutex);
        if(readcount = =0) P(mutex);
        readcount + + ;
        V(rmutex);
        读 F;
        P(rmutex);
        readcount - - ;
        if(readcount = =0) V(mutex);
        V(rmutex);
        P(mutex);
        写 F;
        V(mutex);
    }
}
```

37. 设公共汽车上，司机和售票员的活动分别是：

司机的活动：启动汽车

正常行车

到站停车

售票员的活动:关车门

售票

开车门

在汽车不断地到站、停车、行驶过程中,这两个人的活动有什么关系?用信号量和 P、V 操作实现他们的同步。(华中理工大学 1999 年试题)(哈尔滨工业大学 2000 年研究生试题)

答:司机和售票员之间的活动存在着两种制约关系:①司机只有在售票员关车门后,才能启动汽车;②售票员只有在司机到站停车后,才能开车门。只有在司机和售票员两者的活动得到很好的同步后,公共汽车才能正常行驶。

如果开始时,司机和售票员均已在汽车上,而汽车处于熄火并关闭车门的状态下,则具体的同步算法可描述为:

```
semaphore close=0,stop=0;
driver()|/* 司机 */
    点火发动汽车;
    while(1)|
        P(close);
        正常行车;
        到站停车;
        V(stop);
    |
|
conductor()|/* 售票员 */
    while(1)|
        开车门;
        上下乘客;
        关车门;
        V(close);
        售票;
        P(stop);
    |
|
main()|
    parbegin(driver,conductor);
|
```

38. 假设缓冲区 buf1 和缓冲区 buf2 都无限大,进程 p1 向 buf1 写数据,进程 p2 向 buf2 写数据。现要求 buf1 数据个数与 buf2 数据个数的差保持在[m,n]之间(m 小于 n,皆为正数)。请用信号量描述此同步关系。(浙江大学 2001 年研究生试题)

39. 有一个仓库,可以存放 A 和 B 两种产品,但要求:

(1)每次只能存入一种产品(A 或 B);

(2) $-N<A$ 产品数量 $-$ B 产品数量 $<M$。

其中,N 和 M 是正整数。试用 P,V 操作描述产品 A 与产品 B 的入库过程。(北京大学 1991 年研究生试题)

答:本题中,首先需要设置一个初值为 1 的互斥信号量 mutex,以保证每次只存入一种产品。另外,为了保证"$-N<A$ 产品数量 $-$ B 产品数量 $<M$",还需设置信号量 SA,表示仓库中目前可再存放的 A 产品的数量,其初值为 $M-1$;SB,表示目前还可再存放的 B 产品的数量,其初值为 $N-1$。A 产品入库的过程可描述为:

```
while(1){
    P(SA); /* 还可再存放一个 A 产品吗? */
    P(mutex);
    将 A 产品放入仓库;
    V(mutex);
    V(SB); /* 可存放的 B 产品数量增 1 */
}
```

B 产品入库的过程可描述为:

```
while(1){
    P(SB); /* 还可再存放一个 B 产品吗? */
    P(mutex);
      将 B 产品放入仓库;
    V(mutex);
    V(SA); /* 可存放的 A 产品数量增 1 */
}
```

40. 某工厂有两个生产车间和一个装配车间,两个生产车间分别生产 A,B 两种零件,装配车间的任务是把 A,B 两种零件组装成产品。两个生产车间每生产一个零件后都要分别把他们送到装配车间的货架 F1,F2 上。F1 存放零件 A,F2 存放零件 B,F1 和 F2 的容量均为可以存放 10 个零件。装配工人每次从货架上取一个 A 零件和一个 B 零件后组装成产品。请用 P,V 进行正确管理。(南京大学 1999 年研究生试题)

答:本题是生产者—消费者问题的一种变形,生产者"车间 A"和消费者"装配车间"共享缓冲区"货架 F1";生产者"车间 B"和消费者"装配车间"共享缓冲区"货架 F2"。因此,可为它们设置 6 个信号量,其中,empty1 对应货架 1 上的空闲空间,其初值为 10;full1 对应货架 1 上的 A 产品,其初值为 0;empty2 对应货架 2 上的空闲空间,其初值为 10;full2 对应货架 2 上的 B 产品,其初值为 0;mutex1 用于互斥地访问货架 1,其初值为 1;mutex2 用于互斥地访问货架 2,其初值为 1。

A 车间的工作过程可描述为:

```
while(1){
    生产一个产品 A;
    P(empty1);
    P(mutex1);
```

```
        将产品 A 存放到货架 F1 上;
        V(mutex1);
        V(full1);
    }
```

B 车间的工作过程可描述为:

```
    while(1){
        生产一个产品 B;
        P(empty2);
        P(mutex2);
        将产品 B 存放到货架 F1 上;
        V(mutex2);
        V(full2);
    }
```

装配车间的工作过程可描述为:

```
    while(1){
        P(full1);
        P(mutex1);
        从货架 F1 上取一个 A 产品;
        V(mutex1);
        V(empty1);
        P(full2);
        P(mutex2);
        从货架 F2 上取一个 B 产品;
        V(mutex2);
        V(empty2);
        将取得的 A 产品和 B 产品组装成产品;
    }
```

41. 有一个仓库存放两种零件 A 和 B,最大库容量各为 m 个。有一车间不断地取 A 和 B 进行装配,每次各取一个。为避免零件锈蚀,遵循先入库者先出库的原则。有两种供应商分别不断地供应 A 和 B。为保证齐套和合理库存,当某种零件的数量比另一种的数量超过 n(n<m)个时,暂停对数量大的零件进货,集中补充数量少的零件。试用 P,V 操作正确地实现之。(北京大学 1995 年研究生试题)

答:本题也是生产者—消费者问题的一种变形,两种供应商即两个生产者,车间即消费者。仓库可看成两个缓冲区,buffer1[m]用来存放零件 A,buffer2[m]用来存放零件 B。相应的算法如下:

```
type_A buffer1[m]; /* 仓库中用来存放零件 A 的缓冲区 */
type_B buffer2[m]; /* 仓库中用来存放零件 B 的缓冲区 */

/* 下面变量是为了遵循先入库者先出库的原则而设置的 */
```

```
int in1 = out1 = 0; /* in1、out1 用来指示下一个 A 零件的入库位置和出库位置 */
int in2 = out2 = 0; /* in2、out2 用来指示下一个 B 零件的入库位置和出库位置 */

semaphore empty1 = m, full1 = 0, empty2 = m, full2 = 0, mutex = 1;

/* 下面的信号量用来控制还可入库的零件 A 和 B 的数量，以使它们之差不超过 n */
semaphore SA = n;
semaphore SB = n;

supplierA(){
    while(1){
        P(empty1);
        P(SA);
        P(mutex);
        将零件 A 存放到 buffer1[in1]中;
        in1 = (in1 + 1) % m;
        V(mutex);
        V(full1);
        V(SB);
    }
}
supplierB(){
    while(1){
        P(empty2);
        P(SB);
        P(mutex);
        将零件 B 存放到 buffer2[in2]中;
        in2 = (in2 + 1) % m;
        V(mutex);
        V(full2);
        V(SA);
    }
}
workshop(){
    while(1){
        P(full1);
        P(full2);
        P(mutex);
        从 buffer1[out1]中取出一 A 零件;
```

```
            从 buffer2[out2]中取出一 B 零件;
            out1 = (out1 + 1) % m;
            out2 = (out2 + 1) % m;
            V(mutex);
            V(empty1);
            V(empty2);
            将零件 A 和 B 装配成产品;
        }
    }
    main(){
        parbegin(supplierA, supplierB, workshop);
    }
```

42. 设有一个发送者进程和接收者进程，其流程图如图 2.14 所示。S 是用于实现进程同步的信号量，mutex 是用于实现进程互斥的信号量。试问流程图中的 A, B, C, D 四框中应填写什么？假定缓冲区有无限多个，s 和 mutex 的初值应为多少？

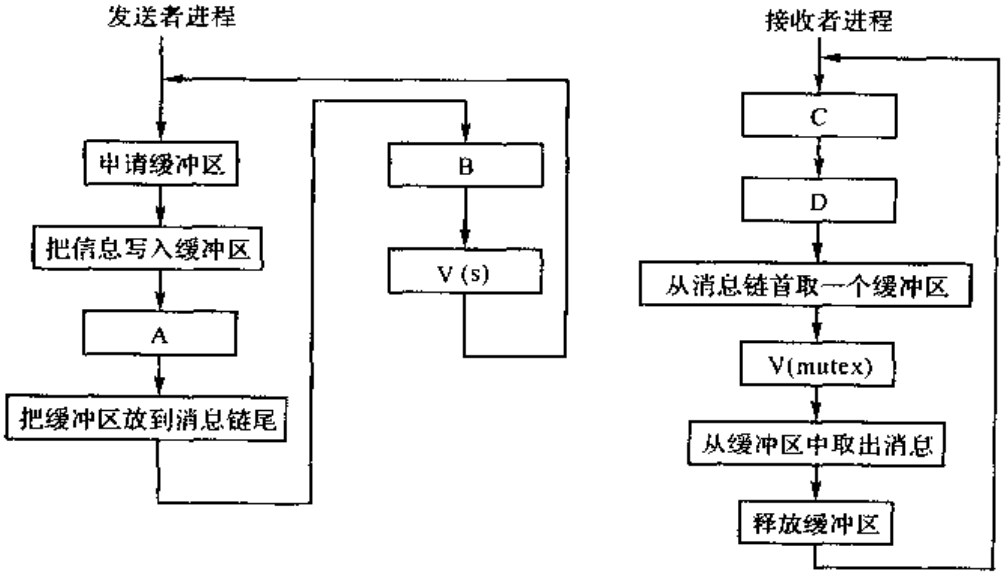


图 2.14 发送者及接收者工作流程图

答：A, B, C, D 四框中分别应该填写：P(mutex)、V(mutex)、P(S)、P(mutex)。S 是一资源信号量，用来表示消息链中消息的个数，故其初值应设置为 0；mutex 是用来实现对消息链互斥访问的互斥信号量，故其初值应设置为 1。

43. 进程 $A_1, A_2, \cdots, A_{n1}$ 通过 m 个缓冲区向进程 $B_1, B_2, \cdots, B_{n2}$ 不断地发送消息。发送和接收工作遵循如下规则：

(1)每个发送进程一次发送一个消息，写入一个缓冲区，缓冲区大小等于消息长度；

(2)对每一个消息，$B_1, B_2, \cdots, B_{n2}$ 都须各接收一次，读入各自的数据区内；

(3)m 个缓冲区都满时，发送进程等待；没有可读的消息时，接收进程等待。

试用 P, V 操作组织正确的发送和接收工作。(北京大学 1994 年研究生试题)

答：本题又是一个生产者—消费者问题的变形。由于每个产品(即消息)必须被 n2 个消费

者(接收者)各消费一次,因此,我们可以将每个存放产品的缓冲单元看成是有 n2 个小格,每个消息都必须被发送者同时写入这 n2 个小格中,而每个接收者从对应自己的小格中取消息。相应的算法如下:

```
messagetype buffer[m]; /* messagetype 为消息的类型 */
int in=0; /* in 指示下一个存放产品的缓冲区位置 */
int out[n2]={0,…,0}; /* out[i]指示 Bi 接收下一个消息的缓冲区位置 */
semaphore empty[n2]={m,…,m};
semaphore full[n2]={0,…,0};
semaphore mutex=1;

Ai(){
    int i;

    while(1){
        构成一消息;
        for(i=0;i++;i<n2) P(empty[i]);
        P(mutex);
        将消息存放入 buffer[in]中;
        in=(in+1)%m;
        V(mutex);
        for(i=0;i++;i<n2) V(full[i]);
    }
}
Bi(){
    while(1){
        P(full[i]);
        P(mutex);
        将消息从缓冲区 buffer[out[i]]中取到自己的数据区;
        out[i]=(out[i]+1)%m;
        V(mutex);
        V(empty[i]);
    }
}
```

44. 如图 2.15 所示,有四个进程 P0, P1, P2, P3 和四个信箱 M0, M1, M2, M3。进程间借助于相邻的信箱传递消息:Pi 每次从 Mi 中取出一条消息,经加工送入 M(i+1)mod 4 中,M0, M1, M2, M3 分别设有 3,3,2,2 个格子,每个格子放一条消息。初始时,M0 装了 3 条消息,其余为空。写出使用信号量实现进程间同步及互斥流程。(国防科技大学 1996 年研究生试题)

答:本题又是一个生产者—消费者问题,其中的每个进程 Pi 既是一个生产者,又是一个消

费者。为了同步上述进程，可设置下列变量和信号量：

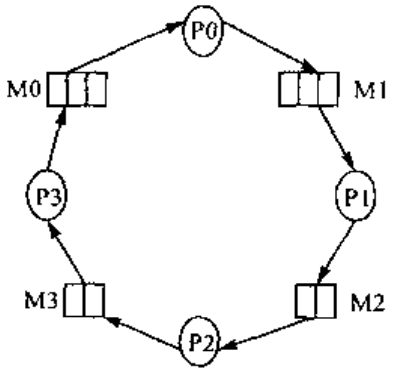


图 2.15 四个并发进程的工作示意图

```
int in[4]={0,0,0,0}, out[4]={0,0,0,0};
int size[4]={3,3,2,2}; /* 各信箱的大小 */
semaphore mutex[4]={1,1,1,1};
semaphore empty[4]={0,3,2,2}, full[4]={3,0,0,0};
```

进程 Pi 的工作流程可描述为：

```
Pi(){ /* 进程 Pi,i=0,1,2,3 */
    while(1){
        P(full[i]);
        P(mutex[i]);
        从信箱 Mi 的第 out[i]格中取出一条消息 message;
        out[i]=(out[i]+1)%size[i];
        V(mutex[i]);
        V(empty[i]);
        对消息 message 进行加工处理;
        P(empty[(i+1)%4]);
        P(mutex[(i+1)%4]);
        将加工后的消息 message 存放到第(i+1)%4 个信箱的第 in[(i+1)%4]格中;
        in[(i+1)%4]= (in[(i+1)%4]+1)%size[(i+1)%4];
        V(mutex[(i+1)%4]);
        P(full[(i+1)%4]);
    }
}
```

45. 进程 P1 和 P2 通过两个缓冲区给进程 P11、P12、P21、P22 传递信息，进程 P11、P12 取进程 P1 的信息，进程 P21、P22 取进程 P2 的信息。假定这两个缓冲区一样大小，所要传递的信息也与缓冲区一样大，同一时刻只能由一个进程往缓冲区中送信息或取信息。试用 PV 操作来实现这 6 个进程之间的同步与互斥关系，只要求写出进程 P1 与 P11 的同步算法。（南京航空航天大学 2002 年研究生试题）

答：在本题中，需设置以下信号量：empty 表示空闲的缓冲区，初值为 2；full1 表示装有 P1 进程信息的满缓冲区，初值为 0；full2 表示装有 P2 进程信息的满缓冲区，初值为 0；mutex 用来实现对缓冲区的互斥操作。另外，整型变量 in 表示存放下一个信息的缓冲区指针，初值为 0；out1 表示 P11、P12 下一次取信息的缓冲区指针，初值为 0；out2 表示 P21、P22 下一次取信息的缓冲区指针，初值为 0。

P1 进程可描述为：

```
while(1){
    P(empty);
    P(mutex);
```

```
        将消息存放入第 in 个缓冲区中;
        out1 = in;
        in = 1 - in;
        V(mutex);
        V(full1);
    }
```

P11 进程和 P12 进程可描述为:

```
    while(1){
        P(full1);
        P(mutex);
        将消息从第 out1 个缓冲区中取出;
        in: = out1;
        out1 = 1 - out1;
        V(mutex);
        V(empty);
    }
```

P2 进程可描述为:

```
    while(1){
        P(empty);
        P(mutex);
        将消息存放入第 in 个缓冲区中;
        out2 = in;
        in = 1 - in;
        V(mutex);
        V(full2);
    }
```

P21 进程和 P22 进程可描述为:

```
    while(1){
        P(full2);
        P(mutex);
        将消息从第 out2 个缓冲区中取出;
        in: = out2;
        out2 = 1 - out2;
        V(mutex);
        V(empty);
    }
```

46. 假定有一个信箱可存放 n 封信。当信箱不满时,发信者可把信件送入信箱;当信箱中有信时,收信者可从信箱中取信。用指针 r、k 分别表示可存信和取信的位置,请用管程(Moni-

tor)来管理这个信箱,使发信者和收信者能正确工作。(南京大学 1997 年研究生试题)

答:本题类似于生产者—消费者问题,此处的信箱相当于生产者—消费者问题中的缓冲区,发信者相当于生产者,收信者相当于消费者,指针 r、k 相当于生产者—消费者问题中的 in、out。具体算法略。

47. 消息缓冲通信技术是一种高级通信机制。(西安交通大学 2000 年试题)

(1)试叙述高级通信机制和低级通信机制 P、V 原语操作的主要区别;

(2)给出消息缓冲机制(有限缓冲)的基本工作原理;

(3)消息缓冲通信机制(有限缓冲)中提供发送原语 send(receiver, a),调用参数 a 表示发送消息的内存首地址,试设计相应的数据结构,并用 P、V 原语操作实现 send 原语。

答:(1)高级通信机制和低级通信机制的主要区别表现在两个方面:①透明性。在高级通信机制中,用户通过系统提供的发送原语 send 和接收原语 receive 进行进程通信,通信过程对用户是透明的。而在在利用 P、V 原语操作进行通信时,必须由用户自己实现整个通信过程,包括共享数据结构的设置、数据的传递、互斥和同步等,即通信过程对用户是不透明的。②效率。高级通信机制每次可传递一大批数据,效率较高。低级通信机制每次只能传递少量的数据,具体的量取决于共享的数据结构,效率较低。

(2)消息缓冲机制(有限缓冲)的基本工作原理如下:操作系统管理由有限个缓冲区组成的缓冲池,其中的每个缓冲区可存放一条消息;在每个进程的 PCB 中设有一个消息缓冲队列,其中有其他进程发送给它的所有消息。用户将需发送的消息存放在自己的发送区中,并调用发送原语将消息发送出去。发送原语须先向系统申请一个空闲的消息缓冲,并将用户发送区的消息复制到该消息缓冲区中,然后再将该消息缓冲区挂到接收进程的消息队列中。接收进程通过接收原语接收消息,即由接收原语从接收进程的消息队列中摘下一个消息缓冲,并将其中的消息复制到接收进程的接收区中,然后再释放消息缓冲区。在通信的过程中,还需通过同步机制保证对消息队列的互斥访问,并实现发送进程和接收进程之间的同步。

(3)通信过程中用到的数据结构主要为消息缓冲区和 PCB。其中消息缓冲是一个结构,其中需包含以下成员:

sender:发送进程的标识符;

size:消息长度;

text:消息正文;

next:指向下一个消息缓冲的指针。

而在 PCB 结构中,除了进程标识符、进程状态、CPU 现场保护信息、CPU 调度和控制信息等信息外,还需增加以下信息:

mq:消息队列队首指针;

mutex:消息队列互斥信号量;

Sm:消息队列资源信号量。

send(receiver, a)原语可描述为:

```
getbuf(a.size, i);          根据 a.size 申请消息缓冲区 i;
```

```
        i.sender = a.sender;
        i.size = a.size;          将发送区 a 中的信息复制到消息缓冲区 i 中;
        i.text = a.text;
        i.next = NULL;
        getpcb(PCBset,receiver,j);   根据进程标识符 receiver 获得接收进程 j 的
PCB;
        P(j.mutex);
        insert(j.mq,i);              将消息缓冲插入接收进程的消息缓冲队列 mq 中;
        V(j.mutex);
        V(j.sm);
```

48. 在消息缓冲通信中，系统设定消息队列和空闲缓冲队列。用 P、V 操作完成发送原语 send(Q,a1)和接收原语 read(P,a2)两流程。(Q、P 分别为接收进程名和发送进程名，a1、a2 分别为发送和接收区首址)(设：消息队列信号量为 S1，初值为 0；空闲缓冲队列信号量为 s2，初值为 n)。(北方交通大学 1996 年研究生试题)

答：根据题意，消息队列和空闲缓冲队列都是临界资源，它们供所有的进程共享，故必须为它们分别设置初值为 1 的互斥信号量 mutex1 和 mutex2；同时，为了避免从空的队列中摘取缓冲区，还必须为它们分别设置初值为 0 的资源信号量 S1 和初值为 n 的资源信号量 S2。相应的发送原语和接收原语可描述为：

```
message_buffer * mq, * fq; /* mq、fq 分别为消息队列和空闲缓冲队列队首指针
*/

semaphore mutex1 = mutex2 = 1;
semaphore S1 = 0, S2 = n;

send(Q,a1){
    message_buffer * buf;

    P(S2);
    P(mutex2);
    buf = remove(fq); /* 从空闲缓冲队列中摘下一个空缓冲 */
    V(mutex2);
    将消息内容从 a1 所指的发送区复制到 buf 所指的消息缓冲中;
    将接收进程的名称 Q 填入 buf 所指的消息缓冲中;
    将现行进程，即发送进程的名称也填入消息中;
    P(mutex1);
    insert(mq,buf); /* 将 buf 所指的消息缓冲插入消息队列 */
    V(mutex1);
    V(S1);
}
```

```
    receive(P,a2){
        message_buffer *buf;


        buf=NULL;
        P(S1);
        P(mutex1);
        buf=find(mq,P); /* 寻找并摘取一个发送者为 P、接收者为现行进程的消息
*/
        V(mutex1);
            将 buf 所指的消息缓冲中的消息复制到 a2 所指的接收区中;
            P(mutex2);
            insert(fq,buf); /* 释放使用完毕的缓冲区 */
            V(mutex2);
            V(S2);
        }
    }
```

49. 在消息传递通信方式下：

(1)发送进程和接收进程在通信过程中可以采用哪 3 种同步方式?

(2)试以下面给出的发送进程和接收进程(将接收到的数据存入 S)为例,说明当接收进程执行到标号为 L2 的语句时,采用这 3 种同步方式,X 的值可能是多少?(中科院计算所 1999 年研究生试题)

```
发送进程 P:
                M:=10;
        L1:  send M TO Q;
        L2:  M:=20;
             goto L1;
接收进程 Q:
                S:=-100;
        L1:  receive S from P;
        L2:  X:=S+1;
```

答:(1)3 种同步方式为:①发送进程发送消息后阻塞,以等待消息被接收进程接收;接收进程接收时需阻塞,以等待发送进程将消息发送给它。②发送进程发送消息后不必阻塞;而接收进程则需阻塞,以等待消息到达。③发送进程发送消息后不必阻塞;接收进程也不需阻塞,当消息未到达时,它将接收到一个空消息。

(2)①采用第一种同步方式时,Q 接收到的肯定是 P 发送的第一个消息,故 X 的值为 11。②采用第二种同步方式时,由于接收进程需阻塞,故它接收到的肯定是一个有效消息。如果该消息系统将发送给同一进程的所有消息按序排队,并规定接收者按顺序接收这些消

息，则 X 的值为 11。如果该消息系统只允许接收进程接收一个最新的消息，则 X 的值可能是 11 或 21。③采用第三种同步方式时，由于接收进程不需阻塞，因此执行到 L2 时，它既可能还没接收到任何有效消息，此时，X 的值为 -99；也可能已接收到一个有效消息，此时，X 的取值可能同②。

50. 某高校计算机系开设网络课并安排上机实习，假设机房共有 2m 台机器，有 2n 名学生选该课，规定：

(1)每两个学生组成一组，各占一台机器，协同完成上机实习；

(2)只有一组两个学生到齐，并且此时机房有空闲机器时，该组学生才能进入机房；

(3)上机实习由一名教师检查，检查完毕，一组学生同时离开机房。

试用 P、V 操作模拟上机实习过程。(北京大学 1997 年研究生试题)

答：本题中，可设置一个隐含的进程 guard，他相当于机房管理员，当有两个学生到达、并有空闲的机器时，他便通知两个学生进入机房。相应的信号量和各个进程描述如下：

```
semaphore computer=2m; /* 对应于计算机的资源信号量 */
semaphore student=0; /* 对应于欲进入机房的学生 */
semaphore enter=0; /* 用来控制学生是否可进入机房 */
semaphore finish=test=0; /* 用来同步学生和教师——教师须检查实习完毕的学生 */
student_i(){/* i=1,2,…, 2n*/
    V(student);          /* 通知管理员来了一个学生 */
    P(enter);            /* 等待进入机房 */
    进入机房上机实习;
    V(finish);          /* 通知教师已完成实验 */
    P(test);     /* 等待检查 */
    离开机房;
    V(computer);     /* 释放所占用的计算机 */
}
guard(){
    int i;
    for(i=0;i++;i<n){
        P(computer);     /* 申请计算机 */
        P(computer);
        P(student);     /* 等学生达到 */
        P(student);
        V(enter);     /* 通知学生进入机房 */
        V(enter);
    }
}
teacher(){
    int i;
    for(i=0;i++;i<n){
```

```
            P(finish);      /* 等学生完成实验 */
            P(finish);
            检查两个学生的实习结果;
            V(test);      /* 通知学生检查完毕 */
            V(test);
        }
    }
```

51. 兄弟俩共同使用一个账号，每次限存或取 10 元，存钱与取钱的进程分别如下所示：(南京大学 2000 年试题)

```
begin
    amount:integer;
    amount:=0;
    cobegin
      process SAVE
        m1:integer;
        begin
          m1:=amount;
          m1:=m1+10;
          amount:=m1;
        end;
      process TAKE
        m2:integer;
        begin
          m2:=amount;
          m2:=m2-10;
          amount:=m2;
        end;
      coend;
    end;
```

由于兄弟俩可能同时存钱和取钱，因此两个进程是并发的。若哥哥先存了两次钱，但在第三次存钱时，弟弟在取钱。请问最后账号 amount 上面可能出现的值？如何用 P、V 操作实现两并发进程的互斥执行？

答：哥哥存了两次钱后，共享变量 amount 的值为 20。哥哥的第三次存钱与弟弟的取钱同时进行，如果两者顺序执行，则最后账号 amount 的值为 20；如果在一个进程的执行过程中，进行 CPU 调度，转去执行另一进程，则最后 amount 的值取决于 amount:=m1 及 amount:=m2 的执行先后次序，若前者先执行，则最后 amount 的值为 10，若后者先执行，则最后 amount 的值为 30。因此，最后账号 amount 上面可能出现的值有 10、20、30。

上述问题中，共享变量 amount 是一个临界资源，为了实现两并发进程对它的互斥访问，可

为它设置一初值为 1 的互斥信号量 mutex，并将上述算法修改为：

```
begin
        amount:integer;
        mutex:semaphore;
        amount:=0;
        mutex:=1;
        cobegin
          process SAVE
            m1:integer;
            begin
          P(mutex);
          m1:=amount;
          m1:=m1+10;
          amount:=m1;
          V(mutex);
        end;
      process TAKE
        m2:integer;
        begin
          P(mutex);
          m2:=amount;
          m2:=m2-10;
          amount:=m2;
          V(mutex);
        end;
      coend;
    end;
```

52. 进程之间存在哪几种相互制约关系？各是什么原因引起的？下列活动分别属于哪种制约关系？（北京理工大学 1996 年研究生试题）

(1)若干同学去图书馆借书；

(2)两队举行篮球比赛；

(3)流水线生产的各道工序；

(4)商品生产和社会消费。

答：进程之间存在着直接制约和间接制约这两种制约关系。其中直接制约(即同步)是由于进程间的相互合作而引起的；而间接制约(即互斥)则是由于进程间共享临界资源而引起的。

(1)主要属于互斥关系，因为书是临界资源。

(2)主要属于互斥关系，因为篮球是临界资源。

(3)主要属于同步关系，各道工序的开始都依赖于前一道工序的完成。

(4)主要属于同步关系,商品没生产出来则不可能消费,商品没消费完则无须再生产。

53.某招待所有 100 个床位,住宿者住入要先登记(在登记表上填写姓名及床位号),离去时要撤销登记(在登记表上删去姓名和床位号)。请给出住宿登记及撤销登记过程的算法描述。(北京邮电大学 1999 年研究生试题)

答:本题中,被住宿者竞争的资源主要有床位和住宿登记表两种,可分别为它们设置初值为 100 的信号量 bed 及初值为 1 的信号量 mutex。住宿登记过程的算法描述如下:

```
P(bed);
P(mutex);
在登记表上填写姓名及床位号;
v(mutex);
```

撤消登记过程的算法描述如下:

```
P(mutex);
在登记表上删去姓名和床位号;
V(mutex);
V(bed);
```

54.有一阅览室,读者进入阅览室必须先在一张登记表(TB)上登记,该表为每一座位设一个表目,读者离开时要消掉其登记信息,阅览室共有 100 个座位。为了描述读者的动作,请用 Pascal 语言和 P、V 操作写出进程间的同步算法。约定:(北方交通大学 1999 年研究生试题)

(1)flag 的值:0 座位空闲,1 座位被占用。

(2)用语句 i=getflag(0)可搜索到一个空座位 i,用语句 i.falg=0 或 1 可给标志位赋值。

(3)用 i=getname(readername)可搜索到某读者所登记的座位号 i;用 i.name=0 或 i.name=readername 可给姓名字段赋值,0 表示消除读者姓名。

(4)计数信号量用 count,互斥信号量用 mutex。

答:本题中,读者要竞争座位、登记表两种资源,故可分别为它们设置初值为 100 的信号量 count,以及初值为 1 的信号量 mutex。读者的动作可描述为:

```
reader(){
    while(1){
        P(count);                /* 申请一个座位 */
        P(mutex);                /* 申请登记表 */
        i=getflag(0);            /* 在登记表上搜索一个空座位 */
        i.flag=1;                /* 登记该座位已被占用 */
        i.name=readername;       /* 登记读者姓名 */
        V(mutex);                /* 释放登记表 */
        进入阅览室,坐下并开始阅览;
        P(mutex);                /* 申请登记表 */
        i=getname(readername);   /* 在登记表上搜索读者登记的座位号 */
        i.falg=0;                /* 撤销登记信息 */
        i.name=0;
```

```
            P(mutex);              /* 释放登记表 */
            V(count);              /* 释放座位 */
            离开阅览室;
        }
    }
```

55. 某车站售票厅，任何时刻最多可容纳 20 名购票者进入，当售票厅中少于 20 名购票者时，则厅外的购票者可立即进入，否则需在外面等待。若把一个购票者看作一个进程，请回答下列问题：

(1)用 P、V 操作管理这些并发进程时，应怎样定义信号量，写出信号量的初值以及信号量各种取值的含义。

(2)根据所定义的信号量，把应执行的 PV 操作填入下述方框中，以保证进程能够正确地并发执行。

```
COBEGIN   PROCESS   PI(I=1,2,……)
        begin  [          ];
               进入售票厅;
               购票;
               退出;
               [          ];
        end;
COEND
```

(3)若欲购票者最多为 n 个人，写出信号量可能的变化范围(最大值和最小值)。

答：(1)本题中，有限的售票厅空间是购票者竞争的资源，故必须为它设置一个信号量 S。由题意可知，该信号量的初值必须置为 20，表示在初始状态下，售票厅可容纳 20 个购票者；在购票过程中，如果 S.value 的值大于 0，则表示该时刻售票厅内还可容纳 S.value 个购票者；否则，表示售票厅已挤满，而 S.value 的绝对值表示等在售票厅外的购票者人数。

(2)前一框中应填 P(S)；后一框中应填 V(S)。

(3)信号量 S 的变化范围为[20 − n, 20]，即最大值为 20，最小值为 20 − n。

56. 描述操作系统中使用公用缓冲池时的数据块插入缓冲队列的过程。(清华大学 1999 年研究生试题)

答：在公用缓冲池中，每个队列都是被多个进程共享的临界资源，故必须为每个队列设置一个初值为 1 的互斥信号量；另外，为了防止进程从空缓冲队列中摘取缓冲，还必须为每个队列设置一个初值为 0 的资源信号量。如果队列 queue 对应的互斥信号量为 mutex，资源信号量为 Sm，则将 buf 所指的数据缓冲插入队列 queue 的操作可描述为：

```
P(mutex);
在 queue 中找到插入位置，修改链接指针以插入 buf;
V(mutex);
V(Sm);
```

57. 有一单向行驶的公路桥，每次只允许一辆汽车通过。当汽车到达桥头时，若桥上无车，便可上桥；否则，需等待，直到桥上的汽车下桥为止。若每一辆汽车为一个进程，请用 P，V 操作

编程实现。

答:在本题中,公路桥是一个临界资源,由于它每次只允许一辆汽车通过,故可为它设置一个初值为 1 的临界资源 mutex。汽车过公路桥的动作可描述为:

```
行驶到桥头;
P(mutex);
上桥行驶;
从另一头下桥;
V(mutex);
```

58. 在一个允许单向行驶的十字路口,分别有若干个由东向西、由南向北的车辆在等待通过十字路口。为了安全,每次只允许一辆车通过(东→西或南→北)。当有车辆需通过时其他车辆需等待,当无车辆在路口行驶时则允许一辆(东→西或南→北)进入。请用 P、V 操作实现能保证安全行驶的自动管理系统。(西安电子科技大学 2001 年研究生试题)

答:本题中,十字路口是一个临界资源,由于它每次只允许一辆车通过,因此,可为它设置一个初值为 1 的信号量 mutex。相关车辆的动作可描述为:

```
由东向西的车辆:
        ⋮
    行驶到十字路口东端;
    P(mutex);
    通过十字路口;
    V(mutex);
        ⋮

由南向北的车辆:
        ⋮
    行驶到十字路口南端;
    P(mutex);
    通过十字路口;
    V(mutex);
        ⋮
```

59. 有桥如图 2.16 所示。(北京大学 1992 年研究生试题)

车流如箭头所示。桥上不允许两车交会,但允许同方向多辆车依次通行(即桥上可以有多个同方向的车)。用 P,V 操作实现交通管理以防止桥上堵塞。

答:为了描述上述同步问题,需设置两个整形变量 countA、countB 分别表示由南往北、由北往南已在桥上行驶的汽车数目,它们的初值为 0;再设置三个初值都为 1 的互斥信号量:SA 用来实现对 countA 的互斥访问,SB 用来实现对 countB 的互斥访问,mutex 用来实现两个方向的车辆对桥的互斥使用。则可将由南往北过桥的车辆描述为:

```
行驶到桥头;
P(SA);
```

```
    if(countA=0) P(mutex);
    countA++;
  V(SA);
  过桥;
  P(SA);
    countA--;
    if (countA=0) V(mutex);
  V(SA);
由北往南的车辆的算法
  行驶到桥头;
  P(SB);
    if(countB=0) P(mutex);
    countB++;
  V(SB);
  过桥;
  P(SB);
    countB--;
    if (countB=0) V(mutex);
  V(SB);
```

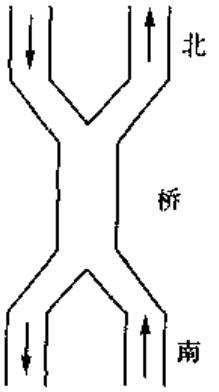


图 2.16　桥的示意图

60. 在一个盒子里，混装了数量相等的围棋白子和黑子。现在要用自动分拣系统把白子和黑子分开。设系统有两个进程 P1 和 P2，其中 P1 拣白子、P2 拣黑子。规定每个进程每次只拣一子。当一进程正在拣子时，不允许另一进程去拣；当一进程拣了一子时，必须让另一进程去拣。试写出这两个并发进程能正确执行的程序。（陕西省 1998 年自考题）

答：本题中，首先需设置两个初值为 0 的信号量 S1 和 S2，来同步 P1 和 P2 的执行；其次还需通过一个初值为 0 的整型变量 count 以及对应它的初值为 1 的互斥信号量 mutex 来控制首个拣棋的进程。如果白子的数目为 n 个，则可将 P1、P2 描述为：

```
int count=0; /* 已启动的拣棋进程数目,用来控制谁先开始拣棋 */
semaphore mutex=1;
semaphore S1=S2=0;

P1(){
    int i;
    P(mutex);
        if(count==0){
          V(S1); /* 若 P1 先到达,则允许 P1 先开始拣棋 */
          count = 1;
        }
    V(mutex);
    for(i=0;i++;i<n){ /* n 为白子的数目 */
```

```
            P(S1);
            拣起一白子;
            V(S2);
        }
    }

    P2(){
        P(mutex);
            if(count = =0) {
            V(S2); /* 若 P2 先到达,则允许 P2 先开始拣棋 */
            count = 1 ;
            }
        V(mutex);
        for(i=0;i++;i<n) {/* 黑子的数目与白子相等 */
            P(S2);
            拣起一黑子;
            V(S1);
        }
    }
```

61. 有一个理发师,一把理发椅和 n 把供等候理发的顾客坐的椅子。如果没有顾客,则理发师便在理发椅上睡觉;当一个顾客到来时,必须叫醒理发师进行理发;如果理发师正在理发时有顾客来到,则如果有空椅子可坐,他就坐下来等,如果没有空椅子,他就离开。为理发师和顾客各编一段程序描述他们的行为,要求不能带有竞争条件。(西安电子科技大学 2000 年研究生试题)(参见 P143,题 7.8)

62. 某寺庙,有小和尚、老和尚若干。有一水缸,由小和尚提水入缸供老和尚饮用。水缸可容 10 桶水,水取自同一井中。水井径窄,每次只能容一个桶取水。水桶总数为 3 个。每次入、取缸水仅为 1 桶,且不可同时进行。试给出有关取水、入水的算法描述。(北京邮电大学 1998 年研究生试题)

答:本题中需设置 5 个信号量:bucket 对应临界资源水桶,初值为 3;jar 用来对水缸进行互斥操作,初值为 1;full 表示水缸中可以使用的水,初值为 0;empty 表示缸中还可入水的容量,初值为 10;well 用来保证互斥地使用水井,初值为 1。有关取水、入水的算法描述如下:

```
semaphore bucket=3, jar=1, full=0, empty=10, well=1;

little_monk(){ /* 小和尚入水算法 */
    while(1){
        P(empty);
        P(bucket);
        P(well);
          从水井中打水;
```

```
            V(well);
            P(jar);
              倒水入缸;
            V(jar);
            V(full);
            V(bucket);
        }
    }

old monk(){ /* 老和尚取水算法 */
        while(1){
            P(full);
            P(bucket);
            P(jar);
              从缸中取水;
            V(jar);
            V(empty);
              从桶中倒水饮用;
            V(bucket);
        }
    }
```

63. 有一间酒吧里有3个音乐爱好者队列，第1队的音乐爱好者只有随身听，第2队只有音乐磁带，第3队只有电池。而要听音乐就必须随身听、音乐磁带和电池这3种物品俱全。酒吧老板一次出售这3种物品中的任意两种。当一名音乐爱好者得到这3种物品并听完一首乐曲后，酒吧老板才能再一次出售这3种物品中的任意两种，于是第2名音乐爱好者得到这3种物品，并开始听乐曲。全部买卖就这样进行下去。(北京大学1999年研究生试题)

试用P、V操作正确解决这一买卖。

答：第1队音乐爱好者要竞争“待出售的音乐磁带和电池”，而且在初始状态下，系统并无“待出售的音乐磁带和电池”，故可为该种资源设置一初值为0的信号量buy1；同样，需设置初值为0的buy2、buy3分别对应“待出售的随身听和电池”、“待出售的随身听和音乐磁带”。另外，为了同步买者的付费动作和卖者的给货动作，还需设置信号量payment和goods，以保证买者在付费后才能得到所需商品。信号量music_over用来同步音乐爱好者听乐曲和酒吧老板的下一次出售行为。

具体的算法描述如下：

```
semaphore buy1 = buy2 = buy3 = 0;
semaphore payment = 0;
semaphore goods = 0;
semaphore music_over = 0;
```

```
fan1(){ /* 对应第 1 队音乐爱好者的算法 */
    while(1){
        P(buy1); /* 等有音乐磁带和电池出售 */
        V(payment); /* 付费 */
        P(goods); /* 取货 */
        欣赏一曲乐曲;
        V(music_over); /* 通知老板乐曲结束 */
    }
}
fan2(){/* 对应第 2 队音乐爱好者的算法 */
    while(1){
        P(buy2); /* 等待有随身听和电池出售 */
        V(payment); /* 付费 */
        P(goods); /* 取货 */
        欣赏一曲乐曲;
        V(music_over); /* 通知老板乐曲结束 */
    }
}
fan3(){/* 对应第 3 队音乐爱好者的算法 */
    while(1){
        P(buy3); /* 等待有随身听和音乐磁带出售 */
        V(payment); /* 付费 */
        P(goods); /* 取货 */
        欣赏一曲乐曲;
        V(music_over); /* 通知老板乐曲结束 */
    }
}
boss(){
    while(1){
        拿出任意两种物品出售;
        if(出售的是音乐磁带和电池) V(buy1);
        else if(出售的是随身听和电池) V(buy2);
        else if(出售的是随身听和音乐磁带) V(buy3);
        P(payment); /* 等待付费 */
        V(goods); /* 给货 */
        P(music_over); /* 等待乐曲结束 */
    }
}
```

64. 有一材料保管员，负责管理纸和笔；另有 A，B 两组学生，A 组学生每个都备有纸，B 组学生

每个都备有笔，但一个学生只要能得到其他一种材料就可以写信。有一个可以放一张纸或一支笔的小盒，当小盒中无物品时，保管员就可以放入一张纸或一支笔供学生取用。当盒中有学生所需的材料时，每次允许一个学生从中取出自己需用的材料，当一个学生从盒中取走材料后，允许保管员再存放一件材料。请用管程写出他们并发执行时能正确工作的程序。(哈尔滨工业大学 1999 年研究生试题)

答：本题中，首先需建立以下的管程：

```
monitor letter_writing
{
    int empty,no_paper,no_pen;
    condition notfull,paper,pen;
    void put(){ /* 向小盒中放材料 */
        if(empty==0) notfull.wait(); /* 等待盒子变空，即不满 */
        empty=0;
        将材料放入盒中;
        if 材料是纸{
            no_paper=0;
            paper.signal();
        } else if 材料是笔{
            no_pen=0;
            pen.signal();
        }
    }
    void getpaper(){ /* 从小盒中取纸 */
            if(no_paper==1) paper.wait(); /* 等待条件"小盒中有纸张"成立 */
            从盒中取得纸;
            no_paper=1;
            empty=1;
            notfull.signal();
        }
        void getpen(){ /* 从小盒中取笔 */
            if(no_pen==1) pen.wait(); /* 等待条件"小盒中有笔"成立 */
            从盒中取得笔;
            no_pen=1;
            empty=1;
            notfull.signal();
        }
        void init(){
          empty=1;
          no_pen=no_paper=1;
        }
```

}

其中，局部变量 empty 表示盒子是否空闲，no_pen 表示盒子中是否无笔，no_paper 表示盒子中是否无纸。条件变量 notfull 表示条件“盒子不满”，pen 表示条件“盒子中有笔”，paper 表示条件“盒子中有纸”。过程 put()向盒子中放材料，getpen()从盒子中取笔，getpaper()从盒子中取纸。相应的进程可描述为：

```
keeper(){ /* 保管员 */
    while(1){
        letter_writing.put();
    }
}
studentA(){ /* A组学生 */
    while(1){
        letter_writing.getpen();
        写信;
    }
}
studentB(){ /* B组学生 */
    while(1){
        letter_writing.getpaper();
        写信;
    }
}
```

65.“吸烟者”问题。假设一个系统有 3 个吸烟者(smoker)进程和一个供货商(Agent)进程。每个吸烟者连续不断地制造香烟并吸掉它。但是，制造一支香烟需要 3 种材料——烟、纸、火柴。一个吸烟者进程有纸，另一个有烟，第三个有火柴。供货商进程可以无限地提供这 3 种材料。供货商将两种材料一起防在桌上，持有另一种材料的吸烟者即可制造一支香烟并吸掉它。当此吸烟者抽香烟时，它发出一个信号通知供货商进程，供货商马上给出另外两种材料，如此循环往复。试编写一个程序使供货商与吸烟者同步执行。

答：略。(参见 P144，题 7.9)

66.现有 100 名毕业生去甲、乙两公司求职，两公司合用一间接待室，其中甲公司准备招收 10 人，乙公司准备要 15 人，招完即止；两公司各有一位人事主管在接待毕业生，每位人事主管每次只可接待一人，其他毕业生在接待室外排成一队等待。试用记录型信号量机制实现对此过程的管理。要求给出您的分析过程，写出需要的假设以及所设计的数据结构和算法。(中科院软件研究所 2000 年试题)

答：由于毕业生只排成一个队列，故需设置一个队列数据结构，其中的每个节点表示一个未完成面试的毕业生。节点中设置了一个标志字段 flag，flag=0 表示该学生还未接受任何公司的面试，其第 0 位若为 1 则表示该学生已接受过 A 公司的面试，第 1 位为 1 则表示该学生已接受过 B 公司的面试。节点中还设置了一个 testing 字段，表示对应的学生是否正在接受面试。每个毕业生到达后，就申请一个空闲的节点，填上自己的信息后，将节点插入毕

业生队列末尾，然后等待面试，当两个公司都面试完后，学生将节点释放，然后离开。甲公司的人事主管负责从队列中按顺序找一个目前没在接受面试、且还未被自己面试过的学生；然后开始进行面试，决定录用与否；面试结束后，如果该毕业生已被录用或者未被录用但已接受过 B 公司的面试，则将其从队列中删除，并通知该毕业生离开。B 公司人事主管的工作过程与 A 公司的人事主管类似。由于毕业生人数为 100，而 A 公司只需招收 10 人，B 公司只需招收 15 人，故应分别设置计数器 wait_A、wait_B 表示等待被 A 公司、B 公司面试的人数，countA、countB 表示已被 A 公司、B 公司招收的人数；并用它们来控制招聘的进程，招聘是否结束由整型变量 overA、overB 来表示。具体的算法描述如下：

```
#define TRUE 1
#define FALSE 0
struct student_node{
    :
    short int flag; /* 01--已被 A 公司面试,02--已被 B 公司面试 */
    short int testing; /* 1--正在接受面试 0-正在等待面试 */
    semaphore test_over; /* 控制该学生面试是否结束,初值为 0 */
    struct student *next; /* 用来排队 */
} *queue;

semaphore mutex;/* 用于实现对毕业生队列的互斥访问 */

int countA=countB= 0; /* A 公司、B 公司已招收的人数 */
int wait_A=wait_B=100; /* 等待 A 公司、B 公司面试的人数 */
semaphore SA=SB=0; /* 用来实现对 wait_A、wait_B 互斥访问 */

int overA=overB=FALSE; /*表示招聘工作是否结束 */

graduate(){ /* 毕业生进程,共 100 个 */
    struct student_node *stu;

    来到接待室外;
    申请一个空闲的学生节点 stu(其中的字段均初始化为 0);
    P(mutex);
    Insert(queue,stu);          /* 将节点 stu 插到 queue 末尾 */
    V(mutex);
    P(stu->test_over);          /* 等待第一次面试 */
    P(stu->test_over);          /* 等待第二次面试 */
    freenode(stu);             /* 释放学生节点 */
    离开;
}
```

```
directorA(){ /* A公司的人事主管 */
    while(overA! =TRUE){
        P(mutex);
        在队列 queue 中找一个未被 A 公司面试过、而且目前没在接受面试的学生 stu;
                /* 即 stu->flag|01==0 ; stu->testing=FALSE */
        stu->flag|=01; /* 被 A 公司面试过 */
        stu->testing=TRUE; /* 目前正在接受面试 */
        V(mutex);
        P(SA);
        wait_A--; /* 等待 A 公司面试的人数减 1 */
        if(wait_A==0) overA=TRUE; /* 如果是最后一个毕业生,则置结束标志 */
        V(SA);
        进行面试;
        if (合格) {
            P(mutex);
            if(stu->flag|02==0){ /* 该毕业生没接受过 B 公司的面试 */
                P(SB);
                wait_B--; /* 不需再参加 B 公司的面试 */
                V(stu->test_over); /* 通知该毕业生 B 公司的面试结束 */
                V(SB);
            }
            countA++; /* 录用人数增加 1 */
            delete(queue,stu); /* 从队列中删除该毕业生节点 */
            V (stu->test_over); /* 通知该毕业生 A 公司的面试结束 */
            V(mutex);
        } else { /* 不合格 */
            P(mutex);
            If(stu->flag|02==-1) { /* 该毕业生已经 B 公司面试 */
            Delete(queue,stu);
            } else
                stu->testing=FALSE; /* 清正在接受面试标志 */
            V (stu->test_over); /* 通知该毕业生 A 公司的面试结束 */
            V(mutex);
        }

        if(countA==10){
            overA=TRUE; /* A 公司招聘结束 */
            P(mutex);
            P(SA);
            while(wait_A>0){
```

```
            找到队列中下一个没被A公司面试过的毕业生节点stu;
            if(stu->flag|02==0 || stu->testing==TRUE) /* 还需B公司面试 */
                stu->flag|=01;
            else { /* 已经过B公司面试 */
                delete(queue,stu);
            }
            V(stu->test_over); /* 通知该毕业生A公司的面试结束 */
            wait_A--;
        }
        V(SA);
        V(mutex);
    }
  }
}

directorB(){ /* B公司的人事主管 */
    while(overB! =TRUE){
        P(mutex);          在队列queue中找一个未被B公司面试过、而且目前没在接
受面试的学生stu;                    /* 即 stu->flag|02==0 ; stu->testing=
FALSE */
        stu->flag|=02; /* 被B公司面试过 */
        stu->testing=TRUE; /* 目前正在接受面试 */
        V(mutex);
        P(SB);
        wait_B--; /* 等待B公司面试的人数减1 */
        if(wait_B==0) overB=TRUE; /* 如果是最后一个毕业生,则置结束标志 */
        V(SB);
        进行面试;
        if (合格) {
            P(mutex);
            if(stu->flag|01==0){ /* 该毕业生没接受过A公司的面试 */
                P(SA);
                wait_A--; /* 不需再参加A公司的面试 */
                V(stu->test_over); /* 通知该毕业生A公司的面试结束 */
                V(SA);
            }
            countB++; /* 录用人数增加1 */
            delete(queue,stu); /* 从队列中删除该毕业生节点 */
            V(stu->test_over); /* 通知该毕业生B公司的面试结束 */
            V(mutex);
```

```
        } else {
            P(mutex);
            If(stu->flag[01 = = 1) { /* 该毕业生已经A公司面试 */
            Delete(queue,stu);
        } else
            stu->testing=FALSE; /* 清正在接受面试标志 */
        V(stu )test_over); /* 通知该毕业生B公司的面试结束 */
        V(mutex);
    }

    if(countB= =15){
        overB=TRUE; /* B公司招聘结束 *
        P(mutex);
        P(SB);
        while(wait_B>0){
            找到队列中下一个没被B公司面试过的毕业生节点stu;
            if(stu->flag[01 = =0 || stu->testing= =TRUE) /* 还需A公司面试 */
                stu->flag[=02; /* B公司不再面试 */
            else{ /* 已经过A公司面试 */
                delete(queue,stu);
            }
            V(stu->test_over); /* 通知该毕业生B公司的面试结束 */
            wait_B--;
        }
        V(SB);
        V(mutex);
    }
  }
}
```

67. 一个主修动物行为学、辅修计算机科学的学生参加了一个课题，调查花果山的猴子是否能被教会理解死锁。他找到一处峡谷，横跨峡谷拉了一根绳索(假设为南北方向)，这样猴子就可以攀着绳索越过峡谷。只要它们朝着相同的方向，同一时刻可以有多只猴子通过。但是如果在相反的方向上同时有猴子通过则会发生死锁(这些猴子将被卡在绳索中间，假设这些猴子无法在绳索上从另一只猴子身上翻过去)。如果一只猴子想越过峡谷，它必须看当前是否有别的猴子在逆向通过。请使用信号量写一个避免死锁的程序来解决该问题。(南京航空航天大学2001年考研试题)

答：略。(请参考74页第59题)

68. 有一个铁笼子，每次只能放入一个动物。猎手向笼中放入老虎，农民向笼中放入猪；动物园等待取笼中的老虎，饭店等待取笼中的猪。请用P，V原语写出能同步执行的程序。

答:略。(请参考50页31题)

2.7.3 进程调度

1. 关于处理机调度,试问:

(1)什么是处理机的三级调度?

(2)处理机的三级调度分别在什么情况下发生?

(3)各级调度分别完成什么工作?(西北大学2000年研究生试题)

答:(1)处理机的三级调度是指,一个作业在运行过程中需要经过高级调度(即作业调度)、中级调度(即进程对换)和低级调度(即进程调度)这三种调度。但是,作业是否需要高级调度和中级调度,将与具体使用的操作系统类型有关。

(2)高级调度主要用在批处理系统中,并在需要从外存的后备队列向内存调入作业运行时发生;中级调度在内存紧张而无法满足运行作业的要求时发生;低级调度是在执行进程运行完毕、执行进程转入阻塞状态、执行进程的时间片用完、有比现行进程更紧迫的进程到达并允许它抢占CPU等情况下发生的。

(3)高级调度的主要工作是根据调度算法决定把外存后备队列中的哪些作业调入内存,并为它们创建进程、分配必要的资源,然后,再将新创建的进程插入就绪队列上等待执行。中级调度的工作是在内存紧张时,将内存中暂时不能运行的进程调出至外存,并在内存空闲时再将外存中具备运行条件的就绪进程调入内存。低级调度的主要工作是根据一定的调度算法,决定就绪进程中的哪一个进程将获得CPU,并将CPU分派给它。

2. 引起进程调度的原因有哪些?

答:引起进程调度的原因有:

(1)进程正常终止或异常终止;

(2)正在执行的进程因某种原因而阻塞:

* 提出I/O请求后被阻塞;
* 在调用wait操作时因资源不足而阻塞;
* 因其他原因执行block原语而阻塞等;

(3)在引入时间片的系统中,时间片用完;

(4)在抢占调度方式中,就绪队列中某进程的优先权变得比当前正在执行的进程高,或者有优先权更高的进程进入就绪队列。

3. 某进程被唤醒后立即投入运行,我们就说这个系统采用的是剥夺调度方法,对吗?为什么?

答:上述说法是错误的。因为,即使在一个不采用剥夺调度策略的系统中,当一个进程被唤醒后,如果系统中没有其他的就绪进程(如其他进程均已运行完毕,或全部处于阻塞状态),则它同样将立即投入运行。

4. 在CPU按优先权调度的系统中:

(1) 没有运行进程是否一定就没有就绪进程?

(2) 没有运行进程,没有就绪进程,或者两者都没有,是否可能?各是什么情况?

(3) 运行进程是否一定是自由进程中优先权最高的?

答:(1)只要有就绪进程,进程调度程序便必将从其中选择优先权最大的进程投入执行。故没有运行进程,则肯定就没有就绪进程。

(2)当系统中只有一个进程且它具备执行条件时;或者,系统中只有一个进程具备执行条件,

而其他的进程均处于阻塞状态时，系统中便会出现只存在运行进程却没有就绪进程的现象。如果系统中的所有进程均处于阻塞状态(如它们都在等待 I/O 的完成)，则系统中便会出现既没有运行进程也没有就绪进程的现象。

系统中不会出现只有就绪进程，却没运行进程的现象。

(3)如果系统采用立即抢占的优先权调度方式，则运行进程将是自由进程中优先权最高的(不考虑切换过程中的暂时状态)；否则，运行进程的优先权可能比某个就绪进程低。

5. 现有两道作业同时执行，一道以计算为主，另一道以输入为主，你将怎样赋予作业进程占有处理器的优先级？为什么？

答：输入为主的作业的优先权应高于计算为主的的作业，这样才能使 CPU 和输入设备都得到有效的利用，从而提高资源的利用率和系统的吞吐量。

6. Kleinrock 曾给出过一个基于动态改变优先数的抢占型优先数调度算法，较大的优先数表明优先数较高。当一个作业等待 CPU 时(在就绪队列中但并未运行)，其优先数依参数 α 而变化；当它运行时，其优先数依参数 β 而变化。所有进程进入就绪队列时，均预置优先数为 0。置定 α 和 β 可得出很多不同的调度算法。

①试问：

a. $\beta>\alpha>0$ 时导出的是什么算法？　　b. $\alpha<\beta<0$ 时导出的是什么算法？

c. $\alpha>\beta>0$ 时导出的是什么算法？　　d. $\beta<\alpha<0$ 时导出的是什么算法？

②假如：

a. $\beta=2$ 优先数/秒 $\alpha=1$ 优先数/秒

b. $\beta=-1$ 优先数/秒 $\alpha=-2$ 优先数/秒

c. $\beta=1$ 优先数/秒 $\alpha=2$ 优先数/秒

d. $\beta=-2$ 优先数/秒 $\alpha=-1$ 优先数/秒

如有二个进程：A 进程在 0 时刻到达，B 进程在 2 秒时刻到达，A、B 进程各需要运行 10 秒。当就绪进程优先数高于(等于)执行态进程优先数 1 时发生抢占型调度，试画出上面四种情况时 0－8 秒时间内 CPU 运行进程的情况。

答：略。(参见 139 页题 6.7)

7. 许多 CPU 调度算法都是含有变量的，例如轮转算法的时间片，多级反馈队列调度法中的队列个数、优先级、各队列的调度算法以及进程在队列之间移动的原则等。也可以说，这些算法通过选取某些变量的不同值可能演变成其他算法，如轮转法当时间片取值无穷大时，便变成 FCFS 法。另外，一种调度算法也可以包含另一种调度算法。下面的各对进程调度算法，如果它们之间有关系的话，是什么关系？

a. 优先数法，最短作业优先法。

b. 多级反馈队列法，FCFS 法。

c. 优先数法，FCFS 法。

d. 轮转法，最短作业优先法。

e. 多队列调度法，多级反馈队列法。

答：略。(参见 139 页题 6.8)

8. 为什么说多级反馈队列调度算法能较好地满足各类用户的需要？

答：对终端型作业用户而言，他们提交的作业大多属于交互型作业，作业通常较小，系统只要能使这些作业在第一个队列所规定的时间片内完成，便可使他们都感到满意。对短批处理

作业用户而言，开始时他们的作业像终端型作业一样，如果仅在第一个队列中执行一个时间片即可完成，便可获得与终端型作业一样的响应时间；对于稍长的作业，通常也只需在第二队列和第三队列各执行一个时间片即可完成，其周转时间仍然很短。对长批处理作业用户而言，他们的作业将依次在第 1，2，…，n 个队列中运行，然后再按轮转方式运行，用户不必担心其作业长期得不到处理；而且每往下降一个队列，其得到的时间片将随着增加，故可进一步缩短长作业的等待时间。

9. 假设就绪队列中有 10 个进程，系统将时间片设为 200ms，CPU 进行进程切换要花费 10ms，试问系统开销所占的比率约为多少？

答：当一个进程获得 CPU 后，它能执行一个时间片，即 200ms；然后，系统进行 CPU 切换，把 CPU 切换给一个新进程，这要花费 10ms。可见，系统开销（即用于 CPU 切换的时间）所占的比率为：10ms/(200ms + 10ms) = 4.8%

10. 在单处理机的分时系统中，分配给进程 P 的时间片用完后，系统进行切换，结果调度到的仍然是进程 P。有可能出现上述情形吗？如果可能请说明理由。

答：有可能出现上述情形。如果在进程 P 的时间片用完时，系统中没有其他的就绪进程；或者，系统采用的是带优先权的轮转调度算法，而 P 恰巧是优先权最高的就绪进程，则调度到的将仍然是 P 进程。

11. 在轮转调度中决定时间片 q 是一个复杂而关键的任务。假定进程间平均文本切换时间是 s，一个受 I/O 束缚的进程在产生一个 I/O 请求前使用的时间为 t(t>>s)，讨论以下每个时间片设置的影响。

(1)q 比 0 稍微大；(2)q=s；(3)s<q<t；(4)q=t；(5)q>t；(6)q=无限大。

答：轮转调度中的时间片会直接影响到 CPU 的利用率。每当执行进程执行完一个时间片 q 后，系统便要进行 CPU 的切换，假如切换需花费的平均时间为 s，则 CPU 浪费在管理开销上的时间大约占 s/(q+s)，而 CPU 的利用率则为 q/(q+s)。另外，当具备运行条件的进程数目为 N 时，一个需 k(k>=⌈t/q⌉)个时间片才能得到响应的进程，在忽略 I/O 所花费时间的情况下，其响应时间 T 可表示为：

$$T \approx k \times N \times (q+s) \approx \begin{cases} N \times (q+s) & \text{当 } k=1 \text{ 时} \\ k \times N \times q + k \times N \times s & \text{当 } k>1 \text{ 时} \end{cases}$$

(1)q 趋近 0 时，CPU 的利用率也趋于 0，即它的时间基本都花在进程切换上。此时，由于 k 趋于无穷大，使得响应时间也将趋于无穷大。

(2)q=s 时，CPU 的利用率约为 50%；而由于 t>>s=q，故一个交互作业在获得响应以前需执行相当多个时间片，使得其响应时间很长。

(3)s<q<t 时，CPU 的利用率将超过 50%；其响应时间也可得当一定的改善，但一个交互作业在获得响应以前仍需执行一个以上的时间片，故响应时间仍可能较长。

(4)q=t 时，由于 q=t>>s，CPU 的利用率将趋近于 100%；而且，此时一个交互作业在获得响应以前平均只需要执行一个时间片，故能有较好的响应时间。

(5)q>t 时，CPU 的利用率将更趋近于 100%；但随着时间片的增大，响应时间也将越变越长。

(6)q 趋于无限大时，轮转调度算法将变成 FCFS 算法，它虽然有较高的 CPU 利用率，但平均响应时间将难以满足交互请求的需要。

12. 试证明，短作业优先的作业调度算法可以得到最短的平均响应时间。（北京大学 1990 年研

究生试题)

答:假设有 n 个作业 $J_1, J_2, \cdots, J_n$,它们各自的运行时间分别为 $T_1, T_2, \cdots, T_n$,当作业的调度次序为 $J_{i1}, J_{i2}, \cdots, J_{in}$时,则平均响应时间 T 为:

$$T = [T_{i1} + (T_{i1} + T_{i2}) + (T_{i1} + T_{i2} + T_{i3}) + \cdots + (T_{i1} + T_{i2} + T_{i3} + \cdots + T_{in})]/n$$
$$= [n \times T_{i1} + (n-1) \times T_{i2} + (n-2) \times T_{i3} + \cdots + 1 \times T_{in}]/n$$

由于 $n>(n-1)>(n-2)>\cdots>1$,因此,当 $T_{i1}<=T_{i2}<=T_{i3}<=\cdots<=T_{in}$时,平均响应时间 T 最短。故短作业优先的作业调度算法可以得到最短的平均响应时间。

13. 有 5 个待运行作业 A, B, C, D, E, 各自估计的运行时间为 9, 6, 3, 5, x, 假定这些作业同时到达,试问采用哪种运行次序可以使得平均周转时间最短? 如果 x 为 7,按上述运行次序,平均周转时间为多少?

答:采用短作业优先调度算法将使作业的平均周转时间最短,故本题的答案与 x 有关:

(1)当 $x<3$ 时,应采用 E, C, D, B, A 的运行次序;

(2)当 $3<=x<5$ 时,应采用 C, E, D, B, A 的运行次序;

(3)当 $5<=x<6$ 时,应采用 C, D, E, B, A 的运行次序;

(4)当 $6<=x<9$ 时,应采用 C, D, B, E, A 的运行次序;

(5)当 $x>=9$ 时,应采用 C, D, B, A, E 的运行次序。

当 x 为 7 时,采用 C, D, B, E, A 的运行次序,平均周转时间为(3 + 8 + 14 + 21 + 30)/5,即 15.2。

14. 设有 4 道作业,它们的提交时间及执行时间如表 2-8 所示:

表 2-8 作业的提交时间和执行时间

作业号	提交时间	执行时间
1	10	2.0
2	10.2	1.0
3	10.4	0.5
4	10.5	0.3

试计算在单道程序环境下,采用先来先服务调度算法和最短作业优先调度算法时的平均周转时间和平均带权周转时间,并指出它们的调度顺序。(时间单位:小时,以十进制进行计算)。

答:采用先来先服务调度算法时,作业的调度顺序为 1, 2, 3, 4,从表 2-9 中可以计算出,作业的平均周转时间和平均带权周转时间分别为 2.8, 5.25。

表 2-9 FCFS 算法下作业的运行情况

作业号	提交时间	开始时间	执行时间	结束时间	周转时间	带权周转时间
1	10	10	2.0	12	2	1
2	10.2	12	1.0	13	2.8	2.8
3	10.4	13	0.5	13.5	3.1	6.2
4	10.5	13.5	0.3	13.8	3.3	11.0

由于在 10:00 时,系统中只有一个作业 1,故先调度它运行,当它运行完毕(即 12:00)时,作业 2、3、4 均已到达,按短作业优先的原则,作业的调度顺序应为 4, 3, 2,因此,采用最短作

业优先调度算法时，作业的调度顺序为：1，4，3，2。表 2-10 中给出了各作业的周转时间和带权周转时间，作业的平均周转时间和平均带权周转时间分别为 2.45，3.85。

表 2-10　SJF 算法下作业的运行情况

作业号	提交时间	开始时间	执行时间	结束时间	周转时间	带权周转时间
1	10	10	2.0	12	2	1
2	10.2	12.8	1.0	13.8	3.6	3.6
3	10.4	12.3	0.5	12.8	2.4	4.8
4	10.5	12	0.3	12.3	1.8	6

15. 设有 4 个作业 J1，J2，J3，J4，它们的到达时间和计算时间如表 2-11 所示。

若这 4 个作业在一台处理机上按单道方式运行，采用响应比高者优先调度算法，试写出各作业的执行顺序、各作业的周转时间及平均周转时间。

表 2-11　作业的到达时间和计算时间

作业	到达时间	计算时间
J1	8:00	2 小时
J2	8:30	40 分钟
J3	9:00	25 分钟
J4	9:30	30 分钟

答：作业的响应比可表示为：

$$\text{响应比} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = \frac{\text{响应时间}}{\text{要求服务时间}}$$

在 8:00，系统中只有一个作业 J1，故系统将它投入运行。在 J1 完成（即 10:00）时，J2，J3，J4 的响应比分别为：(90 + 40)/40，(60 + 25)/25、(30 + 30)/30，即 3.25，3.4、2，故应先将 J3 投入运行。在 J3 完成（即 10:25）时，J2、J4 的响应比分别为(115 + 40)/40、(55 + 25)/25，即3.875、3.2，故应先将 J2 投入运行，待它运行完毕时（即 11:05），再将 J4 投入运行，J4 的结束时间为 11:35。

可见作业的执行次序为 J1，J4，J3，J2，各作业的运行情况如表 2-12 所示，它们的周转时间分别为 120 分钟、155 分钟、85 分钟、125 分钟，平均周转时间为 121.25。

表 2-12　作业的运行情况

作业号	提交时间	开始时间	执行时间	结束时间	周转时间
1	8:00	8:00	2 小时	10:00	120 分钟
2	8:30	10:25	40 分钟	11:05	155 分钟
3	9:00	10:00	25 分钟	10:25	85 分钟
4	9:30	11:05	30 分钟	11:35	125 分钟

16. 在一个批处理系统中，有两个作业进程。有一作业序列，其到达时间及估计运行时间列表如表 2-13 所示。（北京大学 1993 年研究生试题）

系统采用最高响应比优先的作业调度算法（响应比＝等待时间/估计运行时间）。作业进程的调度采用短作业优先的抢占式调度算法。

(1)列出各作业的执行时间（即列出每个作业运行的时间片段，例如作业 i 的运行时间序列为 10:00～10:40,11:00～11:20,11:30－11:50(结束)）。

(2)计算这批作业的平均周转时间。

表 2-13 作业到达时间及估计运行时间

作业	到达时间(时)	估计运行时间(分钟)
1	10:00	35
2	10:10	30
3	10:15	45
4	10:20	20
5	10:30	30

答：上述 5 个作业的运行情况如图 2.17 所示。

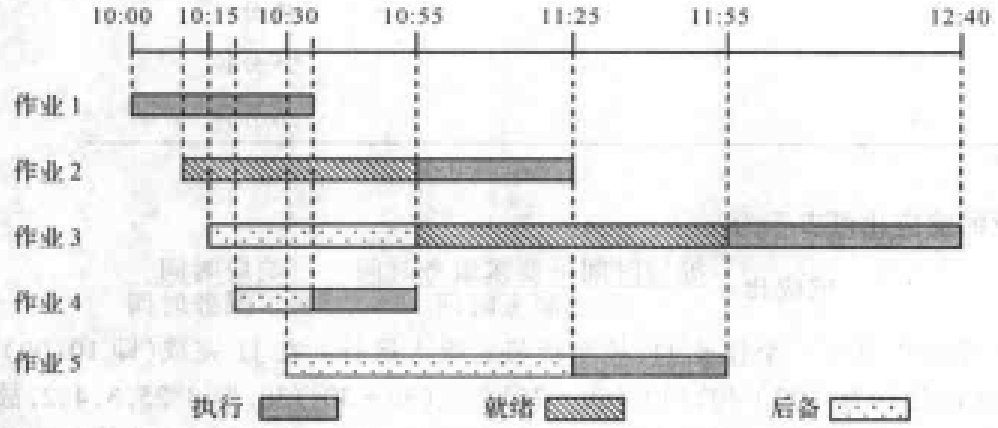


图 2.17 作业的运行情况

在 10:00，因为只有 J1 到达，故将它调入内存，并将 CPU 调度给它。

在 10:10，J2 到达，故将 J2 调入内存，但由于 J1 只需再执行 25 分钟，故 J1 继续执行。

虽然 J3、J4、J5 分别在 10:15、10:20 和 10:30 到达，但因当时内存中已存放了两道作业，故不能马上将它们调入内存。

在 10:35，J1 结束。此时，J3、J4、J5 的响应比（根据题意，响应比＝等待时间/估计运行时间）分别为 20/45、15/20、5/30，故将 J4 调入内存；并将 CPU 分配给内存中运行时间最短者，即 P4。

在 10:55，J4 结束。此时，J3、J5 的响应比分别为 40/40、25/30，故将 J3 调入内存，并将 CPU 分配给估计运行时间较短的 J2。

在 11:25，J2 结束，作业调度程序将 J5 调入内存，并将 CPU 分配给估计运行时间较短的 J5。

在 11:55，J5 结束，将 CPU 分配给 J3。

在 12:40，J3 结束。

通过上述分析，可知：

(1)作业 1 的执行时间片段为：10:00～10:35(结束)。

作业 2 的执行时间片段为：10:55～11:25(结束)。

作业 3 的执行时间片段为：11:55～12:40(结束)。

作业 4 的执行时间片段为：10:35～10:55(结束)。

作业 5 的执行时间片段为：11:25～11:55(结束)。

(2)它们的周转时间分别为：35 分钟、75 分钟、145 分钟、35 分钟、85 分钟，故它们的平均周转时间为 75 分钟。

17. 在一单道批处理系统中，一组作业的提交时间和运行时间如表 2-14 所示。试计算以下 3 种作业调度算法的平均周转时间 T 和平均带权周转时间 W。(西北大学 1998 年研究生试题)

(1)先来先服务；(2)短作业优先；(3)响应比高者优先。

表 2-14　作业提交时间和运行时间表

作业	提交时间	运行时间
1	8.0	1.0
2	8.5	0.5
3	9.0	0.2
4	9.1	0.1

答：采用先来先服务(FCFS)、短作业优先(SJF)、响应比高者优先(HRRN)三种算法时，作业的运行情况分别如表 2-15, 2-16, 2-17 所示。因此：

表 2-15　FCFS 算法下作业的运行情况

作业号	提交时间	开始时间	运行时间	结束时间	周转时间	带权周转时间
1	8.0	8.0	1.0	9.0	1.0	1.0
2	8.5	9.0	0.5	9.5	1.0	2.0
3	9.0	9.5	0.2	9.7	0.7	3.5
4	9.1	9.7	0.1	9.8	0.7	7.0

表 2-16　SJF 算法下作业的运行情况

作业号	提交时间	开始时间	运行时间	结束时间	周转时间	带权周转时间
1	8.0	8.0	1.0	9.0	1.0	1.0
2	8.5	9.3	0.5	9.8	1.3	2.6
3	9.0	9.0	0.2	9.2	0.2	1.0
4	9.1	9.2	0.1	9.3	0.2	2.0

表 2-17 HRRN 算法下作业的运行情况

作业号	提交时间	开始时间	运行时间	结束时间	周转时间	带权周转时间
1	8.0	8.0	1.0	9.0	1.0	1.0
2	8.5	9.0	0.5	9.5	1.0	2.0
3	9.0	9.6	0.2	9.8	0.8	4.0
4	9.1	9.5	0.1	9.6	0.5	5.0
响应比	在时刻 9.0 时			在时刻 9.5 时		
	P2:(0.5+0.5)/0.5=2.0			P3:(0.0+0.2)/0.2=1.0		
	P3:(0.5+0.2)/0.2=3.5			P4:(0.4+0.1)/0.1=5.0		

(1)采用先来先服务算法时,作业的平均周转时间 T 为 0.85,平均带权周转时间 W 为 3.375。

(2)采用短作业优先算法时,作业的平均周转时间 T 为 0.675,平均带权周转时间 W 为 1.65。

(3)采用响应比高者优先算法时,作业的平均周转时间 T 为 0.825,平均带权周转时间 W 为 3.0。

18. 在一个 4 道作业的操作系统中,设在一段时间内先后到达 6 个作业,它们的提交时间和运行时间如表 2-18 所示。(北京大学 1991 年研究生试题)

系统采用短作业优先的调度算法,作业被调度进入运行后不再退出,但每当一作业进入运行时,可以调整运行的优先次序。

(1)按照所选择的调度算法,请分别给出上述 6 个作业的执行时间序列。

(2)计算在上述调度算法下作业的平均周转时间。

表 2-18 作业提交时间和运行时间

作业	提交时间(时)	运行时间(分钟)
JOB 1	8:00	60
JOB 2	8:20	35
JOB 3	8:25	20
JOB 4	8:30	25
JOB 5	8:35	5
JOB6	8:40	10

答:上述 6 个作业的运行情况如图 2.18 所示。可见:

(1)按照所选择的调度算法,6 个作业的执行时间序列分别为:

JOB1:8:00~8:20,9:55~10:35;

JOB2:8:20~8:25,9:25~9:55;

JOB3:8:25~8:45;

JOB4:9:00~9:25;

JOB5:8:45~8:50;

JOB6:8:50~9:00。

(2)在上述调度算法下,各作业的周转时间分别为 155 分钟、95 分钟、20 分钟、55 分钟、15 分钟、20 分钟,因此,它们的平均周转时间为 60 分钟。

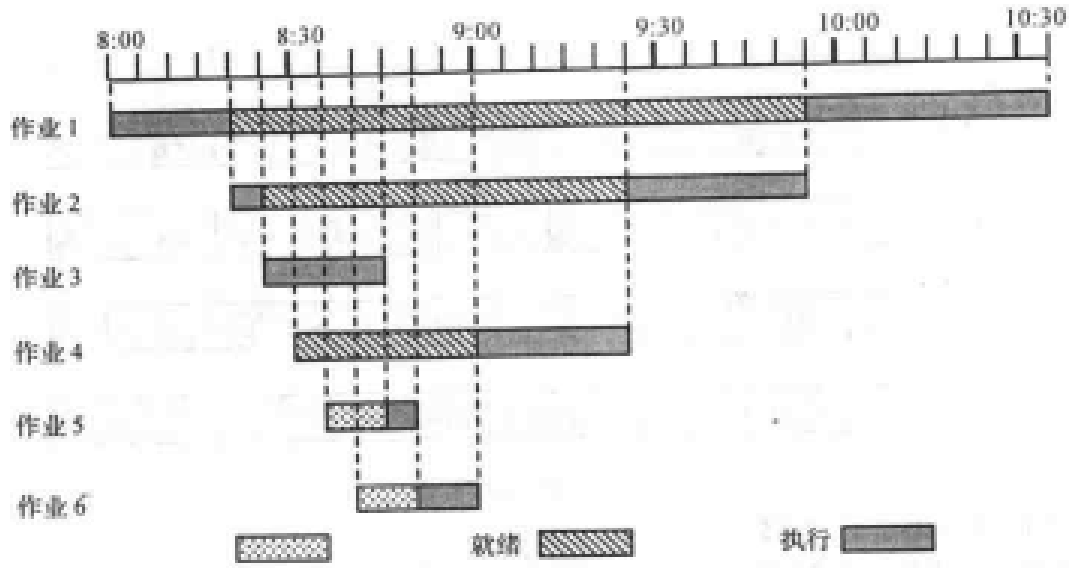


图 2.18 6个作业的运行情况

19. 假定要在一台处理机上执行如表2-19所示的作业，且假定这些作业在时刻0以1,2,3,4,5的顺序到达。(西北工业大学2000年研究生试题)

(1)说明分别使用FCFS、RR(时间片=1)、SJF以及非剥夺式优先级调度算法时，这些作业的执行情况。

(2)针对上述每种调度算法，给出平均周转时间和平均带权周转时间。

表 2-19 作业执行时间和优先级

作业	执行时间	优先级
1	10	3
2	1	1
3	2	3
4	1	4
5	5	2

答：略。(参见137页题6.3)

20. 有5个批处理的作业(A,B,C,D,E)按C,D,B,E,A的顺序几乎同时到达一个计算中心，估计的运行时间分别为2,4,6,8,10分钟，它们的优先级分别为1,2,3,4,5(1为最低优先级)。对下面的每种调度算法，分别计算作业的平均周转时间。

(1)最高优先级优先；

(2)时间片轮转(时间片为2分钟)；

(3)FIFO；

(4)短作业优先。

答：采用上述各种调度算法时，CPU调度的情况如图2.19所示。因此：

在采用最高优先级优先调度算法时，A,B,C,D,E各作业的周转时间分别为：30分钟、28分钟、24分钟、18分钟、10分钟，故此时的作业平均周转时间为22分钟。

在采用时间片轮转调度算法时，各作业的周转时间分别为：10分钟、16分钟、20分钟、26分钟、30分钟，故此时的作业平均周转时间为20.4分钟。

在采用FIFO调度算法时，各作业的周转时间分别为：30分钟、18分钟、6分钟、14分钟、28

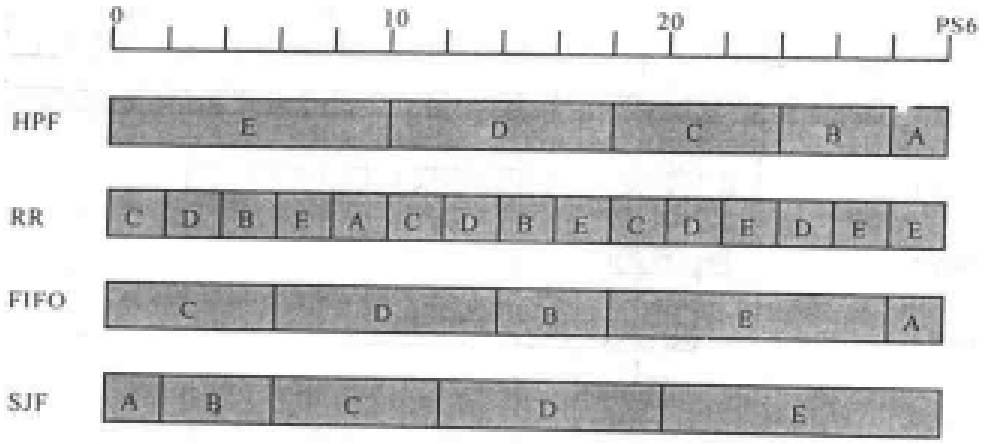


图 2.19 CPU 调度的情况

分钟，故此时的作业平均周转时间为 19.2 分钟。

在采用最高优先级优先调度算法时，各作业的周转时间分别为：2 分钟、6 分钟、12 分钟、20 分钟、30 分钟，故此时的作业平均周转时间为 14 分钟。

21. 有一个具有两道作业的批处理系统，作业调度采用短作业优先的调度算法，进程调度采用以优先数为基础的抢占式调度算法。有如表 2-20 所示的作业序列（表中所列的作业优先数即为进程优先数，优先数越小优先级越高）。（北京大学 1995 年研究生试题）

表 2-20 作业的到达时间、估计运行时间和优先数

作业名	到达时间	估计运行时间	优先数
A	10:00	40 分	5
B	10:20	30 分	3
C	10:30	50 分	4
D	10:50	20 分	6

(1)列出所有作业进入内存时刻及结束时刻；

(2)计算平均周转时间。

答：作业的运行情况如图 2.20 所示。因此，

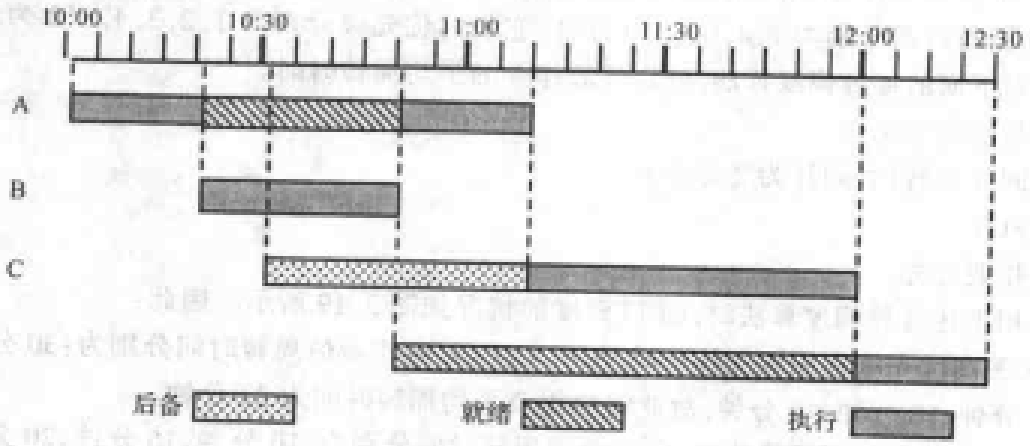


图 2.20 4 个作业的运行情况

(1)A、B、C、D 各作业进入内存时刻分别为 10:00、10:20、11:10、10:50，它们的结束时刻分

别为 11:10、10:50、12:00、12:20。

(2)作业 A、B、C、D 的周转时间分别为 70 分钟、30 分钟、50 分钟、90 分钟，故它们的平均周转时间为 60 分钟。

22. 假定在一个处理机上执行的操作如下：

进程	CPU 运行时间	优先数
p1	10	3
p2	1	1
p3	2	3
p4	1	4
p5	5	2

假定这些进程按 p1, p2, p3, p4, p5 这种次序同时到达。

(1)给定相应的图来分别说明用 FCFS, RR(时间片 = 1), SJF 和非抢占优先调度算法(最小优先数有最高优先权)调度这些进程的情况。

(2)分别给出采用上述调度算法时每个进程的周转时间和带权周转时间。

(3)此时，具有最小平均等待时间的调度算法是那一个？

答：(1)采用各种调度算法时，进程的调度情况如图 2.21 所示。

(2)采用上述调度算法时各个进程的周转时间和带权周转时间如表 2-21 所示。

表 2-21 进程的周转时间 T、带权周转时间 M 和等待时间 D

进程	FCFS			RR(q=1)			SJF			非抢占优先调度		
	T	M	D	T	M	D	T	M	D	T	M	D
P1	10	1	0	2	1.9	9	19	1.9	18	16	1.6	6
P2	11	11	10	7	2	1	1	1	0	1	1	0
P3	13	6.5	11	4	3.5	5	4	2	2	18	9	16
P4	14	14	13	14	4	3	2	2	1	19	19	18
P5	19	3.8	14		2.8	9	9	1.8	4	6	1.2	1
			19									
平均等待时间	9.6			5.4			5			8.2		

(3)从表 2-21 可以看出，具有最小平均等待时间的调度算法是 SJF 算法。

23. 今有三个批处理作业。第一个作业 10:00 到达，需要执行 2 个小时；第二个作业在 10:10 到达，需要执行 1 小时；第三个作业在 10:25 到达，需要执行 25 分钟。分别采取如表 2-22、2-23、2-24 所示的三种作业调度算法：

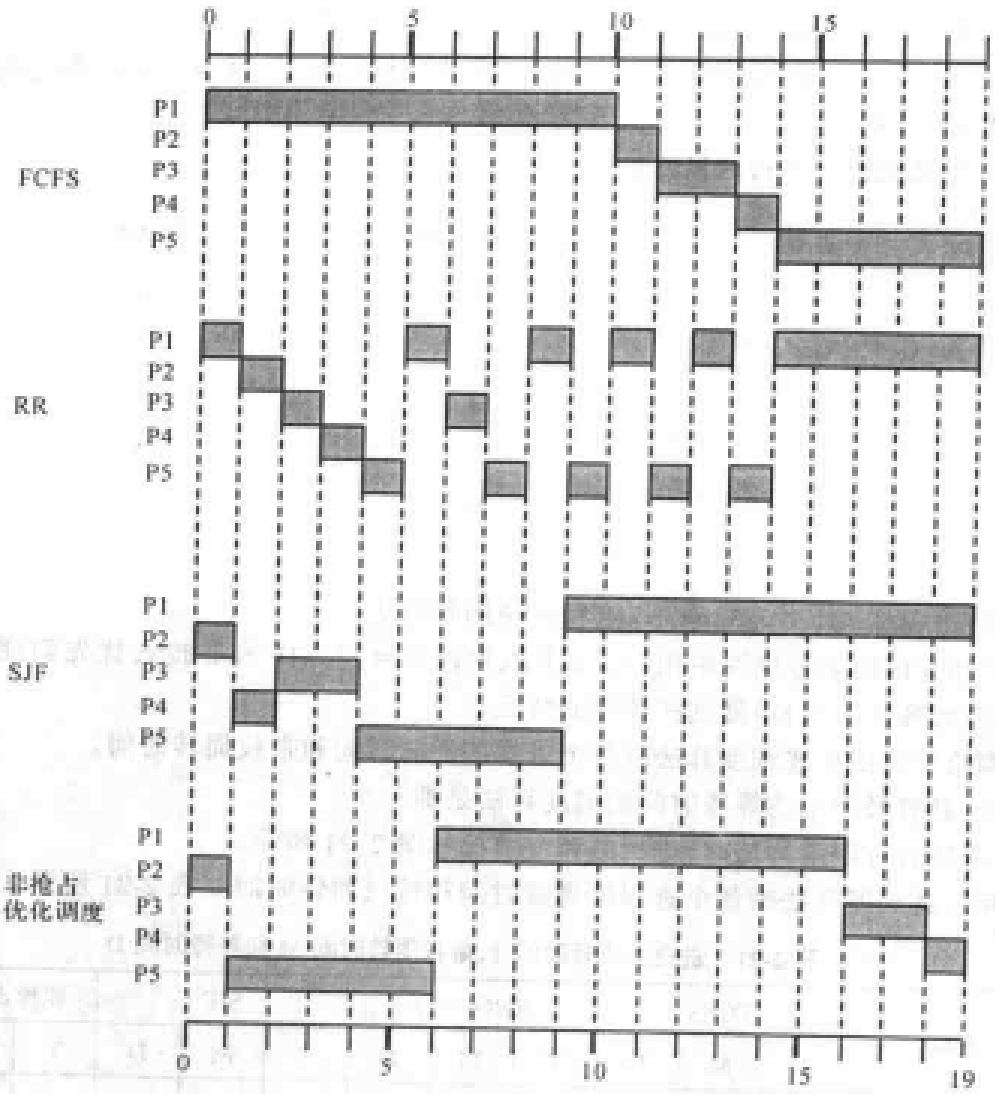


图 2.21 各种调度策略的进程调度情况

表 2-22 调度算法 1

作业号	到达时间	开始执行时间	执行结束时间
1	10:00	10:00	12:00
2	10:10	12:00	13:00
3	10:25	13:00	13:25

表 2-23 调度算法 2

作业号	到达时间	开始执行时间	执行结束时间
1	10:00	11:50	13:50
2	10:10	10:50	11:50
3	10:25	10:25	10:50

表 2-24　调度算法 3

作业号	到达时间	开始执行时间	执行结束时间
1	10:00	10:00	12:00
2	10:10	12:25	13:25
3	10:25	12:00	12:25

(1)计算各调度算法的作业平均周转时间。

(2)调度算法 1、3 分别是什么作业调度算法?

答:(1)采用调度算法 1 时,作业 1,2,3 的周转时间分别为 2 小时、2 小时 50 分、3 小时,故它们的平均周转时间为 2 小时 37 分。采用调度算法 2 时,作业 1,2,3 的周转时间分别为 3 小时 50 分、1 小时 40 分、25 分,故它们的平均周转时间为 1 小时 58 分。采用调度算法 3 时,作业 1、2、3 的周转时间分别为 2.0 小时、3 小时 15 分、2 小时,故它们的平均周转时间为 2 小时 25 分

(2)调度算法 1 是先来先服务算法,调度算法 3 可能是非抢占的短作业优先调度算法或响应比高者优先调度算法(在 12:00 时,作业 2、3 的响应比分别是 2.83、4.8)。

24. 在单 CPU 和两台输入/输出设备(I1, I2)的多道程序设计环境下,同时投入三个作业 J1、J2 和 J3 运行。这三个作业对 CPU 和输入/输出设备的使用顺序和时间如下所示:

J1:I2(30ms);CPU(10ms);I1(30ms);CPU(10ms);I2(20ms)

J2:I1(20ms);CPU(20ms);I2(40ms)

J3:CPU(30ms);I1(20ms);CPU(10ms);I1(10ms)

假定 CPU、I1、I2 都能并行工作,J1 优先级最高,J2 次之,J3 优先级最低,优先级高的作业可以抢占优先级的作业的 CPU 但不抢占 I1 和 I2。试求:

(1)三个作业从投入到完成分别需要的时间;

(2)从投入到完成的 CPU 利用率;

(3)I/O 设备利用率。

答:三个作业的运行情况如图 2.22 所示。因此:

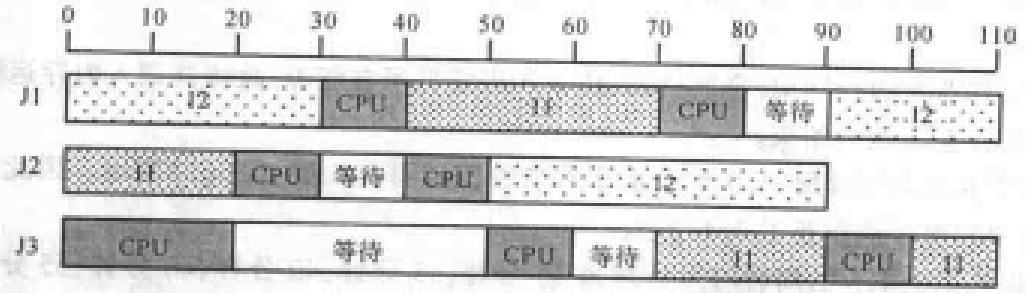


图 2.22

(1)J1、J2、J3 从投入到完成分别需要 110ms、90ms、110ms。

(2)从作业的投入到完成,CPU 的利用率为(20 + 10 × 6)/110,即 72.7%。

(3)设备 I1 的利用率为(20 + 30 + 20 + 10)/110,即 72.7%;I2 的利用率为(30 + 40 + 20)/110,即 81.8%。

25. 在多道程序系统中,供用户使用的内存空间有 100K,磁带机有 2 台,打印机 1 台。系统采用可变式分区分配方式管理内存,对磁带机和打印机采用静态分配方式,并假设输入/输出操作的时间忽略不计。现有一作业序列如表 2-25 所示。(东南大学 1996 年研究生试题)

表 2-25 作业情况表

作业号	到达时间	要求计算时间	要求内存量	申请磁带机数	申请打印机数
1	8:00	25 分钟	15K	1 台	1 台
2	8:20	10 分钟	30K	0 台	1 台
3	8:20	20 分钟	60K	1 台	0 台
4	8:30	20 分钟	20K	1 台	0 台
5	8:35	15 分钟	10K	1 台	1 台

假设作业调度采用 FCFS 算法，优先分配内存的低地址区域且不准移动已在内存中的作业，在内存中的作业平均分配 CPU 时间，试问：

(1)作业调度选中作业的次序是什么？

(2)如果把一个作业的周转时间定义为到达系统至计算完成的时间，则最大和最小的作业周转时间是多少？平均周转时间又是多少？

(3)作业全部执行结束的时间是多少？

第 1 种解答：本参考答案假设在作业调度时，不仅要考虑内存是否足够，而且还要考虑设备的申请情况，只有它们都得到满足时，才将作业调入内存，否则便不调入内存。

在 8:00，作业 1 到达，由于其申请的内存和设备均可得到满足，故被调入内存投入运行。

在 8:20，作业 1 还再需运行 5 分钟；作业 2 到达，但其申请的打印机资源无法得到满足，故作业 2 等待；作业 3 到达，其申请的资源均可得到满足，故将其投入运行。

在 8:30，与作业 3 平分 CPU 时间的作业 1 运行完毕，释放资源；作业 2 则无法得到内存资源，仍需等待；作业 3 还剩 15 分钟；作业 4 到达，其申请资源可得到满足，故将其投入运行。

在 8:35，作业 2 仍等待；作业 3 还剩 12.5 分钟；作业 4 还剩 17.5 分钟；作业 5 到达，其所需的磁带机资源无法得到满足，故作业 5 需等待。

在 9:00，作业 3 运行结束，释放资源；作业 4 还剩 5 分钟；作业 2 所申请的资源能得到满足，故将其调入内存运行；作业 5 由于无法得到打印机资源，仍需等待。

在 9:10，作业 2 还剩 5 分钟；作业 4 运行结束，释放资源；作业 5 由于无法得到打印机资源，仍需等待。

在 9:15，作业 2 运行结束，释放资源；作业 5 可获得所有资源，故将其调入内存运行。

在 9:30，作业 5 运行结束。

在上述作业的运行过程中，5 个作业及系统资源的情况如表 2-26(1)所示。因此：(1)作业调度选中的次序是作业 1,3,4,2,5。

(2)作业 1,2,3,4,5 的周转时间分别为 30 分钟、55 分钟、40 分钟、40 分钟、55 分钟。最大的作业周转时间是 55 分钟，最小的作业周转时间是 30 分钟。

(3)作业全部结束的时间是 9:30。

第 2 种解答：假设作业调度时，只要有足够内存便将作业调入内存，调入内存后再申请设备，并在设备满足不了时，进入阻塞状态。

在 8:00，作业 1 到达，由于其申请的内存和设备均可得到满足，故被调入内存并投入执行。

在 8:20，作业 1 还需再执行 5 分钟；作业 2 到达，其申请的内存能得到满足，故被调入内存，但由于打印机资源无法得到满足，故作业 2 阻塞；作业 3 到达，其申请的内存资源得不到满足，故仍需在后备队列中等待。

表 2-26(1)　作业和资源的情况表

时间	作业情况					内存分配情况（始址，大小，状态）	2 台磁带机	1 台打印机
	J1	J2	J3	J4	J5			
8:00 以前	未到	未到	未到	未到	未到	(0,100K,空闲)	空闲,空闲	空闲
8:00～8:20	运行	未到	未到	未到	未到	(0,15K,作业 1) (15K,85K,空闲)	作业 1,空闲	作业 1
8:20～8:30	运行	等待	运行	未到	未到	(0,15K,作业 1) (15K,60K,作业 3) (75k,25K,空闲)	作业 1,作业 3	作业 1
8:30～8:35	结束	等待	运行	运行	未到	(0.15K,空闲) (15K,60K,作业 3) (75K,20K,作业 4) (95K,5K,空闲)	作业 4,作业 3	空闲
8:35～9:00	结束	等待	运行	运行	等待	(0,15K,空闲) (15K,60K,作业 3) (75K,20K,作业 4) (95K,5K,空闲)	作业 4,作业 3	空闲
9:00～9:10	结束	运行	结束	运行	等待	(0,30K,作业 2) (30K,45K,空闲) (75K,20K,作业 4) (95K,5K,空闲)	作业 4,空闲	作业 2
9:10～9:15	结束	运行	结束	结束	等待	(0,30K,作业 2) (30K,70K,空闲)	空闲,空闲	作业 2
9:15～9:30	结束	结束	结束	结束	运行	(0,10K,作业 5) (10K,90K,空闲)	作业 5,空闲	作业 5
9:30	结束	结束	结束	结束	结束	(0,100K,空闲)	空闲,空闲	空闲

在 8:25,作业 1 运行完毕,释放资源;作业 2 被唤醒;作业 3 仍在后备队列中等待。

在 8:30,作业 2 还剩 5 分钟;作业 3 仍在后备队列中等待;作业 4 到达,其申请的内存和设备资源均可得到满足,故将其调入内存并投入执行。

在 8:35,作业 2 还剩 2.5 分钟;作业 3 仍在后备队列中等待;作业 4 还剩 17.5 分钟;作业 5 到达,其申请的内存能得到满足,故将其调入内存,但其所需的打印机资源无法得到满足,故作业 5 阻塞。

在 8:40,作业 2 运行结束,释放资源;作业 3 仍在后备队列中等待;作业 4 还剩 15 分钟;作业 5 被唤醒。

在 9:10,作业 2 和 5 均运行结束,释放资源;作业 3 被调入内存,并可得到所需的磁带机,故将其投入执行。

在 9:30,作业 3 运行结束,释放资源。至此,所有作业运行完毕。

在上述作业的运行过程中,5 个作业及系统资源的情况如表 2-26(2)所示。因此:

表 2-26(2) 作业和资源的情况表

时间	作业情况					内存分配情况 (始址,大小,状态)	2 台 磁带机	1 台 打印机
	J1	J2	J3	J4	J5			
8:00 以前	未到	未到	未到	未到	未到	(0,100K,空闲)	空闲,空闲	空闲
8:00~8:20	运行	未到	未到	未到	未到	(0,15K,作业 1) (15K,85K,空闲)	作业 1,空闲	作业 1
8:20~8:25	运行	阻塞	后备	未到	未到	(0,15K,作业 1) (15K,30K,作业 2) (45K,55K,空闲)	作业 1,空闲	作业 1
8:25~8:30	结束	运行	后备	未到	未到	(0,15K,空闲) (15K,30K,作业 2) (45K,55K,空闲)	空闲,空闲	作业 2
8:30~8:35	结束	运行	后备	运行	未到	(0,15K,空闲) (15K,30K,作业 2) (45K,20K,作业 4) (65K,35K,空闲)	作业 4,空闲	作业 2
8:35~8:40	结束	运行	后备	运行	阻塞	(0,10K,作业 5) (10K, 5K,空闲) (15K,30K,作业 2) (45K,20K,作业 4) (65K,35K,空闲)	作业 4,作业 5	作业 2
8:40~9:10	结束	结束	后备	运行	运行	(0,10K,作业 5) (10K, 35K,空闲) (45K,20K,作业 4) (65K,35K,空闲)	作业 4,作业 5	作业 5
9:10~9:30	结束	结束	运行	结束	结束	(0,60K,作业 3) (60K,40K,空闲)	作业 3,空闲	空闲
9:30	结束	结束	结束	结束	结束	(0,100K,空闲)	空闲,空闲	空闲

(1)作业的调度选中作业的次序为 1,2,4,5,3。

(2)作业 1,2,3,4,5 的周转时间分别为 25 分钟、20 分钟、70 分钟、40 分钟、35 分钟,故最大周转时间为 70 分钟,最小周转时间为 20 分钟。

(3)作业全部执行结束的时间为 9:30。

26. 假设某多道程序设计系统中有供用户使用的主存 100K,打印机 1 台。系统采用可变分区方式管理主存;对打印机采用静态分配,并假设输入输出操作的时间忽略不计;采用短作业优先的进程调度算法,进程执行时间相同时采用先来先服务算法;进程调度时机选择在执行进程结束时或有新进程到达时。现有一进程序列如表 2-27 所示:

表 2-27 作业情况表

进程号	进程到达时间	要求执行时间	要求主存量	申请打印机数
1	0	8	15K	1 台
2	4	4	30K	1 台
3	10	1	60K	0 台
4	11	20	20K	1 台
5	16	14	10K	1 台

假设系统优先分配主存的低地址区域，且不准移动已在主存中的进程，并考虑到执行进程的要求机时随时间递减的因素：

(1)给出进程调度算法选中进程的次序，并说明理由。

(2)全部进程执行结束所用的时间是多少？

答：(1)P1 到达的时刻 0，系统中只有一个就绪进程 P1，因此此时选中的进程为 P1。

在时刻 4，P1 执行 4 个时间片后，P2 到达，由于 P2 因申请打印机而阻塞，故系统中具备执行条件的仍只有 P1，因此执行进程仍然是 P1。

在时刻 8，P1 结束，P2 将得到 P1 释放的打印机，从阻塞变为就绪，且它是系统中惟一的进程，因此，P2 被选中执行。

在时刻 10，P2 执行 2 个时间片，P3 到达，但因申请内存而阻塞，因此，仍选中 P2 执行。

在时刻 11，P2 执行 3 个时间片，P3 阻塞，P4 到达，但 P4 因申请打印机而阻塞，因此，此时仍然是 P2 执行。

在时刻 12，P2 结束释放内存和打印机，P3 所申请的内存空间仍得不到满足，P4 则将得到打印机转为就绪状态，因此，选中 P4 执行。

在时刻 16，P3 阻塞，P4 执行 4 个时间片，P5 到达，但 P5 因申请打印机而阻塞，因此，仍选中 P4 执行。

在时刻 32，P4 结束，释放内存和打印机，P3 获得足够的内存转为就绪状态，P5 获得打印机转为就绪，但因 P3 要求执行的时间为 1，短于 P5 的执行时间 14，因此选中 P3 执行。

在时刻 33，P3 结束，选中 P5 执行，并在时刻 47，所有进程执行完毕。

从上可看出，选中进程的顺序为 P1，P2，P4，P3，P5。

(2)在时刻 47，所有的进程执行完毕。

27. 图 2.23 中将一组进程分为 4 类，各类进程之间采用优先级调度，而各类进程的内部采用时间片轮转调度。请简述 P1，P2，P3，P4，P5，P6，P7，P8 进程的调度过程。(西安电子科技大学 2001 年研究生试题)

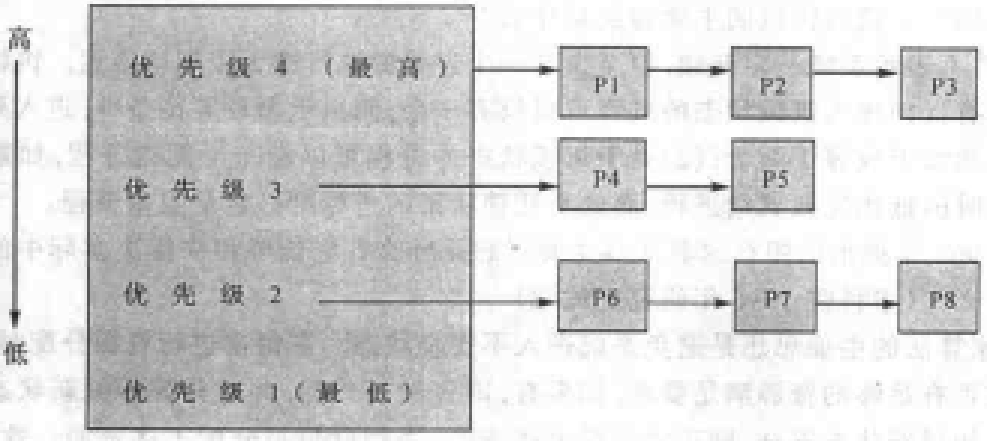


图 2.23　进程调度示意图

答：根据题意可知，该系统采用的是多级队列调度算法。系统首先对优先级 4 队列上的进程 P1，P2，P3 按时间片轮转算法进行调度；仅当优先级 4 队列为空(即 P1，P2，P3 中的任何进程或者运行完毕，或者处于阻塞状态)时，系统才对优先级 3 队列上的进程 P4，P5 按时间片轮转算法进行调度；同样仅当优先级 4 和优先级 3 队列均为空时，才对优先级 2 队列上的进程 P6，P7，P8 按时间片轮转算法进行调度；仅当上述三个队列均为空时，才对优先级 1

队列上的进程按时间片轮转算法进行调度。在上述过程中，当 CPU 在执行较低优先级队列上的进程时，若较高优先级队列上有进程从阻塞状态转变为就绪状态，则系统将在当前时间片用完后，重新回到较高优先级队列上进行进程调度。

2.7.4 死锁

一、问答题

1. 产生死锁的必要条件是什么？解决死锁问题常用哪几种措施？（中科院计算所 1997 年研究生试题）

答：产生死锁的必要条件如下

(1)互斥条件。进程所竞争的资源必须被互斥使用。

(2)请求与保持条件。当前已拥有资源的进程，仍能申请新的资源；而且，当该进程因新的资源被其他进程占用而阻塞时，它对自己已获得的资源仍保持不放。

(3)不剥夺条件。进程已获得的资源，只能在使用完时自行释放。

(4)环路等待条件。存在一个至少包含两个进程的循环等待链，链中的每个进程都正在等待下一个进程所占有的资源。

解决死锁问题常用的措施有如下几种：

(1)死锁的预防。通过事先破坏死锁产生的必要条件来防止死锁的发生。

(2)死锁的避免。在进程申请资源时，通过安全性检查以决定是否可以分配相应资源，避免系统进入不安全状态，防止死锁的发生。

(3)死锁的检测与解除。系统通过死锁检测程序来判断是否有进程已进入死锁状态，如果有，则通过撤销某些死锁进程或者剥夺某些进程的资源来解除死锁现象。

2. 要使一个系统不发生死锁，一般可采用哪些方法？简述它们的实现原理。（中国科技大学 1998 年研究生试题）

答：参考第 1 题。

3. 什么是饥饿？死锁与饥饿的主要差别是什么？

答：饥饿并不表示系统一定死锁，但至少有一个进程的执行被无限期地推迟。饥饿与死锁的主要差别有：(1)进入饥饿状态的进程可以只有一个，而由于循环等待条件，进入死锁状态的进程却必须大于或等于两个；(2)处于饥饿状态的进程可以是一个就绪进程，如静态优先权调度算法时的低优先权就绪进程，而处于死锁状态的进程则必定是阻塞进程。

4. Dijkstra 1965 年提出的银行家算法其主要思想是什么？它能够用来解决实际中的死锁问题吗？为什么？（中科院 1996 年研究生试题）

答：银行家算法的主要思想是避免系统进入不安全状态。在每次进行资源分配时，它首先检查系统是否有足够的资源满足要求，如果有，则先试行分配，并对分配后的新状态进行安全性检查。如果新状态安全，则正式分配上述资源，否则就拒绝分配上述资源。这样，它保证系统始终处于安全状态，从而避免死锁现象的发生。

在银行家算法中，是根据每个进程对资源的最大需求来进行安全性检查的；另外，在整个过程中，进程的数目固定不变。由于实际应用中，在每次进程运行之前，系统难以了解其对资源的最大需求；而在整个过程中，可能不断地有进程终止、或有新进程被创建，所以进程的数目是动态变化的。因此，用银行家算法来解决实际中的死锁问题存在一定的困难。

5. 设系统中有 3 种类型的资源(A, B, C)和 5 个进程 P1, P2, P3, P4, P5, A 资源的数量为 17, B

资源的数量为 5，C 资源的数量为 20。在 T0 时刻系统状态如表 2-28 所示。系统采用银行家算法实现死锁避免策略。（北京大学 1997 年研究生试题）

表 2-28　T0 时刻的系统状态

	最大资源需求量	已分配资源数量	剩余资源数
	A B C	A B C	A B C
P1	5 5 9	2 1 2	2 3 3
P2	5 3 6	4 0 2	
P3	4 0 11	4 0 5	
P4	4 2 5	2 0 4	
P5	4 2 4	3 1 4	

(1)T0 时刻是否为安全状态？若是，请给出安全序列。

(2)在 T0 时刻若进程 P2 请求资源(0,3,4)，是否能实施资源分配，为什么？

(3)在(2)的基础上，若进程 P4 请求资源(2,0,1)，是否能实施资源分配？为什么？

(4)在(3)的基础上，若进程 P1 请求资源(0,2,0)，是否能实施资源分配？为什么？

答：在 T0 时刻，各进程对资源的需求量 Need 可从最大资源需求量 Max 和已分配资源数量 Allocation 获得：Need = Max-Allocation。表 2-29 中增加了各进程对资源的需求量。

表 2-29　T0 时刻的系统状态

	最大资源需求量	已分配资源数量	资源需求量	剩余资源数
	A B C	A B C	A B C	A B C
P1	5 5 9	2 1 2	3 4 7	2 3 3
P2	5 3 6	4 0 2	1 3 4	
P3	4 0 11	4 0 5	0 0 6	
P4	4 2 5	2 0 4	2 2 1	
P5	4 2 4	3 1 4	1 1 0	

(1)对 T0 时刻系统状态的安全性检查如表 2-30 所示，由于此时能找到安全序列＜P4, P5, P1, P2, P3＞，故系统处于安全状态。

表 2-30　安全性检查过程

资源情况		Work A B C	Need A B C	Allocation A B C	Work + Allocation A B C	Finish
进程	P4	2 3 3	2 2 1	2 0 4	4 3 7	True
	P5	4 3 7	1 1 0	3 1 4	7 4 11	True
	P1	7 4 11	3 4 7	2 1 2	9 5 13	True
	P2	9 5 13	1 3 4	4 0 2	13 5 15	True
	P3	13 5 15	0 0 6	4 0 5	17 5 20	True

(2)由于 P2 的请求 Request2(0,3,4)不满足小于等于剩余的可用资源 Available(2,3,3)的条件，故当 P2 提出上述请求时，不能实施资源分配。

(3)由于 P4 的请求 Request4(2,0,1)满足下述两个条件：

Request4(2,0,1)≤Available(2,3,3)

Request4(2,0,1)≤Need4(2,2,1)

故可为 P4 试行分配资源，试行分配后，系统的状态如表 2-31 所示。

表 2-31 为 P4 试行分配资源后的系统状态

	最大资源需求量	已分配资源数量	资源需求量	剩余资源数
	A B C	A B C	A B C	A B C
P1	5 5 9	2 1 2	3 4 7	0 3 2
P2	5 3 6	4 0 2	1 3 4	
P3	4 0 11	4 0 5	0 0 6	
P4	4 2 5	4 0 5	0 2 0	
P5	4 2 4	3 1 4	1 1 0	

对上述状态的安全性检查过程如表 2-32 所示，可见，此时能找到一安全序列＜P4, P5, P1, P2, P3＞，系统的状态安全，故可实施上述资源分配。

表 2-32 安全性检查过程

资源情况 / 进程	Work	Need	Allocation	Work + Allocation	Finish
	A B C	A B C	A B C	A B C	
P4	0 3 2	0 2 0	4 0 5	4 3 7	True
P5	4 3 7	1 1 0	3 1 4	7 4 11	True
P1	7 4 11	3 4 7	2 1 2	9 5 13	True
P2	9 5 13	1 3 4	4 0 2	13 5 15	True
P3	13 5 15	0 0 6	4 0 5	17 5 20	True

(4)由于 P1 的请求 Request1(0,2,0)满足下述两个条件：

Request1(0,2,0)≤Available(0,3,2)

Request1(0,2,0)≤Need1(3,4,7)

故可为 P1 试行分配资源，试行分配后，系统的状态如表 2-33 所示。

表 2-33 为 P1 试行分配资源后的系统状态

	最大资源需求量	已分配资源数量	资源需求量	剩余资源数
	A B C	A B C	A B C	A B C
P1	5 5 9	2 3 2	3 2 7	0 1 2
P2	5 3 6	4 0 2	1 3 4	
P3	4 0 11	4 0 5	0 0 6	
P4	4 2 5	4 0 5	0 2 0	
P5	4 2 4	3 1 4	1 1 0	

对上述状态进行安全性检查，由于此时的 work = Available(0,1,2)已不能满足任何进程的资源需求 Needi(1≤i≤5)，系统状态不安全，故不能实施上述资源分配。

6. 试描述避免死锁的银行家算法。(浙江大学 1997 年研究生试题)

若系统运行中出现如表 2-34 所示的资源分配情况，该系统是否安全？如果进程 P2 此时提出资源申请(1,2,2,2)，系统能否将资源分配给它？为什么？

表 2-34　资源分配情况表

进程	ALLOCATION				NEED				AVAILABLE			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	3	2	0	0	1	2	1	6	2	2
P1	1	0	0	0	1	1	5	0				
P2	1	3	5	4	2	2	5	6				
P3	0	3	3	2	0	0	5	2				
P4	0	0	1	4	0	0	5	6				

答：由于此时能找到安全序列<P0, P1, P3, P2, P4>，故系统处于安全状态。

如果进程 P2 此时提出资源申请(1, 2, 2, 2)，预分配后的资源分配情况如表 2-35 所示：

表 2-35　资源分配情况表

进程	ALLOCATION				NEED				AVAILABLE			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	3	2	0	0	1	2	0	4	0	0
P1	1	0	0	0	1	1	5	0				
P2	2	5	7	6	1	0	3	4				
P3	0	3	3	2	0	0	5	2				
P4	0	0	1	4	0	0	5	6				

不能满足任何进程的资源需求，系统状态不安全，故不能实施上述资源分配。

7. 某系统有 R1, R2 和 R3 共 3 种资源，在 T0 时刻 P1, P2, P3 和 P4 这 4 个进程对资源的占用和需求情况见表 2-36，此时系统的可用资源向量为(2, 1, 2)，问题：(中国科学院软件研究所 1999 年研究生试题)

表 2-36　资源分配情况表

	最大资源需求量			已分配资源数量		
	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0
P2	6	1	3	4	1	1
P3	3	1	4	2	1	1
P4	4	2	2	0	0	2

(1)将系统中各种资源总数，以及此刻各进程对各资源的需求数目用向量或矩阵表示出来；

(2)如果此时 P1 和 P2 均发出资源请求向量 Request(1, 0, 1)，为了保证系统的安全性，应该如何分配资源给这两个进程？说明你所采用策略的原因。

(3)如果(2)中两个请求立即得到满足后，系统此刻是否处于死锁状态？

答：(1)系统中各种资源总数为系统中可用资源数量 Available 加已分配给各个进程的资源数量，即：

(2 1 2) + (1 0 0) + (4 1 1) + (2 1 1) + (0 0 2) = (9 3 6)

各进程对各资源的需求数目 Need 为最大资源需求量 Max 与已分配资源数量 Allocation 之差，即：

$$\begin{bmatrix}3&2&2\\6&1&3\\3&1&4\\4&2&2\end{bmatrix}-\begin{bmatrix}1&0&0\\4&1&1\\2&1&1\\0&0&2\end{bmatrix}=\begin{bmatrix}2&2&2\\2&0&2\\1&0&3\\4&2&0\end{bmatrix}$$

(2)如果 P1 发出资源请求 Request1(1,0,1),由于它满足下列两个条件:

Request1(1,0,1)≤Available(2,1,2)

Request1(1,0,1)≤Need1(2,2,2)

故可为 P1 试行分配资源,分配后的系统状态如表 2-37 所示:

表 2-37 试行分配后的系统状态

	最大资源需求量			已分配资源数量			资源需求量			剩余资源数		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	2	0	1	1	2	1	1	1	1
P2	6	1	3	4	1	1	2	0	2			
P3	3	1	4	2	1	1	1	0	3			
P4	4	2	2	0	0	2	4	2	0			

对上述状态进行安全性检查,由于此时的 work=Available(1,1,1)已不能满足任何进程的资源需求 Needi(1≤i≤4),系统状态不安全,故不能实施上述资源分配,而应恢复到原来表 2-36 所示的状态。

如果 P2 发出资源请求 Request2(1,0,1),由于它满足下列两个条件:

Request2(1,0,1)≤Available(2,1,2)

Request2(1,0,1)≤Need2(2,0,2)

故可为 P2 试行分配资源,分配后的系统状态如表 2-38 所示:

表 2-38 试行分配后的系统状态

	最大资源需求量			已分配资源数量			资源需求量			剩余资源数		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	2	2	2	1	1	1
P2	6	1	3	5	1	2	1	0	1			
P3	3	1	4	2	1	1	1	0	3			
P4	4	2	2	0	0	2	4	2	0			

对上述状态的安全性检查过程如表 2-39 所示,因为,此时能找到一个安全序列<P2, P3, P4, P1>,系统状态安全,故能进行资源的分配。

表 2-39 安全性检查过程

资源情况 / 进程	Work			Need			Allocation			Work + Allocation			Finish
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3	
P2	1	1	1	1	0	1	5	1	2	6	2	3	True
P3	6	2	3	1	0	3	2	1	1	8	3	4	True
P4	8	3	4	4	2	0	0	0	2	8	3	6	True
P1	8	3	6	2	2	2	1	0	0	9	3	6	True

因此，当 P1 和 P2 均发出资源请求(1,0,1)时，应先为 P2 分配资源，而让 P1 等待。

(3)如果立即满足(2)中的两个请求，则系统中剩余的资源数量还有(0,1,0)，而各个进程仍处于就绪或执行状态，故系统并没立即进入死锁状态。只有当上述进程全部因申请资源而阻塞时，系统才进入死锁状态。

8. 一系统具有 150 个存储单元，在 T0 时刻系统按表 2-40 所示分配给 3 个进程。(中科院计算所 1999 年研究生试题)

表 2-40　T0 时刻系统资源分配状态表

进程	Maximum demand	Current allocation
P1	70	25
P2	60	40
P3	60	45

对下列请求应用银行家算法分别分析判定是否安全?

(1)第 4 个进程 P4 到达，最大需求 60 个存储单元，当前请求分配 25 个单元。

(2)第 4 个进程 P4 到达，最大需求 50 个存储单元，当前请求分配 35 个单元。

如果是安全的，请给出一个可能的进程安全执行序列；如果不是安全的，请说明原因。

答：根据题意，在 T0 时刻，系统剩余的存储单元 Available 为(150-25-40-45)，即 40。

(1)P4 的当前请求量 request4 为 25，分别小于它的最大需求量 60 和 Available(40)。故可为它试行分配资源，分配后的新状态如表 2-41 所示。

表 2-41　试行分配后的系统状态

进程	Need	Alocation	Available
P1	45	25	15
P2	20	40	
P3	15	45	
P4	35	25	

对上述状态进行安全性检查(见表 2-42)，此时，能找到一个安全序列<P3, P4, P1, P2>，系统处于安全状态，故可为 P4 分配资源。

表 2-42　安全性检查过程

资源情况 / 进程	Work	Need	Allocation	Work + Allocation	Finish
P3	15	15	45	60	True
P4	60	35	25	85	True
P1	85	45	25	110	True
P2	110	20	40	150	True

(2)P4 的当前请求量 request4 为 35，分别小于它的最大需求量 50 和 Available(40)。故可为它试行分配资源，分配后的新状态如表 2-43 所示。

表 2-43 试行分配后的系统状态

进程	Need	Alocation	Available
P1	45	25	5
P2	20	40	
P3	15	45	
P4	15	35	

对上述状态进行安全性检查，由于此时系统的剩余资源数量 Available(5)已无法满足任一进程对资源的需求量 $Need_i(i=1,2,3,4)$，故新状态不安全，系统不能为 P4 分配上述资源。

9. 设有三个进程 P、Q、R，它们共享 10 个同类资源，每个进程使用资源的最大需求依次为 4，7，8，现假定它们对资源请求的序列如表 2-44 所示。为使系统不发生死锁，当执行到序号为 6 时，进程 P，Q，R 各处于何种状态？获得多少同类资源？

表 2-44 资源请求序列

序号	进程	申请资源数
1	P	2
2	Q	4
3	R	2
4	Q	2
5	R	2
6	P	2
⋮	⋮	⋮

答：根据银行家算法，当执行到序号 3 时，系统状态如表 2-45 所示，此时能找到一安全序列(P，Q，R)，故系统仍处于安全状态。

表 2-45 序号 3 时的资源分配情况

进程	Max	Need	Allocation	Available
P	4	2	2	2
Q	7	3	4	
R	8	6	2	

当执行序号 4 时，可为 Q 试行分配资源，在试行分配后，Available = 0 无法满足任一进程对资源的需求量(此时，$Need_P=2$，$Need_Q=1$，$Need_R=2$)，故应拒绝为 Q 分配所需资源，Q 进入阻塞状态。

执行序号 5 时，可为 R 试行分配资源，在试行分配后，Available = 0 无法满足任一进程对资源的需求量(此时，$Need_P=2$，$Need_Q=3$，$Need_R=4$)，故应拒绝为 R 分配所需资源，R 进入阻塞状态。

执行序号 6 时，可为 P 试行分配资源，在试行分配后，系统状态如表 2-46 所示。此时能找到一个安全序列(P，Q，R)，即系统处于安全状态，故能为 P 分配所需资源。因此，执行到序号 6 时，进程 P、Q、R 分别处于就绪(或执行)、阻塞、阻塞状态，它们获得的资源数量分别为 4，4，2。

表 2-46　序号 6 的资源分配情况

进程	Max	Need	Allocation	Available
P	4	0	4	0
Q	7	3	4	
	R	8	6	2

10. 设系统只有一种资源，进程一次只能申请一个资源。进程申请的资源总数不会超过系统的资源总数。下列情况中哪些会发生死锁？

	进程数	资源总数
(a)	1	1
(b)	1	2
(c)	2	1
(d)	2	2
(e)	2	3

现在假设进程最多需要两个资源，下列情况中哪些会发生死锁？

	进程数	资源总数
(f)	1	2
(g)	2	2
(h)	2	3
(i)	3	3
(j)	3	4

答：(a)、(b)、(f)均不会发生死锁，因为进程的数目小于 2，死锁的必要条件“环路等待”不可能成立。

(c)也不会发生死锁，因为此时每个进程的最大需求不会超过资源总量 1，因此，无论两个进程中的哪一个先得到该资源，它都能运行完毕，并将资源归还给系统，使另一个进程也得到顺利运行。

(d)可能发生死锁。假设，进程所竞争的资源是必须互斥共享并不可剥夺的资源。当两个进程对资源的最大需求量都为 2，并各自均已得到 1 个资源时，这两个进程都将会因申请第 2 个资源的请求无法得到满足而阻塞，那时，它们便进入死锁状态。

(e)可能发生死锁。假设，进程所竞争的资源是必须互斥共享并不可剥夺的资源，而两个进程对资源的最大需求量均为 3。那么，当其中的一个进程得到 1 个资源，而另一个进程得到 2 个资源时，这两个进程均需进一步申请资源才能继续运行，并会因为得不到满足而阻塞，那时，它们将进入死锁状态。

(g)可能发生死锁。具体原因同(d)。

(h)不会发生死锁。因为系统中只有 2 个进程，每个进程最多需要 2 个资源，而系统中总共有 3 个资源，因此必定有一个进程可成功申请到 2 个资源，待它运行完毕归还资源后，另一个进程也可顺利运行并完成。

(i)可能发生死锁。如果 3 个进程各自申请到 1 个资源，那么它们都需再申请一个资源才能顺利运行，当它们提出这些请求时，都将会因为系统无空闲资源而阻塞，从而进入死锁状态。

(j)不会发生死锁。因为系统中只有3个进程,每个进程最多需要2个资源,而系统中总共有4个资源,因此必定有一个进程可成功申请到2个资源,待它运行完毕归还资源后,另两个进程也可顺利运行并完成。

11. 一个操作系统有20个进程,竞争使用65个同类资源,申请方式是逐个进行的,一旦某进程获得它所需要的全部资源,则立即归还所有资源。每个进程最多使用3个资源。若仅考虑这类资源,该系统有无可能产生死锁,为什么?(北京大学1995年研究生试题)

答:若仅考虑这类资源,该系统不可能产生死锁。因为,系统中有足够多的资源,完全能满足各个进程的需要。即使全部20个进程均得到3个资源,系统中仍有5个资源空闲,因此,各个进程在任何时刻都不会因为竞争该类资源而阻塞,更不可能进入死锁状态。

12. 一台计算机有8台磁带机。它们由N个进程竞争使用,每个进程可能需要3台磁带机。请问N为多少时,系统没有死锁危险,并说明原因。(上海交通大学1999年研究生试题)

答:当N为1、2、3时,系统没有死锁的危险。因为1个进程不可能发生死锁;2个进程时,最多需要6台磁带机,因此系统的资源已经足够满足进程的需要;而在3个进程时,由于每个进程最多需要3台磁带机,而系统总共有8台磁带机,因此,必定有2个进程可顺利申请到3台磁带机而不被阻塞,待它们运行完毕释放资源后,另一进程也可以顺利完成。而当N为4时,若每个进程均得到2台磁带机,系统便可能发生死锁。

13. 设系统仅有一类数量为M的独占型资源,系统中N个进程竞争该类资源,其中各进程对该类资源的最大需求为W。当M,N,W分别取下列值时,试判断下列哪些情形会发生死锁?为什么?(上海交通大学1999年研究生试题)

(1)M=2;N=2,W=2;

(2)M=3;N=2,W=2;

(3)M=3;N=2,W=3;

(4)M=5;N=3,W=2;

(5)M=6;N=3,W=3;

答:(1)可能发生死锁。当两个进程各自均已得到1个资源时,这两个进程都将会因申请第2个资源的请求无法得到满足而阻塞,那时,它们便进入死锁状态。

(2)不会发生死锁。因为系统中只有2个进程,每个进程最多需要2个资源,而系统中总共有3个资源,因此必定有一个进程可成功申请到2个资源,待它运行完毕归还资源后,另一个进程也可顺利运行并完成。

(3)可能发生死锁。当其中的一个进程得到1个资源,而另一个进程得到2个资源时,这两个进程均需进一步申请资源才能继续运行,并会因为得不到满足而阻塞,那时,它们将进入死锁状态。

(4)不会发生死锁,因为总共有3个进程,每个进程最大需求为2,而资源总量为5,故必定有一个进程可顺利申请到2个资源,在它完成并释放资源后,其他进程也可顺利完成。

(5)可能发生死锁。当每个进程各自获得2个资源后,该类资源便已被全部分配出去,而每个进程的运行都需要进一步申请资源,并将因系统无空闲资源而阻塞,从而使系统进入死锁状态。

14. 设系统由相同类型的m个资源组成,系统有n个进程,每个进程至少请求一个资源。试分析当m>n和m<=n时每个进程可以请求多少个这类资源,该系统仍无死锁。

答：如果 n>1，那么当 m>n 时，每个进程最多可以请求$\left[\frac{m+n-1}{n}\right]$个这类资源，而该系统仍无死锁。因为，此时，如果将系统资源全部分配出去（即$\sum_{i=1}^{n}Allocation_i=m$），则肯定有一个进程可获得个资源$\left[\frac{m+n-1}{n}\right]$。否则，若：

$$Allocation_i\leqslant\left[\frac{m+n-1}{n}\right]-1\ (\text{for all i, } i=1,2,\cdots,n)$$

则可推出：

$$\sum_{i=1}^{n}Allocation_i\leqslant n\times\left(\left[\frac{m+n-1}{n}\right]-1\right)\leqslant m-1$$

这与资源已全部分配完毕（即$\sum_{i=1}^{n}Allocation_i=m$）的条件不符。因此，肯定有一个进程其 $Allocation_i\geqslant\left[\frac{m+n-1}{n}\right]$，故该进程可以顺利完成，待它释放资源后，其他进程也可获得相应资源，并顺利结束。

而如果允许每个进程最多可以请求$\left[\frac{m+n-1}{n}\right]+1$个这类资源，那么当 m 个资源被 n 个进程平分时，每个进程获得的资源数肯定达不到最大需求$\left[\frac{m+n-1}{n}\right]+1$（若有一个进程的 $Allocation_i=\left[\frac{m+n-1}{n}\right]+1$，则其余 n－1 个进程必定满足：$Allocation_i=\left[\frac{m+n-1}{n}\right]$，此时$\sum_{i=1}^{n}Allocation_i=n\times\left[\frac{m+n-1}{n}\right]\geqslant m+1$，与系统只有 m 个资源的条件不符），此时每个进程都必须申请更多个资源才能继续运行，并将因为系统已无空闲资源而阻塞，从而使整个系统进入死锁状态。

如果 n>1，那么当 m≤n 时，要使系统不进入死锁状态，每个进程最多只可申请 1 个该种资源。因为，此时，至少有 m 个进程的资源请求可得到满足，它们完成后释放资源给其他进程使用，使它们也可顺利完成。如果允许每个进程最多申请 2 个资源，当 m 个资源分配给其中的 m 个进程时（每个进程 1 个资源），所有的进程都需申请更多的资源才能继续运行，故这些进程都将因系统已无空闲资源而阻塞，从而使系统进入死锁状态。

15. 考虑由 n 个进程共享的具有 m 个同类资源的系统，证明：如果对 i＝1，2，…，n，有 need>0 而且所有最大需求量之和小于 m＋n，那么该系统是死锁无关的。（西北工业大学 2000 年研究生试题）

证明：对 n 个进程中的任何 k(1<k≤n)个进程，它们的最大需求之和等于 n 个进程的最大需求量之和与其余 n－k 个进程的最大需求量之和两者之差，由于每个进程的最大需求必定大于等于 1（因为其 need 大于 0），故其余 n－k 个进程的最大需求量之和必定大于等于 n－k。当 n 个进程的最大需求量之和小于 m＋n 时，上述 k 个进程的最大需求量之和必定小于(m＋n)－(n－k)，即 m＋k。也就是说，当 n 个进程的最大需求量之和小于 m＋n 时，其中任何 k(1<k≤n)个进程，它们的最大需求量之和都将小于 m＋k。

对 n 个进程中的任何 k(1<k≤n)个进程，假设它们分别为为 Pj1、…、Pjk，如果它们因竞争上述 m 个资源而发生死锁，则必有：

$$\sum_{i=j1}^{jk}Allocation_i=m\text{（即系统已无空闲的资源）}\quad\text{以及}$$

$Need_i>0$　(for all i, i＝j1，…，jk)　（每个进程都在等待得到更多的资源）

因此可得到：$\sum_{i=j1}^{jk} Need_i \geqslant k$

从而可进一步得到：$\sum_{i=1}^{n} Max_i = \sum_{i=1}^{n} Need_i + \sum_{i=1}^{n} Allocation_i \geqslant k + m$

这个结论与"任何 k(1<k≤n)个进程，它们的最大需求量之和都将小于 m+k"相矛盾。因此，当系统的资源已全部分配出去时，k 个进程中必定存在一个进程，其 $Need_i = 0$，也就是说，它已获得了全部所需资源，能够顺利完成，故系统中不可能发生 k 个进程的死锁现象。由于，1<k≤n，因此，系统中的 n 个进程不可能因为竞争 m 个同类资源而发生任何死锁现象。

16. 某系统有同类型资源 m 个，供 n 个进程共享。如果每个进程最多申请 x 个资源(其中 1≤x≤m)。请证明：当 n×(x-1)+1≤m 时，系统不会发生死锁。(西安理工大学 2000 年研究生试题)

证明：由 $1 \geqslant x \leqslant m$，可得：$x-1 \geqslant 0$

当 1<k≤n 时，显然：$k \times (x-1)+1 \leqslant n \times (x-1)+1$

而由于 $n \times (x-1)+1 \leqslant m$，故 $k \times (x-1)+1 \leqslant m$

即当 n 个进程满足条件 $n \times (x-1)+1 \leqslant m$ 时，n 个进程中的任何 k(1<k≤n)个进程，均满足条件：

$$k \times (x-1)+1 \leqslant m$$

如果 n 个进程中有 k(1<k≤n)个进程(假设它们分别为 P_{j1}、…、P_{jk})因竞争 m 个同类资源而进入死锁状态，则此时必有：

$\sum_{i=j1}^{jk} Allocation_i = m$(即系统已无空闲的资源) 以及

$Allocation_i \leqslant x-1$ (for all i, i=j1, …, jk) (每个进程都没得到足够的资源)

故可得到：

$$m = \sum_{i=jk}^{jk} Allocation_i \geqslant k \times (x-1) < k \times (x-1)+1$$

这显然与条件 $k \times (x-1)+1 \leqslant m$ 不符。因此，当系统资源全部分配出去(即 $\sum_{i=j1}^{jk} Allocation_i = m$)时，k 个进程中必定有一个进程的 $Allocation_i > x-1$，由于它已获得所需的全部资源，它必定可以顺利完成，从而不可能发生 k 个进程的死锁现象。由于 1<k≤n，故系统中的 n 个进程不会发生任何死锁现象。

17. 按序分配是防止死锁的一种策略。什么是按序分配？为什么按序分配可以防止死锁？(南京大学 2000 年研究生试题)

答：所谓按序分配是指，系统将所有的资源按类型进行排序，并给不同的类型赋予不同的序号，而所有的进程对资源的请求，必须严格按资源序号递增的顺序(或严格按资源序号递减的顺序)提出。

在采用按序分配方式时，如果系统要求进程严格按资源序号递增的顺序来申请资源，则只可能存在拥有较低序号资源的进程等待拥有较高序号资源的进程释放资源的现象，而不会存在相反的等待，因此，死锁产生的必要条件之一——"环路等待"条件不可能成立，从而达到防止死锁发生的目的。

18. 试列出两种破坏"循环等待"条件的方法以防止死锁的发生。

答：如果对系统资源已经按类型进行排序，则可通过下述两种办法来破坏"循环等待"条件：

(1)进程必须严格按资源序号递增的顺序申请资源;

(2)当进程申请某个资源时,它必须释放所有较低序号的资源。

19. 考虑下列资源分配策略:对资源的申请和释放可以在任何时刻进行。如果一进程提出的资源请求得不到满足,若此时无由于等待资源而被阻塞的进程,则自己就被阻塞;若此时已有等待资源而被阻塞的进程,则检查所有由于等待资源而被阻塞的进程。如果它们有申请进程所需要的资源,则将这些资源取出分配给申请进程。

例如:考虑一个有3类资源的系统,系统所有可用资源为(4,2,2)。进程A申请(2,2,1),可满足;进程B请求(1,0,1),可以满足;若A再请求(0,0,1),则被阻塞。

此时,若C请求(2,0,0),它可以分到剩余资源(1,0,0),并从A已分到的资源中获得一个资源,于是进程A的分配向量变成(1,2,1),而需求向量变成(1,0,1)。

(1)这种分配方式会导致死锁吗?如果会,请举一个例子;如果不会,请说明产生死锁的哪一个必要条件不成立?

(2)这种分配方式会导致某些进程的无限等待吗?为什么?

答:(略)

20. 假定某计算机系统有R1和R2两类可再使用资源(其中R1有2个单位,R2有1个单位),它们被进程P1和P2所共享,且已知两个进程均以下列顺序使用两类资源:(南开大学1994年研究生试题)

→申请R1→申请R2→申请R1→释放R1→释放R2→释放R1→

试求出系统运行过程中可能到达的死锁点,并画出死锁点的资源分配图(或称进程—资源图)。

答:当两个进程各申请到一个R1资源后,系统便进入不安全状态。此时,无论谁先获得R2资源,它都将因申请另一个R1资源而阻塞,并等待另一个进程释放R1资源。而另一个进程则将等待前一进程释放R2资源。因此进入死锁状态。

死锁点的资源分配图如图2.24所示。

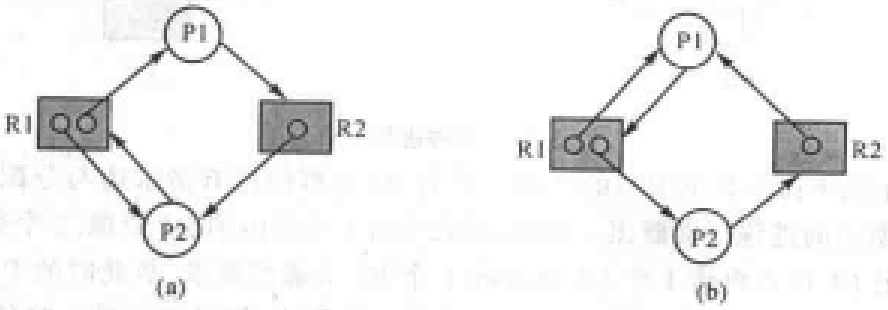


图2.24 死锁点的两种资源分配图

21. 假定某计算机系统有R1设备3台,R2设备4台,它们被P1、P2、P3和P4这4个进程所共享,且已知这4个进程均以下面所示的顺序使用现有设备。(南京大学1995年研究生试题)

→申请R1→申请R2→申请R1→释放R1→释放R2→释放R1→

(1)说明系统运行过程中是否有产生死锁的可能?为什么?

(2)如果有可能的话,请举出一种情况,并画出表示该死锁状态的进程—资源图。

答:(1)系统运行过程中有可能产生死锁。根据题意,系统中只有3台R1设备,它们要被4个进程共享,且每个进程对R1设备的最大需求为2。由于,R1设备数量不足,而且它又是

一个互斥的、不可被剥夺的资源，而系统又没采取任何措施破坏死锁产生的剩余两个必要条件——"请求与保持"条件和"环路等待"条件，因此，在系统运行过程中，可能会发生死锁。

(2)当 P1，P2，P3 进程各得到一个 R1 设备时，它们可继续运行，并均可顺利地申请到一个 R2 设备，当它们第二次申请 R1 设备时，因为系统已无空闲的 R1 设备，故他们全部阻塞，并进入循环等待的死锁状态。这种死锁状态下的进程—资源图如图 2.25 所示。

22. 试化简图 2.26 中的进程—资源图，并利用死锁定理给出相应的结论。

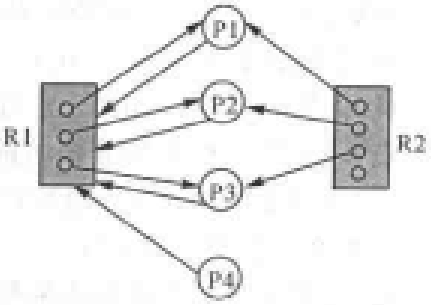


图 2.25 进程—资源图

答：在图(a)中，资源 R1 共 2 个，已分别分配给 P1 进程和 P2 进程；资源 R2 共 3 个，其中 2 个已分别分配给 P1 进程和 P2 进程，还有一个空闲。P1 进程目前已占有 1 个 R1 资源和 1 个 R2 资源，并要申请 1 个 R1 资源和 1 个 R2 资源，由于系统已无 R1 资源，故 P1 进程是一个阻塞结点；P2 进程目前已占有 1 个 R1 资源和 1 个 R2 资源，并要申请 1 个 R2 资源，而该请求能得到满足，故 P2

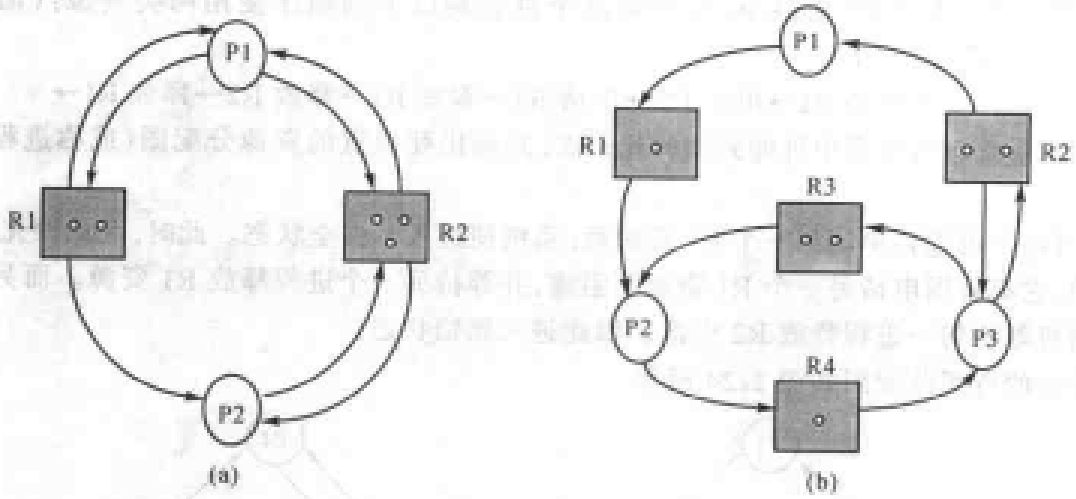


图 2.26 两种进程—资源图

进程是一个既不孤立又不阻塞的结点。消去 P2 进程的所有请求边与分配边，便形成了如图 2.4(a)所示的进程—资源图。此时，系统中有 1 个空闲的 R1 资源，2 个空闲的 R2 资源，因此能满足 P1 进程申请 1 个 R1 资源和 1 个 R2 资源的要求，故此时的 P1 进程也是一个既不孤立又不阻塞的结点，消去其所有请求边与分配边，便形成了图 2.26(b)的进程—资源图。图中所有的进程结点均为孤立结点，即上述进程—资源图是可完全化简的，根据死锁定理，当时的系统没有进入死锁状态。

图 2.6(b)所示的进程—资源图表示，系统有 1 个 R1 资源，并已分配给 P2；有 2 个 R2 资源，并已分别分配给 P1 和 P3；有 2 个 R3 资源，其中 1 个已分配给 P2，1 个仍空闲；1 个 R4 资源，已分配给 P3。进程 P1 还需要 1 个 R1 资源，得不到满足，为阻塞结点；进程 P2 还需要 1 个 R4 资源，也得不到满足，为阻塞结点；进程 P3 需要 1 个 R2 资源和 1 个 R3 资源，因系统已无空闲的 R2 资源，故它也为一个阻塞结点。因此，无法进一步化简该进程—资源图，根据死锁定理，系统处于死锁状态。

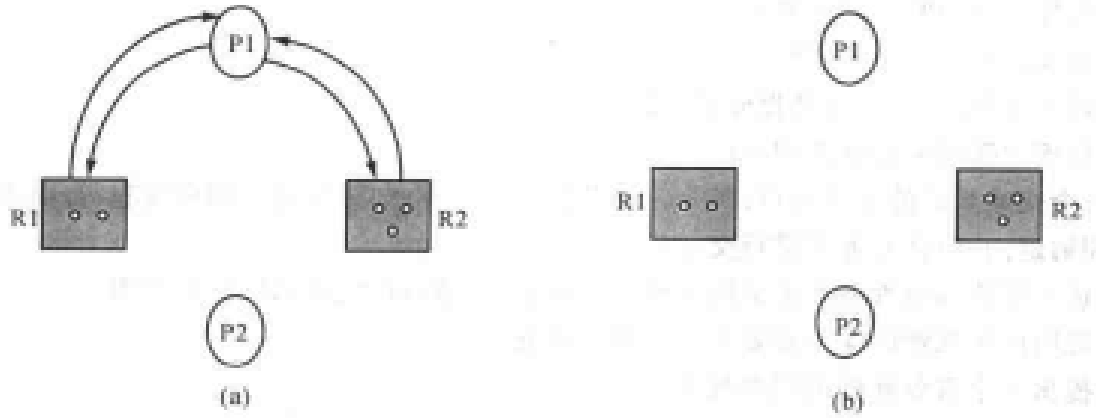


图 2.27 进程—资源图的化简过程

23. 写出两种检测进程死锁的方法及相应的结论(或定理)。(南开大学 1998 研究生试题)

答:(1)利用化简资源分配图的方法进行死锁检测。化简的具体方法如下:

① 在资源分配图中,找一个既不孤立又非独立的进程结点 Pi。在顺利的情况下,Pi 可获得所需资源而继续运行,直至运行完毕,再释放其全部资源,这相当于消去 Pi 的所有请求边和分配边,使之成为孤立结点。

② 不断地重复步骤①进一步简化资源分配图,直至找不到既不孤立又非独立的进程结点为止。此时,如果所有的进程结点都已成为孤立结点,则称上述资源分配图是可完全化简的;否则,则称上述资源分配图是不可完全化简的。

相应的定理是:S 为死锁状态的充分必要条件是 S 状态的资源分配图是不可完全化简的。该定理也称作死锁定理。

(2)利用死锁检测算法进行死锁检测。假设系统中有 n 个进程和 m 类资源,矩阵 Allocation$_{n\times m}$中的第 i 行、第 j 列的元素 a_{ij}表示进程 i 已获得第 j 类资源的数目。Available 是一个 m 个元素的行向量,其中的元素 ai 表示系统中第 i 类资源的可用数目。矩阵 $Q_{n\times m}$是一个请求矩阵,其中 q_{ij}表示进程 i 请求第 j 类资源的数量。死锁检测算法是通过标记没有死锁的进程而进行的。最初,所有的进程都是未标记的,然后执行下列步骤:

① 标记 Allocation 矩阵中一行全为 0 的进程;

② 初始化一个临时向量 W,令 W 等于 Available 向量;

③ 查找满足下列条件的下标 i:进程 i 当前未标记,且 Q 的第 i 行小于等于 W。如果找不到这样的行,则终止算法;

④ 如果找到这样的行,则表示在顺利的情况下,进程 i 可获得所需资源而继续运行,直至运行完毕,再释放其全部资源。因此可标记进程 i 并把分配矩阵中的相应行加到 W 中。返回步骤③。

最后的结论是:S 状态必将进入死锁状态的充分必要条件是,算法的最后结果有未标记的进程。

24. 为了解除死锁,在选取撤消的进程和抢占的进程时,应考虑哪些因素?

答:在选取撤消的进程和抢占的进程时,应考虑以下因素:

(1)优先级;

(2)完成一个进程所需的剩余执行时间;

(3)进程已花费的时间;

(4)进程所占用的资源类型和数量;

(5)完成进程还需要的资源数量;

(6)进程是交互型的、还是批处理型的;

(7)在该次撤消中所涉及到的进程数等。

25. 在一个交通繁忙的十字路口,每个方向只有一个车道,如果车辆只能向前直行,而不允许转弯和后退,并没任何方式进行交通管理。

(1)该十字路口是否会产生交通死锁?若可能,则请画出交通死锁的示意图。

(2)说明产生死锁的 4 个必要条件在此处成立。

(3)提出一个避免死锁的简单规则。

答:(1)该十字路口可能发生交通死锁,其示意图如图 2.28 所示。

(2)在此处,产生死锁的四个必要条件如下:

a. 互斥条件。每个车道的每段道路只能被一辆车占用。

b. 请求与保持条件。每个车队占用了一个车道,并请求前方的车道,即使需等待前方车道上的车队驶离,它仍将持有已占用的车道。

c. 不剥夺条件。在前方的车道被其他车队占用时,因为是单车道,而其他车队又不会后退,所以无法从其他车队处抢占车道。

d. 环路等待条件。向东行驶的车队等待向北行驶的车队让出车道,向北行驶的车队等待向西行驶的车队让出车道,向西行驶的车队等待向南行驶的车队让出车道,而向南行驶的车队则等待向东行驶的车队让出车道。故存在一循环等待链。

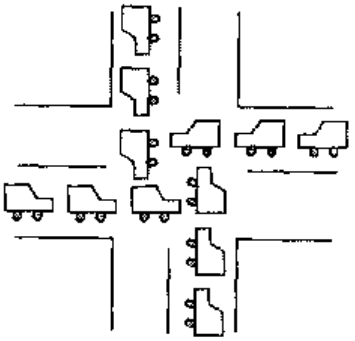


图 2.28 交通锁示意图

(3)可在十字路口设置一交通信号灯,并使南北方向的两个车队和东西方向的两个车队互斥地使用十字路口,便可避免交通死锁。

26. 有 3 个进程 P1、P2 和 P3 并发工作。进程 P1 需用资源 S3 和 S1;进程 P2 需用资源 S1 和 S2;进程 P3 需用资源 S2 和 S3。回答:(西安电子科技大学 2002 年研究生试题)

(1)若对资源分配不加限制,会发生什么情况?为什么?

(2)为保证进程正确地工作,应采用怎样的资源分配策略?为什么?

答:(1)若对资源分配不加限制,上述进程可能会进入死锁状态。因为 P1、P2、P3 是并发工作的,所以完全有可能发生下述情况,此时 P1 获得的 S3,P2 获得了 S1,P3 获得了 S2,但同时它们又循环等待下一个进程释放所占用的资源,从而进入死锁状态。相应的资源分配图如图 2.29 所示。

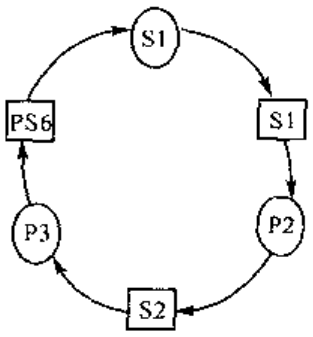


图 2.29 资源分配图

(2)为保证进程正确的工作,可采取下列资源分配策略:

①要求进程一次性地申请它所需的全部资源。这样可破坏"请求与保持"条件。

②要求进程严格按资源号递增的顺序申请资源,即 P1 先申请 S1,再申请 S3;P2 先申请 S1,再申请 S2;P3 先申请 S2,再申请 S2。这样可破坏"环路等待"条件。

③当一个进程申请资源时,如果系统已无此类资源可用,但某个拥有该资源的其他进程处于阻塞状态,则允许前者抢占后者的资源。这样可以破坏"不剥夺"条件。

二、判断题

1. 程序的并发执行失去程序的封闭性和再现性，程序和机器执行程序活动一一对应。（陕西省 1997 年自考题）

答：错。程序和机器执行程序的活动不再一一对应。

2. 进程由进程控制块和数据集以及对该数据集进行操作的程序段组成。（清华大学 1998 年研究生试题）

答：对。

3. 进程具有并发性，它能与其他进程并发运行。（陕西省 1997 年自考题）

答：对。

4. 进程是一个独立的运行单位，也是系统进行资源分配和调度的基本单位。（西安电子科技大学 2000 年研究生试题）

答：对，在没引入线程概念的传统 OS 中，该叙述正确。

5. 进程是程序执行的动态过程，而程序是进程运行的静态文本。（陕西省 1997 年自考题）

答：对。

6. 用户级线程(user-level thread)由操作系统调度，但不能指定在某个处理器上执行。

答：错。用户级线程的管理(包括线程调度)由用户空间的线程库进行，操作系统内核感觉不到线程的存在。

7. 线程是调度的基本单位，但不是资源分配的基本单位。

答：对。

8. 隶属于同一进程的多个线程共享一组 CPU 寄存器值，并共享一个堆栈。

答：错。隶属于同一进程的多个线程可共享进程的地址空间和已打开文件等资源，但必须有自己私有的一组 CPU 寄存器值和一个私有堆栈。

9. 一个进程可以调用创建原语创建一个进程。

答：对。

10. 处于运行状态的进程只能转换为就绪状态或阻塞状态。

答：错。处于运行状态的进程运行结束时不转换为就绪态或阻塞态。

11. 在单处理机上的进程就绪队列和阻塞队列最多只能有一个。（陕西省 1995 年自考题）

答：错。在单处理机上允许有多个就绪队列(如采用多级队列或多级反馈队列进程调度算法的系统中)和多个阻塞队列(如采用信号量进行同步和互斥时，每个信号量都对应有一个阻塞队列)。

12. 进程申请 CPU 得不到满足时，其状态变为等待状态。

答：错。只有就绪状态的进程才能申请 CPU，如申请不到，其状态仍是就绪状态。

13. 进程从运行态转变为就绪态的原因一定是时间片用完。

答：错。也可能是被更高优先权的就绪进程抢占 CPU。

14. 一个进程的状态发生变化总会引起其他一些进程的状态发生变化。

答：错。例如，当一个进程从就绪转变为执行状态时，并不会引起其他进程的状态变化。

15. 如果一个进程正在等待使用处理机，同时除 CPU 以外其他运行条件已满足，则称该进程为就绪状态。

答：对。

16. 一个单处理机中，最多只能允许有两个进程处于运行状态。

答:错。一个单处理机中,最多只能允许有一个进程处于运行状态。

17. 一次仅允许一个进程使用的资源叫临界资源,所以对临界资源是不能实现共享的。

答:错,对临界资源允许共享,但必须互斥地进行。

18. 对临界资源,应采用互斥访问方式来实现共享。

答:对。

19. 进程 A 与进程 B 共享变量 S1,需要互斥;进程 B 与进程 C 共享变量 S2,需要互斥;从而进程 A 与进程 C 也必须互斥。(陕西省 1998 年自考题)

答:错。进程 A 与进程 C 并没有共享同一临界资源,故不必互斥。

20. 临界区是指进程中用于实现进程互斥的那段代码。

答:错。临界区是指进程中用于访问临界资源的那段代码。

21. 信号量是用来表示资源的物理实体,它是一个与队列有关的整型变量。(陕西省 1997 年自考题)

答:错。每个信号量包含一个整型变量和一个队列,故应看成是一个记录型的变量。

22. 一个二元信号量至多能被两个进程访问。

答:错。一个二元信号量可被多个进程共享。

23. 利用信号量(semaphore)解决临界区问题不会发生忙等待(busy waiting)。

答:对。

24. 信号量的初值不能是负的。

答:对。

25. P,V 操作是操作系统中进程低级通信原语。(陕西省 1998 年自考题)

答:对。

26. 用 P,V 操作可以解决进程的同步与互斥问题。

答:对。

27. 使用 P,V 操作后,可以防止系统出现死锁。

答:错。P,V 操作可用来实现同步和互斥,但却不能防止系统出现死锁。如用信号量来实现哲学家就餐时,便可能发生死锁。

28. 只要同一信号量上的 P,V 操作成对地出现就可以防止引起死锁。

答:错。如生产者—消费者问题中将两个 wait 操作互换后,或哲学家就餐问题中的死锁现象。

29. 进程在要求使用某一临界资源时,如果资源正被另一进程所使用,则该进程必须等待;当另一进程使用完并释放后方可使用。这种情况即所谓进程间同步。(陕西省 1995 年自考题)

答:错。这种情况为互斥。

30. 任何时刻,只有一个进程能在管程中执行。

答:对。

31. 为了防止进程长期占用 CPU,在动态优先级调度中,随着进程执行时间的增加,其优先级可渐渐降低。

答:对。

32. 采用简单的时间片轮转调度算法的系统中,也可能出现饥饿现象。

答:错,任何一个就绪进程至多等待(n-1)q 后,一定会得到 CPU。

33. 多级反馈队列(multilevel feedback queue)进程调度算法有利于 I/O-bound 进程的执行。

答:对。

34. 操作系统通过切换进程标识符来进行进程切换。

答：错。进程的切换过程包括现场的保护和恢复，比切换进程标识符要复杂得多。

35. 死锁是一种与时间有关的错误，它与进程推进的速度无关。（陕西省 1996 年自考题）

答：错，死锁与进程的推进顺序有关，而推进顺序又受进程推进速度的影响。

36. 采用资源的静态分配算法可以预防死锁的发生。（西安电子科技大学 2000 年研究生试题）

答：对。

37. 不安全状态指系统中有进程已发生死锁。

答：错。不安全状态指系统有可能发生死锁的状态，而并非是指系统已经发生死锁。

38. 系统处于不安全状态必然导致系统死锁。

答：错。系统的安全性检查算法是根据进程的最大需求来进行的，而进程在运行过程中的资源需求量可小于最大需求，故系统不一定会死锁。

39. 某系统由相同类型的 4 个资源组成，若资源可被 3 个进程申请使用，每个进程最多可申请 2 个资源，则该系统不会发生死锁。

答：对。

40. 在死锁的避免方法中，仅当系统处于安全状态时，才实施分配。

答：对。

41. 一个给定的进程—资源图的全部化简序列导致同一不可化简图。（陕西省 1997 年自考题）

答：对。

42. 死锁定理：S 是一个死锁状态，当且仅当 S 的进程—资源图是可完全化简图。（陕西省 1998 年自考题）

答：错，“可完全化简”应改为“不可完全化简”。

43. 若系统资源分配图中出现了环路，则表明系统发生了死锁。

答：错。上述条件是发生死锁的必要条件而非充分条件。如图 2.30 的系统资源分配图虽出现了环路，但却没发生死锁。

44. 进程之间因为竞争资源可能会引起死锁，死锁的检测修复以及死锁的避免策略是对付死锁最常用的方法，但是前一种方法所付出的代价更大。

答：错。对付死锁的最常用的方法为预防死锁或驼鸟算法。

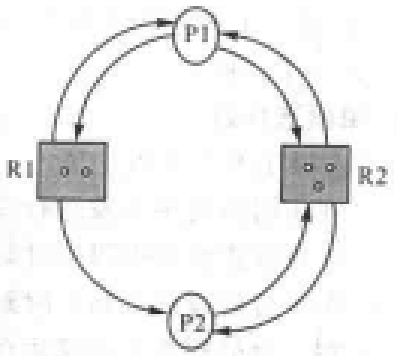


图 2.30　资源分配图

45. 假设有三道作业，它们的提交时间及运行时间由表 2-47 给出，如果采用非多道程序设计并且采用先来先服务和短作业优先算法，它们的调度性能对这个作业流来讲完全一样。

答：错，采用先来先服务时调度次序是 1，2，3，平均周转时间是 2.5 小时，采用短作业优先时调度次序是 3，2，1，平均周转时间是 2.33 小时，故后者性能较好。

表 2-47　作业提交时间和执行时间表

作业	提交时间	执行时间/小时
1	10:00	2
2	10:20	1
3	10:40	0.5

46. 作业调度是处理机的高级调度，进程调度是处理机的低级调度。（西安电子科技大学 2000 年研究生试题）

答：对。

47. 一个作业由若干个作业步组成，在多道程序设计的系统中这些作业步可以并发执行。（西安电子科技大学 2000 年研究生试题）

答：错，作业之间而非作业步之间可并发执行。

48. 在各种作业调度算法中，短作业优先调度算法会使每个作业的等待时间最短。

答：错，短作业调度算法可使平均等待时间最短，但某些长作业的等待时间可能会比其他算法更长。

49. 作业一旦被作业调度选中，即占有了 CPU。

答：错，作业调度只将作业调入内存，并为它创建就绪进程，分配 CPU 由进程调度完成。

50. 在剥夺式进程管理方式下，现运行进程的优先级不低于系统中所有进程的优先级。（陕西省 1997 年自考题）

答：错，现行进程的优先级仅高于就绪进程的优先级，但可低于阻塞进程的优先级。

51. 程序的并发执行是指同一时刻有两个以上的程序，它们的指令在同一个处理器上执行。（西安交通大学 1999 年研究生试题）

答：错，程序的并发执行是指同一时间间隔内有一个以上的程序在同时执行，但由于任一时刻，一个处理器仅能执行一个程序，故在单处理机的环境下，上述程序仅在宏观上同时执行，微观上，它们仍必须交替地执行。

52. 并发是并行的不同表述，其原理相同。（清华大学 1998 年研究生试题）

答：错，并发是指多个事件在同一时间间隔内发生，而并行是指多个事件在同一时刻进行。两者含义不同，如单处理机系统中，多个程序虽然可以并发执行，但却无法并行地执行。

53. 某一进程被中断，转去执行中断处理程序；中断处理程序结束后，一定返回到被中断的程序。（陕西省 1998 年自考题）

答：错。有些操作系统中，在中断处理程序结束的过程中进行进程调度，从而有可能返回到另一个程序。

三、单项选择题

1. 多道程序设计是指_____。（西安电子科技大学 2002 年研究生试题）

A. 在实时系统中并发运行多个程序

B. 在分布式系统中同一时刻运行多个程序

C. 在一台处理机上同一时刻运行多个程序

D. 在一台处理机上并发运行多个程序

答：D

2. 一个进程是_____。（清华大学 1996 年研究生试题）

A. 由协处理机执行的一个程序　　B. 一个独立的程序 + 数据集

C. PCB 结构与程序和数据的组合　　D. 一个独立的程序

答：C

3. 下列几种关于进程的叙述，_____最不符合操作系统对进程的理解。（浙江大学 1998 年研究生试题）

A. 进程是在多程序环境中的完整的程序

B.进程可以由程序、数据和进程控制块描述

C.线程(Thread)是一种特殊的进程

D.进程是程序在一个数据集合上的运行过程,它是系统进行资源分配和调度的一个独立单元

答:A

4.下面对进程的描述中,错误的是_____。

A.进程是动态的概念　　B.进程有生命期

C.进程是指令的集合　　D.进程可以并发执行

答:C

5.进程与程序的重要区别之一是_____。

A.程序有状态而它没有　　B.它有状态而程序没有

C.程序可占有资源而它不可　　D.它能占有资源而程序不能。

答:B.它有状态而程序没有

6.操作系统中,可以并行运行的基本单位是_____,同时,它也是核心调度及资源分配的基本单位。

A.作业　　B.函数　　C.进程　　D.过程

答:C.进程

7.进程是由_____组成的。

A.程序、数据和 PCB　　B.程序、数据和标识符

C.程序、标识符和 PCB;　　D.数据、标识符和 PCB

答:A.程序、数据和 PCB

8.现代操作系统中申请资源的基本单位是_A_,在 CPU 得到执行的基本单位是_B_。_A_是由_C_组成的,它与_B_的区别之一是_D_。(东南大学 2000 年考研试题)

A、B:(1)模块;(2)作业;(3)线程;(4)管程;(5)进程;(6)类程;(7)例程。

C:(1)入口,过程,出口;(2)正文,数据,堆栈;(3)正文段,数据段,PCB;(4)正文,数据,JCB。

D:(1)A 的并发粒度比 B 的大;(2)A 的并发粒度比 B 的小;(3)A 是动态的,而 B 是静态的;(4)A 有后备状态,而 B 没有。

答:A:(5)进程;B:(3)线程;C:(3)正文段,数据段,PCB;D:(1)A 的并发粒度比 B 的大。

9.在多对一的线程模型中,当一个多线程进程中的某个线程执行一个需阻塞的系统调用时,_____。

A.该进程的其他线程仍可继续运行　　B.整个进程都将阻塞

C.该阻塞线程将被撤消　　D.该阻塞线程将永远不可能再执行

答:B

10.用户可通过_____建立和撤消进程。

A.函数调用　　B.宏指令　　C.系统调用　　D.过程调用

答:C.系统调用

11.通常用户进程被建立后,_____。

A.便一直存在于系统中,直到被操作人员撤消;

B.随着进程运行的正常或不正常结束而撤消;

C.随着时间片轮转而撤消与建立;

D. 随着进程的阻塞或唤醒而撤消与建立。

答:B 随着进程运行的正常或不正常结束而撤消;

12. 支持多道程序设计的操作系统在运行过程中,不断地选择新进程运行来实现 CPU 的共享,但其中_____不是引起操作系统选择新进程的直接原因。(复旦大学 1999 年研究生试题)

A. 运行进程的时间片用完　　B. 运行进程出错

C. 运行进程要等待某一时间发生　　D. 有新进程进入就绪状态

答:D

13. 正在执行的进程应终端的请求而暂停下来以便研究其运行情况(执行挂起进程原语),这时进程应转变为_____状态。

A. 静止阻塞　　B. 活动阻塞　　C. 静止就绪　　D. 活动就绪

答:C

14. 处于静止阻塞状态的进程,在进程等待的事件出现后,应转变为_____状态。

A. 执行　　B. 活动阻塞　　C. 静止就绪　　D. 活动就绪

答:C

15. 当_____时,进程从执行状态转变为就绪状态。(西北工业大学 1999 年研究生试题)

A. 进程被调度程序选中　　B. 时间片到

C. 等待某一事件　　D. 等待的事件发生

答:B

16. 在非剥夺调度方式下,当_____时,不会引起一进程从就绪态变为运行态。

A. 一个进程被创建后进入就绪态　　B. 一个进程从运行态变成等待态

C. 运行的进程执行结束　　D. 一个进程从运行态变成就绪态

答:A

17. 进程从运行态到等待态可能是_____。

A. 运行进程执行 P 操作　　B. 进程调度程序的调度

C. 运行进程的时间片用完　　D. 运行进程执行了 V 操作

答:A

18. 采用时间片轮转法分配 CPU 时,当处于运行状态的进程用完一个时间片后,它的状态是_____。

A. 阻塞　　B. 运行　　C. 就绪　　D. 消亡

答:C

19. 在进程状态转换时,状态转换_____一般是不可能发生的。

A. 就绪态→运行态　　B. 运行态→就绪态

C. 运行态→阻塞态　　D. 阻塞态→运行态

答:D

20. 系统中有 n(n>2)个进程,并且当前没有执行进程调度程序,则_____不可能发生。

A. 有一个运行进程,没有就绪进程,剩下的 n-1 个进程处于等待状态

B. 有一个运行进程和 n-1 个就绪进程,但没有进程处于等待状态

C. 有一个运行进程和 1 个就绪进程,剩下的 n-2 个进程处于等待状态

D. 没有运行进程但有 2 个就绪进程,剩下的 n-2 个进程处于等待状态

答:D

21. 并发进程之间_____。(陕西省 1997 年自考题)

A. 彼此无关　　B. 必须同步　　C. 必须互斥　　D. 可能需要同步或互斥

答:D

22. 我们把在一段时间内,只允许一个进程访问的资源,称为临界资源,因此,我们可以得出下列论述,请选择一条正确的论述。

A. 对临界资源是不能实现资源共享的。

B. 对临界资源,应采取互斥访问方式,来实现共享。

C. 为临界资源配上相应的设备控制块后,便能被共享。

D. 对临界资源应采取同时访问方式,来实现共享。

答:B

23. 下面临界区概念论述正确的是_____。

A. 临界区是指进程中用于实现进程互斥的那段程序代码

B. 临界区是指进程中用于实现进程同步的那段程序代码

C. 临界区是指进程中用于实现进程通信的那段程序代码

D. 临界区是指进程中用于访问临界资源的那段程序代码

答:D

24. 进程中_____是临界区。

A. 用于实现进程同步的那段程序　　B. 用于实现进程通讯的那段程序

C. 用于访问共享资源的那段程序　　D. 用于更改共享数据的那段程序

答:D

25. 在多进程的系统中,为了保证公共变量的完整性,各进程应互斥进入临界区。所谓临界区是指_____。(清华大学 1996 年研究生试题)

A. 一个缓冲区　　B. 一段数据区　　C. 同步机制　　D. 一段程序

答:D

26. 在操作系统中解决进程间的两种基本关系__A__,往往运用信号量进行__B__的__C__。例如为保证系统数据库的完整性,可以把信号量定义为对某个库文件或记录的锁,初值为 1,任何进程存取库文件或记录之前先对它执行一个__D__,存取后再执行一个__E__。(东南大学 2000 年考研试题)

A:(1)同步与异步;(2)串行与并行;(3)调度与控制;(4)同步与互斥。

B:(1)消息操作;(2)P－V 操作;(3)开关操作;(4)读写操作;(5)锁。

C:(1)通信原语;(2)调度算法;(3)分配操作;(4)检查操作;(5)释放

D,E:(1)联机操作;(2)V 操作;(3)输出操作;(4)读操作;(5)P 操作;(6)写操作;(7)输入操作;(8)trap 操作。

答案:A:(4)同步与互斥;B:(5)锁;C:(4)检查操作;D:(5)P 操作;E:(2)V 操作。

27. 在操作系统中,信号量表示资源实体,是一个与队列有关的_____变量,其值仅能用 P、V 操作来改变。(陕西省 1995 年自考题)

A. 实体　　B. 整形　　C. 布尔型　　D. 记录型

答:D

28. 用 P、V 操作可以解决_____互斥问题。

A. 某些　　B. 一个　　C. 一切　　D. 大多数

答:C

29. 在操作系统中,P,V 操作是一种_____。(西安交通大学 1999 年研究生试题)

A. 机器指令　　B. 系统调用命令

C. 作业控制命令　　D. 低级进程通信原语

答:D

30. 实现进程之间同步与互斥的通信工具为_____。

A. P、V 操作　　B. 信箱通信　　C. 消息缓冲　　D. 高级通信

答:A

31. 对于记录型信号量,在执行一次 P 操作时,信号量的值应当A;当其值为B 时,进程应阻塞。在执行 V 操作时,信号量的值应当C;当其值为D时,应唤醒阻塞队列中的进程。

A、C:(1)不变;(2)加 1;(3)减 1;(4)加指定数值;(5)减指定数值。

B、D:(1)大于 0;(2)(3)大于等于 0;(4)小于等于 0。

答:A:(3)减 1;B:(4)小于 0;C:(2)加 1;D:小于等于 0。

32. 对两个并发进程,其互斥信号量为 mutex;若 mutex=0,则表明_____。

A. 没有进程进入临界区

B. 有一个进程进入临界区但没进程处于阻塞状态

C. 一个进程进入临界区而另一个进程正处于等待进入临界区状态

D. 有两个进程进入临界区

答:B

33. 设有 5 个进程共享一个互斥段,如果允许有 3 个进程同时进入互斥段,则所采用的互斥信号量的初值应是_____。(陕西省 1996 年自考题)

A. 5　　B. 3　　C. 1　　D. 0

答:B

34. N 个进程共享某一临界资源,则互斥信号量的取值范围为_____。

A. 0～1　　B. －1～0　　C. 1～(N－1)　　D. 0～－(N－1)

答:C

35. 若有 3 个进程共享一个互斥段每次最多允许两个进程进入互斥段,则信号量的变化范围是_____。(陕西省 1998 年自考题)

A. 2,1,0,－1　　B. 3,2,1,0

C. 2,1,0,－1,－2　　D. 1,0,－1,－2

答:A

36. 计算机操作系统中,若 P、V 操作的信号量 S 初值为 2,当前值为－1,则表示有_____等待进程。

A. 0 个　　B. 1 个　　C. 2 个　　D. 3 个

答:B

37. 在 9 个生产者、6 个消费者共享容量为 8 的缓冲器的生产者—消费者问题中,互斥使用缓冲器的信号量 mutex 的初始值为_____。

A. 1　　B. 6　　C. 8　　D. 9

答:A

38. 每个管程管理__A__临界资源,进程要进入管程,其方法是通过调用特定的__B__。管程内部任何时刻__C__进程处于活动状态,活动进程调用管程内的__D__,不活动的进程处在条件变量上,条件变量是与__E__相同的变量。(东南大学2000年研究生试题)

A:(1)一个;(2)二个;(3)若干;(4)全部。

B:(1)入口命令;(2)访管指令;(3)特权指令;(4)原语;(5)trap指令。

C:(1)只允许一个;(2)只允许一对即二个;(3)可允许若干个;(4)理论上可允许无穷多个。

D:(1)类程;(2)过程;(3)函数或过程;(4)P操作和V操作。

E:(1)布尔变量;(2)普通变量;(3)计数信号量类型;(4)队列结构变量。

答:A:(3)若干;B:(1)入口命令;C:(1)只允许一个;D:(2)过程;E:(3)计数信号量类型。

39. 信箱通信是一种_____通信方式。

A. 直接通信 B. 间接通信 C. 低级通信 D. 信号量

答:B

40. 在消息缓冲通信方式中,临界资源为_____。

A. 发送进程 B. 消息队列 C. 接收进程 D. 信箱

答:B

41. 作业周转时间为_____。(陕西省1998年自考)

A. 作业开始时间 - 作业提交时间 C. 作业等待时间

B. 作业等待时间 + 作业执行时间 D. 作业执行时间

答:B

42. 设有4个作业同时到达,每个作业的执行时间均为2小时,它们在一台处理机上按单道方式运行,则平均周转时间为_____。(西安电子科技大学2000年研究生试题)

A. 1小时 B. 5小时 C. 2.5小时 D. 8小时

答:B

43. 现有3个同时到达的作业J1、J2和J3,它们的执行时间分别是T1、T2和T3,且T1<T2<T3。系统按单道方式运行且采用短作业优先算法,则平均周转时间是_____。(西安电子科技大学2002年研究生试题)

A. T1 + T2 + T3 C. (3T1 + 2T2 + T3)/3

B. (T1 + T2 + T3)/3 D. (T1 + 2T2 + 3T3)/3

答:C

44. 在现代操作系统中必不可少的调度是_____。

A. 高级调度; B. 中级调度; C. 作业调度; D. 进程(线程)调度。

答:D

45. 进程调度的对象和任务分别是_____。

A. 作业,从就绪队列中按一定的调度策略选择一个进程占用CPU

B. 进程,从就绪队列中按一定的调度策略选择一个进程占用CPU

C. 进程,从后备作业队列中按一定的调度策略选择一个作业占用CPU

D. 作业,从后备作业队列中按一定的调度策略选择一个作业占用CPU

答:B

46. 在下列调度算法中,对所有进程和作业都是公平合理的调度算法是__A__;最有利于提高

系统吞吐量的作业调度算法是__B__；能兼顾作业等待时间和作业执行时间的调度算法是__C__；最有利于提高资源的使用率，能使短作业、长作业及交互作业用户都比较满意的调度算法是__D__；为实现人机交互作用应采用调度算法是__E__；能对紧急作业进行及时处理的调度算法是__F__。

A,B,C,D,E,F:(1)FCFS 调度算法;(2)短作业优先调度算法;(3)时间片轮转法;(4)多级反馈队列调度算法;(5)高响应比优先算法;(6)基于优先权的剥夺调度算法。

答:A:(1)FCFS 调度算法;B:(2)短作业优先调度算法;C:(5)高响应比优先算法;D:(4)多级反馈队列调度算法;E:(3)时间片轮转法;F:(6)基于优先权的剥夺调度算法。

47. 哪一个说法对剥夺式系统来讲结论正确:_____。(北京理工大学 1999 年研究生试题)

A. 若系统采用轮转法调度进程,则系统采用的是剥夺式调度

B. 若现行进程要等待某一事件时引起调度,则该系统是剥夺式调度

C. 实时系统通常采用剥夺式调度

D. 在剥夺式系统中,进程的周转时间较之非剥夺式系统可预见

答:A

48. 假设就绪队列中有 10 个进程,系统将时间片设为 200ms,CPU 进行进程切换要花费 10ms。则系统开销所占的比率约为_____。

A.1% B.5% C.10% D.20%

答:B

49. 一作业 8:00 到达系统,估计运行时间为 1 小时。若 10:00 开始执行该作业,其响应比是_____。(西安电子科技大学 2002 年研究生试题)

A.2 B.1 C.3 D.0.5

$$\text{响应比}=\frac{\text{响应时间}}{\text{要求服务时间}}=\frac{\text{等待时间}+\text{要求服务时间}}{\text{要求服务时间}}=\frac{2+1}{1}=3$$

答:C

50. 设有 3 个作业,它们的到达时间和运行时间如表 2-48 所示,并在一台处理机上按单道方式运行。如按响应比高者优先算法,则作业执行是次序是_____。(陕西省 1998 年自考题)

A. J1, J2, J3 B. J1, J3, J2

C. J2, J3, J1 D. J3, J2, J1

表 2-48 作业到达时间和运行时间表

作业	到达时间	运行时间
1	8:00	2 小时
2	8:30	1 小时
3	9:30	0.25 小时

答:B

51. 既考虑作业等待时间又考虑作业执行时间的调度算法是_____。

A. 响应比高者优先 B. 短作业优先

C. 优先级调度 D. 先来先服务

答:A

52. 调度算法与作业的估计运行时间有关的是_____算法。

A. 先来先服务　　B. 均衡　　C. 短作业优先　　D. 时间片轮转

答:C

53. 关于优先权大小的论述中,正确的论述是_____。

A. 计算型作业的优先权,应高于I/O型作业的优先权。

B. 用户进程的优先权,应高于系统进程的优先权。

C. 长作业的优先权,应高于短作业的优先权。

D. 资源要求多的作业,其优先权应高于资源要求少的作业。

E. 在动态优先权中,随着作业等待时间的增加,其优先权将随之下降。

F. 在动态优先权中,随着进程执行时间的增加,其优先权降低。

答:F

54. 为多道程序提供的共享资源不足时,可能会出现死锁。但是,不适当的_____也可能产生死锁。

A. 进程调度顺序　　B. 进程的优先级

C. 资源分配方法　　D. 进程推进顺序

答:D

55. 在_____的情况下,系统出现死锁。

A. 计算机系统发生了重大故障

B. 有多个封锁的进程同时存在

C. 若进程因竞争资源而无休止地相互等待对方释放已占有的资源

D. 资源数大大小于进程数或进程同时申请的资源数大大超过资源总数

答:C

56. 操作系统中,"死锁"的概念是指_____。

A. 程序死循环

B. 硬件发生故障

C. 两个或多个并发进程各自占有某种资源而又都等待别的进程释放它们所占有的资源

D. 系统停止运行

答:C

57. 产生死锁的根本原因是__A__和__B__。发生死锁的四个必要条件是:互斥使用条件,__C__条件,不剥夺条件和__D__条件。银行家算法用于__E__死锁。(东南大学2000年考研试题)

A:(1)资源分配不当;(2)系统资源数量不足;(3)作业调度不当;(4)用户数太多。

B:(1)进程推进顺序不当;(2)进程调度失误;(3)进程数太多;(4)CPU太慢。

C:(1)请求和阻塞;(2)请求与释放;(3)请求和保持;(4)释放和阻塞。

D:(1)环路;(2)环路且等待;(3)资源编序;(4)资源无序。

E:(1)避免;(2)预防;(3)控制;(4)模拟;(5)解除。

答:A:(2)系统资源数量不足;B:(1)进程推进顺序不当;C:(3)请求和保持;D:(2)环路且等待。E:(1)避免

58. 死锁预防是保证系统不进入死锁状态的静态策略,其解决办法是破坏产生死锁的四个必要条件之一。下列方法中破坏了"循环等待"条件的是_____。

A. 银行家算法　　B. 一次性分配策略
C. 剥夺资源法　　D. 资源有序分配策略
答:D

59. 死锁与安全状态的关系是:_____。
A. 死锁状态有可能是安全状态　　B. 安全状态有可能成为死锁状态
C. 不安全状态就是死锁状态　　D. 死锁状态一定是不安全状态
答:D

60. 银行家算法是一种_____算法。
A. 死锁解除　　B. 死锁避免
C. 死锁预防　　D. 死锁检测
答:B

61. 设 m 为同类资源数,n 为系统中并发进程数。当 n 个进程共享 m 个互斥资源时,每个进程的最大需求是 w;则下列情况会出现系统死锁的是_____。
A. m=2, n=1, w=2　　B. m=2, n=2, w=1
C. m=4, n=3, w=2　　D. m=4, n=2, w=3
答:D
m=4, n=2, w=3 时,若每个进程各占 2 个资源,那么在它们申请第 3 个资源时,两个进程都将阻塞,从而进入死锁状态。

62. 假设系统由相同类型的 9 个资源被 4 个进程共享,试分析每个进程最多可以请求多少个资源数时该系统仍无死锁。
A. 1　　B. 2　　C. 3　　D. 4
答:C

63. 如果系统的资源分配图_____,则系统处于死锁状态。
A. 出现了环路　　B. 没有环路
C. 每种资源只有一个,并出现了环路　　D. 每个进程结点至少有一条请求边
答:C

64. 下列死锁的论述中,正确的论述是_____。
A. 由于产生死锁的基本原因是系统资源不足,因而预防死锁的最常用方法,是根据系统规模,配置足够的系统资源。
B. 由于产生死锁的另一种基本原因是进程推进顺序不当,因而预防死锁的常用方法,是使进程的推进顺序合法。
C. 因为只要系统不进入不安全状态,便不会产生死锁,故预防死锁的常用方法,是防止系统进入不安全状态。
D. 可以通过破坏产生死锁的四个必要条件之一或其中几个的方法,来预防发生死锁。
答:D

65. 下面的叙述中正确的是_____。
A. 操作系统的一个重要概念是进程,因此不同进程所执行的代码也一定不同
B. 为了避免发生进程死锁,各个进程只能逐个申请资源
C. 操作系统用 PCB 管理进程,用户进程可以从 PCB 中读出与本身运行状况有关的信息
D. 进程同步是指某些进程之间在逻辑上的相互制约关系

答:D

66. 以下叙述中正确的是_____。

A. 调度原语主要是按一定算法从阻塞队列中选择一个进程,将处理机分配给它

B. 预防死锁的发生可以通过破坏产生死锁的4个必要条件之一来实现,但破坏互斥条件的可能性不大

C. 进程进入临界区时要执行开锁原语

D. P,V操作可以防止死锁的发生

答:B

67. 下面关于进程的叙述不正确的是_____。

A. 进程申请CPU得不到满足时,其状态变为就绪状态。

B. 在单CPU系统中,任一时刻有一个进程处于运行状态。

C. 优先级是进行进程调度的重要依据,一旦确定不能改变。

D. 进程获得处理机面运行是通过调度而实现的。

答:C

四、填空

1. 程序顺序执行时具有顺序性、_____和可再现性的特点。

答:封闭性

2. 每个进程都有生命期,即从_____到_____。

答:创建、撤消(或消亡)

3. 进程主要有_____、_____、_____三部分内容组成,其中_____是进程存在的惟一标志,而_____部分则可以为其他进程共享。

答:PCB、程序段、数据段、PCB、程序段

4. 一个被创建的进程处于_____状态。

答:就绪

5. 进程的基本特征有_____、_____、独立、异步及结构特征。

答:动态性、并发性、

6. 进程是一个_____态概念,而程序是一个_____态概念。

答:动、静

7. 进程在运行过程中有三种基本状态,它们是_____、_____、_____。

答:运行、就绪、阻塞

8. 当多个进程等待分配处理机时,系统按一种规定的策略从多个处于_____状态的进程中选择一个进程,让它占有处理机,被选中的进程就进入了_____状态。

答:就绪,执行

9. 当一个进程从_____变成等待状态或进程完成后被撤消时都会产生_____。

答:运行、进程调度

10. 在一个单处理机系统中,若有5个用户进程,且假设当前时刻为用户态,则处于就绪状态的用户进程最多有_____个,最少有_____个。

答:4,0

11. 如果系统中有n个进程,则在等待队列中进程的个数最多为_____个。(北京大学1997年研究生试题)

答：n(如这些进程均在等待 I/O 操作的完成，或它们已全部进入死锁状态)

12. 进程是一个_____的基本单位，也是一个_____和_____的基本单位。

答：独立运行、资源分配、CPU 调度

13. 当系统创建一个进程时，系统就为其建立一个_____，当进程被撤消时就将其收回。(陕西省 1998 年自考题)

答：进程控制块 PCB

14. 将进程的_____链接在一起便形成了进程队列。

答：PCB

15. 进程的队列组织，通常采用_____和_____的形式。(陕西省 1995 年自考题)

答：链接，索引

16. 在多道程序系统中，进程之间存在着的不同制约关系可以划分为两类：_____与_____。

其中前者指进程间具有的一定逻辑关系；后者是指进程间在使用共享资源方面的约束关系。

答：同步、互斥

17. 临界资源的概念是_____，而临界区是指_____。

答：一次仅允许一个进程访问的资源，进程中访问临界资源的那段程序代码

18. 我们把互斥执行的程序段称为_____。

答：临界区

19. 若一个进程已进入临界区，其他欲进入临界区的进程必须_____。

答：等待

20. 用于实现互斥的同步机制必须遵循_____、_____、_____和_____四条准则。

答：空闲让进、忙则等待、有限等待、让权等待

21. 操作系统中，对信号量 S 的 P 原语操作定义中，使进程进入相应等待队列等待的条件是_____。

答：S.value<0

22. 对于信号量可以做_____操作和_____操作，其中前一种操作用于申请资源或阻塞进程，后一种操作用于释放资源或唤醒进程。

答：P、V

23. 在利用信号量实现进程互斥时，应将_____置于_____和_____之间。

答：临界区、P 操作、V 操作

24. 在 P、V 操作中，信号量 S 的物理意义是当信号量 S 值大于零时表示_____；当信号量 S 值小于零时，其绝对值为_____。

答：可用资源的数目，因申请该资源而阻塞的进程数目

25. 有 m 个进程共享同一临界资源，若使用信号量机制实现对临界区资源的互斥访问，则信号量值的变化范围是_____。

答：(1－m)～1

26. 如果信号量的当前值为－4，则表示系统中在该信号量上有_____个等待进程。(北京大学 1997 年研究生试题)

答：4

27. 设有 4 个进程共享一程序段，而每次最多允许两个进程进入该程序段，则信号量的取值范围可能是_____。

答：[－2,2]

28. 实现一个管程时必须考虑的 3 个主要问题包括：_____ __。（西安电子科技大学 1996 年研究生试题）

答：互斥、同步、条件变量

29. 为实现基于消息缓冲队列的进程通信，在进程控制块中应增加_____、_____以及_____三个数据项。

答：消息队列指针、消息队列的互斥信号量、消息队列的资源信号量

30. 用信箱实现通信时，应有_____和_____两条基本原语。

答：发送、接收

31. 信箱逻辑上可分成_____和_____两部分，其中前一部分存放有关信箱的描述，后一部分由若干格子组成，每格存放一信件，格子的数目和大小在创建信箱是确定。

答：信箱头、信箱体

32. 用户与操作系统之间的接口主要分为_____和_____两类 。（中国科技大学 1998 年研究生试题）

命令接口，系统功能调用

33. 所谓系统调用，就是用户在程序中调用_____所提供的一些子功能。

答：操作系统

34. 按命令接口对作业控制方式的不同可将命令接口分为_____和_____。

答：联机命令接口、脱机命令接口

35. 作业调度又称_____，其主要功能是_____，并为作业做好运行前的准备工作和作业完成后的善后处理工作。

答：高级调度，按照一定的调度算法从后备队列中选取作业进入内存

36. 确定作业调度算法时应注意系统资源的均衡使用，使_____作业和_____作业搭配运行。

答：计算型、I/O 型

37. 一个作业可以分成若干顺序处理的加工步骤，每个加工步骤称为一个_____。

答：作业步

38. 在一个具有分时兼批处理的计算机操作系统中，如果有终端型作业和批处理型作业混合同时执行，_____作业应优先占用处理器。

答：终端型

39. _____调度是处理机的高级调度，_____调度是处理机的低级调度。（西安电子科技大学 2001 年研究生试题）

答：作业调度，进程调度

40. 进程调度负责_____的分配工作。

答：处理机

41. 进程的调度方式有两种，一种是_____，另一种是_____。

答：抢占调度方式、非抢占调度方式

42. 在剥夺调度方式中，剥夺的原则有_____、_____和_____等原则。

答:时间片原则、优先权高者优先原则、短作业优先原则

43. 一个作业运行时间假定为 1 小时,它在系统中等待了 2 小时,则该作业的响应比是_____。(陕西省 1997 年自考题)

答:3

44. 如果系统中所有作业是同时到达的,则使作业平均周转时间最短的作业调度算法是_____。(北京大学 1997 年研究生试题)

答:短作业优先调度算法

45. 响应比高者优先算法综合考虑了作业的_____和 _____。

答:等待时间,执行时间

46. 若使当前运行的进程总是优先级最高的进程,应选择_____进程调度算法。(北京大学 1997 年研究生试题)

答:抢占式最高优先权优先

47. 一种最常用的进程调度算法是把处理机分配给具有最高优先权的进程,而确定优先权的方法概括起来不外乎是基于_____特性和_____特性两种方法,前者所得到的是_____优先权,后者得到的是_____优先权。

答:静态、动态、静态、动态

48. 在_____调度算法中,按照进程进入就绪队列的先后次序来分配处理机。

答:先来先服务

49. 设有一组作业,它们的提交时间及运行时间如表 2-49 所示:

表 2-49 作业的提交时间及运行时间

作业号	提交时间	运行时间(分钟)
1	9:00	70
2	9:40	30
3	9:50	10
4	10:10	5

在单道方式下,采用短作业优先调度算法,作业的执行顺序是_____。

答:1,4,3,2

50. 死锁是指在系统中的多个_____无限期的等待永远不会发生的条件。

答:进程

51. 死锁产生的主要原因是_____和_____。

答:竞争资源,进程推进顺序非法

52. 死锁产生的必要条件有四个,即_____、_____、_____、_____。

答:互斥条件、请求与保持条件、不剥夺条件、环路等待条件

53. 对待死锁,一般应考虑死锁的预防、避免、检测和解除四个问题。典型的银行家算法属于_____,破坏环路等待条件是属于_____,而剥夺资源是_____的基本方法。

答:死锁的避免、死锁的预防、死锁的解除

54. 如果要求所有进程一次性申请他所需要的全部资源,若系统有足够的资源分配给进程,便一次把所有的资源分配给该进程,但在分配时只要有一种资源要求不能满足,则资源全不

分配,进程等待。这种死锁预防方法破坏了死锁产生必要条件中的____条件。

答:请求与保持条件

55. 为了破坏环路等待条件,可将所有的_____按类型进行_____,并赋予它们不同的_____。

答:资源,排序,序号

56. 有多种类型的资源可分配给进程使用时,可以采用_____来避免死锁。

答:银行家算法

57. 银行家算法中,当一个进程提出的资源请求导致系统从_____进入_____时,系统就拒绝它的资源请求。

答:安全状态、不安全状态时

58. _____法和_____法是解除死锁的两种常用方法。(陕西省 1997 年自考题)

答:撤消进程,剥夺资源

2.8 OSC 习题与解答

Chapter 4 进程管理

4.1 有些操作系统如 MS-DOS 没有提供并发处理手段。引入并发处理会导致操作系统设计的复杂性。试分析引入并发处理后导致的操作系统设计的三个主要的复杂性。

答:(1)分时技术。分时使得系统中的各个进程有机会获得系统服务。分时技术包括对占用 CPU 的进程的剥夺(如通过系统调用)以及内核的可重入性(从而保证多个进程能并发执行内核代码)。

(2)资源保护。进程和系统资源必须得到保护以保证不会有进程非法访问其他进程或系统的资源。必须限制任何一个进程对内存的使用权限和对外部设备(如磁盘的操作权限。

(3)死锁预防。内核必须经严格设计以预防发生死锁,即不会导致进程间相互等待被对方所占用的资源。

4.2 分析调度的三种形式:短期调度、中期调度和长期调度的差别。

答:短期调度(即 CPU 调度)。从内存中选择就绪的进程并将 CPU 分配给它。

中期调度。通常在分时系统中作为一个中间调度层。指将一个部分执行的进程从内存中换出到外存交换设备中,下次再将其重新调入内存。

长期调度。决定用户提交的进程中哪些被调入内存以待执行。

三者的主要区别在于它们发生的频率。短期调度发生频率最高,以决定哪个进程被执行;长期调度不经常发生,即当一个进程执行完时,决定是否接纳新的进程。

4.3 一台 DEC SYSTEM-20 计算机有多个寄存器组。试描述当一个新进程的上下文信息已经存储在一个寄存器组时,进程上下文切换所发生的动作?若新进程的上下文信息存储内存里,情况又会怎样?

答:只要将当前寄存器组指针指向这个包含新进程上下文信息的寄存器组就可以了,这只需要花费很少时间。

若进程的上下文信息在内存中,则要选择一个寄存器组,将其内容保存在内存中,然后将内存中新进程的上下文信息装入这个寄存器组中。

4.4 阐述发生进程上下文切换时内核的工作。

答:总的来说,操作系统必须保存当前进程的现场、恢复选中进程的现场。现场保存主要包括对 CPU 寄存器值的保存和已分配内存的保存。

上下文切换还包括与计算机体系结构相关的一些操作,如数据及指令高速缓存(cache)的刷新。

4.5 说明下列各组基于消息队列的通信方式的优缺点。

(1)对称通信与非对称　　(2)自动缓冲与透明缓冲

(3)拷贝发送与引用发送　　(4)固定大小的消息与可变大小的消息

答:(1)对称通信与非对称通信

在基于消息传递的进程间通信中,若发送方和接收方都直接给出了对方的身份,则称为对称通信;若发送方给出了接收方身份,而接收方不给出发送方身份,则称为非对称通信。这两种方式统称为直接通信方式。这种直接通讯方式直接在接收方和发送方之间建立通信链路,因而效率都很高。在对称通信方式中,无论发送方或接收方的身份发生变化,通信链路都将重建,而非对称通信方式中只需要发送方了解接收方的新身份,接收方没有变化。

(2)自动缓冲与透明缓冲

自动缓冲指在基于消息队列的间接通信中,存放消息的缓冲区为有限或无限容量,所以可以充分利用缓冲机制,实现发送方或接收方的无阻塞通信。如果缓冲区为零容量,则发送方必须等接收方取走消息,即只能实现阻塞通信。

(3)当发送方发送一个较小的数据包时,发送方将数据拷贝至消息队列,然后接收方从消息队列中拷走,这称为拷贝发送;如果数据包较大,发送方只是把指向数据包的指针和数据包的大小发送给接收者,接收者通过指针访问数据包,这称为引用拷贝。显然引用拷贝比复制拷贝更复杂,但不需要数据拷贝,所以效率高。

(4)固定大小的消息易于管理,而可变大小的消息传递需要更复杂的数据结构。

Chapter 5 线程

5.1 举三个采用多线程比单线程具有更高的性能的程序实例。

答:(1)一个 WEB 服务器中,每一个线程服务于一个客户请求。

(2)一个并行应用程序如矩阵乘法,矩阵中的不同部分可以并行工作。

(3)一个具有交互式 GUI 的应用程序(如调试器),其中一个线程用于监视用户输入、一个线程运行程序、另一个线程进行性能监视。

5.2 举两个采用多线程比单线程不能提高性能的程序实例。

答:(1)任何一种串行执行的程序,如个人退税计算程序。

(2)Shell 程序,如 C-shell, Korn shell。

5.3 用户级线程和核心级线程的主要区别在哪里?两者在什么情况下各有优劣?

答:(1)用户级线程直接由应用程序管理,操作系统核心感知不到它的存在,而核心级线程直接由操作系统管理。

(2)用户级线程由线程库(thread library)来调度,而核心级线程由操作系统调度。

(3)核心级线程不需要绑定到一个特定的进程,而用户级线程只属于某个特定的进程。

5.4 说明内核为核心级线程的上下文切换所作的工作。

答:核心级线程间的上下文切换包括保存落选线程的 CPU 寄存器的值及恢复选中线程的 CPU 寄存器的值。

5.5 说明线程库为用户级线程间的上下文切换所作的工作。

答:用户级线程间的上下文切换与核心级线程间上下文切换类似,虽然用户级线程间切换依赖于线程库及用户级线程与核心级线程的映射关系。

总的来说,用户级线程间上下文切换指将 LWP(轻量级进程,属核心级线程)的一个用户级线程替换为另一个用户级线程,其工作包括保存落选线程的寄存器状态、恢复选中线程的寄存器状态。

5.6 一个线程被创建时需要哪些资源?与一个进程被创建时需要的资源相比有什么不同?

答:线程比进程小,创建时占用的资源也少。进程创建时,需要进程控制块(PCB)。PCB 往往是一个较大的数据结构,包括对内存映象、打开文件表、环境变量等的管理。管理内存映象表是一件很费时的事情。创建用户线程或核心线程只需要一个较小的数据结构,以存放寄存器组的值、栈以及线程优先级。

5.7 写一个多线程程序用于生成一个 fibonacci 序列,要求从键盘输入生成的 fibonacci 序列个数,然后生成一个 Pthread 线程计算 fibonacci 序列,输出到屏幕。

```
/**
   * A pthread program illustrating how to
   * create a simple thread and some of the pthread API
   * This program implements the fibonacci function where
   * the fibonacci series generation operation is run as a separate thread.
   * Usage on Solaris/Linux/Mac OS X:
   *
   * gcc thread.c -lpthread
   * a.out <number>
   **/

#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */

void *runner(void *param); /* the thread */

main(int argc, char *argv[])
{
pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of attributes for the thread */

    if (argc != 2) {
```

```
        fprintf(stderr, "usage: a.out <integer value> \n");
        exit();
      }

      if (atoi(argv[1]) < 0) {
      fprintf(stderr, "Argument %d must be non-negative \n", atoi(argv[1]));
        exit();
      }

      /* get the default attributes */
      pthread_attr_init(&attr);

      /* create the thread */
      pthread_create(&tid, &attr, runner, argv[1]);

      /* now wait for the thread to exit */
      pthread_join(tid, NULL);

      printf("sum = %d \n", sum);
}

/**
  * The thread will begin control in this function
      */
      void *runner(void *param)
      {
    int upper = atoi(param);
    int i;
    int fib1 = 1, fib2 = 1, fib3;
      printf("fibonacci series: %ld, %ld", 1, 1);
      if (upper > 0) {
        for (i = 1; i <= upper - 2; i++)
          {
            fib3 = fib1 + fib2;
            printf(" %ld", fib3);
            fib1 = fib2;
            fib2 = fib3;
          }
      }
```

```
    pthread_exit(0);…
}
```

Chapter 6 CPU 调度

6.1 CPU 调度算法决定了进程执行的顺序。若有 n 个进程需要调度，有多少种可能的调度顺序？

答：n 个就绪进程被调度，则可能的调度顺序有 n! 种。

6.2 说明抢占式调度与非抢占式调度的区别。为什么说计算中心不适合采用非抢占式调度？

答：抢占式调度允许一个正在执行中的进程暂时中止，将 CPU 交给其他进程。非抢占式调度保证一个进程在执行期不会被剥夺 CPU。

在一个计算中心，多个用户会提交作业给系统，如果采用非抢占式调度，则一个用户的作业只有等到当前占用 CPU 的作业完成后才有可能得到调度，这对用户来说是无法接受的，所以在计算中心，非抢占式调度是不合适的。

6.3 设有一组进程，它们需要占用 CPU 的时间及优先级如下所示：

进程	CPU 时间	优先级
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

假设各进程在时刻 0 按 P1、P2、P3、P4、P5 的顺序到达。

a. 画出分别采用调度算法 FCFS(先来先服务)、SJF(最短作业优先)、非抢占式优先级(nonpreemptive priority，数值小的优先级大)及 RR(时间片轮转，时间片为 1)时的调度顺序甘特图(gantt chart)。

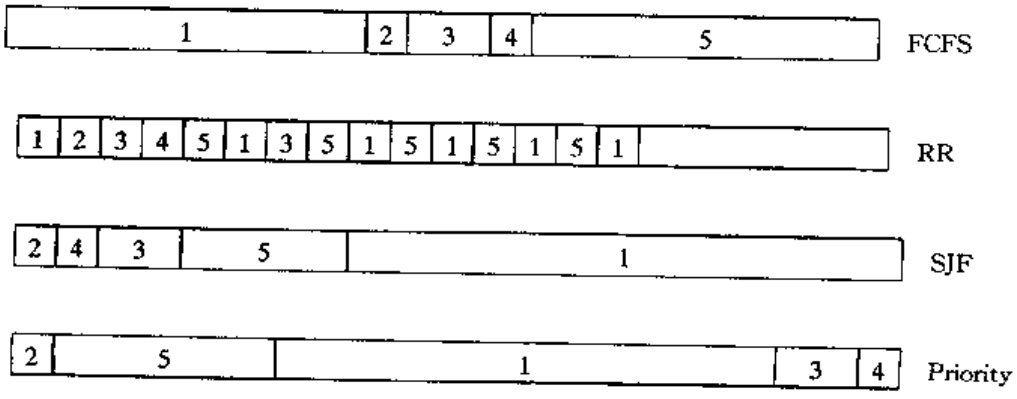
b. a 中各种调度算法下每个进程的周转时间(turnaround time)各为多少？

c. a 中各种调度算法下每个进程的等待时间(waiting time)各为多少？

d. A 中哪个调度算法可以得到最小的平均等待时间？

答：

a. 四个甘特图分别是：



b. 各进程的周转时间分别如下：

	FCFS	RR	SJF	Priority
P_1	10	19	19	16
P_2	11	2	1	1
P_3	13	7	4	18
P_4	14	4	2	19
P_5	19	14	9	6

c. 各进程的等待时间分别如下：

	FCFS	RR	SJF	Priority
P_1	0	9	9	6
P_2	10	1	0	0
P_3	11	5	2	16
P_4	13	3	1	18
P_5	14	9	4	1

d. 由 c 可知，SJF 调度算法可以获得最小的平均等待时间。

6.4 设有一组进程，它们的到达时刻和需要占用 CPU 的时间分别如下：

Process	Arrival Time	Burst Time
P_1	0.0	8
P_2	0.4	4
P_3	1.0	1

假设采用非抢占式调度策略，回答下列问题：

a. 若采用 FCFS 调度算法，各进程的平均周转时间是多少？

b. 若采用 SJF 调度算法，各进程的平均周转时间是多少？

c. SJF 算法往往能提高性能，但考虑 b 中，在 0 时刻选择 P1 是因为我们无法预知有两个更短的进程很快会到达。如果调度程序在 0 时刻等待 1 个时间单位，然后开始调度，则情况就不一样了。由于在时刻 1 之前，进程 P1 和 P2 都将等待，所以它们的等待时间会变长。我们称这种调度算法为预知调度算法。给出采用此预知调度算法时各进程的平均周转时间。

答：a. FCFS，调度顺序：0.0 − P1 − 8 − P2 − 12 − P3 − 13

平均周转时间 = (8 + 12 − 0.4 + 13 − 1.0)/3 = 31.6/3 = 10.53

b. SJF，调度顺序：0.0 − P1 − 8 − P3 − 9 − P2 − 13

平均周转时间 = (8 + 9 − 1.0 + 13 − 0.4)/3 = 28.6/3 = 9.53

c. 预知调度算法，调度顺序：0 − 1 − P3 − 2 − P2 − 6 − P1 − 14

平均周转时间 = (14 + 6 − 0.4 + 2 − 1)/ 3 = 20.6 /3 = 6.86

注意：请记住周转时间指进程执行完成时刻减去进程的到达时刻。

6.5 考虑一种 RR(时间片轮转)调度算法的变种，算法中就绪队列中存放的是各个指向进程控制块 PCB 的指针。回答下面问题：

a. 若就绪队列中有两个指针指向同一个 PCB，会产生什么现象？

b. 说明采用这种实现方法的优缺点。

c. 请你改变一种策略，能达到与 a 相同的效果，但不需要两个相同的指针。

答：a. 这相当于将那个进程的优先级提升了，因为它得到了比其他进程更多的 CPU 时间。

b. 这种实现方法的优点是可以让重要的进程获得更多的 CPU 时间，即可以很容易地提升重要进程的优先级，缺点是其他进程可能会产生饥饿现象。

c. 一种改进的策略是让就绪队列中各进程的时间片可以是不一样的，即重要进程的时间片大，一般进程的时间片小。这样，那些重要的进程虽然在就绪队列中也只出现一次，但它们获得的 CPU 时间会多些。

6.6 在多级队列调度算法中(multilevel queueing)，若每一级的时间片不一样，会有怎么的好处？

答：对那些需要交互性比较好的进程，可以使它们出现在时间片小的队列中，这样它们可以更频繁地获得 CPU 服务。相反，对那些不需要频繁交互的进程，可以使它们出现在时间片相对大的队列中，这样不会导致频繁的进程上下文切换，系统的效率也会提高。

6.7 考虑下面的基于动态优先级的抢占式进程调度算法，其中优先级数值大的表示进程的优先级高。若一个进程在等待占用 CPU(即处于就绪队列中)，则随着时间的推移，其优先级会按速率 α 变化；若进程正在执行，其优先级会按速率 β 变化。所有进程在到达就绪队列时的优先级均为 0。显然，参数 α 及 β 的取值不同，会导致不一样的调度结果。试问：

a. 若 $\beta>\alpha>0$，调度算法达到怎样的效果？

b. 若 $\alpha<\beta<0$，调度算法达到怎样的效果？

答：a. 若 $\beta>\alpha>0$，则随着时间的推移，先到达就绪队列的进程的优先级会高于后到达的进程，且正在执行的进程的优先级会大于就绪队列中进程的优先级，则当前进程会一直占用 CPU，直到结束；进程结束后，在就绪队列中，由于先到达的进程的优先级高，会被调度，这就是 FCFS(First Come, First Served)调度算法。

b. 若 $\alpha<\beta<0$，则随着时间的推移，正在执行的进程的优先级以及就绪队列中进程的优先级都会降低，但执行进程的优先级高于就绪队列进程中的优先级，而在就绪队列中，先到达的进程的优先级低于后到达进程。由于采用抢占式调度，在每个抢占点，当前进程会被新到达的进程抢占，这就是 LIFO(Last In, First Out)调度算法。

6.8 很多 CPU 调度算法都带参数。比如 RR(时间片轮转)调度算法中，有一个参数代表时间片大小；多级反馈队列调度算法的参数包括队列数、每个队列的调度策略、进程在队列间迁移的准则等。由于参数的不同，一个调度算法可能导致和其他调度算法一致的结果，如 FCFS 算法就是当时间片为无穷大时的 RR 算法，下面各组调度算法有这样的关系吗？

a. 基于优先级的调度算法与 SJF 调度算法

b. 多级反馈队列调度算法与 FCFS 调度算法

c. 基于优先级的调度算法与 FCFS 调度算法

d. RR 与 SJF

答：a. 若以进程的作业时间(需要占用 CPU 的时间)作为进程的优先级，则基于优先级的调度算法就是 SJF 调度算法；

b. 多级反馈队列调度算法中最低一级的队列(即 MLFQ)采用的算法就是 FCFS 算法；

c. 若以进程在就绪队列中逗留的时间长度作为进程的优先级，则基于优先级的调度算法就是 FCFS 调度算法；

d.两者没有关系

6.9 假设一个 CPU 调度算法会优先考虑在近期占用 CPU 时间最少的进程，分析为什么这样的调度算法有利于受 I/O 约束的进程，但又不会使受 CPU 约束的进程饥饿？

答：由于受 I/O 约束的进程占用 CPU 的时间较少，所以它们会得到优先考虑。但这不会导致受 CPU 约束的进程饥饿，因为受 I/O 约束的进程经常会放弃 CPU，等待 I/O 完成。

6.10 说明下面的调度算法在照顾短作业方面的差异：

a.FCFS

b.RR

c.多级反馈队列

答：a.FCFS 不会照顾短作业，而是照顾先到达的作业。任何在一个长作业之后到达的短作业都不会比之前的长作业优先调度；

b.RR 也不会照顾短作业，而是对所有作业一视同仁，以时间片进行轮转；

c.多级反馈队列会照顾短作业，因为一个长作业在用完当前时间片后，会进入下一调度队列，其优先级会降低。

Chapter 7 进程同步

7.1 什么叫“忙等待”(busy waiting)？操作系统中还有其他的等待吗？忙等待能完全消除吗？

答：忙等待是指在临界区问题中，若一个进程在临界区，当其他进程欲进入临界区时，反复检测是否可以进入，即忙于等待临界区中的进程离开。

在操作系统中，还有其他场合会出现这种等待，如程序控制 I/O 技术中，I/O 程序等待 I/O 操作的完成。忙等待是不可能完全消除的。

7.2 说明为什么自旋锁不适合于单处理机系统而在多处理机系统中是有用的？

答：当一个进程欲访问已被其他进程锁定的资源时，进程循环检测该锁是否被释放，这种技术称为自旋锁(spin lock)。施加自旋锁意味着进程一直占用 CPU。在单处理机系统中，如果进程循环等待其他进程释放锁，因为系统只有一个 CPU，拥有锁的进程无法得到 CPU 而释放该锁，因此，该进程应该阻塞，而不是循环等待。而在多处理机系统中，循环等待被其他进程施加的锁是可行的，因为其他进程很有可能在别的 CPU 上执行，从而很快释放该锁。

7.3 证明在下面所示的面包房算法(bakery algorithm)中，若进程 Pi 在临界区而进程 Pk(k〈〉i)已经求得了 number[k]，且 number[k]〈〉0，则

(number[i],i) 〈 (number[k],k)。

面包房算法：

对于进程 i：

do {

```
choosing[i] = true;
number[i] = max( number[0], number[1],...,number[n-1]) + 1;
choosing[i] = false;
for ( j = 0 ; j < n ; j + + ) {
    while ( choosing[j]);
```

```
        while (( number[j] ! = 0 )&&((number[j],j) \ (number[i],i)));
    }
    〈 critical section〉
    number[i] = 0;
} while (1);
```

证：

考虑两种情形：

(1)Pk 求解 number[k]时，Pi 已经在临界区，则 number[k] 〉 number[i]，所以(number[i],i) 〈 (number[k],k)满足；

(2)Pk 求解 number[k]时，Pi 未在临界区，则有两种可能：

a.Pi 未执行 for 循环当 j= =k 时的循环体，则如果(number[i],i) 〈 (number[k],k)不满足，因为 number[k]〈〉0，所以进程 Pi 会止步于 for 循环体中的 j=k 时的第二个 while 循环。既然 Pi 能进入临界区，则上式一定满足。

b.Pi 已执行 for 循环当 j= =k 时的循环体，则 number[k] 〉 number[i]，所以(number[i],i) 〈 (number[k],k)满足。

7.4 第一个公认的解决两进程(线程)临界区问题的算法是由 Dekker 提出的，故称为 Dekker 算法。算法如下所示。说明该算法满足临界区问题的三个必要条件。

Dekker 算法：

共享变量：

```
int turn, 初值为 0;
Boolean flag[2], 初值均为 false;
Pi(i=0,1):
while(1) {
    flag[ i ] = true;
    while (flag[j]) {
        if (turn= =j) {
        flag[ i ] = false;
        while ( turn = = j );
        flag[ i ] = true;
        }
    }
    CS
    turn = j;
    flag[ i ] = false;
    RS
}
```

答：

1.互斥(mutual exclusion)

A) Pi 和 Pj 不可能同时进入 CS：

turn= =j 和 turn= =i 不会同时满足。所以，当两进程都想进入 CS 时，只有编号为 turn

的进程能进入。

B)一个进程(如 Pi)先进入 CS,则 flag[i]为 true,则 Pj 无法再进入。

2.前进(progress)

A)Pi 申请进入,Pj 不想进入,则 flag[j] = = false,Pi 顺利进入 B)Pi 和 Pj 都想进入,则 turn = = j 时 Pj 进入,turn = = i 时 Pi 进入

3.有限等待(bounded waiting)

Pi 离开 CS 后,置 turn = j, flag[i] = false,这时 Pj 可以进入.如果 Pj 没有进入,Pi 又想进入,即执行了 flag[i] = true,则由于 turn = = j,所以 Pi 无法进入,而 Pj 可以进入。即满足先来先服务,故不会导致无限等待。

7.5 第一个公认的 n 进程临界区问题的解决方法是由 Eisenberg 和 McGuire 提出的。该算法中,n 个进程共享下面的变量:

```
enum pstate { idle, want_in, in_cs };
pstate flag[n];
int turn;
```

算法描述如下:

```
for process i ;

While(1) {
    while(1) {
      flag[i] = want_in;
      j = turn;
      while (j ! = i) {
        if (flag[j]! = idle) j = turn;
        else j = (j+1) % n;
      }
      flag[i] = in_cs; j = 0;
      while ((j<n)&&(j = = i || flag[j]! = in_cs)
          j + + ;
      if ((j) = n)&&(turn = = i||flag[turn] = = idle))
          break;
    }
    turn = i ;          CS
    j = (turn+1) % n;
    while(flag[j] = = idle) j=(j+1) % n;
    turn=j; flag[i]=idle;          RS
}
```

证明该算法满足临界区问题的所有要求。

证:

1. 互斥(mutual exclusion)

A) 两个进程 Px 和 Py 不可能同时进入 CS:

turn = = x 和 turn = = y 不会同时满足,故 Px 和 Py 的两个 if 条件不会都满足

B) 一个进程(如 Px)先进入 CS,则 turn = = x, flag[turn] = = in_cs,则其他进程无法进入

2. 前进(progress)

A)Px 申请进入,其他进程不想进入,则其他进程 flag 均为 idle,则 Px 的 j> = n 成立,if 条件成立,跳出 while,进入 CS

B)若干进程想进入,如 Pturn 也想进入,则 Pturn 优先进入;否则,编号在 turn 后的最小号的想进入的进程进入 CS

3. 有限等待(bounded waiting)

Pi 离开 CS 时,置 turn 为想进入的进程中大于 turn 的最小欲进入之进程号,该进程就进入。进程 Pk 不会饿死,最多等待(k-turn)个进程的 CS 执行

7.6 如果经常屏蔽中断,则可能会影响系统时钟,为什么? 如何才能尽量降低这种影响?

答:如果经常屏蔽中断,则时钟中断可能没法得到及时响应,显然系统时钟会受影响。所以应该尽量缩短屏蔽中断的时间,这样,即使偶尔几次时钟中断得不到及时响应,但这种现象只存在很短时间,系统时钟很快会得到修正。

7.7 说明如果 wait 和 signal 操作不是原子操作,就没法实现互斥。

答:如果 wait 和 signal 不是原子操作,则对信号量的 counter 分量的操作也就不是原子操作,这样无法实现正确的互斥。

7.8 理发问题:一个理发店内有一张沙发,可以坐 n 个等待理发的顾客,理发室内有一张理发椅。当没有顾客时,理发师就休息;当一个新顾客进来时,如果所有的座位上都有人,他就离开理发店;如果沙发上有空位,他就坐下;如果理发师在休息,顾客就唤醒他让他为其理发。试用信号量来同步理发师和顾客的行为。

```
Semaphore max;          //初始 n+1,表示理发店可以容纳总人数
Semaphore chair;        //初始 n,空闲的椅子
Semaphore barber;       //初始 1,表示理发椅空闲
Semaphore finished;      //初始 0,表示一次理发结束
Semaphore ready; //初始 0,表示客人准备就绪
Customer:
    While(1){
    wait(max);
    wait(chair);
    wait(barber);
    signal(chair);
    signal(ready);
    ...barbered ...
    wait(finished);
    signal(max);
}
```

```
Barber:
While(1){
    wait(ready);
    ...barbering...
    signal(finished);
    signal(barber);
}
```

7.9 抽烟问题:有一个烟草代理和三个抽烟者。抽烟者若要抽烟,必须具有烟叶、烟纸和火柴。三个抽烟者中,一人缺烟叶、一人缺烟纸、一人缺火柴。烟草代理会源源不断地分别供应烟叶、烟纸和火柴,并将它们放在桌上。如果他放的是烟叶,则缺烟叶的抽烟者会拾起烟叶,制作香烟,然后抽烟;其他类推。试用信号量同步烟草代理和三个抽烟者。

```
Semaphore smoker[3];          //初始 0,三个抽烟者
Semaphore material[3];        //初始 0,三种原料
Semaphore agent;              //初始 1,供应商
Int turn;                     //初始 0,轮到谁

Agent:
While(1){
    wait(agent);
    signal(smoker[turn]);
    signal(material[(turn+1)%3]);
    signal(material[(turn+2)%3]);
    turn= (turn+1)%3;
}

Smoker-i:
While(1){
    wait(smoker[i]);
    wait(material[(i+1)%3]);
    wait(material[(i+2)%3]);
    signal(agent);
}
```

7.11 写一个管程(monitor)用于实现有限缓冲区的生产者-消费者问题,要求将有限缓冲区置于管程内。

```
Monitor bounded_buffer
{
    item pool[n];
    int count, in, out;
```

```
    condition full, empty;
    void get_item()
    {
        if (count = =0) full.wait();
        get_from_buffer();
        count - - ;
          empty.signal();
    }
    void put_item()
    {
        if (count = =n) empty.wait();
        put_to_buffer();
        count + + ;
        full.signal();
    }
    void init() { count = in = out = 0;}
}

Producer process:
While(1) {
    produce an item;
    bounded_buffer.put_item();
}

Consumer process:
While (1) {
    bounded_buffer.get_item();
    consume the item;
}
```

7.12 为什么 Solaris 2 和 Windows NT 都实现了多种加锁机制。说明在什么情况下会分别采用旋转锁(spinlocks)，互斥量(mutexes)，信号量(semaphores)，自适应互斥量(adaptive mutexes)以及条件变量(condition variables)来实现加锁机制。

答：这些操作系统提供了多种加锁机制让应用程序开发者使用。旋转锁用于在多处理机系统中的加锁，施加旋转锁的线程会在短时间内处于循环检测状态，而不是进入等待队列；互斥量用于对资源加锁。Solaris2 利用自适应互斥量来实现多处理机环境下的旋转锁；利用信号量和条件变量实现进程间的同步更高效，因为往往需要较长时间占用某些资源，而若采用旋转锁则效率不高。

Chapter 8 死锁问题

8.1 试举例说出三种与计算机系统环境无关的死锁现象。

答:a.两辆汽车相向行驶欲通过一座只有单个车道的桥梁;

b.一人从楼梯上方往下走,另一人从楼梯下方往上走,而楼梯只能过一人;

c.两列车相向行驶,欲通过单个轨道。

8.2 是否可能存在一种死锁现象,只涉及单个进程?

答:不可能,因为死锁的一个必要条件是保持等待,即涉及死锁的每个进程都占用某些资源,同时又在等待其他进程所占用的资源。

8.3 考虑图 2.31 的交通阻塞死锁现象,回答下面问题:

a.说明在此情况下,发生死锁的四个必要条件都满足;

b.试增加一个简单的约束,以排除死锁现象。

答:

a.图 2.31 中,将每一个车队看成一个进程,每一个十字路口看作一个资源。则该系统处于死锁状态,因为:

1)满足互斥:每个十字路口是临界资源,一个车队占用了一个路口,其他车队就无法占用该车队,即对它们的访问是互斥的;

2)满足保持等待:每个车队都占用了一个路口,又在等待被前方车队占用的路口;

3)满足非剥夺:如果一个车队占用了一个路口,另一车队无法抢占这个路口;

4)循环等待:显然系统的资源分配图中存在一个回路。

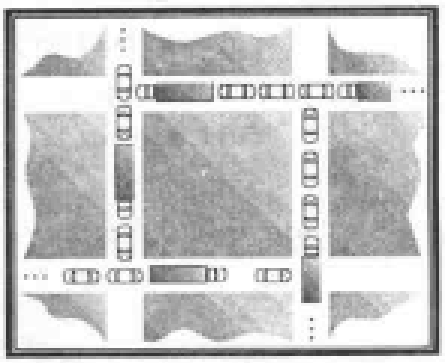


图 2.31

b.增加约束:只有前方两个路口都空闲时,才能占用第一个路口。

8.4 假设系统处于非安全(unsafe)状态,证明系统中的进程仍然有可能执行完,而系统不会进入死锁状态。

答:非安全状态表示系统中找不到一个安全序列,系统中所有的进程能按此序列顺序执行完毕。但实际上,系统中的进程不一定非要按顺序执行完。所以,不安全状态并不表示系统一定会进入死锁状态。

8.5 略

8.6 略

8.7 略

8.8 设一系统有相同类型的 4 个资源被 3 个进程共享,每个进程最多会申请 2 个资源,证明该系统不会发生死锁。

答:如果系统发生死锁,必然是系统中的 3 个进程都占用了 1 个资源,然后申请另外 1 个资源。因为系统中共有 4 个资源,必然会有一个进程得到 2 个资源,则这个进程不再需要更多的资源,它必然会执行完,然后释放所有的资源。这样就不会导致死锁。

8.9 设一系统有 m 个相同类型的资源被 n 个进程共享。每个进程每次只能申请或释放一个资源。证明如果下面两条件满足,则系统不会发生死锁:

a.每个进程最多申请的资源数在 1 到 m 之间

b.所有进程最多申请的资源总数小于 m + n

证：

记每个进程最大申请资源数为 Max_i，已分配资源数为 $Allocation_i$，尚需分配的资源数为 $Need_i$。

则：

a. $\sum_{i=1}^{n} Max_i < m + n$

b. $\sum_{i}^{n} Allocatin_i = m$,

假设系统发生了死锁，则：

c. $\forall i, Max_i \geqslant 1$

由 a 和 b 可得：

$\sum Need_i + \sum Allocation_i = \sum Max_i < m + n$

$\sum_{i=1}^{n} Need_i < n$

则必然存在一个进程，它尚需要的资源数 Needi = 0。由 c 可知，此进程已分配的资源数 Allocationi〉0。显然该进程能顺利执行完并释放它所占用的资源。这样系统就不会发生死锁。

8.10 一个系统能测试得到哪些进程是饥饿的吗？如果是，请解释如何测试？如果不是，请说明系统如何处理饥饿问题？

答：可以在进程的 PCB 中设置一个记录进程最后一次被调度的时间的数据成员，当调度程序选中该进程时，就将当前时刻记录在该数据成员中。系统中的记账管理部件可以定期检索每一个就绪进程的最后一次调度时刻，如果该时刻离当前时刻超过了系统预先设定的一个阈值，我们可以认为该进程处于饥饿状态，但显然这种方法的代价是很高的。

8.11 考虑下面的资源分配策略：资源的申请和释放可以在任意时刻进行。如果一个进程请求的资源得不到满足，则系统检查那些处于阻塞状态的进程，如果它们占有了该进程请求的资源，则释放这些资源，分配给请求进程，且释放资源的进程的资源请求向量作相应的增加。

例如，一个系统有三类资源，初始可用资源向量为(4,2,2)。如果进程 P0 请求资源(2,2,1)，它会得到相应的资源。如果 P1 请求(1,0,1)，也会得到资源。如果 P0 再请求资源(0,0,1)，它将被阻塞，因为无可用资源。但如果随后进程 P2 请求资源(2,0,0)，它会得到可用的资源(1,0,0)，然后从进程 P0 那里得到资源(1,0,0)，因为这时 P0 处于阻塞状态。P0 的已分配资源向量变为(1,2,1)，而其请求资源向量变为(1,0,1)。问：

a.这种资源分配策略会导致死锁吗？如果会，举例说明。如果不会，指出它不满足死锁的哪个必要条件？

b.这种资源分配策略会导致无限期的等待吗？说明你的理由。

答：a.不会导致死锁，因为该策略允许资源剥夺。

b.会导致无限期等待，因为可能存在某些进程，它们所占用的资源会被其他进程反复地剥夺。

8.12　略

8.13 设有一系统在某时刻的资源分配情况如下：

进程号	已分配资源	最大请求资源	剩余资源
	A B C D	A B C D	A B C D
P0	0 0 1 2	0 0 1 2	1 5 2 0
P1	1 0 0 0	1 7 5 0	
P2	1 3 5 4	2 3 5 6	
P3	0 6 3 2	0 6 5 2	
P4	0 0 1 4	0 6 5 6	

问题：

a. 系统中各进程尚需资源数各是多少?

b. 当前系统安全(safe)吗?

c. 如果此时进程 P1 提出资源请求(0,4,2,0),系统能分配给它吗?

答：

a. 尚需资源数矩阵如下：

A B C D

0 0 0 0

0 7 5 0

1 0 0 2

0 0 2 0

0 6 4 2

b. 系统是安全的,因为可以找到一个安全序列:P0, P2, P1, P3, P4

c. 如 P1 申请(0,4,2,0),新的状态为

	Allocation	Max	Need	Available
P0	0 0 1 2	0 0 1 2	0 0 0 0	1 1 0 0
P1	1 4 2 0	1 7 5 0	0 3 3 0	
P2	1 3 5 4	2 3 5 6	1 0 0 2	
P3	0 6 3 2	0 6 5 2	0 0 2 0	
P4	0 0 1 4	0 6 5 6	0 6 4 2	

该状态是安全的,存在安全序列如 P0,P2,P3,P1,P4,所以可以分配资源给 P1。

第3章

存储管理

3.1 概述

计算机系统的主要目的是为了执行程序。这些执行中的程序以及它们访问的数据都是存在主存储器中的。

为了提高 CPU 的使用率，加快对用户的反应速度，计算机必须使多个进程同时存在于内存中。目前，存在许多存储管理的方案，每种方案的有效性取决于具体的应用环境。究竟选择哪一种方案则需要考虑很多因素，尤其是系统的硬件设计。每一种方案都需要相应的硬件支持。

既然主存储器对于永久地保存所有数据和程序而言显得太小，计算机系统就需要提供第二级的存储设备来备份主存储器。现代计算机系统采用磁盘作为主要的信息存储媒介(包括程序和数据)。文件系统提供了存储和访问磁盘上的数据和程序的机制。文件是其创建者所定义的相关信息的集合。文件被操作系统映射到具体的物理设备上。为了方便使用，文件一般按照目录的方式组织起来。

3.2 内存管理

用于多道程序设计的操作系统的内存管理算法，包括从简单的单用户系统的方法到分段且分页的方法。一个特定系统所采用方法的最大决定因素是它所能提供的硬件支持。CPU 所产生的每个内存地址都必须先进行合法性检查，才可映射成为物理地址。检查是不能通过软件来(有效地)实现的。因此，我们受到可用硬件的约束。

各种内存管理算法(连续分配、分页、分段、及分段加分页)在许多方面都呈现出不同的特点。在比较不同的内存管理策略时，需要考虑如下几点：

- 硬件支持。对单分区和多分区方案，只需要一个基址寄存器或一对基址和界限寄存器就够了；而对于分页和分段，需要映射表以定义地址映射。
- 性能。随着内存管理算法变得越来越复杂，逻辑地址到物理地址的映射所需要的时间也更长。对于简单系统，我们只需要对逻辑地址进行比较和加减操作(这些较为简单)。如果表能通过快速寄存器来加以实现，那么分页和分段操作也会很快。不过，如果表在内存中，那么访问内存的速度就大大地降级。TLB(Translation Look-aside Buffer)可以用来改善性能。
- 碎片。如果多道程序的级别很高，那么多道程序系统通常会更有效地执行。对于给定的一组进程，我们可以通过将更多进程装入内存以增加多道程序的程度。为了完成这一任务，

我们必须降低内存浪费或碎片。采用固定大小分配单元(如单个分区和分页)的系统会有内部碎片问题。采用可变大小分配单元(如多个分区和分段)的系统会有外部碎片问题。

● 重定位。外部碎片问题的解决方案之一是紧缩(Compaction)。紧缩涉及到在内存中移动程序而不影响程序。这种方案要求在执行时逻辑地址能动态地进行重定位。如果地址只能在装入时进行重定位,那么就不能采用紧缩。

● 交换。任何算法都可加上交换操作。进程可以定时地(由操作系统来定,通常由 CPU 调度策略来决定)从内存交换到外存,之后再交换到内存。这种方法能允许更多进程(它们不能同时装入内存)并发运行。

● 共享。另一个增加多道程序程度的方法是让不同用户共享代码和数据。共享通常要求分页或分段,以便共享较小的信息区域。共享在有限内存情况下能容纳更多的进程,但是共享的程序和数据需要仔细设计。

● 保护。如果提供了分页或分段,那么用户程序的不同区域可以声明为只可执行、只读或可读可写。这种限制对于共享代码和数据是必要的,对于常见程序设计错误能提供简单的运行时的检查。

3.3 虚拟内存

很可能需要执行这样一个进程,其逻辑地址空间大于可用物理地址空间。程序可通过覆盖技术来执行这样的进程,但是这通常是个困难的编程任务。而虚拟内存允许将大逻辑地址空间映射到小的物理内存。虚拟内存允许极大的进程运行,且提高了多道程序的程度,增加了 CPU 使用率。再者,它使得程序员不必考虑内存的可用性。

纯粹的请求页面调度只有在引用页时才调入页。第一次引用就会产生页错误。操作系统检查内部表以确定该页在备份存储上的位置。接着,它找到空闲帧并从备份存储中读入页。更新页表以反映这一修改,并重启产生页错误的指令。这种方法允许一个进程运行,即使其完整内存映像不能同时在内存中。只要缺页率足够低,那么性能就可接受。

我们使用请求页面调度以降低分配给进程的帧数。这种安排能增加多道程序的程度(允许更多进程同时能执行),且(至少从理论上说)增加了系统 CPU 的使用率。即使进程内存需求超过了总的物理内存,也能允许进程执行。这些进程可在虚拟内存中执行。

如果总的内存需求超过了物理内存,那么有可能必须置换内存中的页以便为新页所用。我们可以使用各种页置换算法。FIFO(First In First Out,先进先出)页置换算法容易编程,但有 Belady 异常。最优页置换算法需要预知将来。LRU(Least-Recent-Used,最近最少使用)页置换算法能近似最优算法,但是它也很难实现。绝大多数页置换算法如二次机会算法,都是 LRU 置换的近似。

除了页置换算法,还需要帧分配策略。分配可以是固定的,如局部页置换算法,或动态的如全局置换。工作集模型假定进程在局部内执行。工作集是位于当前局部所有页的集合。相应地,每个进程应该为其当前工作集分配足够多的帧。

如果进程没有足够多的内存用于其工作集,那么它就会颠簸(Thrashing)。为进程提供足够多的内存以避免颠簸可能需要进程交换和调度。

除了要求我们解决页置换和帧分配的主要问题外,合理设计调页系统还要求我们考虑页大小、I/O、加锁、预约式页面调度、进程创建、程序结构、颠簸和其他问题。虚拟内存可作为计算机系统存储层次的一层。每层都有其自己的访问时间、大小和代价参数。

3.4 习题与解答

一、单项选择题

1. 测得某个采用按需调页(Demand-paging)策略的计算机系统部分状态数据为:CPU 利用率 20%,用于对换空间的硬盘利用率 97.7%,其他设备的利用率 5%。由此断定系统出现异常。此种情况下_____能提高利用率。(浙江大学 1998 年研究生试题)

A. 安装一个更快的硬盘　　B. 通过扩大硬盘容量增加对换空间

C. 增加运行进程数　　D. 加内存条来增加物理空间容量

答:D

分析:利用率 = 忙时间/(忙时间 + 空时间)。对换空间的磁盘利用率已达 97.7%,说明交换操作非常频繁,故断定物理内存严重短缺。

2. 存储管理方案中,_____可采用覆盖技术。(北方交通大学 2000 年试题)

A. 单一连续区存储管理　　B. 可变分区存储管理

C. 段式存储管理　　D. 段页式存储管理

答:A

分析:覆盖技术是早期在单一连续区存储管理中使用的扩大存储容量的一种技术。

3. 页式虚拟存储管理的主要特点是_____。(西安电子科技大学 2000 年研究生试题)

A. 不要求将作业装入到主存的连续区域

B. 不要求将作业同时全部装入到主存的连续区域

C. 不要求进行缺页中断处理

D. 不要求继续页面置换

答:B

分析:页式存储管理的特点是不要求作业装入到内存的连续区域,而页式虚拟管理的特点是不要求作业同时全部装入到内存的连续区域。

4. 在可变式分区分配方案中,某一作业完成后,系统收回其主存空间并与相邻空闲区合并,为此需修改空闲区表,造成空闲区数减 1 的情况是_____。(西安电子科技大学 2002 年研究生试题)

A. 无上邻空闲区也无下邻空闲区

B. 有上邻空闲区但无下邻空闲区

C. 有下邻空闲区但无上邻空闲区

D. 有上邻空闲区也有下邻空闲区

答:D

分析:将上邻空闲区、下邻空闲区和回收区合并为一个空闲区,因此空闲区数减少了一个。

5. 为使虚存系统有效地发挥其预期的作用,所运行的程序应具有的特性是_____。(中科院计算所 1999 年研究生试题)

A. 该程序不应含有过多的 I/O 操作

B. 该程序的大小不应超过实际的内存容量

C. 该程序应具有较好的局部性(Locality)

D. 该程序的指令相关不应过多

答:C

分析:虚拟存储器是基于程序的局部性原理的。

6. 某虚拟存储器系统采用页式内存管理,使用 LRU 页面替换算法,考虑下面的页面访问地址流(每次访问在一个时间单位中完成):

1 8 1 7 8 2 7 2 1 8 3 8 2 1 3 1 7 1 3 7

假定内存容量为 4 个页面,开始时是空的,则页面失效次数是_____。

(中科院计算所 1999 年研究生试题)

A. 4　　B. 5　　C. 6　　D. 7

答:B

分析:见表 3-1

表 3-1

页面走向	1	8	1	7	8	2	7	2	1	8	3	8	2	1	3	1	7	1	3	7
4 内存块	1	8	1	7	8	2	7	2	1	8	3	8	2	1	3	1	7	1	3	7
		1	8	1	7	8	2	7	2	1	8	3	8	2	1	3	1	7	1	3
				8	1	7	8	8	7	2	1	1	3	8	2	2	3	3	7	1
						1	1	1	8	7	2	2	1	3	8	8	2	2	2	2
缺页	缺	缺		缺							缺						缺			

7. 采用_____不会产生内部碎片。

A. 分页式存储管理　　B. 分段式存储管理

C. 固定分区式存储管理　　D. 段页式存储管理

答:B

8. 把作业地址空间中使用的逻辑地址变成内存中物理地址称为_____。

A. 加载　　B. 地址映射

C. 物理化　　D. 逻辑化

答:B

9. 联想存储器在计算机系统中是用于_____的。

A. 存储文件信息　　B. 与主存交换信息

C. 地址变换　　D. 存储通道程序

答:C

10. 在下列存储管理方案中,不适应于多道程序设计的是_____。

A. 单一连续区分配　　B. 固定式分区分配

C. 可变式分区分配　　D. 段页式存储管理

答:A

11. 下面哪种内存管理方法有利于程序的动态链接?_____

A. 分段存储管理　　B. 分页存储管理

C. 可变式分区管理　　D. 固定式分区管理

答:A

12. 在存储管理方案中,_____可与覆盖技术配合。

A. 页式管理　　B. 段式管理

C. 段页式管理　　D. 可变分区管理

答：D

13. 在存储管理中，采用覆盖与交换技术的目的是_____。

A. 节省主存空间　　B. 物理上扩充主存容量

C. 提高 CPU 效率　　D. 实现主存共享

答：A

14. 虚拟存储器的最大容量_____。

A. 为内外存容量之和　　B. 由计算机的地址结构决定

C. 是任意的　　D. 由作业的地址空间决定

答：B

15. 在虚拟存储系统中，若进程在内存中占 3 块(开始时为空)，采用先进先出页面淘汰算法，当执行访问页号序列为 1,2,3,4,1,2,5,1,2,3,4,5,6 时，将产生_____次缺页中断。

A. 7　　B. 8　　C. 9　　D. 10

答：D

16. 设内存的分配情况如图 3.1 所示。若要申请一块 40K 字节的内存空间，若采用最佳适应算法，则所得到的分区首址为_____。

A. 100K　　B. 190K　　C. 330K　　D. 410K

答：C

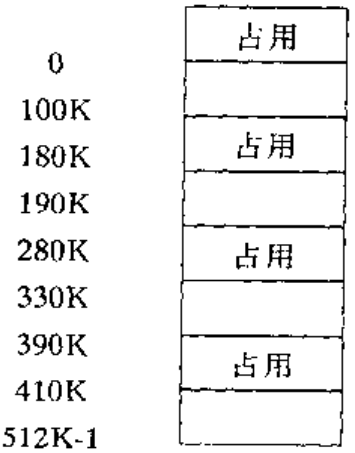


图 3.1　内存分配情况

17. 较好地解决了“零头”问题的存储管理方法是_____。

A. 页式存储管理　　B. 段式存储管理

C. 多重分区管理　　D. 可变式分区管理

答：A

18. 对操作系统而言，系统“抖动”现象的发生是由_____引起的。

A. 置换算法选择不当　　B. 交换的信息量过大

C. 内存容量不足　　D. 请求页式管理方案

答：A

19. 在可变式分区存储管理中的拼接技术可以_____。

A. 集中空闲区　　B. 增加主存容量

C. 缩短访问周期　　D. 加速地址转换

答：A

20. 在固定分区分配中，每个分区的大小是_____。

A. 相同　　B. 随作业长度变化

C. 可以不同但预先固定　　D. 可以不同但根据作业长度固定

答：C

21. 实现虚拟存储器的目的是_____。

A. 实现存储保护　　B. 实现程序浮动

C. 扩充辅存容量　　D. 扩充主存容量

答：D

22. 采用段式存储管理的系统中，若地址用 24 位表示，其中 8 位表示段号，则允许每段的最大长度是_____。

A. 2^{24}　　B. 2^{16}　　C. 2^{8}　　D. 2^{32}

答：B

23. 作业在执行中发生了缺页中断，经操作系统处理后，应让其执行_____ 指令。

A. 被中断的前一条　　B. 被中断的

C. 被中断的后一条　　D. 启动时的第一条

答：B

24. 首次适应算法的空闲区是_____。

A. 按地址递增顺序连在一起　　B. 始端指针表指向最大空闲区

C. 按大小递增顺序连在一起　　D. 寻找从最大空闲区开始

答：A

25. 在分页系统环境下，程序员编制的程序，其地址空间是连续的，分页是由_____完成的。

A. 程序员　　B. 编译地址

C. 用户　　D. 系统

答：D

26. 在请求分页存储管理中，若采用 FIFO 页面淘汰算法，则当可供分配的页帧数增加时，缺页中断的次数_____。

A. 减少　　B. 增加

C. 无影响　　D. 可能增加也可能减少

答：D

27. 在段页式存储管理系统中，内存等分成_____，程序按逻辑模块划分成若干_____。

A. 块　　B. 基址　　C. 分区　　D. 段　　E. 页号　　F. 段长

答：A D

28. 虚拟存储管理系统的基础是程序的_____理论。

A. 局部性　　B. 全局性

C. 动态性　　D. 虚拟性

答：A

29. 下述_____页面淘汰算法会产生 Belady 现象。

A. 先进先出　　B. 最近最少使用

C. 最不经常使用　　D. 最佳

答：A

30. 某段表的内容如表 3-2 所示：

表 3-2

段号	段首址	段长度
0	120K	40K
1	760K	30K
2	480K	20K
3	370K	20K

一逻辑地址为(2154)，它对应的物理地址为_____。

A. 120K + 2　　B. 480K + 154

C. 30K + 154　　D. 2 + 480K

答：B

31. 在一页式存储管理系统中，页表内容如表 3-3 所示：

表 3-3

页号	块号
0	2
1	1
3	3
4	7

若页的大小为 4K，则地址转换机构将逻辑地址 0 转换成的物理地址为_____。

A. 8192　　B. 4096　　C. 2048　　D. 1024

答：A

32. 如果一个程序为多个进程所共享，那么该程序的代码在执行的过程中不能被修改，即程序应该是_____。

A. 可执行码　　B. 可重入码

C. 可改变码　　D. 可再现码

答：B

二、选择题

1. 支持程序放在不连续内存中存储管理方法有_____。（西安电子科技大学 2001 年研究生试题）

A. 可变式分区分配　　B. 多重分区分配

C. 分页式分配、段页式分配　　D. 分段式分配

答：①C　②D

分析：分段式可将主程序、子程序、数据等分放在不连续空间的区段，但对一个程序而言它只能放在一个连续空间中。

2. 一进程刚获得三个主存块的使用权，若该进程访问页面的次序是{1, 3, 2, 1, 2, 1, 5, 1, 2, 3}，当采用先进先出调度算法时，发生缺页次数是 ① 次，而采用 LRU 算法，缺页数是 ② 。（西安电子科技大学 2001 年研究生试题）

A.1 B.3 C.4 D.5 E.6

答:①E ②D

3.使用_____方法可以实现虚存。(西安电子科技大学 2001 年研究生试题)

A.分区靠拢 B.覆盖、交换

C.联想寄存器 D.段靠拢

答:B

分析:虚拟存储器扩充内存的基本方法是将一些页或段从内存中调入、调出,而调入、调出的基本手段则是覆盖与交换。

4.在下列存储管理方案中,能实现虚存的有_____。(陕西省 1998 年自考题)

A.分区分配 B.分页存储管理

C.请求分页存储管理 D.分段存储管理

答:C

5.虚拟存储器的物质基础是_____。(陕西省 1998 年自考题)

A.大容量的辅存、一定容量的主存、地址变换机构

B.高速缓存

C.联想存储器

答:C

6.若存储单元长度为 n,该块存储的作业长度为 m,则剩下长度为 n-m 的空间称为该单元的内零头。下面存储分配方法中,_____存在内零头。

A.固定式分区、页式管理、段页式管理 B.可变式分区

C.动态分区 D.段式管理

答:A

分析:固定式分区既存在内零头也存在外零头,而凡涉及到页的都有内零头。

7.在下面关于存储功能的论述中,正确的是_____。

A.即使在多道程序环境下用户也可以编制用物理地址直接访问内存的程序

B.内存分配的基本任务是为每道程序分配内存空间,其追求的目的则是提高内存利用率

C.为提高内存保护的灵活性,内存保护通常由软件完成

D.地址映射是指将程序中逻辑地址转变为内存空间的物理地址

答:B

8.下面关于虚拟存储器的论述中,正确的是_____。

A.在段页式系统中以段为单位管理用户的逻辑空间,以页为单位管理内存的物理空间;有了虚拟存储器才允许用户使用比内存更大的地址空间

B.为了提高请求分页系统中内存的利用率允许用户使用不同大小的页面

C.为了能让更多的作业同时运行,通常只装入 10%~30%的作业即启动运行

D.最佳适应算法是实现虚拟存储器的常用算法

答:A

三、填空题

1.在采用请求分页式存储管理的系统中,地址变换过程可能会因为_____、_____和_____等原因而产生中断。(中国科技大学 1998 年研究生试题)

答:地址长度越界 缺页 访问权限错误

2. 在分区分配算法中，首次适应算法倾向于优先利用内存中_____部分的空闲分区，从而保留了_____部分的大空闲区。（中科院计算所 1999 年研究生试题）

答：低地址　高地址

3. 把作业装入内存中随即进行地址变换的方式称为_____，而在作业执行期间，当访问到指令或数据时才进行地址变换的方式称为_____。（西安电子科技大学 2000 年研究生试题）

答：静态重定位　动态重定位

4. 存储管理应实现的功能是：主存空间的分配与保护、_____、主存空间的共享与_____。（西安电子科技大学 2002 年研究生试题）

答：主存空间地址重定位　主存的扩充

5. 分区存储管理方案不能实现虚拟的原因是_____。（西安交通大学 1999 年研究生试题）

答：作业地址空间不能超过存储空间

6. 设有 8 页的逻辑空间，每页有 1024 字节，它们被映射到 32 块的物理存储区中。那么，逻辑地址的有效位是_____位，物理地址至少是_____位。（西北工业大学 2000 年研究生试题）

答：13，15

7. 设一段表为（见表 3-4）

表 3-4

段号	基地址	段长
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

那么，逻辑地址（2，88）对应的物理地址是_____。逻辑地址（4，100）对应的物理地址是_____。（西北工业大学 2000 年研究生试题）

答：178　超过段长而产生越界中断

8. 虚拟存储器通常由_____和_____两级存储系统组成。为了在一台特定的机器上执行程序，必须把_____映射到这台机器主存储器的_____空间上，这个过程称为_____。

答：主存　辅存　逻辑地址　物理地址　地址变换

9. 静态重定位是由专门设计的_____完成的，而动态重定位是靠_____来实现的。

答：重定位装配程序　硬件地址变换机构

10. 在段式存储管理中，段的保护措施通常有_____和_____两种。

答：越界保护　存取控制

11. 在可变分区存储管理中，分区的保护通常采用_____和_____两种方式。（中国科技大学 1998 年研究生试题）

答：界地址　存储键

12. 多道程序设计的引入给存储管理提出了新的课题，应考虑的三个问题是_____、

_____和_____。(西安电子科技大学 2000 年研究生试题)

答:存储器分配　虚存管理　存储保护

13. 在存储管理方案中,可用上、下限地址寄存器存储保护的是_____。(西安电子科技大学 2000 年研究生试题)

答:分区分配

14. 采用交换技术获得的好处是以牺牲_____为代价的。(西安交通大学 1999 年研究生试题)

答:CPU 时间

15. 用户编程时使用_____地址,处理机执行程序时使用_____地址。(西安理工大学 2001 年研究生试题)

答:逻辑　物理

16. 在请求分页系统中,反复进行"入页"和"出页"的现象称为_____。(陕西省 1997 年自考题)

答:系统抖动

17. 地址再定位的两种方式是_____和_____。(陕西省 1997 年自考题)

答:静态重定位　动态重定位

18. 在页式和段式管理中,指令的地址部分其结构形式分别为_____和_____。

答:页号及页内位移　段号及段内位移

19. 将作业地址空间中的逻辑地址转换为主存中的物理地址的过程称为_____。

答:地址变换

20. 在请求页式管理中,页面置换算法常用的是_____和_____。

答:先进先出　最近最久未使用

21. 段表表目的主要内容包括_____。

答:段号、段在内存的起始地址、段长度

22. 假设某程序的页面访问序列为 1,2,3,4,5,2,3,1,2,3,4,5,1,2,3,4 且开始执行时主存中没有页面,则在分配给该程序的物理块数是 3 且采用 FIFO 方式时缺页次数是_____;在分配给该程序的物理块数是 4 且采用 FIFO 方式时,缺页次数是_____。在分配给该程序的物理块数是 3 且采用 LRU 方式时,缺页次数是_____。在分配给该程序的物理块数为 4 且采用 LRU 方式时,缺页次数是_____。

答:13　14　14　12

23. 分区管理中采用"首次适应"分配算法时,应将空闲区按_____次序登记在空闲区表中。

答:地址递增

24. 页表表目的主要内容包括_____。

答:页号和块号

25. 主存中一系列物理存储单元的集合称为_____。

答:物理存储空间

26. 静态重定位在_____时进行;而动态重定位在_____进行。

答:程序装入内存　程序执行时

27. 在虚存管理中,虚拟地址空间是指逻辑地址空间,实地址空间是指_____;前者的大小只受_____限制,而后者的大小受_____。

答：物理地址空间　机器的地址长度　物理内存大小限制

28. 在段式虚拟存储管理中，程序所使用的最大段数以及段的最大长度是由_____来决定。

答：逻辑地址结构

29. 在段页式存储管理系统中，每道程序都有一个_____表和一组_____表。

答：段　页

30. 若选用的_____算法不合适，可能会出现抖动现象。

答：页面置换

31. 在页式存储管理系统中，常用的页面淘汰算法有：_____，选择淘汰不再使用或最远的将来才使用的页；_____，选择淘汰在主存驻留时间最长的页；_____，选择淘汰离当前时刻最近的一段时间内使用最少的页。

答：最佳算法　先进先出算法　最近最少使用

32. 对图 3.2 所示的内存分配情况(其中，灰色部分表示一占用块，空白部分表示空闲块)，若要申请 30K 的存储空间，使首地址最大的分配策略是_____。

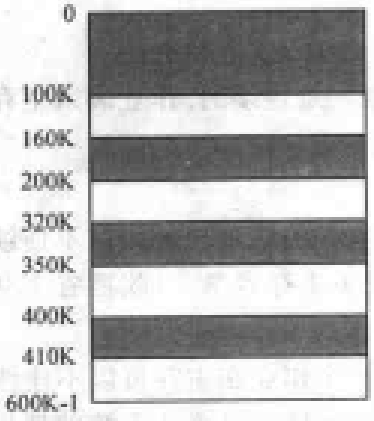


图 3.2　内存分配状态图

答：Last fit

33. 在虚拟段式存储管理中，若逻辑地址的段内地址大于段表中该段的段长，则发生_____。

答：地址越界中断

34. 在请求页式存储管理中，若采用 FIFO 页面淘汰算法，则当分配的页面数增加时，_____的次数可能增加也可能减少。

答：缺页中断

四、判断题

1. 请求分页存储管理系统，若把页面的大小增加一倍，则缺页中断次数会减少一半。(西安交通大学 1999 年研究生试题)　(　)

答：错

分析：增加页面大小可以减少缺页中断次数，但不存在反比关系。

2. 一个虚拟的存储器，其地址空间的大小等于辅存的容量加上主存的容量。(西安电子科技大学 2000 年研究生试题)　(　)

答：错

分析：虚存大小与地址结构即地址地址总线的位数有关。

3. 每个作业都有自己的空间，地址空间中的地址都是相对于起始地址"0"单元开始的，因此逻

辑地址就是相对地址。(西安电子科技大学 2001 年研究生试题) ( )

答:错

4. 在分页存储管理中,减少页面大小,可以减少内存的浪费。所以页面越小越好。(西安理工大学 2000 年研究生试题) ( )

答:错

分析:页面变小将导致页表的增大,即页表占用内存的增大。

5. 虚拟存储器的基本思想是把作业地址空间和主存地址空间视为两个不同的地址空间,前者称为虚存,后者称为实存。(西安理工大学 2000 年研究生试题) ( )

答:正确

6. CPU 的地址空间决定了计算机的最大存储容量。(东北大学 1999 年研究生试题) ( )

答:正确

分析:CPU 指令中的地址位数决定了内存可直接寻址空间的大小。

7. 采用虚拟存储器技术,用户编写的应用程序其地址空间是连续的。(东北大学 1999 年研究生试题) ( )

答:正确

8. 采用分页存储管理时要求逻辑地址是连续的,作业装入主存后的绝对地址也是连续的。(陕西省 1998 年自考题) ( )

答:错

分析:分页存储管理中作业装入主存的绝对地址可以不连续。

9. 分页存储管理方案在逻辑上扩充了主存容量。(陕西省 1997 年自考题) ( )

答:错

分析:分页存储管理方案解决了一个作业在主存可以不连续存放的问题。

10. 由分页系统发展为分段系统进面发展为段页式系统的原因是既满足用户的需要又提高内存的利用率。 ( )

答:正确

11. 虚拟地址即程序执行时所要访问的内存地址。(清华大学 1998 年研究生试题) ( )

答:错

12. 为了使程序在内存中浮动,编程时都使用逻辑地址。因此必须在地址转换后才能达到主存的正确地址。(清华大学 1998 年研究生试题) ( )

答:正确

13. 按最先适应算法分配的分区,一定与作业要求的容量大小最接近。(西安理工大学 2001 年研究生试题) ( )

答:错

14. 页表(PMT)作用是实现逻辑地址到物理地址的映射。(西安理工大学 2001 年研究生试题) ( )

答:正确

15. 虚拟存储器容量可比主存容量大,也可比主存容量小。(电子科技大学 1996 年研究生试题) ( )

答:正确

16. 分区分配是能够满足多道程序要求的一种较为简单的存储管理技术。(陕西省 1997 年自

考题) ( )

答:正确

17. 在分页存储管理中,由于页面置换算法选择不当,就会出现系统抖动现象。 ( )

答:错(应为请求分页系统)

18. 采用最先适应算法时,如果有一个空白区其容量正好满足作业的要求,则它必被选中。(陕西省1996年自考题) ( )

答:错(应为最佳适应算法)

19. 在请求分页存储方案中,页面大小固定,分页的活动对用户来说是可见的。(陕西省1995年自考题) ( )

答:错

20. 提高内存利用率主要是通过内存分配的功能来实现。 ( )

答:正确

五、问答题

1. 存储管理的主要研究内容是什么?

答:存储管理的主要研究内容是:

(1)内存的分配和回收。记录内存空间的使用情况,实施内存的分配,回收系统或用户释放的内存空间。

(2)地址变换。即利用地址变换机构,将逻辑地址转换为物理地址。

(3)存储扩充。借助于虚拟存储技术,从逻辑上扩充内存,为用户提供比内存空间大的地址空间。

(4)存储保护。保证进入内存的各道作业都在自己的存储空间内运行,互不干扰。

2. 考虑一个由8个页面、每页1024个字节组成的逻辑空间,把它映射到容量为32个物理块的存储器中,试问逻辑地址和物理地址分别是多少位?为什么?

答:逻辑地址为13位,物理地址为15位。

分析:因为每页大小为1024字节,故页内地址需要10个二进制位描述;作业的逻辑地址空间有8页,页号需要3个二进制位。由此可知,逻辑地址共需13位。

因为每个物理块与页大小相同,即大小为1024字节,故块内地址需要10个二进制描述;内存空间容量为32块,块号需要5个二进制位。由此可知,物理地址共需要15位。

3. 在某请求分页管理系统中,一个作业共5页,作业执行时依次访问如下页面:1,4,3,1,2,5,1,4,2,1,4,5,若分配给该作业的主存块数为3,分别采用FIFO,LRU页面置换算法,试求出缺页中断的次数及缺页率。

答:(1)采用FIFO页面置换算法,其缺页情况如表3-5所示:

表3-5 FIFO页面置换算法的缺页情况

页面走向	1	4	3	1	2	5	1	4	2	1	4	5	
块1	1	1	1		2	2	2	4	4			4	
块2		4	4		4	5	5	5	2			2	
块3			3		3	3	1	1	1			5	
缺页	缺	缺	缺		缺	缺	缺	缺	缺			缺	

分析：缺页中断次数为 9，缺页率为 9/12 = 75%。

(2)采用 LRU 页面置换算法，其缺页情况如表 3-6 所示。

表 3-6 LRU 页面置换算法的缺页情况

页面走向	1	4	3	1	2	5	1	4	2	1	4	5
块 1	1	1	1		1	1		1	1			1
块 2		4	4		2	2		4	4			4
块 3			3		3	5		5	2			5
缺页	缺	缺	缺		缺	缺		缺	缺			缺

缺页中断次数为 8，缺页率为 8/12 = 67%。

4. 假定某页式管理系统中，主存为 128KB，分成 32 块，块号为 0，1，2，3，…，31；某作业有 5 块，其页号为 0，1，2，3，4，被分别装入主存的 3，8，4，6，9 块中。有一逻辑地址为[3，70]。试求出相应的物理地址(其中方括号中的第一个元素为页号，第二个元素为页内地址，按十进制计算)，并画图说明地址变换过程。

分析：块大小为 128KB/32 = 4KB，因为块与页面大小相等，所以每页为 4KB。第 3 页被装入到主存第 6 块中，故逻辑地址[3，70]对应的物理地址为 4KB×6 + 70 = 24576 + 70 = 24646。其地址变换过程如图 3.3。

答：相应的物理地址为 24646。

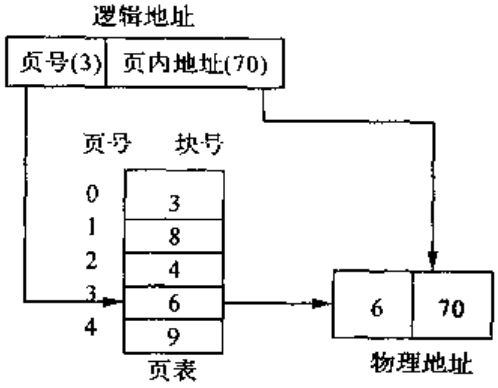


图 3.3 页式系统的地址变换过程

5. 在某段式存储管理系统中，有一作业共 4 段，段号为 0，1，2，3，段表如表 3-7 所示。

表 3-7 某系统段表

段号	段长	主存起始地址	状态
0	500	1500	0
1	400	2600	0
2	120	–	1
3	85	380	0

试计算逻辑地址[0，45]，[1，50]，[2，60]，[3，90]相应的主存地址。当无法进行地址变换时，应说明产生何种中断(方括号内分别为段号和段内地址，按十进制)。

答：

(1)对于逻辑地址[0, 45]，因 45＜500，其逻辑地址合法，对应的主存地址为 1500 + 45 = 1545。

(2)对于逻辑地址[1, 50]，因 50＜400，其逻辑地址合法，对应的主存地址为 2600 + 50 = 2650。

(3)对于逻辑地址[3, 90]，因 90＞85，其逻辑地址非法，所以产生地址越界中断。

6. 什么是地址重定位？有哪几种常用的地址重定位方法？

答：由于将作业装入到与其地址空间不一致的存储空间所引起的对有关地址部分的调整过程称为地址重定位。

常用的地址重定位方法有两种，即静态重定位和动态重定位。静态重定位是在程序运行之前由链接装入程序进行的重定位。动态重定位是在程序执行过程中，每当访问指令或数据时，将要访问的程序或数据的逻辑地址转换成物理地址。

7. 在一个分区存储管理系统中，按地址从低到高排列的空闲分区的长度分别是：10KB、4KB、20KB、18KB、7KB、9KB、12KB、15KB。对于下列顺序的段请求：12KB、10KB、15KB、18KB 分别使用首次适应算法、最佳适应算法、最坏适应算法和下次适应算法，试说明空间的使用情况。

答：为描述方便起见，对空闲分区进行编号，其编号如表 3-8 所示。

表 3-8　分区编号

分区号	分区长度
1	10KB
2	4KB
3	20KB
4	18KB
5	7KB
6	9KB
7	12KB
8	15KB

(1)首次适应算法要求空闲分区按地址递增的次序排列，在进行内存分配时，总是从空闲分区表首开始顺序查找，直至找到第一个能满足大小要求的空闲分区为止。对于段请求 12KB，选中的是 3 号分区，进行分配后 3 号分区还剩下 8KB；对于段请求 10KB，选中的是 1 号分区，因 1 号分区与申请的长度相等，应从空闲分区表中删除 1 号分区；对于段请求 15KB，选中的是 4 号分区，进行分配后 4 号分区还剩下 3KB；对于段请求 18KB，系统已没有空闲分区能满足其要求，让其等待。显然采用首次适应算法进行内存分配时，无法满足所有的段请求。

(2)最佳适应算法总是将作业放入主存中与作业大小最接近的空闲区中，对于段请求 12 KB，选中的是 7 号分区，因 7 号分区与申请空间长度相等，应从空闲分区表中删除 7 号分区；对于段请求 10KB，选中的是 1 号分区；对于段请求 15KB，选中的是 8 号分区；对于段请求 18KB，选中的是 4 号分区；显然采用最佳适应算法进行内存分配时，可以满足所有的段请求。因 1 号分区、8 号分区、4 号分区分别与申请的长度相等，应分别从空闲分区表中删去。

(3)最坏适应算法总是选择最大的空闲区分配给作业。对于段请求 12KB,选中的是 3 号分区,进行分配后 3 号分区还剩下 8KB;对于段请求 10KB,选中的是 4 号分区,进行分配后 4 号分区还剩下 8KB;对于段请求 15KB,选中的是 8 号分区,因 8 号分区与申请的长度相等,应从空闲分区表中删除 8 号分区;对于段请求 18KB,系统已没有空闲分区能满足其要求,让其等待。显然采用最坏适应算法进行内存分配时,无法满足所有的段请求。

(4)下次适应算法是首次适应算法的变形,在为作业进行内存分配时,总是从上次找到的空闲分区的下一个空闲分区开始顺序查找,直至找到第一个能满足大小要求的空闲分区为止。对于段请求 12KB,选中的是 3 号分区,进行分配后 3 号分区还剩下 8KB;对于段请求 10KB,选中的是 4 号分区,进行分配后 4 号分区还剩下 8KB;对于段请求 15KB,选中的是 8 号分区,因 8 号分区与申请的长度相等,应从空闲分区表中删除 8 号分区;对于段请求 18KB,系统已没有空闲分区能满足其要求,让其等待。显然采用下次适应算法进行内存分配时,无法满足所有的段请求。

8. 某请求分页存储管理系统使用一级页表,假设页表总在主存中。

(1)如果一次存储器访问需要 200ns,那么访问一个数据需要多长时间?

(2)现在增加一个快表,在命中或失误时均有 20ns 的开销,假设快表的命中率为 85%,那么访问一个数据的时间为多少?

答:

(1)由于访问一个数据需要访问存储器两次,一次取页表信息,一次取数据信息。故访问一个数据的时间是 $200\times2=400(\text{ns})$

(2)增加了快表后,先在快表中取页表信息,若页表项不在快表中,再到内存取页表信息。故访问一个数据的时间是 $0.85\times(200+20)+0.15\times(20+200+200)\approx250\ (\text{ns})$

9. 在动态分区存储管理中,主要使用的数据结构有哪些?常用哪几种方式寻找和释放空闲区?这些方法有何优缺点?

答:为了实现动态分区存储管理,系统中必须配置相应的数据结构,用来记录内存的使用情况,为分配内存提供依据。常用的数据结构有:

(1)空闲分区表。该表用于记录内存中的所有空闲分区,每个空闲分区在该表中占有一个表项,每个表项至少包括分区号、分区起始地址、分区大小。

(2)空闲分区链。用链指针将内存中空闲分区链结起来,构成空闲分区链。即在每个分区的起始部分,设置一些用来控制分区分配的信息,包括分区状态、分区大小以及用于链接各分区的指针。

常用的寻找空闲分区的方法及其优缺点如下:

①首次适应算法。该算法在进行内存分配时,总是从空闲分区表首开始顺序查找,直到找到第一个能满足其大小要求的空闲分区为止。然后,再按照作业大小,从该分区中划出一块内存空间分配给请求者,余下的空间仍然留在空闲分区表中。该算法要求空闲分区按地址递增的次序排列。

该算法的特点是优先利用内存低地址部分的空闲分区,从而使内存高地址部分保存大的空闲区。但由于不断地划分低地址部分,使低地址端留下许多无法利用的小空闲区,而每次查找又都是从低地址部分开始,这无疑增加了查找可用空闲分区的开销。

②循环首次适应算法。该算法是首次适应算法的变形。在为进程分配内存空间时,不再每次从空闲分区表首开始查找,而是从上次找到的空闲分区的下一个空闲分区开始查找,直到

找到第一个能满足其大小要求的空闲分区为止。

该算法的特点是使存储空间的利用更加均衡，不致使小的空闲区集中在存储区的一端，但这会导致内存中缺乏大的空闲分区。

③最佳适应算法。该算法在进行内存分配时，总是能把既满足作业要求又与作业大小最接近的空闲分区分配给作业。按这种方式为作业分配内存时，要求空闲分区按容量大小递增的次序排列。

该算法的特点是若存在与作业大小一致的空闲分区，则它必然被选中；若不存在与作业大小一致的空闲分区，则只划分比作业稍大的空闲分区，从而保留了大的空闲区。但空闲区一般不可能正好和作业申请的内存空间大小相等，因而将其分割成两部分时，往往使剩下的空闲区非常小，从而在存储器中留下许多难以利用的小空闲区。

④最坏适应算法。该算法总是将最大的空闲区分配给作业。按这种方式为作业分配内存时，要求空闲分区按容量大小递减的次序排列。

该算法的特点是，总是挑选满足作业要求的最大空闲区分配给作业，这样使分给作业后剩下的空闲区也比较大，于是也能装下其他作业。但出于总是首先选择最大的空闲区分配并对其进行划分，当有大作业到来时，其存储空间的申请往往得不到满足。

根据释放区与已有空闲区的相邻情况，采用以下几种方法：

(1)释放区上邻一个空闲分区。此时应将释放区与上邻分区合并成一个连续的空闲区。合并分区的首地址为空闲区的首地址，其大小为二者之和。

(2)释放区下邻一个空闲分区。此时应将释放区与下邻分区合并成一个连续的空闲区。合并分区的首地址为释放区的首地址，其大小为二者之和。

(3)释放区上、下邻空闲分区。此时应将释放区与上、下邻分区合并成一个连续的空闲区。合并分区的首地址为与释放区上邻空闲区的首地址，其大小为三者之和，且应将与释放区下邻接的空闲区从空闲区表(或空闲分区链)中删去。

(4)释放区不与任何空闲分区相邻。这时应为释放区单独建立一个新表项，填写分区大小及起始地址等信息，并将其加入到空闲分区表(或空闲分区链)中的适当位置。

10. 交换扩充了主存，因此交换也实现了虚拟存储器，对吗？请说明理由。

答：这种说法不正确。交换是把内存中暂时不能运行的进程或暂时不用的程序和数据换出到外存上，以腾出足够的内存空间，把已具备运行条件的进程或进程所需的程序、数据换入内存。交换是提高内存利用率的有效措施。

而虚拟存储器是指仅把作业的一部分装入内存便可运行作业的存储器系统，是指具有请求调入功能和置换功能，能从逻辑上对内存容量进行扩充的一种存储器系统。虚拟存储系统的实现，都是建立在离散分配存储管理的基础之上。

虽然交换能提高内存利用率，但仅使用交换技术，仍然无法实现仅把作业的一部分装入内存便运行作业，故交换并不能实现虚拟存储器。

11. 某请求页式存储管理，允许用户编程空间为32个页面(每页1KB)，主存为16KB。如有一个用户程序有10页长，且某时刻该用户页面映射表如下表所示。如果程序执行时遇到以下两个虚地址：0AC5H、1AC5H，试计算它们对应的物理地址。

0AC5H对应物理地址的二进制表示(如表3-9)：

表 3-9 页面映射表

虚页号	物理块号
0	8
1	7
2	4
3	10

答:由题目所给条件可知,每页大小为 1KB。

(1)逻辑地址 0AC5H 的二进制表示如图,由此可知逻辑地址 0AC5H 的页号为 2,查页表知该页存放在第 4 号物理块中,其物理地址的二进制表示如图 3.5 所示。所以逻辑地址 0AC5H 对应的物理地址为 12C5H。

页号 p	页内位移 w
000010	1011000101

逻辑地址 0AC5H 的二进表示

块号	块内位移
000100	1011000101

图 3.5

(2)逻辑地址 1AC5H 的二进制表示如图 3.6,由此可知逻辑地址 1AC5H 的页号为 6,页表中没有该页号,此时发生缺页中断,由缺页中断处理程序将缺页调入内存后再进行地址变换。

页号 p	页内位移 w
000110	1011000101

图 3.6

12. 设正在处理器上执行的一个进程的页表如表 3-10 所示。表中的虚页号和物理块号是十进制数,起始页号(块号)均为 0。所有的地址均是存储器字节地址。页的大小为 1024 字节。

(1)详述在设有快表的请求分页存储器管理系统中,一个虚假地址转换成物理内存地址的过程。

(2)表 3-10 中的虚地址对应什么物理地址:①5499;②2221。

表 3-10 一个进程的页表

0	1	1	0	4
1	1	1	1	7
2	0	0	0	–
3	1	0	0	2
4	0	0	0	–
5	1	0	1	0

答:

(1)在设有快表的请求分页存储管理系统中,一个虚地址转换成物理内存地址的过程为:当 CPU 给出逻辑地址后,地址变换机构自动将逻辑地址划分为页号和页内位移两部分。然后将页号与快表中的所有页号进行并行比较,若快表中有与此匹配的页号,则表示所要访问的页表项在快表中,于是取出该页对应的物理块号,与页内地址拼接形成物理地址。同

时还应修改该页表项中的访问位，对于写指令还需要将修改位置成1。若快表中的所有页号与所查找页号不匹配，则还需再访问主存中的页表。若该页在内存，则从页表中取出物理块号，与页内地址拼接形成物理地址。若该页不在内存，则产生缺页中断，请求操作系统将缺页调入内存，再按前述方式进行地址变换。如果地址变换是通过查找内存中页表完成的，则还应将这次所查到的页表项存入快表中，若快表已满，则必须按某种置换算法淘汰一个表项，以腾出位置存入此页表项。

(2)在本题中，系统页面大小为1024字节。

① 对于逻辑地址5499，其页号为：int(5499/1024)=5

其页内位移为：5499%1024=379

查页表知第5页在第0个物理块中，所以物理地址为：1024×0+379=379

② 对于逻辑地址2221，其页号为：int(2221/1024)=2

其页内位移为：2221%1024=173

查页表可知，第2页不在内存中，此时发生缺页中断，由缺页中断处理程序将缺页调入后再进行地址变换。

13. 某操作系统采用可变分区分配存储管理方法，操作系统占用低地址部分的126KB。用户区大小为386KB，且用户区始址为126KB，用空闲分区表管理空闲分区。若分配时采用分配空闲区高地址部分的方案，且初始时用户区的386KB空间空闲，对下述申请序列：

作业1申请80KB，作业2申请56KB，作业3申请120KB，作业1完成并释放80KB，作业3完成并释放120KB，作业4申请156KB，作业5申请80KB。

试用首次适应算法处理上述作业序列，并回答下列问题：

(1)画出作业1、2、3进入主存后，主存的分配情况。

(2)画出作业1、3完成后，主存的分配情况。

(3)画出作业4、5进入系统后的内存分布情况。

答：为描述方便起见，本题用"(分区首址。分区长度)"的形式描述系统中的分区。由题中所给条件可知，最初系统中只有一个空闲区，大小为386KB，始址为126KB，即(126KB，386KB)。

采用首次适应算法时的操作流程如下：

操作	已分配区	空闲区
初始	无	(126KB，386KB)
作业1申请80KB	(432KB，80KB)	(126KB，306KB)
作业2申请56KB	(432KB，80KB) (376KB，56KB)	(126KB，250KB)
作业3申请120KB	(432KB，80KB) (376KB，56KB) (256KB，120KB)	(126KB，130KB)
作业1释放80KB	(376KB，56KB) (256KB，120KB)	(126KB，130KB) (432KB，80KB)
作业3释放120KB	(376KB，56KB)	(126KB，250KB) (432KB，80KB)
作业4申请156KB	(376KB，56KB) (220KB，156KB)	(126KB，94KB) (432KB，80KB)

作业 5 申请 80KB (376KB, 56KB) (126KB, 14KB)
(220KB, 156KB) (432KB, 80KB)
(140KB, 80KB)

(1)作业 1,2,3 进入主存后,主存的分配情况如图 3.7 所示(用灰色表示空闲空间)

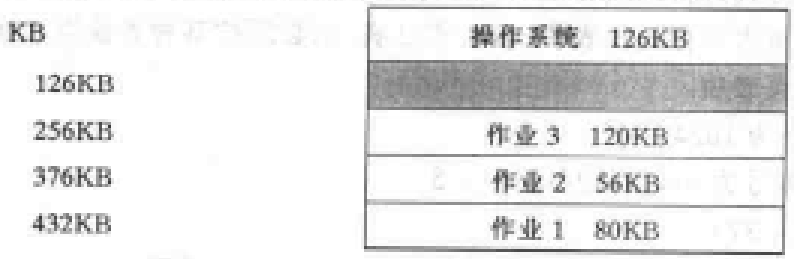


图 3.7 作业 1,2,3 进入主存后的内存分配情况

(2)作业 1,3 完成后,主存的分配情况如图 3.8 所示(用灰色表示空闲空间)

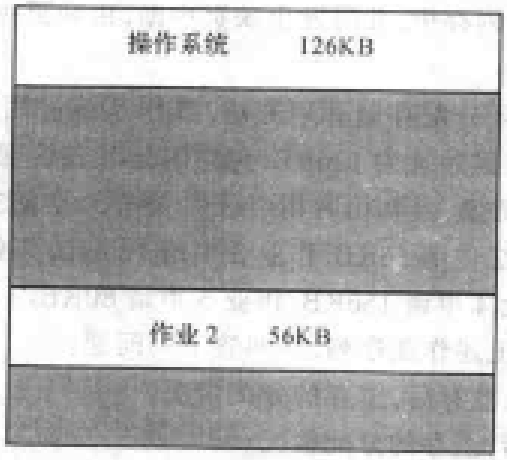


图 3.8 作业 1,3 完成后的内存分配情况

(3)作业 4,5 进入系统后的内存分配情况如图 3.9 所示(用灰色表示空闲空间)

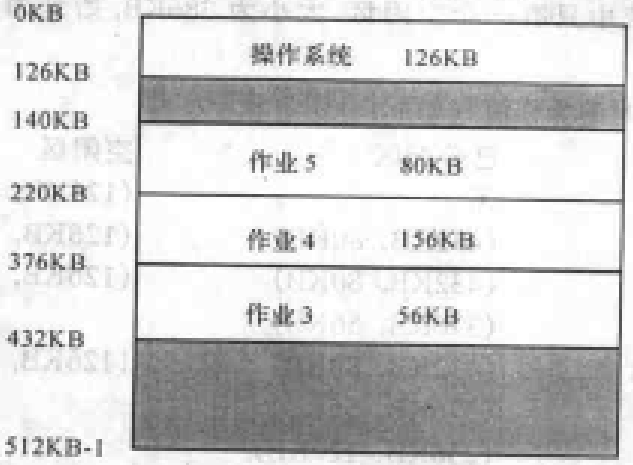


图 3.9 作业 4,5 进入系统后的内存分配情况

14. 存储管理研究的主要课题有哪些?(陕西省 1997 年自考题)

答:存储管理研究的主要的课题可归纳为如下的 4 个方面。

(1)存储分配问题:研究存储共享和各种分配算法。

(2)地址再定位问题:研究各种地址变换以及静态或动态再定位方法。

(3)存储保护问题:研究保护各类程序、数据区的方法。

(4)存储扩充问题:主要研究统一管理内、外存之间程序和数据自动交换的虚拟存储器及其各种调度算法。

15. 什么叫重定位?动态地址重定位的特点是什么?(中科院软件所 1993 年研究生试题)

答:重定位是指作业装入与其地址空间不一致的物理空间所引起的地址变换过程。重定位分为静态重定位和动态重定位两种形式。静态重定位是在作业装入内存时进行地址重定位,而动态重定位则是在程序执行中访问指令或数据时进行地址重定位。动态重定位的特点是:

(1)地址变化机构由硬件实现;

(2)在程序的执行过程中进行地址变换;

(3)程序可以在内存中移动且也可不必连续存放。

16. 什么是动态链接?用何种内存分配方法可以实现这种链接技术?(南开大学 1995 年研究生试题)

答:动态链接就是指作业运行过程中需要调用某一程序模块时,再去装入并链接它;对于未使用的程序模块,这种动态链接法就不会链接它。

在分段存储管理系统中,由于地址空间是二维的,且每一段是一组有意义的信息或具有独立功能的程序段,因而可在作业运行过程中当调用到一个程序段或数据段时再进行动态装入和链接。所以,采用段式内存分配方法可以实现动态链接技术。

17. 在存储管理中,覆盖和对换技术所要解决的是什么问题?各有什么特点?(中国科技大学 1996 年研究生试题)

答:覆盖技术和对换技术是两种扩充内存的技术。覆盖技术主要用于早期的操作系统中,而对换技术在现代操作系统中仍有较强的生命力。

覆盖技术的基本思想:一个程序不需要把所有的指令和数据都装入内存,而是将程序划分为若干个功能相对独立的程序段,按照程序的逻辑结构让那些不会同时执行的程序段共享同一块内存区。这样使用户感觉到内存扩大了,从而达到内存扩充的目的。

对换是指先将内存某部分的程序或数据写入外存交换区,然后再从外存交换区中调出指定的程序或数据到内存中来并让其执行的一种内存扩充技术。

覆盖技术要求程序员提供一个清晰的覆盖结构,即由程序员来完成把一个程序划分为不同的程序段并规定好它们的执行及覆盖顺序。操作系统根据程序员提供的覆盖结构来完成执行过程中程序段的覆盖。因此,程序员必须熟悉计算机的硬件及内存情况,这对程序员的要求较高。

与覆盖技术相比,对换则完全由操作系统实现,它不要求程序员做特殊的工作,整个过程对程序员是透明的。对换主要是在进程或作业之间进行,而覆盖则主要在同一个作业或同一个进程内进行。此外,覆盖只能在彼此无关的不同覆盖段之间进行。

18. 何谓虚拟存储器?举一例说明操作系统是如何实现虚拟内存的。(西安交通大学 1999 年研究生试题)

答:为了给大作业(其地址空间超过主存可用空间)用户提供方便,而由操作系统把主存和辅存统一管理起来并实现自动交换.即一个大作业在执行时,一部分地址空间在主存,而另一部分在辅存,当访问的信息不在主存时由操作系统(而不是由程序设置的 I/O 指令)将其从辅存调入主存.从效果上看,该计算机系统好象为用户提供了一个容量比主存大得多

的存储器，这个存储器称为虚拟存储器。

例如，请求分页存储管理系统就是一种虚拟存储器。请求分页系统在作业运行之前只要求把当前需要的一部分页面装入主存，另一部分页面放在辅存。当运行到不在主存的页面时，产生一个"缺页"中断。这时，由系统把所需要的页面由辅存调入主存，如果内存空间不够，则先将内存中一些不用的页面交换到辅存，再将所需页面调入主存。当然，这里还存在一个从逻辑地址到物理地址的映射问题，这也是由系统提供的动态地址变换机构完成的。即动态地址变换机构自动地将作业的逻辑地址划分为页号和页内地址两部分，经过地址变换机构查找页表，自动地将页号换成与之对应的块号，然后将块号和页内地址拼接就得到了要访问的内存物理地址，从而真正实现了虚拟存储器。

19. 覆盖技术与虚拟存储技术本质有何不同？交换技术与虚存中使用的调入/调出技术有何相同与不同之处？（西安交通大学 2000 年研究生试题）

答：覆盖技术与虚拟存储技术的本质不同如下：

(1)覆盖技术与虚拟存储技术本质的不同在于覆盖程序段的长度要受主存容量大小的限制，而虚拟存储器的最大长度不受主存容量的限制而只受计算机结构的制约。

(2)覆盖技术中的覆盖段由用户设计，即用户要参与对主存的划分操作，而虚拟存储器技术由系统提供逻辑空间给用户使用，由逻辑空间到主存物理空间的地址变换工作完全由系统完成。

交换技术是将主存中处于就绪队列或阻塞队列中的进程暂时由主存调至辅存，以便使更多的作业被选中调入主存来达到提高系统效率的目的。虚存中使用的调入/调出技术是从存储角度考虑利用辅存来扩充内存，即提供一个大于实际主存的逻辑空间给用户使用。两者的相同之处是：都采用交换技术来交换主存与辅存的信息；不同之处是：交换技术并未提供大于实际主存的逻辑空间给用户使用，并且该技术也不是直接面向用户的(用于进程的交换)；而虚存技术则是提供大于主存的逻辑空间给用户使用，是直接面向用户的。

20. 在虚拟页式存储系统中引入了缺页中断：（西安交通大学 2000 年研究生试题）

(1)试说明为什么引入缺页中断？

(2)缺页中断的实现由哪几部分组成？并分别给出其实现方法。

答：(1)虚拟页式存储系统是在页式存储系统的基础上实现虚拟存储器的，也即一作业在执行时并不是所有的页面均放入主存，那么当访问的页面不在主存时，则产生中断并由系统响应中断将该缺页由辅存调至主存，然后恢复中断并继续执行。所以，对于虚拟页式存储系统，系统必须引入相应的缺页中断。

(2)缺页中断的实现由硬件和软件两部分共同完成：

①硬件方面：CPU 在执行一条指令时首先形成操作数的有效地址，然后计算页号，检查页表，看该页是否在主存中。如果该页在主存则进行地址变换，按变换后的地址取出操作数并完成该指令的功能，然后继续执行下一条指令，如果该页不在主存，则引起缺页中断并进入缺页中断处理程序。

②软件方面：在中断处理程序中，首先利用存储器分块表(MBT)检查主存是否有空闲块，如果没有则选择主存中的一页淘汰(该页被修改过还需写入辅存)，并修改页表(PMT)和存储器分块表，此时便出现了空闲块。在有空闲块的情况下，系统根据页表中提供的该页磁盘地址将该页调入主存的空闲块中，然后修改 PMT 和 MBT，最后再重新执行刚才被中断的指令。

21. LRU 算法的基本思想是什么？有什么特点？给出该算法的流程图。(中科院软件所 1996 年研究生试题)

答：LRU 算法的基本思想是：根据局部性原理，依照一个作业在执行过程中已执行过的页面访问踪迹来推测未来的页面走向，即认为过去一段时间里不曾被访问过的页面，在最近的将来可能也不会再被访问。所以，LRU 算法即为：当需要置换一个页面时，选择在最近一段时间最久不用的页面予以淘汰。

LRU 算法能够普遍适用于不同类型的程序。但是，LRU 算法的实现比较困难，这是因为要不断地对前面访问过的页面历史加以记录和更新。如果这种不断地修改完全由软件实现，则系统开销太大；如果由硬件实现则将增加硬件成本。因此，在实际应用中得到推广的是一种简单有效的 LRU 近似算法。

LRU 近似算法的流程如图 3.10 所示，实际上已退化为 NRU(访问位为 0 表示最近没有访问)。

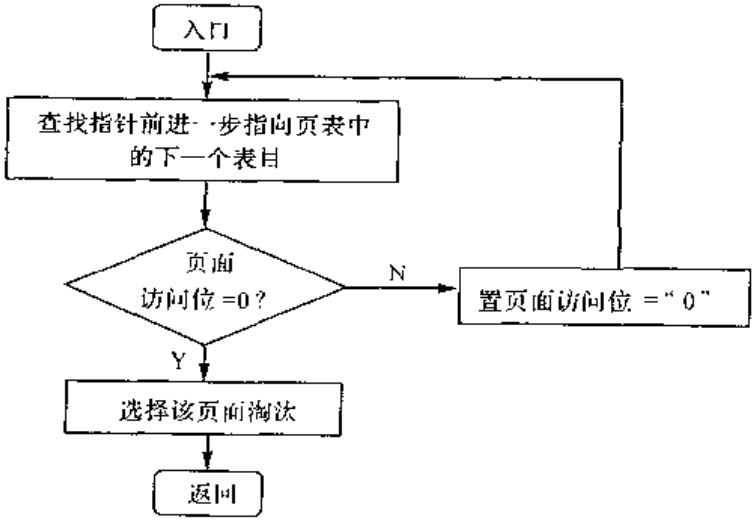


图 3.10

22. 选择页面大小是一个如何进行权衡的问题，试指出大页面和小页面各自的优点。

答：较大页面的优点如下：

(1)页表的大小与页面的大小成反比，较大的页面可节省实现地址映射所需的存储空间及其他资源；

(2)较大的页面可使虚拟存储器的实现更加简单；

(3)在主存和辅存之间传送较大页面比传送较小页面更加有效；

(4)联想存储器(也称快表)的项数有限，对于同样项数的快表，较大页面意味着可高效实现更多存储空间的地址变换，从而减少快表失效的次数。

较小页面的优点如下：

(1)节省存储空间，即减少内零头(内碎片)；

(2)节省进程的启动时间，许多进程都比较小，所以采用小页面可加快进程的调用。

23. 试论述虚拟存储器的优点。

答：虚拟存储器的优点如下：

(1)扩充主存空间。虚拟存储器将主存和辅存统一管理，使得超过主存空间的程序也能运行。

(2)多个进程可以共享主存空间。虚拟存储器把主存空间划分为较小的块(页面或段),并以块为单位分配给各进程。这样,多个进程就可以共享一个较小的主存空间,从而实现并发执行。此外,大多数虚拟存储器还可以减少程序的启动时间,因为这些程序不必等到全部程序和数据都装入主存就可开始执行。

(3)用户不必做存储管理工作。在非虚拟存储中,用户必须对大作业完成所谓的程序覆盖工作,而虚拟存储器解决了这个问题,它自动地对主、辅存进行管理,从而免除了用户的负担。

(4)采用动态重定位技术简化了程序的装入。动态重定位技术是在程序执行中真正访问指令和数据时才进行地址变换而形成物理地址的。动态重定位使得同一程序可以很方便地装入到主存中任何不同的位置上运行。

24. 什么是局部性原理?什么是抖动,有什么办法减少系统的抖动现象?

答:局部性问题是指在所有程序的执行过程中,在一段时间内 CPU 总是集中地访问程序中某一部分而不是对程序的所有部分具有平均的访问概率。虚拟存储器就是基于局部性原理而实现的。

抖动是指当给进程分配的内存小于所要求的工作区时,由于内、外(即主、辅)存之间交换频繁,访问外存的时间和输入输出的时间大大增加,反而造成 CPU 有效处理时间的降低,使得系统性能下降。如在页面置换算法设置不好的情况下,就会出现同一个页面频繁地在主、辅存之间换进和换出这种"抖动"现象。

为了防止抖动的产生,在页面置换中,一方面要采用合适的置换算法,另一方面在进行页面淘汰或置换时,可以将缺页的进程锁住,不让其置换出页面(因该进程在运行),从而防止抖动的发生。此外,防止抖动发生的另一方法是设置较大的内存工作区。

25. 在内存管理中,"内零头"和"外零头"各指什么?在固定式分区分配、可变式分区分配、页式虚拟存储系统、段式虚拟系统中,各会存在何种零头?为什么?(上海交通大学 1997 年研究生试题)

答:在页式存储系统中,一个作业的地址空间不一定是页的整数倍,因而最后一页往往是装不满的。这样造成的零头称为"内零头"。在分区式存储系统中,由于对空闲区频繁的申请和释放,最后会形成在分区之间存在着一些无法使用的过小空闲区,这些小空闲区就称为"外零头"。更为规范的"内零头"和"外零头"定义如下。

(1)内零头(也称内碎片):若存储单元长度为 n,该块存储的作业长度为 m,则剩下的长度为 n - m 的空间称为该单元的内零头。

(2)外零头(也称外碎片):若存在这样一些存储块,其长度总和满足某作业的存储要求,但在该系统所采用的调度算法下无法选出合适的存储块满足该作业,则称这些块为外零头。

在固定式分区分配中两种零头均会存在,因为空间划分是固定的,无论作业长短,存储单元均不会随之变化,若作业短而存储块长则产生内零头,若作业长而存储块短则产生外零头。

在可变式分区分配中只有外零头而无内零头,因为空间划分是依作业长度进行的,故不存在内零头,但划分后的剩余部分可能太小而无法使用则成为外零头。

在页式虚存中存在内零头而无外零头,因存储空间与作业均分为等长单位(块和页),所以不存在无法分配的单元,但作业长度并不是正好为页面大小的整数倍,因而在最后一页会出现剩余空间,即内零头。

在段式虚存中会存在外零头而无内零头,因段式空间的划分类似可变分区,即按段的实际

大小划分，故不存在内零头，但仍然存在划分后的剩余部分可能太小而无法使用的问题，即外零头。

26. 在虚拟段式存储系统中引入段的动态链接：

(1)试说明为什么引入段的动态链接；

(2)请给出动态链接的一种实现方法。(北京大学 1997 年研究生试题)

答：(1)一个大型作业可能由若干个程序模块组成。静态链接的方法是，分别对这些程序模块编译形成多个相当于起始地址为"0"的浮动地址程序，最后再链接装配程序根据当前机器实际内存的情况把这些浮动地址的程序链接装配成一个可在内存中直接运行的可执行目标程序。这种静态链接方法既复杂又费时，有时甚至比实际运行这个作业所花费的时间还要长，并且，经常出现一些被链接好的程序模块在作业某次运行过程中根本不用的情况。所以，最好是在程序的执行过程中用到哪个程序模块时，再去动态装入并链接它，这就是动态链接的方法。在分段存储管理系统中，由于地址空间是二维的，且每段是一组有意义的信息或具有独立功能的程序段，为动态链接提供了必要的条件，因此可在作业运行过程中当调用到某一程序段或数据段时，再进行动态装入和链接。

(2)MULTICS 系统采用的动态链接方法是附加两个硬件设施：间接编址指示位和链接中断位。链接中断位设在间接字中，而间接编址是指指令中地址，是存放直接地址的所在单元地址。

例如，图 3.11 所示的[MAIN]段程序中遇到对另一段产生的符号形式调用(如 LOAD1、[X]|<Y>)时，汇编程序将其汇编成一条间接寻址指令(见图 3.11(b))，且将间接字中链接中断位置"1"，指令中的"*"表示其后的地址 1004 为间接地址。

当程序段[MAIN]装入主存并执行到 LOAD *1,0|1004 时就会遇到链接障碍而发生链接中断信号。操作系统处理这个因需要链接而引起的中断，即从中断信息中找到[0]段的 1004 号单元处的符号串[X]|<Y>，由此从辅存调入[X]段并分配给它一个段号(这里假定[X]=1)，同时得出<y>在[X]段的偏移(这里假定为 120)，其后修改间接字使其指向[X]|<Y>在主存的当前位置"1|120"，并置 L=0。链接完成后，原 0 段 100 号单元处被中断的指令重新启动运行。

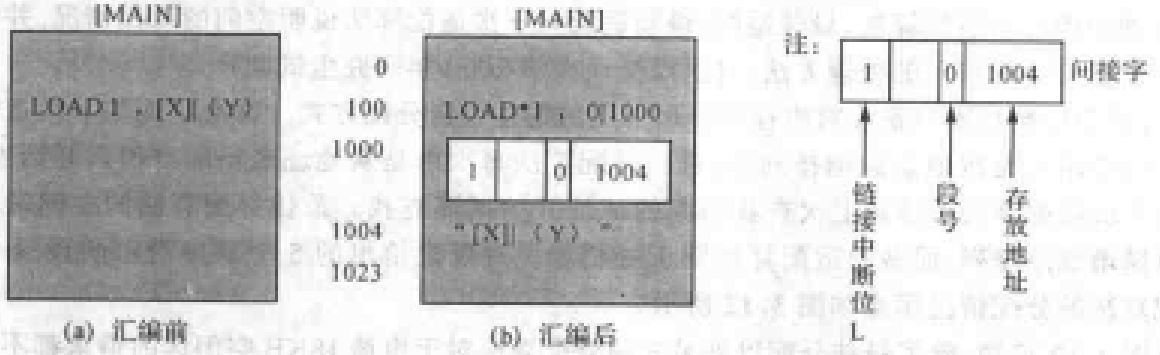


图 3.11

27. 段页式管理的主要缺点是什么？有何改进方法？

答：段页式管理的主要缺点是对内存中指令或数据进行存取时，至少需要对内存进行 3 次访问。第一次是由段表地址寄存器取得段表始址后访问段表，并由此取出对应段的页表在内存的始址。第二次是根据页表始址访问页表得到所要访问的指令或数据的物理地址。在访问了段表和页表后，第三次才根据得到的物理地址去访问对应内存单元中的指令或数

据。显然,这大大降低了 CPU 执行指令的速度。

改进的方法是在硬件上增加联想寄存器(存储器),在联想寄存器中存放当前最常用的段号 S、页号 P 和对应的内存物理块号及其他控制项。当需要访问内存中某一单元时,可在通过段表、页表进行内存地址查找的同时,根据联想寄存器查找其段号和页号。如果所要访问的段和页地址在联想寄存器中,则系统终止对内存中的段表和页表访问,而直接把联想寄存器中的值(即物理块号)与页内偏移地址 d 拼接而得到要查找的内存物理地址。由于联想寄存器的查找速度比分两次查内存的段表和页表快得多,因此使得 CPU 执行指令的速度没有明显下降。

28. 给出 3 种存储保护方法,并说明各适用于何场合?(陕西省 1997 年自考题)

答:(1)上、下界寄存器,适用于分区式分配方案的保护。即为每个进程设置一对上、下界寄存器,上、下界寄存器中装有被保护的程序和数据段的起始地址和终止地址。在程序执行过程中,只要有对内存的访问操作就进行访问地址的合法性检查,检查经过重定位之后的内存地址是否在上、下界寄存器所规定的范围之内,若在规定范围内则访问合法,否则为非法访问并产生越界中断。

(2)在页式存储管理和段式存储管理中,硬件控制寄存器中保存的页表始址和页表长度或者段表始址和段表长度,这两项内容本身就限定了一个作业所允许访问的页和段的个数,如超出则产生越界中断。此外,还可在段表中增加段的长度及访问权限两栏,访问地址越出当前段的长度或访问请求超越了访问权限则引起越界中断或越权中断。

(3)保护键法,一般用在重要状态字的内存保护区。保护键法是一种软件保护方法,它为每一个被保护的存储块分配一个单独的保护键。在程序状态字中则设置相应的保护开关位,对不同的作业(或进程)赋予不同的开关代码并与被保护的存储块中的保护键相匹配。保护键可以设置成对读写同时保护,或只对读写进行单项保护的方式。如果保护开关位与访问的存储块保护键不匹配,则产生错误中断。

29. 在一个使用交换技术(Swapping)的系统中,按地址从低到高排列的内存空间长度是:10KB、4KB、20KB、18KB、7KB、9KB、12KB 和 15KB。对于下列顺序的段请求:

(1)12KB (2)10KB (3)15KB (4)18KB (5)12KB

分别使用首次适配算法、最佳适配、最差适配和下次适配算法说明空间的取用情况,并说明对暂不能分配情况的处理方法。(上海交通大学 2000 年研究生试题)

答:使用交换技术的系统对内存的分配采用的是连续区分配方式。首次适配与下次适配算法的空闲区是按地址递增排列的,首次适配算法每次总是从地址最小的空闲区开始查找,而下次适配算法总是从上次查找结束的地址开始循环查找。最佳适配算法的空闲区按容量递增顺序排列,而最差适配算法则正好相反。对题设给出的 5 个顺序请求的段,4 种分配算法的分配情况示意如图 3.12 所示。

由图 3.12 可知,除了最佳分配以外另 3 种分配算法对于申请 18KB 空闲区的请求都不能满足。对于不能满足请求的进程,只能将其挂入等待内存资源的队列,待以后回收空闲区时,如果存在回收区与空闲区相邻的情况,则合并为一个空闲区,当空闲区大于 18KB 时可唤醒这个等待进程。

30. 在一分页存储管理系统中,逻辑地址长度为 16 位,页面大小为 2048 字节,对应的页表如表 3-11 所示。现有逻辑地址为 0A5CH 和 2F6AH,经过地址变换后所对应的物理地址各是多少?

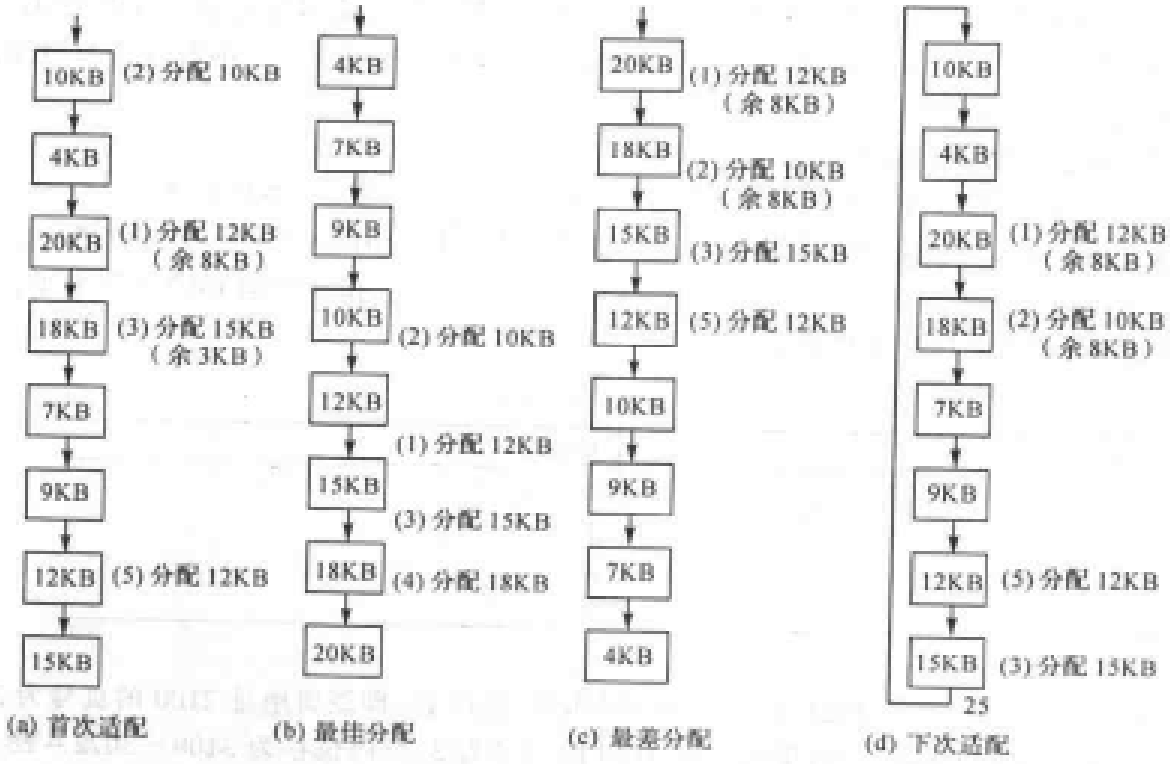
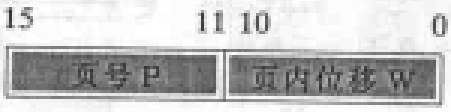


图 3.12　4 种空闲区分配算法的分配情况

表 3-11　页表

页号	块号
0	5
1	10
2	4
3	7

答：由题设条件可知，分页存储管理系统的逻辑地址结构为：



逻辑地址 0A5CH 的二进制表示如下：

$$\underbrace{0000\ 10}_{P}\ \underbrace{10\ 0101\ 1100}_{W}$$

逻辑地址 0A5CH 的页号为 1，从上表所示可知该页对应的物理块号 B 为 10。所以，将二进制表示中的页号换为块号得：

$$\underbrace{0101\ 00}_{B}\ \underbrace{10\ 0101\ 1100}_{W}$$

用 16 进制表示即为 525CH。

逻辑地址 2F6AH 的二进制表示如下：

$$\underbrace{0010\ 1}_{P}\ \underbrace{111\ 0110\ 1010}_{W}$$

即，逻辑地址 2F6AH 的页号为 5，查上表知已超过页表长度，故产生越界中断。

31. 在某个采用页式存储管理系统中，现有 J1、J2 和 J3 共 3 个作业同驻内存。其中 J2 有 4 个页面，被分别装入到主存的第 3，4，6，8 块中。假定页面和存储块的大小均为 1024 字节，主存容量为 10KB 字节。

(1)写出 J2 的页面映像表；

(2)当 J2 在 CPU 上运行时，执行到其地址空间第 500 号处遇到一条传送指令：

MOV 2100, 3100 请用地址变换图计算 MOV 指令中的两个操作数的物理地址。

（南开大学 1997 年研究生试题）

答：(1)J2 的页表如表 3-12 所示。

表 3-12 页表

页号	块号
0	3
1	4
2	6
3	8

(2)本题的页面大小为 1024 字节，可知页内位移为 10 位，即逻辑地址 2100 的页号为 2，页内位移为 2100 − 2048 = 52；逻辑地址 3100 的页号为 3，页内位移为 3100 − 3072 = 28。逻辑地址 2100 的地址变换过程如图 3.13 所示。

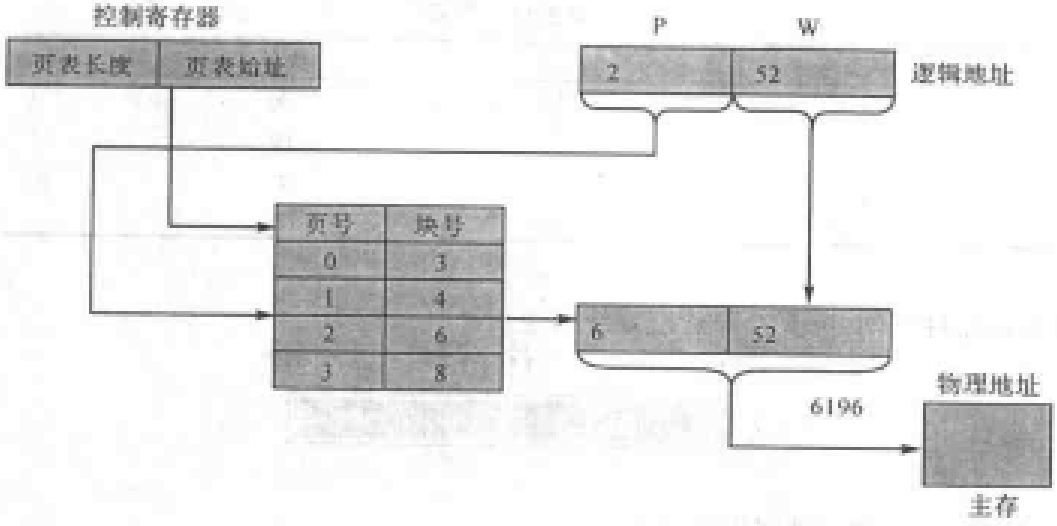


图 3.13 页式地址变换示意图

根据图 3.13 所示可得出逻辑地址 2100 所对应的物理地址为 6196，同理，逻辑地址 3100 所对应的物理地址为 8220。地址变换过程如图 3.14 所示。

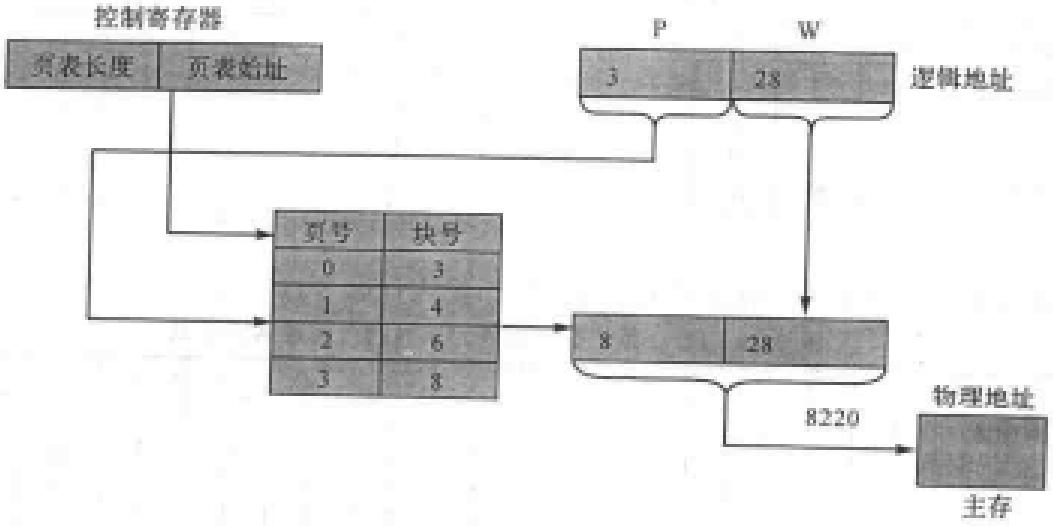


图 3.14 页式地址变换示意图

32. 一台计算机有 4 个页框，装入时间、上次引用时间、它们的 R(读)与 M(修改)位如表 3-13 所示(时间单位：滴答)，请问 NRU、FIFO、LRU 和第二次机会算法将替换哪一页？(上海交通大学 1999 年研究生试题)

表 3-13 作业的页表

页	装入时间	上次引用时间	R	M
0	126	279	0	0
1	230	260	1	0
2	120	272	1	1
3	160	280	1	1

答：(1)NRU 算法是从那些最近一个时期内未被访问过的页中任选一页淘汰。根据上表所示，只有第 0 页的 R 和 M 位均为 0，故第 0 页是最近一个时间内未被访问的页，所以 NRU 算法将淘汰第 0 页。

(2)FIFO 算法淘汰最先进入内存的页。由表 3-13 所示可知，第 2 页最先进入内存(装入时间最小)，故 FIFO 算法将淘汰第 0 页。

(3)LRU 算法淘汰最近最久未用的页。根据表 3-13 所示，最近最久未使用的页(上次引用时间最小)是第 1 页，故 LRU 算法将淘汰第 1 页。

(4)第二次机会算法是淘汰一个自上一次对它检查以来没有被访问过的页。根据表 3-13 所示可知，自上一次对它检查以来只有第 0 页未被访问过(R 和 M 均为 0)，故第二次机会算法将淘汰第 0 页。

33. 在请求分页存储管理方式中，若采用先进先出(FIFO)页面淘汰算法，会产生一种奇怪的现象：分配给作业的实页越多，进程执行时的缺页反而越高。试举一例说明这种现象。(中山大学 1997 年研究生试题)

答：我们以下面的页号走向为例：

4,3,2,1,4,3,5,4,3,2,1,5

若内存分配给该作业 3 个存储块，则页面置换情况如表 3-14 所示。

表 3-14 FIFO 算法在内存块为 3 时的页面置换情况表

页面走向	4	3	2	1	4	3	5	4	3	2	1	5
3 内存块	4	3	2	1	4	3	5	5	5	2	1	1
		4	3	2	1	4	3	3	3	5	2	2
			4	3	2	1	4	4	4	3	5	5
缺页	缺	缺	缺	缺	缺	缺	缺			缺	缺	

而内存分配给该作业 4 个存储块的页面置换情况表如表 3-15 所示。

表 3-15 FIFO 算法在内存块为 4 时的页面置换情况表

页面走向	4	3	2	1	4	3	5	4	3	2	1	5
4 内存块	4	3	2	1	1	1	5	4	3	2	1	5
		4	3	2	2	2	1	5	4	3	2	1
			4	3	3	3	2	1	5	4	3	2
				4	4	4	3	2	1	5	4	3
缺页	缺	缺	缺	缺			缺	缺	缺	缺	缺	缺

从上面的例子可以看出，当内存中为作业提供 4 个内存块时，出现了刚被淘汰的页面随后又要访问的反复调出、调进的现象。这种现象表明：FIFO 算法具有一种异常现象，即分配给作业的存储块数增加时，可能会出现缺页次数没有减少反而增加的情况。

34. 假定某操作系统存储器采用页式存储管理，页的大小为 64 字节，假定一进程的代码段的长度为 702 个字节，页表如表 3-16 所示，该进程在联想存储器中的页表如表 3-17 所示。现进程有如下的访问序列：其逻辑地址为八进制的 105，217，567，1120，2500。试问给定的这些地址能否进行转换？若能，请说明地址转换过程及相应的物理地址。若不能，则说明理由。（北京邮电学院 2000 研究生试题）

表 3-16 内存中的页表

页号	页帧号
0	F0
1	F1
2	F2
3	F3
4	F4
5	F5
6	F6
7	F7
8	F8
9	F9
10	F10

表 3-17 联想存储器中的页表

页号	页帧号
0	F0
1	F1
2	F2
3	F3
4	F4

答:由题设可知,页面的大小为 64 字节,故页内位移的长度为 6 个二进制位,该进程所需的页数为 702÷64=11 页,其编号为 0~10。设页号为 P,页内位移为 W,则有:

(1)八进制逻辑地址 105 的二进制表示如下:

$$\underbrace{0\,0\,1}_{P}\quad\underbrace{0\,0\,0\,1\,0\,1}_{W}$$

即逻辑地址 105 的页号为 1,此页号可在联想寄存器的页表中找到,得页帧号为 F1,页内位移为 5。

(2)八进制逻辑地址 217 的二进制表示为:010 001 111

也即,逻辑地址 217 的页号为 2,此页号可在联想寄存器的页表中找到,得页帧号为 F2,页内位移为 15(十进制)。

(3)八进制逻辑地址 567 的二进制表示为:101 110 111

也即,逻辑地址 567 的页号为 5,此页号可在内存中的页表中找到,得页帧号为 F5,页内位移为 55(十进制)。

(4)八进制逻辑地 1120 的二进制表示为:001 001 010 000

也即,逻辑地址 1120 的页号为 9,此页号可在内存中的页表中找到,得页帧号为 F9,页内位移为 16(十进制)。

(5)八进制逻辑地址 2500 的二进制表示为:010 101 000 000

即,逻辑地址 2500 的页号为 21,此页号已超过页表的长度,故产生越界中断。

35. 已知某系统页面长为 4KB,页表项 4B,采用多层分页策略映射 64 位虚拟地址空间。若限定最高层页表占 1 页,问它可以采用几层分页策略。(浙江大学 2000 年研究生试题)

答:由页面长 4KB 知,页面长为 2^{12}字节,页内偏移量占 12 位。由每一页表项占 4B 知,每页可有表项 2^{10}个,最高层页表的页号占 10 位。由于最高层页表占 1 页,也即该页最多存放页表项个数为 2^{10}。每项指向一页,每页又存放页表项个数为 2^{10},依次类推,最多可采用的分页策略层数为[(64-12)÷10]=6

36. 有一矩阵 var A:array[1..100,1..100] of integer 以行为先进行存储。有一个虚拟存储系统,物理内存共有三页,其中一页用来存放程序,其余两页用于存放数据。假设程序已在内存中占一页,其余两页空闲。

程序 A:

```
for i:=1 to 100 do
    for j:=1 to 100 do
      A[i,j]:=0;
```

程序 B:

```
for j:=1 to 100 do
    for i:=1 to 100 do
        A[i,j]:=0;
```

若每页可存放200个整数,程序A和程序B的执行过程各会发生多少次缺页?试问:若每页只能存放100个整数呢?以上说明了什么问题?(西北工业大学1999年研究生试题)

答:由题设条件可知,每个内存页(块)可存放200个数组元素,共有两个内存页(块)可用来存放数组信息,数组中的元素按行编址。

对于程序A,数组元素的访问顺序是:A[1,1],A[1,2],…,A[1,99],A[1,100]

A[2,1],A[2,2],…,A[2,99],A[2,100]

……

A[100,1],A[100,2],…,A[100,99],A[100,100]

显然,数组元素的存储顺序与访问顺序一致。这样,每访问两行数组元素后将产生一次缺页中断(两个内存页交替调入数组元素),则访问全部数组元素将产生50次缺页中断。

对于程序B,数组元素的访问顺序是:

A[1,1],A[2,1],A[3,1],…,A[99,1],A[100,1]

A[1,2],A[2,2],A[3,2],…,A[99,2],A[100,2]

……

A[1,100],A[2,100],A[3,100],…,A[99,100],A[100,100]

即数组元素的存储顺序(按行存储)与访问顺序(按列访问)不一致。这样,每访问两个数组元素后将产生一次缺页中断,则访问全部数组元素将产生5000次缺页中断。

若每页只能存放100个整数,对程序A则变为每访问一行数组元素后将产生一次缺页中断,访问全部数组元素将产生100次缺页中断;而对程序B则变为每访问一个数组元素后将产生一次缺页中断,访问全部数组元素将产生10000次缺页中断。

以上结果说明,缺页的次数和数据存放的方法及程序执行的顺序都有很大关系。此外,我们也可看出,缺页中断次数不多的情况下,减小页面的大小影响不大,但缺页中断次数很多的情况下,页面的减小必然带来严重的影响。

37. 考虑下列程序:(北京大学1991年研究生试题)

```
VAR     A:ARRAY[1..100,1..150] OF integer;
        B:ARRAY[1..150,1..200] OF integer;
        C:ARRAY[1..100,1..200] OF integer;
i,j,k:integer;
  FOR i:=1 TO 100 DO
    FOR j:=1 TO 200 DO
      FOR k:=1 TO 150 DO
        C[i,j]:= C[i,j]+ A[i,k]* B[k,j];
```

假设矩阵A和B的初值已置好,矩阵C初始为0,各矩阵均以页为单位连续存放。又假定一个整数占一个字,代码以及变量i、j和k存放在其他页面里,并且存取变量i、j和k时不存在缺页问题。

主存初始为空,在请求分页存储管理中,页面淘汰算法为FIFO。

(1)作业分配10个页面,每个页面为100字,给矩阵A、B和C使用。问执行上面程序时,

缺页次数是多少？当执行完程序时，留在内存的 10 个页面各属于哪些矩阵？

(2)当作业分配两个页面，每个页面为 500 字，给矩阵 A、B 和 C 使用。问执行上面程序时，缺页次数是多少？当执行完程序时，留在内存的两个页面各属于哪些矩阵？

答：(1)假定矩阵是按行存储的，且每页均从页面首址开始存放，则 A、B、C 各矩阵的存储情况如表 3-18 所示。

表 3-18　矩阵存储情况表

矩阵	行数	列数	总字数	总页数	每行页数
A	100	150	15000	150	1.5
B	150	200	30000	300	2
C	100	200	20000	200	2

假设程序的执行的顺序为：读 A、读 B、计算 A×B、读 C、计算 C+A×B、写 C，则程序执行中对存储器的访问顺序为读 A、读 B、读 C 和写 C。

由于每页可存放 100 个字，由上表可知：矩阵 A 占用 150 页、矩阵 B 占用 300 页，矩阵 C 占用 200 页。假设矩阵 A 占用的页面为 1～150，矩阵 B 占用的页面为 151～450 页，矩阵 C 占用的页面为 451～650 页。其存储示意如图 3.15 所示。

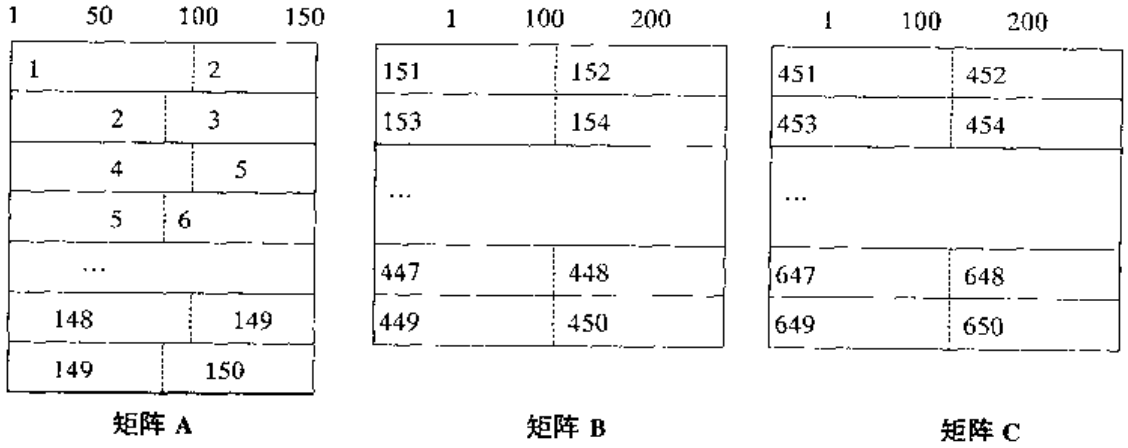


图 3.15　A,B,C 各矩阵对应各行的页号示意

程序对矩阵 A 和 C 的访问是按行访问，即矩阵 A 和 C 的存放顺序与访问顺序相同。程序对矩阵 B 的访问是按列访问，即矩阵 B 的存放顺序与访问顺序不一致。即访问顺序是访问某列的第一个元素后再访问该列 的第 2 个元素、第 3 个元素、… …并且，由于矩阵 B 每行必须用两页存储，故一列第一个元素与第 2 个元素存储在不同的页中，也即按列顺序访问时，每次对矩阵 B 的访问实际上都要访问与前一次访问不同的页。

程序中三重 FOR 循环执行的次数为 100×200×150 = 30000000 次，每次需要依次访问矩阵 A、B 和 C。只要不跨页，每次访问矩阵 A 和 C 时无须调入新页，但每次访问矩阵 B 中的元素都需要调入新页。由于系统只有 10 个页面，所以每次访问矩阵 B 时，被访问元素所在的页面都不在内存中。

采用 FIFO 算法，当循环次数为(n×9)+1 或(n×100)+1 时，读 A、读 B 与读 C/写 C 都会出现缺页，而其他情况只有在读 B 时会出现缺页。(n×9)+1 时的情况是由于矩阵 B 所用的页面占用了内存所有 10 个页面而造成的缺页，而(n×100)+1 时的情况则是需要读 A 或 C 新的一页数据造成的缺页。根据这个规律可得出缺页的次数为：

$$(100\times200\times150)+(333333+29999-3333)\times2=3719998(次)$$

最后留在内存中的10个页面，一个属于矩阵A，8个属于矩阵B，1个属于矩阵C。

(2)如果每个页面为500字时，则矩阵A占用30页，矩阵B占用60页，矩阵C占用40页。由于内存中仅两个页面，故每次访问都将出现缺页，也即缺页次数为：

$$3000000\times3=9000000(次)$$

38. 有一页式系统，其页表存放在主存中。

(1)如果对主存的一次存取需要1.5微秒，试问实现一次页面访问时存取时间是多少？

(2)如果系统加有快表且平均命中率为85%，而页表项在快表中的查找时间忽略为0，试问此时的存取时间为多少？

答：页表在主存时，则实现一次存取需两次访问主存：第一次是访问页表获得所需访问的物理地址，第二次才是根据这个物理地址存取数据。

(1)因页表在主存，所以CPU必须两次访问主存，即实现一次页面访问的存取时间是：

$$1.5\times2=3(\mu s)$$

(2)系统增加了快表后，在快表中找到页表项的概率为85%，所以实现一次页面访问的存取时间为：

$$0.85\times0+0.85\times1.5+(1-0.85)\times2\times1.5=1.725(\mu s)$$

39. 在一个段式存储管理系统中，其段表如表3-19所示。试求表3-20所示中的逻辑地址所对应的物理地址是什么？

表3-19 段表

段号	内存起始地址	段长
0	210	500
1	2350	20
2	100	90
3	1350	590
4	1938	95

表3-20 逻辑地址

段号	内存起始地址	段长
0	430	
1	10	
2	500	
3	400	
4	112	
	32	

答：(1)由表3-19可知：第0段内存始址为210，段长为500，故逻辑地址[0,430]是合法地址，其对应的物理地址为210+430=640。

(2)由表3-19可知：第1段的内存始址为2350，段长为20，故逻辑地址[1,10]是合法地址，其对应的物理地址为2350+10=2360。

(3)由表 3-19 可知:第 2 段的内存始址为 100,段长为 90,故逻辑地址[2,500]的段内位移 500 已超过了段长,故为非合法地址。

(4)由表 3-19 可知:第 3 段的内存始址为 1350,段长为 590,故逻辑地址[3,400]是合法地址,其对应的物理地址为 1350 + 400 = 1750。

(5)由表 3-19 可知:第 4 段的内存始址为 1938,段长为 95,故逻辑地址[4,112]的段内位移 112 已超过了段长,故为非合法地址。

(6)由表 3-19 可知:不存在第 5 段,故逻辑地址[5,32]为非合法地址。

40. 假定有一个开方程序 SQRT 被两个进程共享,开方程序如下:

```
(1)SQRT(x,y)
(2)if x<0 then goto (SQRT,L);
(3)y:="x 开方的结果";
(4)return;
(5)(SQRT,L):"报告出错";
(6)return
```

如系统采用段式管理,应如何安排程序?为什么?(国防科技大学 1997 年研究生试题)

答:由于该共享程序引用了自身的某个地址(语句(2)引用该程序自身),则各共享进程必须用同一段号来共享这一段,否则会出现问题。例如,作业 J1、J2 分别将共享段 SQRT 安排在逻辑空间的第 1 段和第 0 段(如下图所示),则会出现这样的问题:SQRT 段调入主存时应将语句(2)的符号地址转换成逻辑地址,即把(SQRT,L)换成(段号,L),那么究竟把 SQRT 换成 1 还是 0?若与作业 J1 相一致则应换成(1,L),这时,当 J2 执行到语句(2)时,若 x<0 则执行 goto(1,L),按照段式系统的工作原理,则应先查段表项 1,然后合成物理地址。这显然会造成错误,即转移到作业 J2 的第一段中去。若将(SQRT,L)换成(0,L)的话,则作业 J1 执行时也会遇到类似的问题。故共享段引用自身,则各作业必须使用同一段号来共享该段。

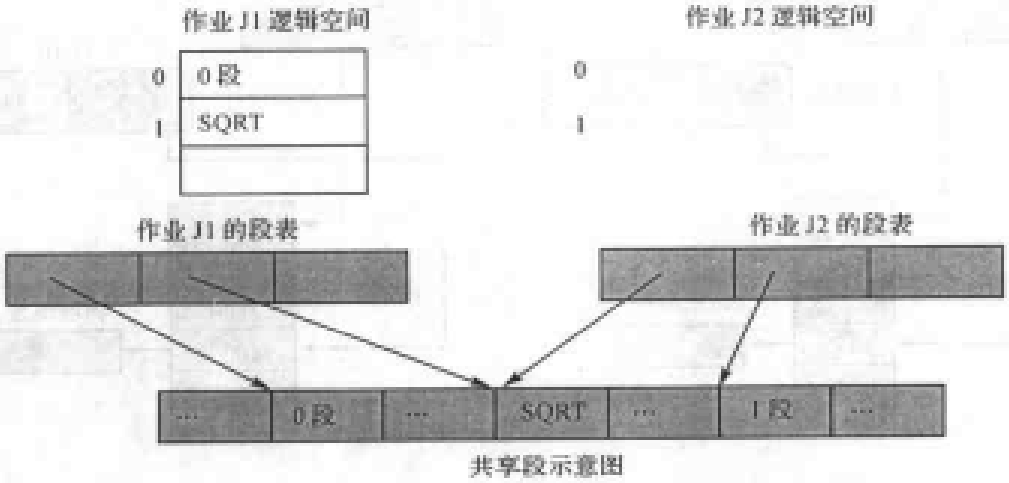


图 3.16 共享段示意图

41. 图 3.17 所示是一种段页式管理配置方案,一页大小为 1KB。

(1)根据给出的虚地址写出物理地址。

(2)描述地址变换过程。

答:物理地址为:14673。

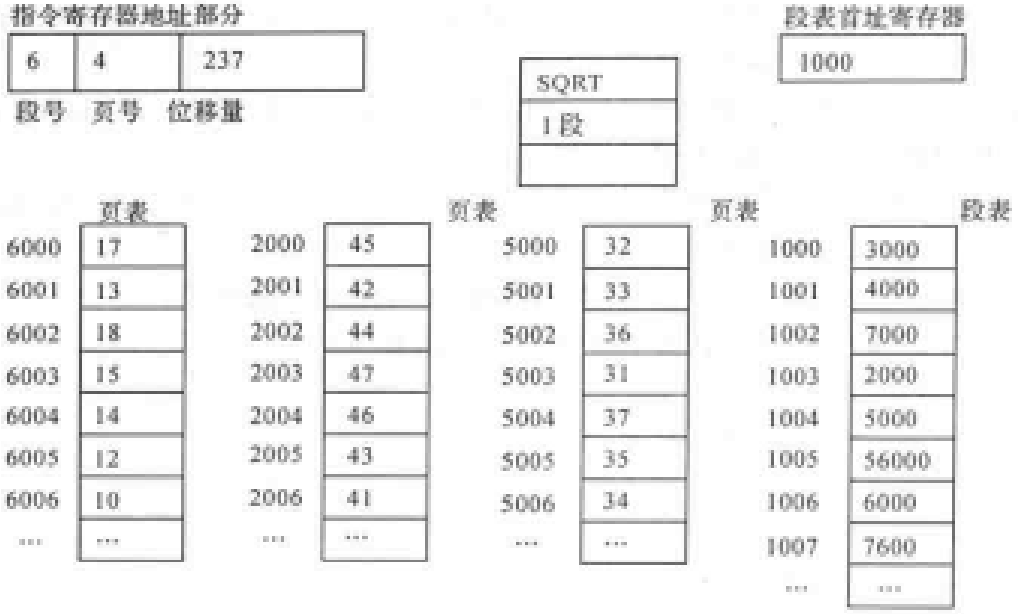


图 3.17 段页式管理配置示意图

分析：地址变换过程为：段号 6 与段表首址寄存器值 1000 相加得 1006，在段表 1006 项查得页表首址为 6000。这时页号 4 与页表首址 6000 相加得 6004，进而查页表项 6004 内容为 14，即块号为 14，该块的始址为 14×1024(每块大小)=14336，加上位移量 337 即得物理地址为 14673。

42. 图 3.18 所示分别给出了段式或页式两种地址变换示意(假定段式变换对每一段不进行段长越界检查，即段表中无段长信息)。

(1)指出这两种变换各属于何种存储管理。

(2)计算出这两种变换所对应的物理地址。

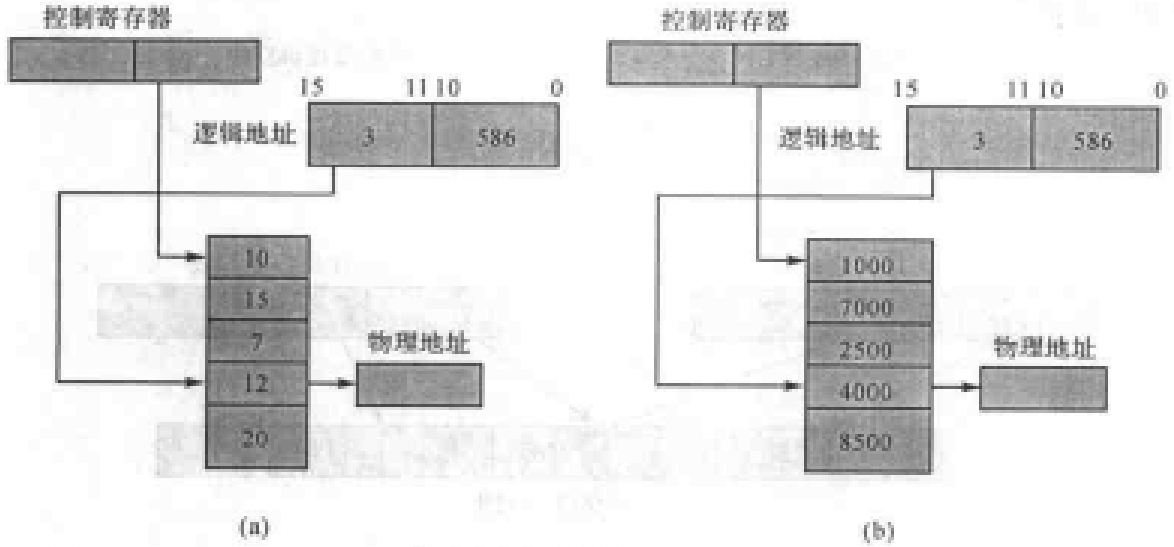


图 3.18 两种地址变换示意图

答：(1)由图 3.18 所示中的逻辑地址结构可知：段或页的最大个数为 32。那么，如果图 3.18(a)是限式管理，段始址 12 加上偏移量 586，远超过了第 1 段的段始址 15，超过了第 4 段的段始址 20。所以，图 3.18(a)是页式变换，而图 3.18(b)满足段式变换。对于页式管

理，由逻辑地址的位移量位数可知，一页的大小为 2KB。

(2)对图 3.18 的页式地址变换，其物理地址为 $12 \times 2048 + 586 = 25162$。

对上图的段式地址变换，其物理地址为 $4000 + 586 = 4586$。

43. 在现代计算机系统中，存储器是十分重要的资源，能否合理有效地使用存储器，在很大程度上反映了操作系统的性能，并直接影响到整个计算机系统作用的发挥。请问：

(1)主存利用率不高主要体现为哪几种形式？

(2)可以通过哪些途径来提高主存利用率？(西北大学 2000 年研究生试题)

答：(1)内存利用率不高主要表现为以下形式：

① 内存中存在着大量的、分散的和难以利用的碎片；

② 暂时或长期不运行的程序和数据占据了大量的内存空间；

③ 作业较大时造成了内存的作业过少，当它们被阻塞时将造成 CPU 空闲，从而也降低了内存的利用率；

④ 内存中存在着重复的拷贝。

(2)可分别采用下述途径来提高内存的利用率：

① 将连续分配方式改为离散分配方式以减少内存的零头；

② 增加对换机制，将那些暂时不能运行的进程或暂不需要的程序和数据换出至外存，以腾出内存来装入运行的进程；

③ 引入虚拟存储机制，使更多的作业能够装入内存，提高 CPU 和内存利用率；

④ 引入动态链接机制，当程序在运行中需要调用某段程序时才将该程序装入内存，从而避免装入不会用到的程序段和数据；

⑤ 引入存储器共享机制，允许一个正文段或数据段被若干进程共享以消除内存中的重复拷贝现象。

44. 虚拟存储器与一般存储器(磁盘作外存)的主要差别是什么？(清华大学 1992 年研究生试题)

答：虚拟存储器主要由主存和联机工作的外存共同组成。对虚拟存储器，用户可用机器指令地址码对整个程序统一编址，即拥有比主存容量更大的虚拟空间。外存可以是虚拟存储器的一个组成部分，也可以是用于保存数据和程序的外部存储器，是真实的物理设备。

45. 在以进程为单位进行对换时，是否每次都将整个进程换出？为什么？

答：在以进程为单位进行对换时，并非每次都要将整个进程换出。这是因为：

(1)从结构上看，进程是由程序、数据和进程控制块组成。其中，进程控制块通常都长驻内存而不被换出。

(2)如果进程对应的程序或数据正被其他进程所共享，则也不能换出。

46. 在页式虚拟管理的页面替换算法中，对于任何给定的驻留集尺寸，在什么样的引用串情况下，FIFO 与 LRU 替换算法一样(即被替换的页和页故障完全一样)？举例说明。(国防科技大学 1998 年研究生试题)

答：由于驻留集尺寸任意，若要 FIFO 与 LRU 算法完全一样(发生缺页时被替换的页均一样)，则访问串必须具有一定的特点。现分析如下：FIFO 是替换最早进入主存的页，而 LRU 是替换上次访问以来最久未被访问的页，二者要相同，就必须有先进入主存页在该次缺页之前不再被访问，即该页也同时是最久未被访问的页。这意味着，某页被替换出主存之前仅被访问过一次(非连续的)。由此得到访问串的规律为：当出现缺页中断时，最先进

入主存的页仅被访问过一次。

例如，合法驻留集为4，对如下访问串：

1 2 3 4 1 2 3 6

当6号页被调入主存时，应在1，2，3，4页中选择一个淘汰，FIFO选择的是最先进入主存的1号页，而LRU则选择最久未用的4号页。原因是1号页已被再次访问，而不是自上次访问以来最久未被访问页3。如果去掉对1号页的第二次访问，即访问串为：

1 2 3 4 2 3 6

则LRU也会选择1号页，因为此时1号页已是自上次访问以来最久未被访问的页3。由此类推，对于驻留集尺寸任意情况下整个访问串中的每一页都应有这个特点，则访问串中的所有页号应均不同，如果相同则必须连续的排在一起。

47. 在页式虚存管理系统中，假定驻留集为m个页帧(初始所有页帧均为空)，在长为p的引用串中具有n个不同页号(n>m)，对于FIFO、LRU两种页面替换算法，试给出页故障数的上限和下限，说明理由，并举例说明。(国防科技大学2000年研究生试题)

答：发生页故障(缺页中断)的原因是当前访问的页不在主存，需将该页调入主存。此时不管主存中是否已满(已满则先调出一页)，都要发生一次页故障。即无论怎样安排，n个不同的页号在首次进入主存必须要发生一次页故障，总共发生n次，这就是页故障的下限。虽然不同页号数为n，小于或等于总长度p(访问串可能会有一些页重复出现)，但驻留集m<n，所以可能会有某些页进入主存后又被调出主存，当再次访问时又发生一次页故障的现象，即有些页可能会出现多次页故障。极端情况是每访问一个页号时，该页都不在主存，这样共发生p次故障。所以，对于FIFO与LRU替换算法，页故障数的上限均为p，下限均为n。例如，当m=3，p=12，n=4时，有如下访问串：

1 1 1 2 2 3 3 3 4 4 4 4

则页故障数为4，这恰好是页故障数的下限n值。

又如：访问串为：

2 3 4 1 2 3 4 1 2 3 4

则页故障数为12，这恰好是页故障数的上限p值。

48. (1)请说明页式虚拟存储器的地址变换过程，并指出哪些操作是由操作系统完成的？

(2)在上述地址变换过程中，进程状态有无可能发生变化？如可能，则指出在哪些点上可能发生变化？(上海交通大学1998年研究生试题)

答：(1)页式虚拟存储器的地址变换如下。

① 由分页地址变换机构将访问的逻辑地址分为页号和页内位移两部分。

② 将页号与页表长度比较，若大于或等于页表长度，则表明本次访问地址已超过了进程的地址空间，此时撤消该进程并给出越界错误信息；否则转③。

③ 查找页表，看该页是否在内存。若在内存则转⑥，否则挂起该进程并产生缺页中断后转④。

④ 执行缺页中断程序，查找空闲块，若内存中无空闲块则按置换算法淘汰内存中的一页，请求I/O将该页装入空闲块。

⑤ 更新页表中的有关信息。

⑥ 分页地址变换机构形成访问该页的内存物理地址(即将该页对应的块号与页内位移拼接)。

⑦ 重新启动挂起进程，执行刚才因缺页而中断的指令。

页式虚拟存储器的地址变换过程对用户是透明的，整个变换过程是由操作系统和 CPU 的 MMU 协调完成。其中，第④步操作以操作系统为主。

(2)上述地址变换过程中，进程的状态可能发生变化，发生变化的情况如下。

① 当本次访问地址超越进程的地址空间时，该进程被撤消。

② 在产生缺页中断及处理过程中，该进程处于挂起状态。

③ 为了提高系统利用率，某些算法在内存空闲块紧张时将当前运行进程整个交换到辅存上，这时该进程变为阻塞态。

49. 类似于请求分页存储管理中的请求式调页那样，在请求分段存储管理中也可以采用请求式调段策略。试给出合理的段替换算法，并说明在段替换过程中会出现哪些在页面替换过程中不出现的问题。

答：(1)采用 FIFO 替换算法。FIFO 查找第一个能容纳即将进入段的空闲段，为避免内部存储碎片，可把该段的未被占用的部分并入空闲空间表中。若找不到满足要求的段，在不允许重定位的情况下，则选择两个或多个连续的段，它们是"最靠近空闲空间表首部"，合并它们能容纳即将进入的段。如果允许重定位，则重新安排内存，使得能够容纳即将进入段的开始 N 个段在内存中是连续的。同样，这两种情况下都应将剩余空间并入空闲空间表。

(2)采用 LRU 替换算法。LRU 选择一个未被使用的、时间周期最长的、可容纳即将进入段的这样一个段，并将剩余空间并入到空闲空间表中。如果没有满足要求的段，则在不能重定位的情况下选择一组内存连续的，且合并起来满足要求的"最老的"段。若可以重定位，则重新安排这组最老的段，使其在内存中是连续的，它们合并后可以满足即将进入段的要求。

在段替换过程中，必须要考虑段的大小变化，但在页面替换中页面的大小是固定的。因此，当找到的空闲段容量小于即将进入的段的容量时，就需要分别考虑能够重定位与不能重定位时的情况。此外，由于段的大小不固定，就可能需要替换若干个连续的段才能满足要求，所以替换算法应尽可能简单，在这种情况下，FIFO 算法最佳。

50. 有一虚拟存储器，它所提供的作业空间最大可达 4096 个页，每页为 2048 个字节。已知主存容量为 220 位，快表用按地址访问的联想存储器构成，行数为 32，快表的地址是经过散列技术形成的。

(1)请给出逻辑地址和物理地址结构。

(2)画出经过快表的地址变换示意图。

答：

(1)已知作业空间最大可达 4096 个页，则页号占用的二进制位数为 12；每页为 2048 字节，则页内位移量占用的二进制位数为 11。由主存容量为 220 位，块内位移为 11 位(块与页的大小相同)，可知，块号占用的二进制位数为 9，逻辑地址结构与物理地址结构如图 3.19 所示。

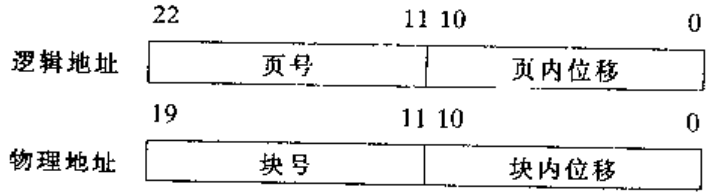


图 3.19

(2)根据逻辑地址结构和物理地址结构画出地址变换示意图 3.20 如下。

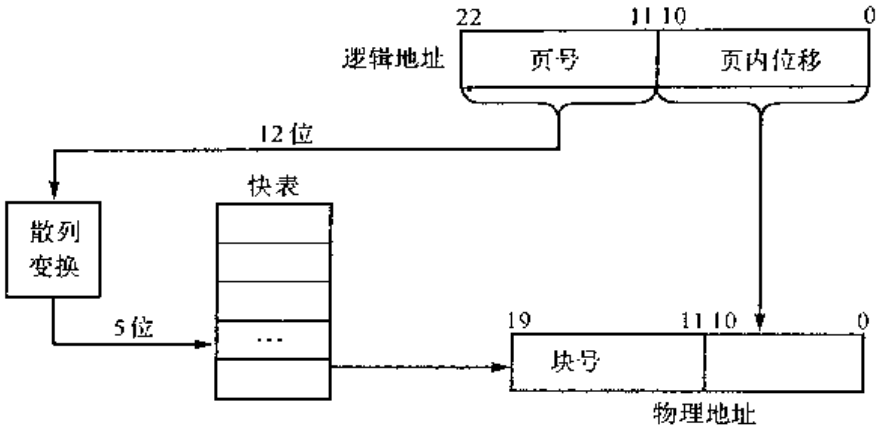


图 3.20

51. 有两台互联的计算机,现需要把这两台计算机的主存储器以一个统一的逻辑地址空间呈现在应用程序面前。那么,在通信系统的支持下,需要什么样的硬软件支持?操作系统应如何实现逻辑地址到物理地址的变换(图示并配以必要说明)?(电子科技大学 1996 年研究生试题)

答:与虚拟存储管理类似,需要注意硬件和软件设施之间的协调。具体实现是:在段表中增设一个标识位以表示该段是否存在于本机主存,同时硬件应能够产生缺段中断。在通信系统支撑下,实现逻辑地址到物理地址的变换过程如图 3.21 所示。

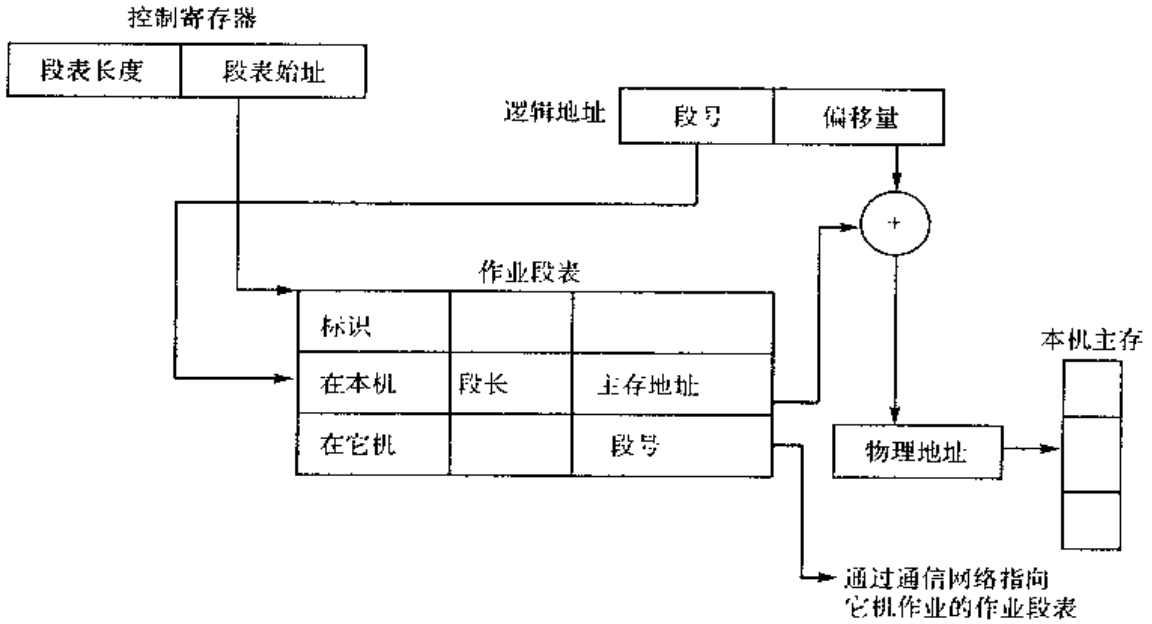


图 3.21

52. 内存保护是否可以完全由软件完成?

答:内存保护的主要任务是确保每道程序都只能在自己的内存区内运行,这就要求系统能够对每条指令所访问的地址进行越界检查。如果发生越界,系统应能立即发现并发出越界中断请求。假设这种检查是用软件实现的,那么每执行一条程序指令时,系统都要花若干条指令去检查是否越界,这无疑降低了程序的运行效率。因此,越界检查通常是由硬件实现的,使程序指令与硬件越界检查能并行执行,而不降低程序的运行效率。当然,对出现越

界时的处理需要与软件配合完成。所以，内存保护功能是由硬件和软件协同完成。

53. 什么叫重定位？采用内存分区管理时，如何实现程序运行时的动态重定位？（西北工业大学2000年研究生试题）

答：所谓地址重定位，就是当一个程序装入到其与地址空间不一致的存储空间而进行的地址变换过程，即将地址空间给出的逻辑地址映射到内存的物理地址上。地址重定位又分为静态重定位和动态重定位两种方式，前者的重定位是在程序装入内存时进行的，后者的重定位是在程序的运行中进行的。

采用内存分区管理时，可在硬件上设置一个"重定位寄存器"来实现程序运行中的动态重定位，此时程序执行的物理地址是通过程序的逻辑地址与重定位寄存器中的基址这两者之和得到的，其地址变换过程由地址变换机构自动实现。

54. 在采用首次适应算法的分区管理中，回收内存时有可能出现哪几种情况应怎样处理这些情况？

答：在采用首次适应算法时，空闲区按地址递增的顺序链成一个队列，此时回收内存时可能出现下述4种情况。

(1)回收区前邻空闲区。此时应将回收区与前邻空闲区合并，并将这个前邻空闲区的大小修改为两者之和。

(2)回收区后邻空闲区。此时应将回收区与后邻空闲区合并，但应修改后邻空闲区的始址为回收区的始址，大小为两者之和。

(3)回收区前后均邻空闲区。此时应将3个分区合并，取消后邻空闲区，修改前邻空闲区的大小为三者之和。

(4)回收区前后均不邻空闲区。此时应为回收区设置一个空闲区表项，填入回收区的始址与大小并插入空闲区队列。

55. 采用可变分区方式管理主存时，引入移动技术有什么优点？在采用移动技术时应注意哪些问题？（南京大学1997年研究生试题）

答：在可变分区方式中，存在大量的不能被利用的小分区。可能会出现这种情况，当一个作业申请空闲区时，虽然内存中总的空闲存储空间是够用的，但由于都是一些小分区却无法装入该作业。为了利用这些无用的小分区，可以采用的一种方法是将内存中的作业进行移动(Compaction)，使这些分散的小分区拼接成一个大分区，从而可以将作业装入大分区。这就是引入移动技术的优点。需要注意的是，由于移动后用户程序在内存中的位置发生了变化，如不对用户程序和数据的地址进行修改，则程序将无法执行。为了使程序能够正常运行，必须进行重定位，即在系统中增加一个重定位寄存器，用它来装入程序的内存起始地址。这样，程序执行时所访问的内存地址是逻辑地址与重定位寄存器中的基址相加得到的。

56. 某系统采用页式存储管理，并采用LFU算法淘汰页面。

(1)写出LFU算法的基本实现方法和原理。

(2)如果系统发生抖动，应如何解决？（清华大学1994年研究生试题）

答：(1)LFU是最不经常使用页面的淘汰算法，当需要淘汰某一页时，首先选择到目前为止被访问次数最少的一页予以淘汰。其实现方法是在页表中每一页增设一个访问计数器，即每当一页被访问时，该页对应的页表计数器加1；在出现缺页中断时淘汰计数值最小的那一页，同时将所有的计数器清零。

(2)LFU 算法发生抖动时应采取增加工作集的方法来减少抖动。在实际系统中为了防止抖动的发生,常常采用把缺页进程锁住不让其换出的方法,而调入的页则总是占据那些暂不执行进程所占有的内存空间,从而扩大缺页进程的工作集。

57. 交换扩充了主存,因此,交换也实现了虚拟存储器,对吗?(清华大学 1995 年研究生试题)

答:该说法有误。交换是将内存中暂不运行的进程或暂不使用的程序及数据交换至外存,以腾出足够的内存空间将具备运行条件的进程或进程所需的程序及数据换入内存。交换只是提高内存利用率的一种技术。虚拟存储器是指仅把作业的一部分装入内存便可以运行的一种存储器管理系统。虚拟存储器除了具有将程序与数据换入换出的功能外,还包括实现逻辑地址到物理地址的变换机构及内存保护功能,即从逻辑上实现对内存容量的扩充,使超过内存容量的作业也能够运行。由于交换技术不能实现装入作业的一部分即可运行这一功能,所以交换并不能实现虚拟存储器。

58. 为什么说请求分页管理可以实现虚拟存储器?(清华大学 1996 年研究生试题)

答:虚拟存储器是指仅将作业的一部分装入内存即可运行的存储器系统,虚拟存储器具有请求调入功能和置换功能,并从逻辑上实现了对内存容量的扩充。虚拟存储器是建立在离散分配存储管理方式基础上的。

请求分页管理建立在纯分页系统基础上,而纯分页系统本身是一种离散分配存储系统(一作业分配在内存的存储块可不连续)。此外,在请求分页式管理下,可先将作业的一部分调入内存,当需要作业的其他部分时,则发出缺页中断,由操作系统将所需要的页面调入内存;而对长期不用的页面,操作系统又将其换出内存。因此,请求分页式管理实现了虚拟存储器的请求调入和置换功能,实现了逻辑上既具有大容量的内存,又装入一部分作业即可运行的特点,所以请求分页管理实现了虚拟存储器。

59. 以分页存储管理为例,说明动态地址映射的实现方法。(中山大学 1996 年研究生试题)

答:动态地址映射是在程序执行过程中将逻辑地址转换为物理地址。页式地址映射是通过页表完成的。当 CPU 给出逻辑地址之后,地址变换机构自动将其分为页号和页内位移两部分;然后在页表中查找该页号,找到后,读出与该页号对应的物理块号,并与页内位移拼接成需要访问的内存物理地址。

60. 分页存储管理中,页表的功能是什么?当系统的地址空间变得非常大时(如 32 位地址空间),会给页表的设计带来什么样的新问题?请给出一种解决方案,分析它的优点和缺点。(中国科技大学 1996 年研究生试题)

答:在分页系统中,允许将进程中的每一页离散地存储在内存中的任一物理块上。为了能保证进程的正常运行,系统建立了一个页表,该页表记录了进程地址空间所有页对应内存中的物理块号,进程执行时则通过查找页表来确定该页在内存中的实际位置(地址)。因此页表的作用就是实现出页号到物理块号的地址映射。

当地址空间变得非常大时,页表也随之变得非常庞大。对于一个具有 32 位地址空间的分页系统,如果设页面大小为 4KB,则页表的表项可达 1M 个。假设每个页表项占 4 个字节(32 位),则每个进程的页表项就要占用 4MB 的内存空间,并且这个空间还要求是连续的,这显然是不现实的。要解决这个问题,一种方法是采用地址散列变换的办法,因为一个进程实际占用的存储空间并不很大,所以可以采用地址散列变换的方法将页号映射到一个较小的页表中。另一种办法可采用多级页表,即:

(1)页表所需空间也采用离散分配;

(2)将当前所需的那部分页表装入内存，其余不用的表项驻留在外存，仅当需要时调入内存。

具体实现是采用两级页表，并对页表也进行分页，使每个页表大小与内存物理块大小相同，并为它们进行编号，将各页面放入不同的物理块中。此外，为这些离散分配的页表再建立一个外层页表。这样，进程运行中的地址变换过程为：根据逻辑地址，首先将其分解为外层页号、内层页号和页内位移；接着查找外层页表，找到其页号对应的物理块号(存放着内层页表)，然后再在这物理块中查内层页号得到要查找的内存物理块号，并将其与页内位移拼接得到物理地址。

61. 简述 LRU、NRU、和 LFU 这 3 种页面置换算法的思想，并给出一种可能的实现方案。(中国科技大学 1998 年研究生试题)

答：LRU 算法利用"最近的过去"作为"最近的将来"的一种近似，选择最近最久未被使用过的页面予以淘汰。LRU 算法赋予每个页面一个访问字段，用其记录该页面自上次被访问以来所经历的时间 T。当需要淘汰一个页面时，总是选择现有页面中 T 值最大的页面予以淘汰。

NRU 算法总是选择在最近一个时期内未被访问过的页面予以淘汰。采用该算法时，应在页表中给每一页增设一个访问位。当某页被访问时其访问位置为 1。系统周期性地对所有访问位清零。当需要淘汰一页时，从访问位为 0 的页面中选择一页予以淘汰。

LFU 算法总是选择在最近时期使用最少的页面予以淘汰。采用该算法时，应在页表中给每一页增设一个访问计数器，每当该页被访问时，访问计数器加 1，而发生缺页中断时，则淘汰计数器值最小的那一页，同时对所有计数器清零。

62. 局部与全局页面淘汰算法有何区别？为什么在多道系统中用局部页面淘汰算法？

(西北大学 1998 年研究生试题)

答：局部页面置换只是在当前进程中分配的页面里选择一个页面进行置换，全局页面置换是针对整个内存用户区的页面置换。在多道程序系统中采用局部页面置换的原因是：即使某个进程发生了抖动现象，也不至于影响到其他进程，这种方法可以把抖动限制在一个较小的范围内。

63. 何为静态链接？何为动态链接？请求页式采用什么链接技术？在虚拟段式系统中，缺页中断和链接中断各完成何种功能？(北方交通大学 1996 年研究生试题)

答：通常的用户源程序经过编译后会形成若干个目标程序，必须再经过链接以形成可执行程序后方可执行，这种在装入时进行的链接称为静态链接。动态链接是指在作业运行之前并不将几个目标程序链接起来，而是将主程序所对应的目标程序装入内存并启动运行。当运行中需要调用某段时，才将该段(目标程序)调入内存并进行链接。可见，动态链接要求以段作为管理单位。因此，请求页式只能采用静态链接。在虚拟段式系统中，链接中断是在所访问的段不在内存时，在系统的现行分段表中建立一个表目(若曾调入过，则只需改变状态)，为调进的段建立一张页表，并在其段表的相应表目中登记此页表的始址。而缺页中断是当所访问页不在内存时在内存中找出空闲的存储块。如果没有，则调用置换算法移去内存中的一页，然后调进所需的页面，最后修改相应的页表表目。

64. 解决大作业和小内存的矛盾有哪些途径？简述其实现思想。(上海交通大学 1997 年研究生试题)

答：早期解决大作业和小内存矛盾的途径是采用覆盖技术，即一个程序不需要把所有的指

令和数据都装入内存，而是将程序划分为若干个功能相对独立的程序段，并根据程序的逻辑结构让那些不会同时执行的程序段共享同一块内存区(称为覆盖区)。这样，在程序执行中，根据需要再调入相应的程序段到覆盖区，由此达到小内存运行大作业的目的。

现代操作系统解决大作业和小内存矛盾的途径是采用虚拟存储器。虚拟存储器是指仅将作业的一部分装入内存便可运行的存储器系统，虚拟存储器具有请求调入功能和置换功能，并且包括能够实现逻辑地址到物理地址的变换机构。这样，虚拟存储器从逻辑上实现了对内存容量的扩充。虚拟存储器都是建立在离散分配存储管理方式基础上的，如请求页式虚拟存储器，可以将页面离散地分配在内存中，段式虚拟存储器则是将段离散地分配在内存中，而段页式虚拟存储器则包括了段式和页式两者的特点。

65. 某系统采用固定式分区管理，内存分区示意如图 3.22 所示。现有大小为 1KB、9KB、33KB、121KB 的多个作业要求进入内存，试画出它们进入内存后的空间分配情况，并说明主存浪费有多大？

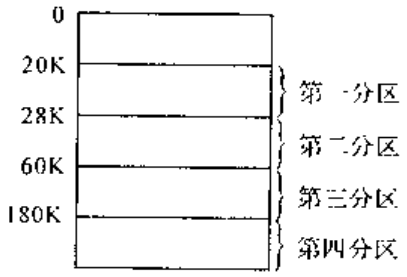


图 3.22

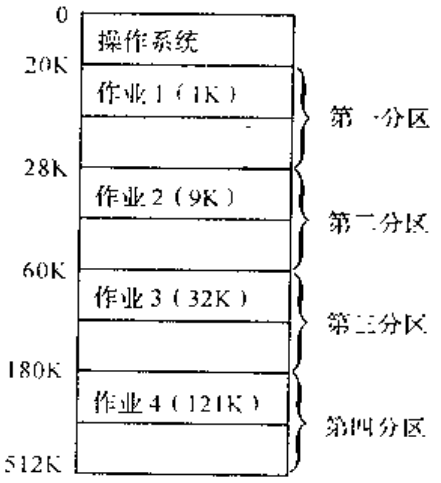


图 3.23

答：作业进入系统后内存分配情况如图。主存空间浪费 328KB。

66. 某系统的空闲分区表如表 3-21 所示，采用可变式分区管理策略，现有如下作业序列：96KB、20KB、200KB。若用首次适应算法和最佳适应算法来处理这些作业序列，则哪一种算法可满足该作业序列请求，为什么？

表 3-21 空闲分区表

分区号	大小	起始地址
1	32KB	100 KB
2	10 KB	150 KB
3	5 KB	200 KB
4	218 KB	220 KB
5	96 KB	530 KB

答：采用最佳适应算法可满足该作业序列需求，采用首次适应算法无法满足该作业序列需求。(具体步骤略)

67. 若某页式管理系统的主存容量为 64KB，被分成 16 块，其块号为 0、1、2、…、15，设某作业有

4 页，其页号为 0、1、2、3，被分别装入主存的 2、4、1、5 块中，试问：

(1)该作业的总长度(字节)是多少？

(2)请写出该作业每一页主存中的起始地址。(陕西省 1996 年自考题)

答：(1)64/16 = 4KB，即主存一块长度为 4KB，由于页式管理系统作业中的页长度等于主存中的块长度，故该作业的总长度为 4KB×4 = 16KB。

(2)该作业每一页在主存中的起始地址分别为：4KB×2 = 8KB、4KB×4 = 16KB、4KB×1 = 4KB、4KB×5 = 20KB。

68. 在一分页存储管理系统中，页面大小为 4KB，系统中的地址寄存器占 24 位，给定页表如表 3-22 所示。现给定一逻辑地址，页号为 3，页内地址为 100，计算相应的物理地址，并画图说明地址变换过程。(注：本题数值采用 10 进制)(陕西省 1996 年自考题)

表 3-22

页号	块号
0	3
1	4
2	9
3	7
·	·
·	·
·	·

答：相应的物理地址为：4096×7 + 100 = 28772

地址变换过程如图 3.24 所示。

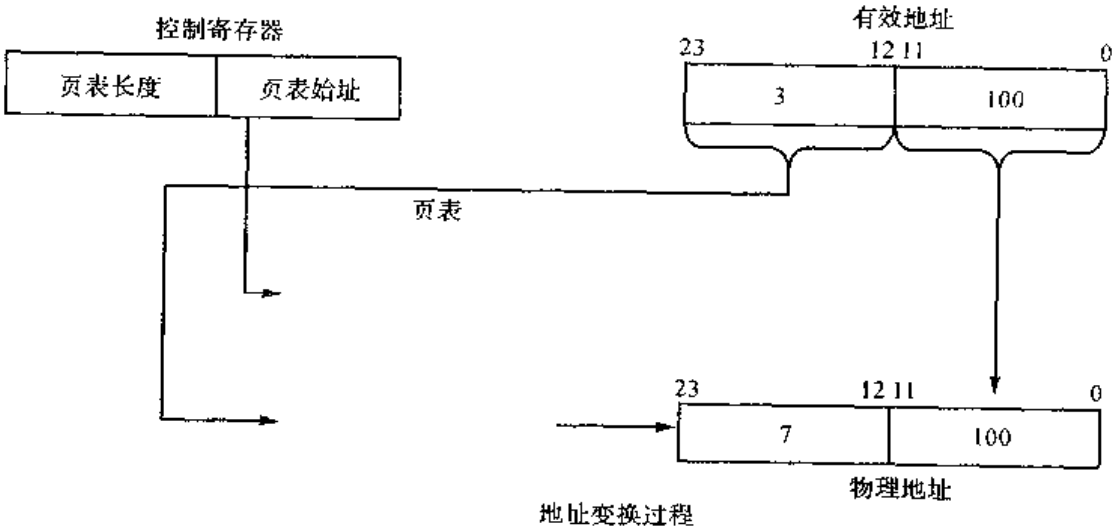


图 3.24　地址变换过程

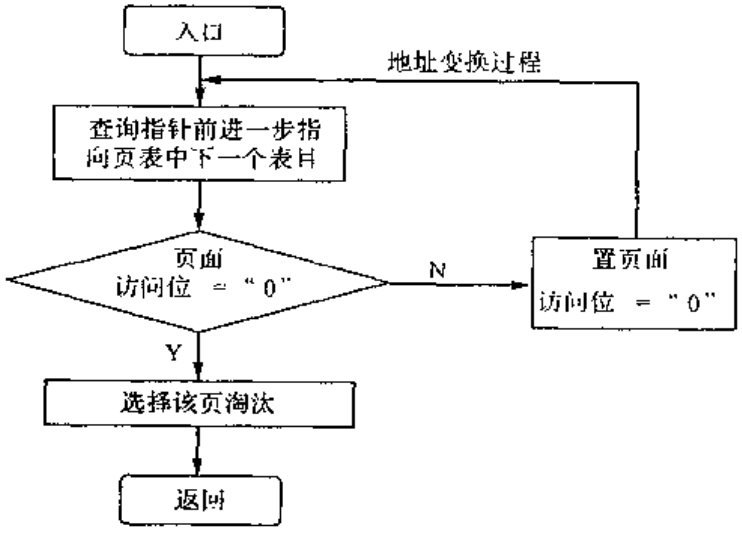
69. 设计一个 LRU 淘汰算法的近似算法(指出所需硬件支持、数据结构及软件处理流程或类 Pascal 描述)。(西北大学 1998 年研究生试题)

块号	页号	指针(指向块号)
0		
1	3	3
2		
3	4	5
4	1	∧
5	2	4

淘汰指针

图 3.25

答:在页表中的每一存储块号后设一"页面访问"位。当一存储块中的页面被访问时,其相应的"页面访问"位由硬件自动置为"1",并由页面管理软件周期性地把所有的页面访问位重新置为"0"。这样,在时间 T 内,某些被访问的页面其对应访问位为"1",而未被访问的页面其对应访问位为"0"。当需要置换某一页时,选择其访问位为"0"的页面,处理流程如图 3.25。该算法就是循环地查寻页面访问位为"0"的页面。在查找过程中,那些被访问的页所对应的访问位重新被置为"0"。由此可见,实际上这种近似 LRU 的算法已退化为"最近不用"(NRU)算法。



NRU 算法

图 3.26 NRU 算法

70. 某一时刻,存储分块表的状态如图 3.26 所示,现有作业 J 的页 5 需要入页,试问按 FIFO 算法,应淘汰哪一页? 并画出淘汰后的存储分块表。

答:应淘汰第 3 页,淘汰后的存储分块表示意图如图 3.27 所示:

块号	页号	指针
0		
1	5	∧
2		
3	4	5
4	1	1
5	2	4

淘汰指针

图 3.27 存储分块示意图

71. 在一个请求分页系统中,假如系统分配给一个作业的物理块数为 3,并且此作业的页面走向为 2,3,2,1,5,2,4,5,3,2,5,2。试用 FIFO 和 LRU 两种算法分别计算出程序访问过程

中所发生的缺页次数。(中科院软件所 1999 年研究生试题)

答:采用先进先出(FIFO,表 3-23)淘汰算法产生的缺页为 9 次。采用最近最少使用(LRU,表 3-24)淘汰算法产生的缺页次数为 7 次。

表 3-23　FIFO

走向	2	3	2	1	5	2	4	5	3	2	5	2
块 1	2	3		1	1	1		4		4	4	2
块 2		2		3	3	2		2		2	5	5
块 3				2	5	5		5		3	3	3
缺页	缺	缺		缺	缺	缺		缺		缺	缺	缺

表 3-24　LRU

走向	2	3	2	1	5	2	4	5	3	2	5	2
块 1	2	2	3	3	2	1	5	2	4	5	3	3
块 2		3	2	2	1	5	2	4	5	3	2	5
块 3				1	5	2	4	5	3	2	5	2
缺页	缺	缺		缺	缺		缺		缺	缺		

72. 纯分页系统和请求式分页系统的主要差别是什么？假定在一个请求页式存储管理系统中，某作业 J 所涉及的页面依次为：

3,2,1,4,4,5,3,4,3,2,1,5

并已知主存中有 3 个可供作业 J 使用的空白存储块(块的大小与页面大小相同)，试说明采用 FIFO 和 LRU 两种算法进行页面置换时，缺页中断的次数各是多少？(南开大学 1998 年研究生试题)

答:纯分页系统和请求式分页系统的主要差别是:纯分页系统要求将作业的页面在运行前一次性全部装入内存，而请求式分页系统则在运行前只装入作业的一部分页面即可运行，当需要访问不在内存的页面时，再由系统将其调入。

作业 J 采用先进先出(FIFO,表 3-25)淘汰算法产生的缺页为 9 次，采用最近最少使用(LRU,表 3-26)淘汰算法产生的缺页也为 9 次。

表 3-25　FIFO

走向	3	2	1	4	4	5	3	4	3	2	1	5
块 1	3	2	1	1		1	3			3	3	5
块 2		3	2	2		5	5			5	1	1
块 3			3	4		4	4			2	2	2
缺页	缺	缺	缺	缺		缺	缺			缺	缺	缺

表 3-26 LRU

走向	3	2	1	4	4	5	3	4	3	2	1	5
块 1	3	3	3	2		1	4	5	5	4	3	2
块 2		2	2	1		4	5	3	4	3	2	1
块 3			1	4		5	3	4	3	2	1	5
缺页	缺	缺	缺	缺		缺	缺			缺	缺	缺

73. 一进程已分配到 4 个页帧(page frame)，如表 3-27 所示(所有数字都为 10 进制数，且以 0 开始)。当进程访问第 4 页时，产生缺页中断，请分别用 FIFO(先进先出)、LRU(最近最少使用)、NRU(最近不用)算法，决定缺页中断服务程序选择换出的页面。(浙江大学 2000 年研究生试题)

表 3-27 进程使用页帧情况表

虚拟页号	页帧	装入时间	最近访问时间	访问位	修改位
2	0	60	161	0	1
1	1	130	160	0	0
0	2	26	162	1	0
3	3	20	163	1	1

答：(1) FIFO 算法：根据算法规则，最先进入的页帧号应最先替换。因此访问第 4 页时，缺页中断程序选择的是 3 号页帧。由于该页帧的修改位是 1，在换出主存后应进行回写，即重新保存。

(2)LRU 算法：根据算法规则，最近一次访问时间离当前最远的页帧应当被选择换出，因此缺页中断程序选择的是 1 号页帧。

(3)NRU 算法：根据算法规则，应当选择访问位为 0 的页帧换出主存，因此缺页中断程序选择 0 号页换出。

74. 对访问串 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5，指出在驻留集大小分别为 3、4 时，使用 FIFO 和 LRU 替换算法的页故障数。结果说明了什么？(国防科技大学 1996 年研究生试题)

答：采用 FIFO 算法，当 m = 3 时，页故障数 = 9；当 m = 4 时，页故障数 = 10；采用 LRU 算法，当 m = 3 时，页故障数 = 10；当 m = 4 时，页故障数 = 8。

结果说明：FIFO 算法有 Belady 奇异现象，即不满足驻留集增大页故障数一定减少的规律。此外，在 m = 3 时 LRU 算法的页故障率要比 FIFO(表 3-28)算法多，所以 LRU(表 3-29)算法并不总优于 FIFO 算法。

表 3-28(1) FIFO

走向	1	2	3	4	1	2	5	1	2	3	4	5
块 1	1	2	3	3	3	2	2			2	4	
块 2		1	2	2	1	1	1			3	3	
块 3			1	4	4	4	5			5	5	
缺页	缺	缺	缺	缺	缺	缺	缺			缺	缺	

表 3-28(2)　FIFO

走向	1	2	3	4	1	2	5	1	2	3	4	5
块 1	1	2	3	4			4	4	4	3	3	3
块 2		1	2	3			3	3	2	2	2	2
块 3			1	2			2	1	1	1	1	5
块 4				1			5	5	5	5	4	4
缺页	缺	缺	缺	缺			缺	缺	缺	缺	缺	缺

表 3-29(1)　LRU

走向	1	2	3	4	1	2	5	1	2	3	4	5
块 1	1	1	1	2	3	4	1	2	5	1	2	3
块 2		2	2	3	4	1	2	5	1	2	3	4
块 3			3	4	1	2	5	1	2	3	4	5
缺页	缺	缺	缺	缺	缺	缺	缺	缺	缺	缺	缺	缺

表 3-29(2)　LRU

走向	1	2	3	4	1	2	5	1	2	3	4	5
块 1	1	1	1	1	2	3	4	4	4	5	1	2
块 2		2	2	2	3	4	1	2	5	1	2	3
块 3			3	3	4	1	2	5	1	2	3	4
块 4				4	1	2	5	1	2	3	4	5
缺页	缺	缺	缺	缺			缺			缺	缺	缺

75. 有一请求分页式存储管理系统，页面大小为每页 100 字节，有一个 50×50 的整型数组按行连续存放，每个整数占两个字节，将数组初始化为 0 的程序描述如下：

```
VAR a:array[1..50,1..50] of integer;
    i,j:integer
for i:=1 to 50 do
for j:=1 to 50 do
a[i,j]:=0;
```

若在程执行时内存只有一个存储块用来存放数组信息，试问该程序执行时产生多少次缺页中断？

答：由题可知，该数组中有 2500 个整数，每个整数占用 2 个字节，共需存储空间 5000 字节；而页面大小为每页 100 字节，所以数组占用 50 页存储空间。由于初始化程序是按行进行的，因此每次缺页中断调进一页后，位于该页内的数组元素全部被赋了 0，然后再调入下一页，因此总的缺页次数为 50 次。

76. 在一分页式存储管理系统中，某作业的页表如表 3-30 所示。已知页面大小为 1024 字节，试将逻辑地址 1011，2148，3000，4000，5012 转化为相应的物理地址。

表 3-30 页表

页号	块号
0	2
1	3
2	1
3	6

答:逻辑地址与物理地址的转换如表 3-31。

表 3-31 逻辑地址转换为物理地址

逻辑地址	物理地址
1011	3059
2148	1124
3000	1976
4000	7072
5012	越界

77. 设一作业共有 5 页(0～4),其中程序占 3 页(0～2 页)、常数占一页(第 3 页)、工作单元占一页(第 4 页),他们依次放在外存的 45,46 页和 98,99,100 页。现程序段已分配在内存的 7,10,19 页,而常数区和工作区尚未获得内存。请回答下述问题:

(1)页表应包含哪些项目?填写此页表。若工作区分配到内存的第 9 页,则页表如何变化?

(2)在运行中,因需要使用常数而发生中断,假定此时内存无空闲页面,需要把第 9 页淘汰,操作系统应如何处理?页表又发生什么变化?

答:(1)页表及其变化如表 3-32。

表 3-32 页表

作业页号	在内存标志	存取方式	外存页号	内存页号	修改标志
0	1(在内存)	执行	45	7	
1	1	执行	46	10	
2	1	执行	98	19	
3	0→1	读	99	9	
4	0→1→0	读/写	100	9	

(2)在把第 9 页淘汰之前,先检查其修改标志,若此页内存已发生过写操作,则说明与外存对应的页面副本不一致,必须重新写回外存,然后才能分配给常数区。

78. 设在处理机上执行的一个进程的页表如表 3-33 所示,表中的虚页号和物理块是十进制数,起始页号(块号)均为 0,所有的地址均是存储器字节地址,页的大小为 1024 字节。

(1)详述在设有快表的请求分页存储管理系统中,一个虚地址转换成物理地址的过程。

(2)下列虚地址对应于什么物理地址:5499,2221。(中科院计算所 1999 年研究生试题)

表 3-33　进程的页表

虚页号	状态位	访问位	修改位	物理块号
0	1	1	0	4
1	1	1	1	7
2	0	0	0	—
3	1	0	0	2
4	0	0	0	—
5	1	0	1	0

答：(1)进行地址变换时首先检索快表(联想存储器)，如果在快表中找到所要访问的页号(页表项)，便修改页表项中的访问位；对于写指令还要置修改位为“1”；最后利用该页表项中给出的物理块号和页内位移形成要访问的物理地址。

如果在快表中未找到该页的页表项，则到内存中的页表中查找(实际上是与快表同时进行)，并从找到的页表项中的状态来了解该页是否已调入内存。若该页已调入内，则将该页的页表写入快表；如果快表已满则按某种算法淘汰快表中的一个页表项，然后再写入该页的页表项。若该页尚未调入内存，则产生缺页中断并由系统从外存把该页调入内存。

(2)$5499 = 1024 \times 5 + 379$，查表知虚页号 5 对应的物理块号为 0，也即物理地址为 379。

$2221 = 1024 \times 2 + 173$，查表知虚页号 2 应的物理块号为空，故产生缺页中断，待将该页调入内存后再确定其物理地址。

79. 假定某操作系统存储器采用页式存储管理，一进程在联想存储器中的页表如表 3-34 所示，不在联想存储器的页表项如表 3-35 所示。(国防科技大学 1998 年研究生试题)

假定该进程体(程序与数据)代码长度为 320 字节，每页 32 字。现有逻辑地址(八进制)为 101、204、576，如果上述逻辑地址能翻译成物理地址，则说明翻译的过程，并指出具体的物理地址；如果上述逻辑地址不能翻译成物理地址，说明为什么？

表 3-34　联想存储器中的页表

页号	页帧号
0	f1
1	f2
2	f3
3	f4

表 3-35　内存中的页表

页号	页帧号
4	f5
5	f6
6	f7
7	f8
8	f9
9	f10

答：一页大小等于 32 字节，也即其二进制长度为 5；由此得逻辑地址结构：低 5 位为页内位

移,其余高位为页号。

101(八进制)=001000001(二进制),则页号为2,在联想存储器中,对应的页帧号为f3,即物理地址为(f3)1。

204(八进制)=010000100(二进制),则页号为4,不在联想存储器中,查内存的页帧表得页帧号为f5,并用其更新联想存储器中的一项,最终的物理地址为(f5)4。

576(八进制)=101111110(二进制),则页号为11,已超出页表范围,即产生越界中断。

80. 假定一个分页系统的页表存放在内存中,试问:

(1)如果访问内存一次需要花费1.2μs,那么存取一次数据至少需多少时间?

(2)如果增加联想存储器,其命中率可达75%,如果联想存储器中的查找时间可以忽略,那么存取一次数据的平均时间是多少?

答:(1)在分页存储系统中,每存取一次数据至少要访问两次内存。所以存取一次数据至少需要时间为:$2\times1.2=2.4(\mu s)$。

(2)增加联想存储器后存取一次数据的平均时间是:$75\%\times0+(1-75\%)\times1.2+1.2=1.5(\mu s)$。

81. 现有一请求页式系统,页表保存在寄存器中。若有一个可用的空页或被置换的页未被修改,则它处理一个缺页中断需要8ms;若被置换的页已被修改,则处理一缺页中断因增加写回外存时间而需要20ms,内存的存取时间为1μs。

假定70%被置换的页被修改过,为保证有效存取时间不超过2μs,可接受的最大缺页中断率是多少?

答:设最大缺页中断率为ρ,则有:

$$(1-\rho)\times1\mu s+0.3\rho\times8ms+0.7\rho\times20ms=2(\mu s)$$

即: $$-\rho+2400\rho+14000\rho=1$$

解得: $$\rho\approx0.00006$$

82. 试述缺页中断与一般中断的主要区别。

答:缺页中断作为中断,同样需要经历保护CPU现场、分析中断原因、转缺页中断处理程序进行处理、恢复CPU现场等步骤。但缺页中断又是一种特殊的中断,它与一般中断的主要区别是:

a)在指令执行期间产生和处理中断信号。通常,CPU都是在一条指令执行完后去检查是否有中断请求到达。若有便去响应中断;否则继续执行下一条指令。而缺页中断是在指令执行期间,发现所要访问的指令或数据不在内存时产生和处理的。

b)一条指令在执行期间可能产生多次缺页中断。例如,对于一条读取数据的多字节指令,指令本身跨越两个页面,假定指令后一部分所在页面和数据所在页面均不在内存,则该指令的执行至少产生两次缺页中断。

83. 一请求分页存储管理系统,页面大小为每页100字节。有一个50×50的整型数组按行连续存放,每个整数占两个字节,将数组初始化为0的程序描述如下:

```
int a[50][50];
    int i, j;
    for (i=0;i(=49;i++)
      for (j=0;j(=49;j++)
        a[i][j]=0;
```

若在程序执行时内存中只有一个存储块用来存放数组信息，试问该程序执行时产生多少次缺页中断？

答：由题目可知，该数组中有2500个整数，每个整数占用2个字节共需存储空间5000个字节；而页面大小为每页100字节，数组占用空间50页。假设数组从该作业的第m页开始存放，则数组分布在第m页袂第m+49页中，它在主存中的排列顺序为：

a[0][0],a[0][1],…,a[0][49]　　第m页
a[1][0],a[1][1],…,a[1][49]　　第m+1页
…
a[49][0],a[49][1],…,a[49][49]　　第m+49页

由于该初始化程序是按行进行的，因此每次缺页中断调进一页后，位于该页内的数组元素全部赋予0值，然后再调入下一页，所以涉及的页面走向为m,m+1,…,m+49，故缺页次数为50次。

84. 试述分页系统和分段系统的主要区别

答：分页和分段有许多相似之处，比如两者都不要求作业连续存放。但在概念上两者完全不同，主要表现在以下几个方式：

(1)页是信息的存储单位，分页是为了实现非连续分配，以解决内存碎片问题，或者说分页是由于系统管理的需要。段是信息的逻辑单位，它含有一组意义相对完整的信息，分段的目的是为了更好地实现共享，满足用户的需要。

(2)页的大小固定且由系统确定，将逻辑地址划分为页号和页内地址是由机器硬件实现的，而段的长度却不固定，决定于用户所编写的程序，通常由编译程序在对源程序进行编译时根据信息的性质来划分。

(3)分页的作业地址空间是一维的。分段的地址空间是二维的。

85. 设有一页式存储管理系统，向用户提供的逻辑地址空间最大为16页，每页2048字节，内存总共有8个存储块，试问逻辑地址至少为多少位？内存空间有多大？

答：本题中，每页2048字节，所以页内位移部分地址需要占据11个二进制位；逻辑地址空间最大为16页，所以页号部分地址需要占据4个二进制位。故逻辑地址至少应为15位。由于内存共有8个存储块，在页式存储管理系统中，存储块大小与页面的大小相等，因此内存空间为16K。

86. 某操作系统采用可变分区分配存储管理方法，用户区为512K且始值为0，用空闲分区表管理空闲分区。若分配时采用分配空闲低地址部分的方案，其初始时用户区的512K空间空闲，对下述申请序列：

申请300K，申请100K，释放300K，申请150K，申请30K，，申请40K，申请60K，释放30K

回答下列问题：

(1)采用首次适应算法，空闲分区中有那些空块(给出始址，大小)？

(2)采用最佳适应算法，空闲分区中有那些空块(给出始址，大小)？

(3)如再申请100K，针对(1)和(2)各有什么结果？

答：(1)采用首次适应算法，在完成了题目所给的系列申请及释放内存操作后，内存分配情况如图3.28所示(用灰色表示空闲空间)，空闲分区表如表3-36所示。

表 3-36

分区	大小	起始地址
0	30K	150K
1	20K	280K
2	112	400K

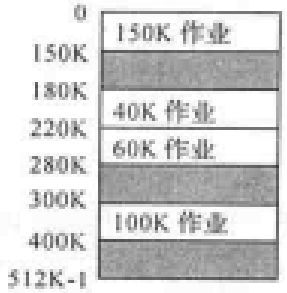


图 3.28 采用首次适应算法的内存分配情况



图 3.29 采用最佳适应算法的内存分配情况

(2)采用最佳适应算法,完成了题目所给的系列申请及释放内存操作后,内存分配情况如图 3.29 所示(用灰色表示空闲空间),空闲分区表如表 3-37 所示:

表 3-37

分区	大小	起始地址
0	30K	400K
1	42K	470K
2	90K	210K

(3)如再申请 100K 空间,由上述结果可知,采用首次适应算法后剩下的空闲分区能满足这一申请要求;而采用最佳适应算法后剩下的空闲分区不能满足这一申请要求。

87. 有一页式系统,其页表存放在主存中。

(1)如果对主存的一次存取需要 1.5 微秒,试问实现一次页面访问的存取时间是多少?

(2)如果系统加有快表,平均命中率为 85%,当页表项在快表中时,其查找时间忽略为 0,试问此时的存取时间为多少?

答:若页表存放在主存中,则要求实现一次页面访问需访问内存两次,一次是访问页表,确定所存取页面的物理地址,第二次才根据该地址存放页面数据。

(1)由于页表存放在主存,因此 CPU 必须访问主存两次才能获得所需数据,所以实现一次页面访问的存取时间是 $1.5\times2=3(\mu s)$

(2)在系统增加了快表后,在快表中找到页表项的概率为 85%,所以实现一次页面访问的存取时间为 $0.85\times1.5+(1-0.85)\times2\times1.5=1.725(\mu s)$

88. 在一个请求分页存储管理系统中,一个作业的页面走向为 4,3,2,1,4,3,5,4,3,2,1,5,当分配给作业的物理块数分别为 3、4 时,试计算采用下述页面淘汰算法时的缺页率(假设开始执行时主存中没有页面),并比较结果。

(1)最佳置换淘汰算法。

(2)先进先出淘汰算法。

(3)最近最久未使用淘汰算法。

答:(1)根据所给页面走向,使用最佳页面淘汰算法时,页面置换情况如表 3-38 所示:

表 3-38(1)

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	4			4			2	2	
块 2		3	3	3			3			3	1	
块 3			2	1			5			5	5	
缺页	缺	缺	缺	缺			缺			缺	缺	

缺页率为:7/12

表 3-38(2)

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	4			4				1	
块 2		3	3	3			3				3	
块 3			2	2			2				2	
块 4				1			5				5	
缺页	缺	缺	缺	缺			缺				缺	

缺页率为:6/12

由上述结果可以看出,增加分配作业的内存块数可以降低缺页率。

(2)根据给页面走向,使用先进先出页面淘汰算法时,页面置换情况如表 3-39 所示:

表 3-39(1)

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	1	1	1	5			5	5	
块 2		3	3	3	4	4	4			2	2	
块 3			2	2	2	3	3			3	1	
缺页	缺	缺	缺	缺	缺	缺	缺			缺	缺	

缺页率为:9/12

表 3-39(2)

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	4			5	5	5	5	1	1
块 2		3	3	3			3	4	4	4	4	5
块 3			2	2			2	2	3	3	3	3
块 4				1			1	1	1	2	2	2
缺页	缺	缺	缺	缺			缺	缺	缺	缺	缺	缺

缺页率为:10/12

由上述结果可以看出,对先进先出算法而言,增加分配作业的内存块数反而使缺页率上升,这种异常现象称为 Belady 现象。

(3)根据所给页面走向,使用最近最久未使用页面淘汰算法时,页面置换情况如表 3-40:

表 3-40(1)

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	1	1	1	5			2	2	2
块 2		3	3	3	4	4	4			4	1	1
块 3			2	2	2	3	3			3	3	5
缺页	缺	缺	缺	缺	缺	缺	缺			缺	缺	缺

缺页率为:10/12

表 3-40(2)

走向	4	3	2	1	4	3	5	4	3	2	1	5
块 1	4	4	4	4			4			4	4	5
块 2		3	3	3			3			3	3	3
块 3			2	2			5			5	1	1
块 4				1			1			2	2	2
缺页	缺	缺	缺	缺			缺			缺	缺	缺

缺页率为:8/12

由上述结果可以看出,增加分配作业的内存块数可以降低缺页率。

89. 在一分页存储管理系统中,逻辑地址长度为 16 位,页面大小为 4096 字节,现有一逻辑地址为 2F6AH,且第 0,1,2 页依次存放在物理块 5,10,11 中,问相应的物理地址为多少?

答:由题目所给条件可知,本页式系统的逻辑地址结构。如图 3.39

逻辑地址 2F6AH 的二进制表示如下:

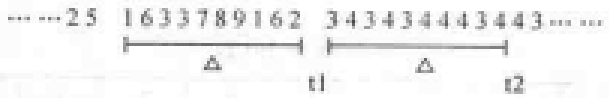
$$\underbrace{0010}_{P}\ \underbrace{111101101010}_{W}$$



图 3.39

由此可知逻辑地址 2F6AH 的页号为 2,该页存放在第 11 号物理块中,用十六进制表示块号为 B,所以物理地址为 BF6AH。

90. 在虚拟页式存储管理中,为解决抖动问题,可采用工作集模型以决定分给进程的物理块数,有如下页面访问序列:



窗口尺寸 △ =9,试求 t1、t2 时刻的工作集。

答:一个进程在时间 t 的工作集可形式化地定义为:

W(t,h)={在时间 t−h 到 t 之间所访问的一串页面}

其中,h 为工作集窗口尺寸。

由题目所给条件可知,t1 时刻的工作集为:{1,2,3,6,7,8,9}

t2 时刻的工作集为:{3,4}

91. 在一个请求分页系统中,假定系统分配给一个作业的物理块数为 3,并且此作业的页面走向为 2,3,2,1,5,2,4,5,3,2,5,2。试用 FIFO 和 LRU 两种算法分别计算出程序访问过程中所发生的缺页次数。(中国科学院软件研究所 1999 年试题)

答:在本题中,分配给作业的物理块数为3。

(1)根据所给页面走向,使用FIFO算法时,页面置换情况如表3-41所示:

表3-41

走向	2	3	2	1	5	2	4	5	3	2	5	2
块1	2	2		2	5	5	5		3		3	3
块2		3		3	3	2	2		2		5	5
块3				1	1	1	4		4		4	2
缺页	缺	缺		缺	缺	缺	缺		缺		缺	缺

缺页次数为:9

(2)根据所给页面走向,使用LRU算法时,页面置换情况如表3-42所示:

表3-42

走向	2	3	2	1	5	2	4	5	3	2	5	2
块1	2	2		2	2		5		5	5		
块2		3		3	5		2		3	3		
块3				1	1		4		4	2		
缺页	缺	缺		缺	缺		缺		缺	缺		

缺页次数为:7

92. 在采用页式存储管理的系统中,某作业J的逻辑地址空间为4页(每页2048字节),且已知该作业的页面映象表(即页表)如表3-43所示:

表3-43

页号	块号
0	2
1	4
2	6
3	8

试借助地址变换图(即要求画出地址变换图)求出有效逻辑地址4865所对应的物理地址。(南开大学1994年研究生试题)

答:在本题中,一页大小为2048字节,则逻辑地址4865的页号及页内位移为:

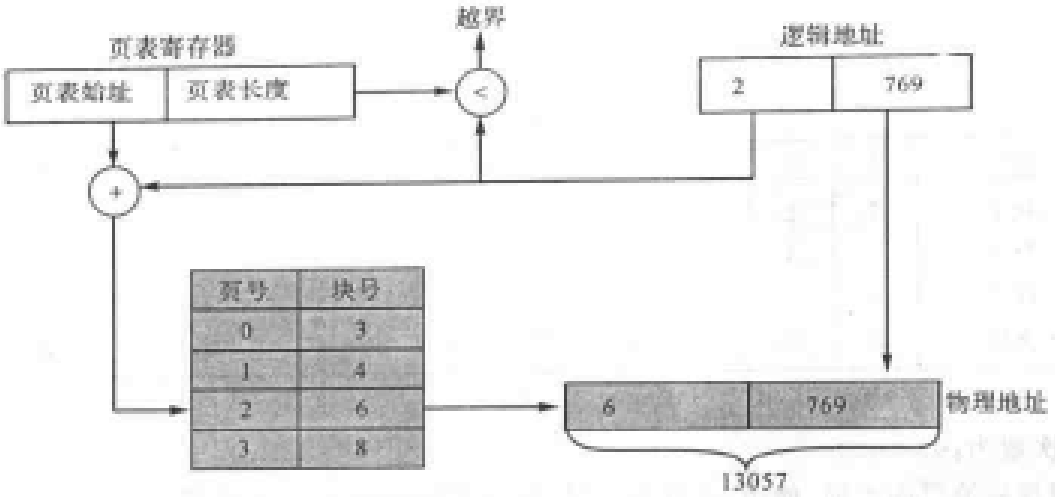
页号　　$4865/2048=2$

页内位移　　$4865-2048\times 2=769$

然后,通过页表查知物理块号为6,将物理块号与逻辑地址中的页内位移拼接,形成物理地址,即:

$$6\times 2048+769=13057$$

其地址变换过程如图3.31所示。



页式存储管理系统中4865的地址变换示意图

图 3.31 页式存储管理系统中 4865 的地址变换示意图

93. 有一 OS 采用段式管理，用户区主存为 512KB，空闲块链入空闲块表，分配时截取空块的前半部分(小地址部分)。初始时全部空闲。在执行了如下申请，释放操作序列后：

req (300KB), req(100KB), release(300KB), req(150KB), req(50KB), req(90KB)

(1)采用最先适配，空闲表中有哪些空块。(指出大小及始址)

(2)采用最佳适配，空闲表中有哪些空块。(指出大小及始址)

(3)若随后又要申请 80KB，针对上述两种情况会产生什么后果？这说明了什么问题？

94. 在某个采用页式存储管理的系统中，现有 J1, J2, J3 共 3 个作业同驻主存。其中 J2 有 4 个页面，被分别装入到主存的第 3, 4, 6, 8 块中。假定页面和存储块的大小均为 1024 字节，主存容量为 10K 字节。

(1)写出 J2 的页面映像表；

(2)当 J2 在 CPU 上运行时，执行到其地址空间第 500 号处遇到一条传输指令

MOV 2100, 3100

请你用地址变换图计算出 MOV 指令中两个操作数的物理地址。(南开大学 1997 年研究生试题)

95. 覆盖技术与虚拟存储技术有何本质不同？交换技术与虚拟存储器中使用的调入/调出技术有何不同？

(西安交通大学 2000 年研究生试题)

3.5 OSC 习题与解答

Chapter 9

9.1 请列举逻辑地址和物理地址的两个不同之处。

答：不同点之一，逻辑地址是由编程语言、编译系统或连接程序生成的地址，在众多存储管理系统里，它往往不能直接访问物理内存；物理地址就是直接访问物理内存的地址。

不同点之二，逻辑地址决定于操作系统，进程的逻辑地址空间是连续的；物理地址决定于计算机硬件系统的存储系统，物理地址空间可以是不连续的。

9.2 请解释内部碎片与外部碎片的差别。

答：内部碎片是处于区域内部或页面内部的存储块。占有这些区域或页面的进程并不使用这个存储块。而在进程占有这个存储块时，系统无法利用它。直到进程释放它，或进程结束时，系统才有可能利用这个存储块。

外部碎片是处于任何已分配区域或页面外面的空闲存储块。这些存储块的总和可以满足当前申请的长度要求，但是由于这些它们的地址不连续，或其他原因，使得系统无法满足当前申请。

9.3 请描述下列存储分配算法：

A. First fit

B. Best fit

C. Worst fit

答：A. First-fit：从头到尾搜索整个有效空间。找到第1个满足条件的存储块时，即返回。

B. Best-fit：搜索整个有效空间。找出满足条件的存储块中，最小的一个，返回。

C. Worst-fit：搜索整个有效空间。找出满足条件的存储块中，最大的一个，返回。

(采用这种策略，是考虑到，被选中的存储块割去所申请的长度后，留下的部分还相对较大，比 Best-fit 算法留下的部分更有用)

9.4 一个进程被换出内存，它就失去了使用 CPU 的机会。除了换出内存这种情形，请列举其他一种情形，进程虽然失去了使用 CPU 的机会，但它并没有被换出内存。

答：当产生一次中断，CPU 转入中断响应程序时，进程虽然失去了使用 CPU 的机会，但它并没有被换出内存。

9.5 某系统有五个固定分区，其长度依次为 100K，500K，200K，300K，600K。今有四个进程，对内存的需求分别是 212K，417K，112K，426K。当分别用 First-fit，Best-fit，Worst-fit 算法响应这四个进程的内存申请时，请分别给出系统的内存分配动态。哪种算法最有效？

答：First-fit：

212K 进程装到 500K 分区

417K 进程装到 600K 分区

112K 进程装到 200K 分区

426K 进程暂时等待

Best-fit：

212K 进程装到 300K 分区

417K 进程装到 500K 分区

112K 进程装到 200K 分区

426K 进程装到 600K 分区

Worst-fit：

212K 进程装到 600K 分区

417K 进程装到 500K 分区

112K 进程装到 388K 分区

426K 进程暂时等待

仅就本题为例，Best-fit 算法是最好的。

9.6 某系统把任一程序都分成代码和数据两部分。CPU 知道什么时候要指令(如取指令周期)，什么时候要数据(如取数据周期或存数据周期)。所以，需要有两组寄存器(基地址寄存器，界限寄存器)，一组用于指令，另一组用于数据。用于指令的(基地址寄存器，界限寄存器)是只读的，以便于多用户的程序共享。请分析这种策略的优缺点。

答：这种策略的优点主要是有效地共享代码和数据。例如，内存中保留一份编辑软件或编译软件地代码，由所有需要编辑软件或编译软件的进程共享它。再一个优点就是便于代码保护，免受非法访问。这种策略的缺点是，它要求任一进程的代码和数据严格分离。

9.7 页面长度为什么正好是 2 的次幂？

答：根据页式存储管理的机理，逻辑地址首先被分成页号和页内偏移量，一般需要一次乘除操作和一次加减操作。如果页面长度正好是 2 的次幂，那么，只需要一次“移位”操作就能取出页号，一次“与”操作就能取出偏移量。效率得到显著提高。

9.8 一个逻辑地址空间由 8 个页面组成，页面长 1024 字。这些页面映射到总长 32 页帧的物理地址空间。

A. 逻辑地址应该占多少位？

B. 物理地址应该占多少位？

答：A. 逻辑地址占 13 位

B. 物理地址占 15 位

9.9 分页式存储管理系统中，进程为什么不能访问不属于它的存储空间？操作系统如何允许进程访问不属于它的存储空间？请分析为什么可以，或为什么不可以。

答：分页式存储管理系统中的逻辑地址由页号和页内偏移量组成。通过进程的页表才可以查到逻辑地址的页号所对应的物理页帧号。因为所有页表的内容都是由操作系统控制的，操作系统就有可能限制任一进程，仅让进程访问已经分配给它的页帧。进程根本无法访问不属于它的页面，因为该页面不在此进程的页表里。

为了让进程访问不属于它的存储空间，可以在进程页表里增加额外的页表项，这些页表项就指向那些物理空间。此法有助于进程间的通信和数据共享，使得共享进程用各自不同的逻辑地址访问相同的物理地址。

9.10 考虑一个分页式存储管理系统，其页表常驻内存。

A. 如果内存访问耗时 200 ns，那么，访问内存中的数据需要多长时间？

B. 如果引入关联寄存器(associative registers)，而且 75% 的页面可以从关联寄存器中找到，那么，此时的有效访问时间(effective memory reference time)为多少？(假设访问关联寄存器的时间可以忽略)

答：A. 400ns，其中，200 纳秒访问页表，200 纳秒访问内存中的数据。

B. Effective access time = 0.75 * 200ns + 0.25 * 400ns = 250ns

9.11 如果对经典的分页式存储管理策略的页表作细微改造，允许同一页表的两个页表项指向同一个物理页帧，由此会有什么结果？怎么利用这种结果，减少内存复制操作(将一个存储段的内容复制到另一个存储段)的时间？在经过改造的存储系统里，修改一个页面中几个字节的值，会对其他页面产生什么影响？

答：让同一页表的两个页表项指向同一个物理页帧，用户可以利用此特点共享该页帧的

代码或数据。如果代码是可重入的，如编辑软件、编译软件、数据库管理系统等，这种方法可节省大量的内存空间。

实现内存"复制"操作时，不需要将页面的内存逐字节复制，而只要在页表里，将指向该页面的指针复制到代表目的地址的页表项。

在这种系统里，如果通过一个页表项修改了一个页面的几个字节，那么通过共享该页面的其他页表项表示的地址，所访问的地址值也跟着变化了。

9.12 为什么有些系统既用分段式存储管理策略，又用分页式存储管理策略？

答：实际系统中经常混合使用分段式存储管理和分页式存储管理，以取长补短。当分页式系统的页表过长时（实际上进程真正使用的地址范围并没有这么多），将进程空间按照其类别（代码、数据、堆栈、堆等）分割成段，采用段表来索引这些段内的页表项，可以节省空间和时间；将不用的空闲页表项，则归到一个段中，置段表项起始地址为0。

相反，当分段式系统的段过长时，分配这样的连续空间相对变得困难。此时，对这种段进一步加以分页，分配给它的页帧则不必要求连续。用此法，同时也避免了外部碎片的产生。

9.13 分段式存储管理系统中，怎么让两个不同的进程共享同一个存储段？

答：分段式存储管理系统的段表项包含了段起始地址和段的长度。两进程共享某一段，就是让进程各自的段表项拥有相同的段起始地址和段长度。还有，共享段的段号，在两个进程里必须是统一的。

9.14 解释采用段式存储管理方式实现可重入模块的共享，比采用页式存储管理方式来得容易。

答："段"是一种逻辑上的理解。我们可以把具有完整逻辑意义的模块看作一个段，它在所有共享进程的段表中，分别只占用一个段表项。"页"是一种物理上的划分，它强调页长度的一致。于是不得不把具有完整逻辑意义的共享模块分割成固定长度的若干页。共享该模块的进程，需要为这若干页分配若干页表项。而且，页表项在各个共享进程的页表中，页号是对应相等的。

9.15 在动态链接段式存储管理系统里，各进程分别用不同的段号访问共享段是可行的。

A. 设计一个类似的系统，让各进程以静态连接方式，分别用不同的段号访问共享段。

B. 描述一个页式存储管理方式，使各进程分别用不同的页号共享页面。

答：MULTICS 操作系统关于这两种需求的解决方案是。每个进程都对应4个寄存器。一个寄存器保存进程的程序段地址，一个寄存器保存堆栈的基地址，一个寄存器保存全程变量空间的基地址。这么做的用意在于，进程的所有访问都间接地通过寄存器实现，此寄存器映射到当前页或当前段。为不同进程服务时，只要更改寄存器的值就行了，无需更改进程内部的页号或段号。

9.16 观察下列段表：

段	基地址	段长度
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

下列逻辑地址对应的物理地址是什么?

A.0,430

B.1,10

C.2,500

D.3,400

E.4,112

答:

A.219 + 430 = 649

B.2300 + 10 = 2310

C.这是个非法地址

D.1327 + 400 = 1727

E.这是个非法地址

9.17 观察如图 3.31 所示的 Intel 地址翻译机制。

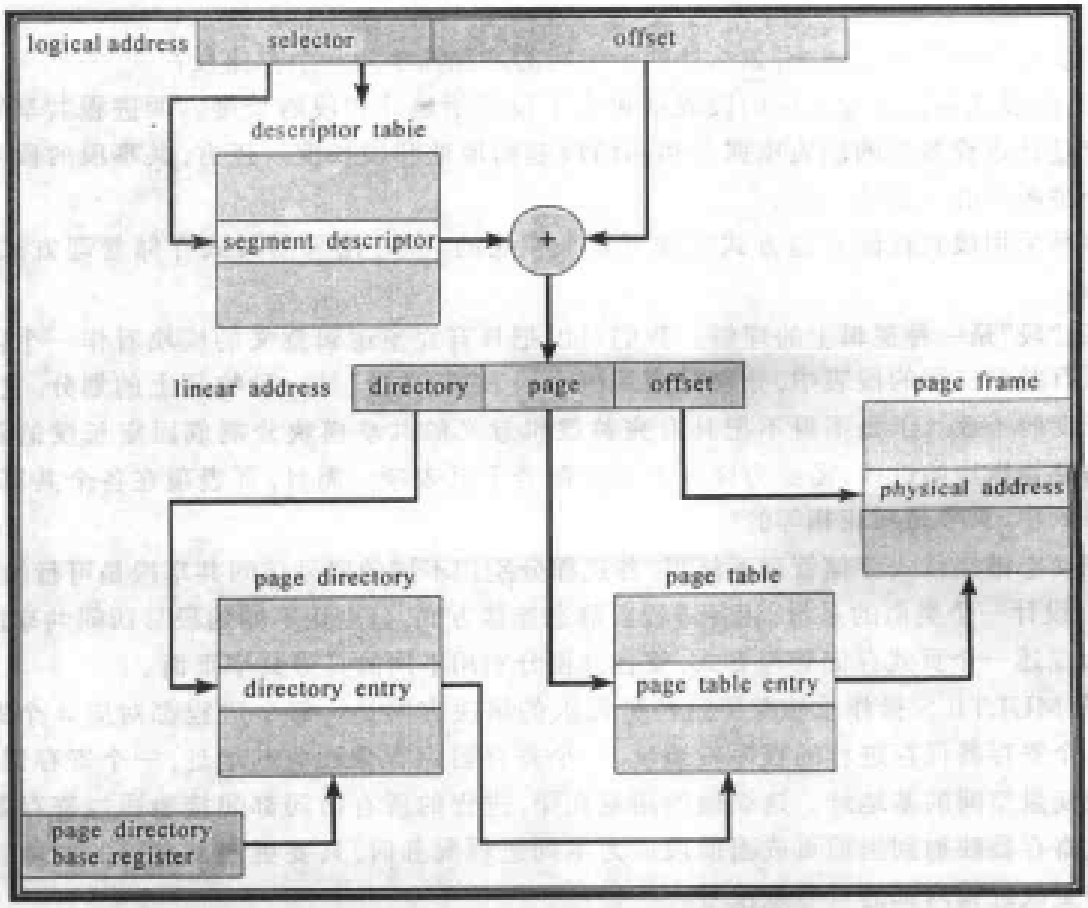


图 3.31

A.描述 Intel 80386 将一个逻辑地址翻译成物理地址的所有步骤。

B.这么复杂的内存地址翻译硬件对操作系统有什么好处?

C.这种地址翻译硬件有什么不利的地方?

答:

A.段选择字是对段描述符表的索引。从段描述符得到的基地址,加上逻辑地址里的偏移量,就是线性地址。线性地址包含页目录号、页号和页内偏移量,通过页目录号索引页目

录中的页目录项，其内容就是指向页表的基地址。页号索引页表中的页表项，其内容包含了对应的页帧号。加上线性地址提供的页内偏移量，就得到了物理地址。

B. 这种内存地址翻译虽然复杂，但是带来了灵活性。多数操作系统的存储管理机制都可以通过这套硬件实现。所以，相对于软硬件混合型实现方式，效率更高，操作系统内核更简单。

C. 因为需要查找多张表，地址翻译所消耗的时间相对较长。虽然 cache 可以缩短时间，但是当没有命中 cache 时，此缺陷还是相当明显。

9.18 IBM/370 机器用钥匙(key)保护内存段。一把钥匙就是一个 4 位的二进制数。每 2K 内存块对应一把钥匙(叫存储钥匙)。CPU 也有一把钥匙(叫保护钥匙)与之对应。仅当两把钥匙相同时，或当它们均为 0 时，才允许执行存储指令。下列哪一种存储管理策略能够充分利用这种硬件？

A. 裸机

B. 单用户系统

C. 进程总数固定的多用户系统

D. 进程总数可变的多用户系统

E. 页式存储管理

F. 段式存储管理

答：

A. 没有必要保护，将钥匙置为 0

B. 可以利用这种硬件，当进入管理模式时，将钥匙置为 0

C. 存储块的长度必须是 2K 字节的倍数。以存储块为单位分配钥匙

D. 存储块的长度必须是 2K 字节的倍数。以存储块为单位分配钥匙

E. 页帧的长度必须是 2K 字节的倍数。以页为单位分配钥匙

F. 段长度必须是 2K 字节的倍数。以段为单位分配钥匙

Chapter 10

10.1 缺页中断在什么情况下发生？描述操作系统的缺页中断服务程序所采取的动作。

答：当现在访问的页面不在内存时，就会产生缺页中断。

操作系统的缺页中断服务程序所采取的动作：

(1)因为访问非法地址也会产生类似的中断，所以，操作系统首先检查本次访问的地址是不是非法地址。如果是非法地址，则强行终止程序的运行。

(2)如果是合法地址，那就是缺页中断了。此时，操作系统必须寻找一个空闲页帧。如果找不到空闲页帧，则调用页面置换算法腾出一个空闲页帧。

(3)申请一次 I/O 操作，将引起缺页中断的页面调入刚刚找到的空闲页帧上。

(4)I/O 操作完成后，修改进程的页表和其他相关数据结构。

(5)重新执行那条引起缺页中断的指令。

10.2 假设有 m 个页帧(初始时全空)，一进程的页面引用串含 n 个页号，长度为 p。对任何页面置换算法来讲，

A. 缺页中断次数的下界是多少？

B. 缺页中断次数的上界是多少？

答：

A. 下界是 n。因为页帧在初始时全空，所以所有页面至少需要装入一次。

B. 上界是 p。在最坏情况下，每次访问页面时，该页面总是不在内存。

10.3 某一计算机有 218 字节的物理空间，提供用户 232 字节的虚拟空间。虚拟内存采用页式管理系统，页长 4096 字节。一用户进程有一虚拟地址 286405718。请解释，系统如何找出对应的物理位置。特别请区分哪些由软件完成，哪些由硬件完成。

答：用二进制改写该虚拟地址，0001 0001 0001 0010 0011 0100 0101 0110

因为页长 $4096 = 2^{12}$，由 $32 - 12 = 20$ 知，页表长 2^{20} 个页表项。所以底端 12 位"0100 0101 0110"就是页内偏移量，高端 20 位"0001 0001 0001 0010 0011"就是页号。根据页号在页表中找到对应的页表项。从页表项取出页帧号(本题里，页帧号为 6 位)。将页帧号作为高端的 6 位，将页内偏移量作为底端的 12 位，两者组合，就是物理地址。

上述操作最好全部由硬件完成。但这决定于 CPU 之 MMU 的支持程度。

10.4 在"按需调页"环境里，下列程序设计技术和数据结构哪些是"好"的，哪些是"不好"的。请解释你的理由。

A. 堆栈

B. Hash 函数索引的符号表

C. 顺序搜索

D. 二进制搜索

E. 纯代码

F. 矢量操作

G. 间接寻址

答："按需调页"环境里，页内地址是连续的，而页间地址不连续。当页面不在内存时，会引起缺页中断，相对消耗很多的时间。

A. 好。栈顶操作一般在当前页进行，此前已驻留内存。只有当栈顶跨页面时，才会引起缺页中断。

B. 不好。Hash 函数产生的索引地址是随机的，可能会频繁缺页。

C. 好。搜索一般在当前页进行，此前已驻留内存。只有当跨页面搜索时，才会引起缺页中断。

D. 不好。二进制搜索是跳跃式的，可能会频繁缺页。

E. 好。纯代码基本上是顺序执行的。其跳转指令全是相对跳转的，范围一般在一个页面之内。只有当跨页面跳转时，才会引起缺页中断。

F. 好。一个矢量的各分量均顺序排列，一般在同一页面内。

G. 不好。存放间接地址的页面，存放直接地址的页面，以及存放内容的页面没有规律，它们可能不在同一页面。

10.5 假设一个"按需调页"虚拟存储空间，页表由寄存器保存。在存在空闲页帧的条件下，处理一次缺页的时间是 8ms。如果没有空闲页面，但待换出页面并未更改，处理一次缺页的时间也是 8ms。如果待换出页面已被更改，则需要 20ms。访问一次内存的时间是 100ns。假设 70% 的待换出页面已被更改，请问缺页率不超过多少，才能保证有效访问时间小于或等于 200ns?

答：

设缺页率为P。题目并没有明确，当缺页中断时，内存中是否有空闲页帧，所以假设内存总是忙的。

访问内存中页面：(1－P) * 100ns

页面不在内存，但不需要保存待换出页面：P * (1－70%) * (8ms+100ns)

页面不在内存，但需要保存待换出页面：P * 70% * (20ms+100ns)

所以，有效访问时间=(1 － P) * 100ns + P * (1－70%) * (8ms+100ns) + P * 70% * (20ms+100ns)=200ns

P=0.000006

10.6 考虑下列页面置换算法。根据缺页率，给它们用五分制排个序。指出哪些算法存在Belady异常现象。

A. LRU

B. FIFO

C. Optimal

D. Second-chance

答：见表3-44

表3-44

顺序	算法	Belady异常
1	Optimal	No
2	LRU	No
3	Second-chance	Yes
4	FIFO	Yes

10.7 实现虚拟存储管理既有好处，也要付出代价。请分别列举这些好处和代价。有没有可能付出的代价超出了所带来的好处。如果可能超出，可以采取什么措施避免吗？

答：好处：内存利用率提高；进程的逻辑空间大于计算机系统的物理空间；同时装入更多的进程。

代价：额外的硬件开销；访问内存相对较慢。

当然有可能存在所付出的代价超出了所带来的好处。如果这样，可以简化存储管理算法(甚至不用虚拟存储管理)，或者加强CPU的MMU。

10.8 某操作系统支持页式虚拟存储管理，其中央处理器的周期是1μs。当不是处于同一页面时，访问另一个页面耗时1微秒。一个页面含1000字。换页设备是个磁鼓，其转速为3000转/分钟，传输率1000000字/秒。还测得如下数据：1%的指令要访问不处于同一页面的其他页面内容，这当中，80%的被访问页面已经在内存中。需要新页面时，50%的被换出页面已经修改过了。

请计算该系统的有效指令时间，假设系统只有一个CPU，而且它在磁鼓传输数据时是空闲的。

答：磁鼓传输一个页面的时间=1000字/1000000字/每秒=1000μs

访问处于同一页面的时间=(1－1%) * 1μs

有效指令时间=(1－1%) * 1μs+1% * (1μs + 80% * 1μs+(1－80%)
(50% * (1μs+1000μs) + 50% * (1μs+1000μs * 2)))

=0.99+0.01*(1+0.8+0.2*(500.5+1000.5))

$= 0.99 + 0.01 * (1 + 0.8 + 0.2 * 1501)$

$= 0.99 + 0.01 * 302$

$= 4.01\mu s$

10.9 对一个"按需调页"系统测得如下数据：

CPU 利用率 20%

用作页面交换的磁盘的利用率 97.7%

其他 I/O 设备利用率 5%

下列措施中，哪些会改善 CPU 利用率(如果有的话)，请说明理由：

A. 安装一个更快的 CPU

B. 安装一个更大容量的磁盘用作页面交换

C. 增加并发进程数

D. 减少并发进程数

E. 安装更多内存

F. 安装更快的硬盘，或安装更多的硬盘和控制器

G. 增加一个预取页面算法

H. 增加页面长度

答：首先判断系统正在频繁地进行换页操作。所以，减少并发进程数会显著地减少换页操作，提高 CPU 的利用率。其他措施也有些效果，例如，安装更多内存。

A. 安装一个更快的 CPU。没用。

B. 安装一个更大容量的磁盘用作页面交换。没用，交换空间本来就足够了。

C. 增加并发进程数。没用，情况将会更糟。

D. 减少并发进程数。效果明显。

E. 安装更多内存。可能会有效果，因为空闲页帧增加了，换页的几率将相对减少。

F. 安装更快的硬盘，或安装更多的硬盘和控制器。效果不明显。

G. 增加一个预取页面算法。效果不确定。

H. 增加页面长度。如果顺序访问居多，则会减少缺页次数。如果随机访问居多，因为单个页面占用更大的物理空间，页帧总数减少，所以缺页次数会增加；因为页面长度增加，页面的传输时间会增加。综上，此方案的效果不确定。

10.10 看一个二维矩阵 A：

```
int A[][] = new int[100][100];
```

其中，A[0][0]位于页式存储系统(页面长 200)的 200 地址处。一个进程在第 0 页(地址范围 0 到 199)，操作该矩阵。也即，指令取自第 0 页。如果有 3 个物理页帧，处理进程位于第 1 个页帧，其他两个页帧开始是空的。再假设采用 LRU 算法，下列矩阵初始化操作会引起多少次缺页：

A.
```
for(int j = 0; j < 100; j++)
    for(int i = 0; i < 100; i++)
      A[i][j] = 0;
```

B.
```
for(int i = 0; i < 100; i++)
    for(int j = 0; j < 100; j++)
      A[i][j] = 0;
```

答:不妨假定采用C语言。C语言中,数组按行排列;每个整型数占4字节,数组A共占40000个字节,即200页。

A.对数组的访问是跳跃的。即,第200号地址,第600号地址,第204号地址,第604号地址,第208号地址,第608号地址,……

当可分配页帧数为2时,每次关于j的for循环,有2次缺页。所以,共有200次缺页。

B.对数组的访问是按顺序的。下标i和j递增时,仅当需要访问后续页面时,才会引起缺页。所以,共有200次缺页。

10.11 考察下列页面引用串:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

若分别采用如下的页面置换算法,会有多少次缺页?假设页帧数分别为1,2,3,4,5,6,7个。假设初始时物理页帧全是空的。

A.LRU

B.FIFO

C.最优算法

答:见表3-45

表3-45

页帧数	LRU	FIFO	Optimal
1	20	20	20
2	18	18	15
3	15	16	11
4	10	14	8
5	8	10	7
6	7	10	7
7	7	7	7

10.12 假设你想使用的页面置换算法要求有引用位reference bit(例如second-chance算法或工作集模型),但是硬件不提供。设想一下,你如何仿真这个引用位,请计算一下代价。或者,如果无法仿真,请解释你的理由。

答:

可以用"有效"位(valid/invalid bit)仿真reference bit。初始时,置该位为0。第1次访问该页面将引起中断。操作系统完成换页操作后,将置该位为1。也即,reference bit置成1。

10.13 你设计了一个你认为最优的页面置换算法。但是不幸的是,某些测试表明,该算法存在Belady异常。这是最优算法吗?请解释理由

答:不是最优算法。根据最优算法的定义来理解。

10.14 假设你的页面置换算法定期检查每个页面。如果某个页面自上次检查后一直没被使用,则丢弃它。相对于LRU或second-chance算法,你的算法好在哪里,不好在哪里?

答:此算法的定期检查的周期是算法的关键。如果该周期足够短,那么此算法接近second-chance算法。如果该周期足够长,那么该算法接近LRU。

10.15 段式虚拟存储管理基本与页式虚拟存储管理一样,只是采用了变长度的"页"。请参考FIFO和LRU,设计两种段置换算法。提示:因为段长度不定,被选中的段可能不足以腾出足够的空间给换入的段;应分别考虑可重定位的段和不可重定位的段。

答:FIFO:找出第1个足够容纳换入段的空闲物理内存段。否则,找出地址连续的若干空闲物理内存段,它们的总和足够容纳换入段。否则,如果允许重定位,则重定位N个内存段,使得腾出的空闲段足够容纳换入段。剩下的挂到空闲队列。

LRU:找出一个最长时间不用的,而且足够容纳换入段的空闲物理内存段。否则,找出地址连续的若干最"老"的空闲物理内存段,它们的总和足够容纳换入段。否则,如果允许重定位,则重定位N个最"老"的内存段,使得腾出的空闲段足够容纳换入段。剩下的挂到空闲队列。

10.16 页面置换算法必须尽可能减少缺页次数。为达到此目的,我们可以将页面均匀地分配到各个物理页帧,而不是集中分发到少数几个物理页帧。我们可以为每个物理页帧捆绑一个计数器,记录装到此页帧的累计页面数。然后,决定换出页面时,只要搜索计数值最小的那个页帧。

A.请按这个思路,设计一个页面置换算法,特别请说明:(1)计数器的初值;(2)计数器什么时候递增;(3)计数器什么时候递减;(4)怎么选择换出页面。

B.对一个只有4个页帧的内存,采用你的算法处理如下的引用串,会产生几次缺页:

1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 7, 8, 9, 5, 4, 5, 4, 2.

C.对上述例子,如果用最优算法,会产生几次缺页?

答:

A.按此思路设计的页面置换算法:

i.计数器的初始值为0

ii.当有一新页面与此页帧关联时,计数器递增

iii.与此页帧关联的某个页面不再需要时,计数器递减

iv.每次选择换出页面时,总选择使用最小计数器值的页帧的页面。如果这样的页面不惟一,则用FIFO

B.13次

C.11次

10.17 一页式虚拟存储系统,用于页面交换的磁盘的平均访问、传输时间是20ms。页表保存在主存,访问时间1μs。也就是说,每引用一次指令或数据,需要访问两次内存。为改善性能,我们可以增设一个关联寄存器。如果页表项在关联寄存器里,则只要访问一次内存就够了。假设80%的访问,其页表项在关联寄存器中;剩下的20%里,10%的访问(即总数的2%)会产生缺页。请计算有效访问时间。

答:有效访问时间 $= 80\% * 1\mu s + (1-80\%)((1-10\%) * 1\mu s * 2 + 10\% * (1\mu s * 2 + 20ms))$

$= 0.8 + 0.2 * (0.9 * 2 + 0.1 * 20002)$

$= 0.8 + 0.2 * 2002$

$= 401.2\mu s$

10.18 一个页式虚拟存储系统,其并发进程数固定为4个。最近测试了它的CPU利用率和用于页面交换的磁盘的利用率,得到的结果就是下列3组数据中的1组。针对每一组数据,说明系统发生了什么事情?增加并发进程数能提升CPU的利用率吗?页式虚拟存储系统有用吗?

A.CPU利用率13%;磁盘利用率97%

B. CPU 利用率 87%；磁盘利用率 3%

C. CPU 利用率 13%；磁盘利用率 3%

答：

A. 系统在抖动。增加并发进程数反而会恶化系统性能。

B. 系统正常。不需要采取什么措施。

D. CPU 没有充分利用。应该增加并发进程数。

10.19 我们有个操作系统，本来使用基地址寄存器和界限寄存器。改装机器后，换成页表。可不可以利用页表仿真基地址寄存器和界限寄存器，怎么做？或者，为什么不能仿真？

答：无法仿真。因为页面长度要求固定。而段的长度是可变的。

10.20 抖动(thrashing)的原因是什么？系统怎么检测抖动状态？如果发生抖动，系统如何解决这个问题？

答：产生抖动的原因是系统没有为进程分配最低限度的页帧，使得进程运行时频繁换页。系统检测抖动的简易办法是，连续检测 CPU 利用率和并发进程数。当并发进程数均匀增加时，CPU 利用率没有正常升高，反而大幅度下降，就要引起怀疑了。解决抖动问题的简易办法是，适当减少并发进程数。

第4章

文件系统

4.1 概述

计算机系统的主要目的是为了运行程序。这些运行中的程序以及它们所访问的数据，都必须存储于主存储设备中。

为了提高 CPU 的利用率以及加快对用户反映的速度，计算机必须在内存中保持多个进程。目前存在很多不同的存储管理方案，每一种方案所对应的算法的有效性都是针对于某一个特定应用环境的。究竟采用什么方案需要考虑很多因素，尤其是系统的硬件设计。每一种算法都需要特定的硬件支持。

既然主存储器相对于永久地保存数据和程序而言显得太小，计算机系统就需要提供第二级的存储设备以备份主存储器。现代计算机系统通常采用磁盘作为主要的信息(包括数据和程序)存储媒介。文件系统就为访问这些磁盘上的数据和程序提供了一种机制。文件是由它的创建者定义的一组相关信息的集合。文件被操作系统映射到具体的物理设备上去。为了方便使用，文件通常以目录的形式组织起来。

4.2 文件系统接口

文件是由操作系统所定义和实现的抽象数据类型。它由一系列逻辑记录组成。逻辑记录可以是字节、行(固定或可变长度)或更为复杂的数据项。操作系统可自己支持各种记录类型或让应用程序提供支持。

操作系统的主要任务是将逻辑文件概念映射到物理存储设备如磁盘或磁带。由于设备的物理记录大小可能与逻辑记录大小不一样，所以可能有必要将多个逻辑记录合并以存入物理记录。同样，这个任务可以由操作系统自己完成或由应用程序来提供。

文件系统的每个设备都有内容的卷表或设备目录，以列出设备上文件的位置。另外，可创建目录以组织文件。多用户系统的单层结构目录会有命名问题，因为每个文件必须具有惟一文件名。双层结构目录通过为每个用户创建独立目录来解决这个问题。每个用户有其自己的目录，包含自己的文件。目录可通过名称列出其文件，目录包括许多文件信息如文件在磁盘上的位置、长度、拥有者、创建时间、上次使用时间等。

双层结构目录的自然扩展是树型结构目录。树型结构目录允许用户创建子目录以组织其文件。无环结构目录允许共享子目录和文件，但是使得搜索和删除复杂了。通用图结构目录在

文件和目录共享方面提供了更为完全的灵活性，但是有时需要采用垃圾收集以恢复未使用的空间。

磁盘分为一个或多个分区，每个分区可包括一个文件系统。文件系统可安装到系统命名结构中，以使其可用。命名方案因操作系统而异。一旦安装，分区内的文件就可使用。文件系统可以卸载以禁止访问或用于维护目的。

文件共享依赖于系统所提供的语义。文件可有多个读者、多个作者或有限共享。分布式文件系统允许客户机安装来自服务器的分区或目录，只要能从网络访问即可。远程文件系统在可靠性、性能和安全方面有些挑战。分布式信息系统维护用户、主机、访问信息以便客户机和服务器能共享状态信息以管理使用和访问。

由于文件是绝大多数计算机的主要信息存储机制，所以需要文件保护。文件访问可以针对每种访问类型如读、写、执行、添加、删除、列表等分别加以控制。文件保护可以由口令、访问控制列表和其他特定技术来实现。

4.3 文件系统实现

文件系统持久驻留在外存上，外存设计成可以持久地容纳大量数据。最常用外存介质是磁盘。物理磁盘可划分成分区(Partition)，以控制介质的使用，允许在同一磁盘上支持多个可能不同的文件系统。这些文件系统安装在逻辑文件系统结构上，然后才可以使用。文件系统通常按层结构或模块结构来加以实现。低层处理存储设备的物理属性。高层处理符号文件名和文件逻辑属性。中间层将逻辑文件概念映射到物理设备属性。

每个文件系统类型都有其结构和算法。VFS(Virtual File System，虚拟文件系统)层允许上层统一地处理每个文件系统类型。即使远程文件系统也能集成到文件系统目录结构中，通过VFS接口采用标准系统调用进行操作。

文件在磁盘上有三种不同空间分配方法：连续的，链接的，或索引分配。连续分配有外部碎片问题。链接分配的直接访问效率低。索引分配可能因其索引块而浪费一定空间。连续分配可采用区域来扩展，以增加灵活性和降低外部碎片。索引分配可按簇(为多个块)来进行，以增加吞吐量和降低所需索引条目的数量。按大簇的索引分配与采用区域的连续分配相似。

空闲空间分配方法也影响磁盘使用效率、文件系统性能、外存可靠性。所使用的方法包括位向量和链表。优化方法包括组合、计数和FAT(File Allocation Table，文件分配表)(将链表放在一个连续区域内)。

目录管理程序必须考虑效率、性能和可靠性。哈希表是最为常用的方法；它快速且高效。然而，表损坏和系统崩溃可能导致目录信息与磁盘内容不一致。一致性检查程序，如UNIX的系统程序fsck，或MS-DOS的chkdsk，可用来修补损坏。操作系统备份工具允许磁盘数据复制到磁带，可以从数据损失甚至整个磁盘的故障(因硬件失败、操作系统错误或用户错误)中恢复。

网络文件系统，如NFS使用客户机—服务器方法允许用户访问远程机器的文件和目录，就好像本地文件系统一样。客户机上的系统调用转换成网络协议，再转换成服务器的文件系统操作。网络和多客户访问在数据一致性和性能方面，增加了挑战。

由于文件系统在系统操作中的重要位置，其性能和可靠性十分关键。日志结构和缓存等技术可帮助改善性能，日志结构和RAID可提高可靠性。

4.4 习题与解答

一、选择题

1. 按逻辑结构划分，文件主要有两类：__(1)__和__(2)__。文件系统的主要目的是__(3)__。

(1)、(2)：A. 网状文件　B. 只读文件　C. 读写文件
D. 记录式文件　E. 索引文件　F. 流式文件

(3)：A. 实现对文件的按名存取　B. 实现虚拟存储器
C. 提高外围设备的输入输出速度　D. 用于存储用户文件

答：(1) D　(2)F　(3)A

2. 从下面关于连续结构文件和链接结构文件的论述中，选出一条正确的论述。

A. 连续结构文件适于建立在顺序存储设备上，而不适合建立在磁盘上
B. 在显式链接结构文件中是在每个盘块中设置一链接指针，用于将文件的所有盘块链接起来
C. 连续结构文件必须采用连续分配方式，而链接结构文件和索引结构文件则都可采取离散分配方式
D. 在 MS-DOS 中采用的是隐式链接文件结构

答：C

3. 从下面关于索引文件的论述中，选出一条正确的论述。

A. 索引文件中，索引表的每个表项中含有相应记录的关键字和存放该记录的物理地址
B. 文件进行检索时，首先从 FCB 中读出文件的第一个盘块号；而对索引文件进行检索时，应先从 FCB 中读出文件索引块的开始地址
C. 对于一个具有三级索引的文件，存取一个记录通常要访问三次磁盘
D. 在文件较大时，无论是进行顺序存取还是随机存取，通常都是以索引文件方式最快

答：B

4. 在文件系统中，设置一张__(1)__表，它是利用二进制的一位来表示磁盘中一个块的使用情况；一张__(2)__表，其中存放着文件中下一个盘块的物理地址。

(1)、(2)：A. 文件描述符表　B. 链接指针表　C. 文件表　D. 空闲区表　E. 位示图

答：(1)E　(2)B

5. 在文件系统中是利用__(1)__来管理文件，为了允许不同用户的文件使用相同的文件名，通常在文件系统中采用__(2)__；在目录文件中的每个目录项通常就是__(3)__；在 UNIX 系统中的目录项则是__(4)__。

(1)：A. 文件控制块　B. 索引结点　C. 符号名表　D. 目录

(2)：A. 重名翻译　B. 多级目录
C. 文件名到文件物理地址的映射表　D. 索引表

(3)、(4)：A. FCB　B. 文件表指针　C. 索引结点　D. 文件名和文件物理地址　E. 文件名和索引结点指针。

答：(1)D　(2)B　(3)A　(4)E

6. 系统级安全管理的主要方法是__(1)__，以及在__(1)__后进行__(2)__；在__(2)__时系统要求输入__(3)__和__(4)__。

(1)、(2)：A.指定用户上机时间 B.限定用户在指定终端上机 C.登录 D.注册 E.机器自检。

(3)、(4)：A.注册名 B.机器名 C.口令 D.终端号

答：(1)D (2)C (3)A (4)C

7.用户请求使用一个已存在的文件时，正确的操作次序为_____。

A.读/写→关闭 B.打开→读/写 C.打开→读/写→关闭

答：C

8.一个采用二级索引文件系统(每块大小为2KB，每块地址占用4字节)管理的最大的文件是__(1)__。存取一块盘块信息通常要访问__(2)__次磁盘。

(1)：A.8GB B.4GB C.2GB D.1GB E.512MB F.256MB G.128MB H.64MB I.以上都不是；

(2)：A.1 B.2 C.3 D.4 E.0

答：(1)E (2)C

9.系统为了管理文件，设置了专门的数据结构—文件控制块(FCB)。FCB是在执行下列哪一个系统调用时建立的？_____。

A.create B.open C.read D.write

答：A

10.文件系统中若文件的物理结构采用连续结构，则文件控制块FCB中关于文件的物理位置信息应包括_____。

(1)首块地址 (2)文件长度 (3)索引表地址

A.只有(1) B.(1)和(2) C.(1)和(3) D.(2)和(3)

答：B

11.操作系统为保证未经文件拥有者授权，任何其他用户不能使用该文件，所提供的解决方法是_____。

A.文件保护 B.文件保密 C.文件转储 D.文件共享

答：A

12.UNIX系统中，把输入输出设备看作是_____。

A.普通文件 B.目录文件 C.索引文件 D.特殊文件

答：D

13.在文件系统管理中，有三类常用的空闲块管理方法，即位图法，空闲块链表链接法和_____。

A.一级目录法 B.多级目录法 C.分区法 D.索引法

答：D

14.位示图方法可用于_____。

A.盘空间的管理 B.盘的驱动调度

C.文件目录的查找 D.页式虚拟存贮管理中的页面调度

答：A

15.存放在磁盘上的文件_____。

A.既可随机访问又可顺序访问 B.只能随机访问

C.只能顺序访问 D.必须通过操作系统访问

答:A

16. 文件系统中,设立打开文件(Open)系统调用的基本操作是_(1)_,关闭文(Close)系统调用的基本操作是__(2)__。

(1)A. 把文件信息从外存读到内存

B. 把文件的控制管理信息从外存读到内存

C. 把文件的 FA7 表信息从外存读到内存

D. 把磁盘的超级块从外存读到内存

(2)A. 把文件的最新信息从内存写入外存

B. 把文件当前的控制管理信息从内存写入外存

C. 把位示图从内存写回外存

D. 把超级块的当前信息从内存写回外存

答:(1)B (2)B

17. 下面说法正确的是_____。

A. 文件系统负责文件存储空间的管理但不能实现文件名到物理地址的转换

B. 在多级目录结构中对文件的访问是通过路径名和用户目录名进行的

C. 文件可以被划分成大小相等的若干物理块且物理块大小也可任意指定

D. 逻辑记录是对文件进行存取操作的基本单位

答:D

18. 根据文件的内部结构,可以把文件分为记录式文件和流式文件两类。记录式文件是一个有相同类型记录的有序集合,流式文件内部没有记录概念,最常见的流式文件是字符流文件,可看成是__(1)__的有序集合。一般来说文件名及其属性可以收纳在__(2)__中,以便查找。__(2)__本身也是一种__(3)__。用户可以通过键盘命令和系统调用两种方式,使用文件系统,当用户通过键盘命令修改文件时,应使用编辑命令;在程序内使用文件之前,需先使用__(4)__语句。

(1):A. 字符串	B. 数组	C. 记录	D. 页面
(2):A. 目录	B. 索引	C. 字典	D. 作业控制块
(3):A. 数组	B. 记录	C. 文件	D. 数据集合
(4):A. 打开	B. 读	C. 建立	D. 写

答:(1)A (2)A (3)C (4)A

19. 同一文件在不同的存储介质上_____的组织形式。

A. 应该有相同 B. 可以有不同 C. 必定用不同

答:B

20. 为防止系统故障造成系统内文件受损,常采用_____方法来保护文件。

A. 存取控制矩阵 B. 备份 C. 口令 D. 加密

答:B

21. 操作系统中对数据进行管理的部分叫做_____。

A. 数据库系统 B. 文件系统

C. 检索系统 D. 数据存储系统

答:B

22. 文件系统是指_____.

A. 文件的集合　　B. 文件的目录
C. 实现文件管理的一组软件　　D. 文件、管理文件的软件及数据结构的总体
答:D

23. 从用户角度看,引入文件系统的主要目的是_____。
A. 实现虚拟存储　　B. 保存系统文档
C. 保存用户和系统文档　　D. 实现对文件的按名存取
答:D

26. 为了对文件系统中的文件安全管理,任何一个用户在进入系统时都必须进行注册,这一级安全管理是_____安全管理。
A. 系统级　　B. 目录级
C. 用户　　D. 文件级
答:A

27. 为了解决不同用户文件的"命名冲突"问题,通常在文件系统中采用_____。
A. 约定的方法　　B. 多级目录
C. 路径　　D. 索引
答:B

28. 一个文件的绝对路径名是从_____开始,逐步沿着每一级子目录向下追溯,最后到指定文件的整个通路上所有子目录名组成的一个字符串。
A. 当前目录　　B. 根目录
C. 多级目录　　D. 二级目录
答:B

29. 对一个文件的访问,常由_____共同限制。
A. 用户访问权限和文件属性　　B. 用户访问权限和用户优先级
C. 优先级和文件属性　　D. 文件属性和口令
答:A

30. 磁盘上的文件以_____为单位读写。
A. 块　　B. 记录
C. 柱面　　D. 磁道
答:A

31. 磁带上的文件一般只能_____。
A. 顺序存取　　B. 随机存取
C. 以字节为单位存取　　D. 直接存取
答:A

32. 使用文件前必须先_____文件。
A. 命名　　B. 建立
C. 打开　　D. 备份
答:C

33. 文件使用完毕后应该_____。
A. 释放　　B. 关闭
C. 卸下　　D. 备份

答:B

34. 位示图可用于_____。

A. 文件目录的查找　　B. 磁盘空间的管理

C. 内存空间的共享　　D. 实现文件的保护和保密

答:B

35. 按物理结构划分，则_____不是属于文件的物理结构。

A. 索引　　B. 读写

C. 连续　　D. 链接

答:B

36. 在文件系统中，文件的不同物理结构有不同的优缺点。在下列文件的物理结构中，_____不具有直接读写文件的任意一个记录的能力。

A. 连续结构　　B. 链接结构

C. 索引结构　　D. Hash 结构

答:B

37. 在下列文件的物理结构中，_____不利于文件长度动态增长。

A. 连续结构　　B. 链接结构

C. 索引结构　　D. Hash 结构

答:A

38. 如果文件采用直接存取方式且文件大小不固定，则宜选择_____文件结构。

A. 直接　　B. 连续

C. 链接　　D. 索引

答:D

39. 文件系统采用二级目录结构，这样可以_____。

A. 缩短访问文件存储器时间　　B. 实现文件共享

C. 节省主存空间　　D. 解决不同用户之间的文件名冲突问题

答:D

40. 常用的文件存取方法有两种：顺序存取和_____存取。

A. 流式　　B. 串联

C. 顺序　　D. 随机

答:D

41. 下列叙述中正确的五项是_____。

A. 在磁带上的顺序文件中插入新的记录时，必须复制整个文件

B. 由于磁带的价格比磁盘便宜，用磁带实现索引文件更经济

C. 在索引顺序文件的最后添加新的记录时，必须复制整个文件

D. 在磁带上的顺序文件的最后添加新的记录时，不必须复制整个文件

E. 顺序文件是利用磁带的特有性质实现的，因此顺序文件只有存放在磁带上

F. 索引顺序文件既能顺序访问，又能随机访问

G. 直接访问的文件也能顺序访问，但一般效率较差

H. 变更磁盘上的顺序文件的记录内容时，不一定要复制整个文件

I. 在磁盘上的顺序文件中插入新的记录时，必须复制整个文件

J. 索引顺序文件是一种特殊的顺序文件，因此通常存放在磁带上

答：A D F G H

42. 如果文件系统中有两个文件重名，不应采用_____。

A. 一级目录结构　　B. 树型目录结构

C. 二级目录结构　　D. A 和 C

答：A

二、填空题

1. 索引文件大体上由_____区和_____区构成。其中_____区一般按关键字的顺序存放。

答：索引　数据　索引

2. 对操作系统而言，打开文件广义指令的主要作用是装入_____目录表。

答：文件

3. 磁盘文件目录表的内容至少应包含_____和　　。

答：文件名，文件存储在磁盘上的地址

4. 操作系统实现按名存取进行检索的关键在于解决文件名与_____的转换。

答：文件存储地址

5. 在文件系统中，若按逻辑结构划分，可将文件划分成_____和记录式文件两大类。

答：流式文件

6. 按用户对文件的存取权限将用户分为若干组，同时规定每一组用户对文件的访问权限。这样，所有用户组存取权限的集合称为该文件的_____。

答：存取控制表

7. _____是指避免文件拥有者或其他用户因有意或无意的错误操作使文件受到破坏。

答：文件保护

8. 从文件管理角度看，文件由_____和文件体两部分组成。

答：文件控制块

9. 文件转储的方法有两种：全量转储和_____。

答：增量转储

10. 磁盘与主机之间传递数据是以_____为单位进行的。

答：数据块

11. 在文件系统中，要求物理块必须连续的物理文件是_____。

答：连续文件

12. 文件系统为每个文件另建立一张指示逻辑记录和物理块之间的对应关系表，由此表和文件本身构成的文件是_____。

答：索引文件

13. 文件的结构就是文件的组织形式，从用户观点出发所看到的文件组织形式称为文件的_____组织形式；从实际观点出发，文件在外存上的存放称为文件的_____。

答：逻辑结构　物理结构

三、判断题

1. 树型结构的文件系统中，设置当前目录有利于加快文件的查找速度。

答：对

2.顺序文件适合于建立在顺序存储设备上,而不适合建立在磁盘上。

答:错

3.操作系统对数据的安全已采取了足够的防范措施,因此不必担心系统内的数据丢失。

答:错

4.同一文件在不同的存储介质上可以采用不同的物理组织方式,可以用不同的文件名答:对

5.在文件的直接存取方法中,允许任意存取文件中的一个记录。

答:对

6.文件目录存放在内存的一个固定区域。

答:错

7.在文件系统支持下,用户也需知道文件存放的物理地址。

答:错

8.连续文件在顺序存取时,其存取速度较慢。

答:对

9.对文件进行检索时,总是将当前目录作为检索的起始点。

答:错

10.文件系统中不同目录下的文件名称不能相同。

答:错

11.多级目录结构中,对文件的访问是通过路径名和文件名进行的。

答:对

12.对文件实施检索时,检索的起始点一定是根目录。

答:错

13.操作系统的文件系统不包括负责管理文件的系统软件。

答:错

14.以逻辑地址为顺序对文件进行操作的方式是随机存取方式。

答:错

四、问答题

1.什么是文件、文件系统?

答:文件是具有名字的存储在某种物理介质的一组相关信息的集合。

文件系统是指操作系统中与文件管理有关的那部分软件以及被它们管理的文件和文件控制信息的集合。

2.文件系统必须完成哪些工作?

答:文件系统必须完成的工作有:

(1)文件的存取:有文件的顺序存取和随机存取两种方式。

(2)目录管理:建立新文件时,应将与该文件的一些属性登记在文件目录中;读文件时,从文件目录中查找指定文件是否存在并核对是否有权使用。

(3)文件的组织:当用户要求保存文件时,必须把逻辑文件转换成物理文件;当用户要求读文件时,又把物理文件转换成逻辑文件。

(4)文件存储空间的管理:必须记住哪些存储空间已被占用,那些存储空间时空闲的。

(5)文件操作:提供基本文件操作,如建立、打开、读、写、关闭和删除等操作。

(6)文件共享、保护、保密:实现文件的共享,对文件提供安全保护措施。

3.什么是文件目录？什么是目录文件？试说明文件目录的作用，它一般应包括哪些信息？

答：在文件系统中，文件目录记录文件的管理信息，又称为文件控制块。文件系统又把同一卷上的若干文件的文件目录组成一个独立的文件，这个文件全部由文件目录组成，称为目录文件。文件目录和目录文件是两个不同的概念。文件目录记录文件的管理信息，用于对单个文件的控制。目录文件是部分文件目录组成的文件，用于文件系统的管理。

文件目录用于对单个文件的控制，一般它包含文件的名字、文件长度、文件存放在外存的物理地址，以及文件属性和文件建立修改的日期、时间等信息。

4.对空闲磁盘空间的管理采用哪几种方式？简述它们的特点。

答：常用的外存空闲存储空间管理方法：空闲表法、位示图法、空闲块链接法、成组链接法。

(1)空闲表法

空闲表法属于连续分配方法，它为外存上所有空闲区建立一张空闲表，每个空闲区对应一个空闲表项，其中包括序号、该空闲区的第一盘块号，该区的空闲盘块数等信息，再将所有空闲区按起始盘块号递增的次序排列。

(2)位示图法

位示图是利用二进制的一位来表示磁盘中一个块的作用情况，当其值为0时表示对应盘块空闲；值为1时盘块已分配。磁盘上所有盘块都有一个二进制位与之对应，这样，由所有盘块所对应的位形成了一个集合称为位示图，位示图用磁盘块存放，称为位图块。

(3)空闲块链接法

空闲块链接法是将磁盘上所有空闲盘区链接在一个队列中，称为空闲块链。请求分配时从链头依次摘下适当数目的空闲盘块来分配，回收时将回收空闲盘块链入空闲块链尾部。

(4)成组链接法

UNIX 操作系统采用成组链接法管理磁盘空闲块。

成组链接法将磁盘空闲块分成若干组，如将每100个盘块作为一组，该组空闲块总数和各空闲块块号存入下一组的第一个空闲块中。最后不满100块的那组空闲块总数和各空闲块块号记入磁盘区专用管理块的空闲块管理的数据结构。由于文件系统要安装时，磁盘专用块拷入内存系统缓冲区，所以成组链接法分配和回收操作大部分情况只与内存进行读写，所以速度较快。

5.文件系统外存储器的管理与内存管理有何异同点？

答：文件系统外存储器的管理与内存管理的异同点如下。

(1)主要任务：内存管理的主要任务是为多道程序的运行提供良好的环境，而外存管理的主要任务是为文件系统提供存储空间。

(2)基本功能：内存管理的基本功能包括内存空间的分配、回收、内存保护以及内存扩充等，而外存管理的基本功能则只是对外存空间的管理。

(3)分配方式：内存和外存管理中都可以采用连续分配方式，且都以离散分配方式为主。

(4)分配算法与机制：对于连续分配方式，内存与外存管理中的分配和回收算法类似，主要有最先适应算法和最佳适应算法等。在离散分配方式中，两者所采用的机制不同，内存管理主要是利用页、段表，而外存管理则主要利用文件分配表或位示图等。

(5)分配单位：内存以字节为分配单位，而外存则以盘块为分配单位。

6.为保证文件系统的安全性，可以采取哪些措施？

答：为保证文件系统的安全性可以采取对文件的保护和保密等措施。

实现文件保护措施可以从两方面来考虑：一是防止系统故障(包括软、硬件故障)造成的破坏；二是防止用户共享文件时可能造成的破坏。一种方法可以建立文件副本和定时转储；另一种方法可以采用树型文件目录、存取控制表、存取控制矩阵等。

实现文件保密措施包括隐藏文件目录、设置口令和对文件进行加密等方法。

7. 把一些外部设备也看成文件，这样做可以给用户带来什么好处？

答：从文件系统的角度看，把一些外部设备也看成文件，用户就可以用统一的观点去看待和处理驻留在各种存储设备上的信息而无需考虑保存文件的设备差异，这样做会给用户带来很大的方便。

8. 简述文件的检索过程

答：一般文件的检索过程如下：

(1)由用户提供的路径和文件名，在目录文件中找到该文件目录。

(2)由文件目录(文件控制块)中所描述的信息，查找到该文件存储的物理位置(如首块号)。

(3)根据文件存放的物理结构及其相应的存取方式，访问该文件。

9. 试述文件管理系统设置打开文件、关闭文件操作的原因。

答：文件系统中要设置打开文件和关闭文件操作。原因是：用户进程存取一个文件，系统首先要检索目录结构，按名查找该文件的文件控制块。打开文件的基本思想是，按指定文件名检索目录结构，把找到的文件控制块读入并保存到内存，此后每次存取该文件时，就无需再执行按名查找过程，可以直接在内存找到它的文件控制块，从而加快了存取速度。文件打开后，可以对该文件进行读、写等操作。当一个文件不再存取时，需要关闭该文件，释放占用的系统资源，并将文件控制块的内容复制到外存储器中。这样做，一方面提高了资源利用率，另一方面保证了数据的安全(对于延迟写的系统)。

10. 许多操作系统中提供了文件重命名功能，它能赋予文件一个新名字。若进行文件复制，并给文件起一个新名字，然后删除旧文件，也能达到给文件重命名的目的。试问这两种方法在实现上有何不同？

答：使用文件重命名功能时，用户必须提供两个参数：旧文件名和新文件名。实现该功能时，系统使用旧文件名查找文件目录，若找到旧文件名所在的目录，则将目录表中文件名对应的值改为新文件名。从实现过程看，文件重命名功能完成的工作是修改目录表中的文件名项，除文件名外，文件的其他特性都未改变。

后一种实现方法，先进行文件复制并给文件起一个新名字，这是系统完成了一次物理文件的复制工作，然后删除旧文件。虽然这样也能达到文件重命名的目的，但其实现过程比前一种要复杂、费时，并且新文件与旧文件的物理存放地址也不一样。

11. 一个文件系统中有一个 20MB 大文件和一个 20KB 小文件，当分别采用连续、链接、链接索引、二级索引和 LINUX 分配方案时，每块大小为 4096B，每块地址用 4B 表示，问：

(1)各文件系统管理的最大的文件是多少？

(2)每种方案对大、小两文件各需要多少专用块来记录文件的物理地址(说明各块的用途)？

(3)如需要读大文件前面第 5.5KB 的信息和后面第(16M + 5.5KB)的信息，则每个方案各需要多少次盘 I/O 操作？

答：本题是帮助读者深入比较文件物理结构的各种方案：连续分配、链接分配、二级索引分配、链接索引分配和 LINUX 的直接间接混合分配，明确各种分配方案的优缺点。

(1)a. 各种分配方案的文件系统可管理的最大文件为

b. 连续分配：理论上是不受限制，可大到整个磁盘文件区。

链接分配：由于块的地址为 4 字节，所以能表示的最多块数为 $2^{32}=4G$，而每个盘块中存放文件大小为 4092 字节。

链接分配可管理的最大文件为：4G * 4092B = 16368GB

链接索引：由于块的地址为 4 字节，所以最多的链接索引块数为 232 = 4G，而每个索引块有 1023 个文件块地址的指针，盘块大小为 4KB。

链接索引分配可管理的最大文件为：4G × 1023 * 4KB = 16368TB

二级索引：由于盘块大小为 4KB，每个地址用 4B 表示，一个盘块可存 1K 个索引表目。

二级索引可管理的最大文件容量为 4KB × 1K × 1K = 4GB。

LINUX 混合分配：LINUX 的直接地址指针有 12 个，还有一个一级索引，一个二级索引，一个三级索引。因此可管理的最大文件为 48KB + 4MB + 4GB + 4TB。

(2)连续分配：对大小两个文件都只需在文件控制块 FCB 中设二项，一是首块物理块块号，另一是文件总块数，不需专用块来记录文件的物理地址。

链接分配：对大小两个文件都只需在文件控制块 FCB 中设二项，一是首块物理块块号，另一是文件总块数；同时在文件的每个物理块中设置存放下一个块号的指针。

链接索引：对 20KB 小文件只有 5 个物理块大小，所以只需一块专用物理块来作索引块，用来保存文件的各个物理块地址。对于 20MB 大文件共有 5K 个物理块，由于链接索引的每个索引块只能保存(1K - 1)个文件物理块地址(另有一个表目存放下一个索引块指针)，所以它需要 6 块专用物理块来作链接索引块，用于保存文件各个的物理地址。

二级索引：对大小文件都固定要用二级索引，对 20KB 小文件，用一个物理块作第一级索引，用另一块作二级索引，共用二块专用物理块作索引块，对于 20MB 大文件，用一块作第一级索引，用 5 块作第二级索引，共用六块专用物理块作索引块。

LINUX 的混合分配：对 20KB 小文件只需在文件控制块 FCB 的 i_addr[15]中使用前 5 个表目存放文件的物理块号，不需专用物理块。对 20MB 大文件，FCB 的 i_addr[15]中使用前 12 个表目存放大文件前 12 块物理块块号，用一级索引块一块保存大文件接着的 1K 块块号，还要用二级索引存大文件以后的块号，二级索引使用第一级索引 1 块，第二级索引 4 块。总共也需要 6 块专用物理块来存放文件物理地址。

(3)连续分配：为读大文件前面和后面信息都需先计算信息在文件中相对块数，前面信息相对逻辑块号为 5.5K/4K = 1(从 0 开始编号)，后面信息相对逻辑块号为(16M + 5.5K)/4K = 4097。再计算物理块号 = 文件首块号 + 相对逻辑块号，最后化一次盘 I/O 操作读出该块信息。

链接分配：为读大文件前面 5.5KB 的信息，只需先读一次文件头块得到信息所在块的块号，再读一次第 1 号逻辑块得到所需信息。而读大文件 16MB + 5.5KB 处的信息，逻辑块号为(16M + 5.5K)/4092 = 4107，要先把该信息所在块前面块顺序读出，共花费 4107 次盘 I/O 操作，才能得到信息所在块的块号，最后化一次 I/O 操作读出该块信息。所以总共需要 4108 次盘 I/O 才能读取(16MB + 5.5KB)处信息。

链接索引：为读大文件前面 5.5KB 处的信息，只需先读一次第一个索引块得到信息所在块的块号，再读一次第 1 号逻辑块得到所需信息，共花费 2 次盘 I/O 操作。为读大文件后面 16MB + 5.5KB 处的信息，需要先化 5 次盘 I/O 操作依次读出各索引块，才能得到信息所在块的块号，再化一次盘 I/O 操作读出该块信息。共花费 6 次盘 I/O 操作。

二级索引：为读大文件前面和后面信息的操作相同，首先进行一次盘 I/O 读第一级索引块，然后根据它的相对逻辑块号计算应该读第二级索引的那块，第一级索引块表目号＝相对逻辑块号/1K，对文件前面信息 1/1K＝0，对文件后面信息 4097/1K＝4，第二次根据第一级索引块的相应表目内容又花一次盘 I/O 读第二级索引块，得到信息所在块块号，再花一次盘 I/O 读出信息所在盘块，这样读取大文件前面或后面处信息都只需要 3 次盘 I/O 操作。

LINUX 混合分配：为读大文件前面 5.5KB 处信息，先根据它的相对逻辑块号，在内存文件控制块 FCB 的 i_addr 第二个表目中读取信息所在块块号，而只花费一次盘 I/O 操作即可读出该块信息。为读大文件后在(16MB＋5.5KB)信息，先根据它的相对逻辑块号判断要读的信息是在二级索引管理范围内，先根据 i_addr 内容花一次盘 I/O 操作读出第一级索引块，再计算信息所在块的索引块号在第一级索引块的表目号为(4097-9-1024)/1024＝3，根据第一级索引块第 3 个表目内容再花费一次盘 I/O 操作，读出第二级索引块，就可以得到信息所在块块号，最后花一次盘 I/O 读出信息所在盘块，这样总共需要 3 次盘 I/O 操作才能读出文件后面的信息。

综上所述，归纳如表 4-1 所示：

表 4-1

		连续分配	链接分配	链接索引	二级索引	LINUX
管理最大文件		不受限制	16368GB	16368TB	4GB	48K＋4M＋4G＋4TB
管理用的专用块数	20KB 文件	0	0	1	2	0
	20MB 文件	0	0	6	6	6
读 20MB 文件某处信息	5.5KB	1	1＋1	1＋1	2＋1	1
	16MB＋5.5KB	1	4107＋1	5＋1	2＋1	2＋1

12. 在实现文件系统时，为加快文件目录的检索速度，可利用“文件控制块分解法”。假设目录文件存放在磁盘上，每个盘块 512 字节。文件控制块占 64 字节。其中文件名占 8 字节。通常将文件控制块分解成两部分，第一部分占 10 字节(包括文件名和文件内部号)，第二部分占 56 字节(包括文件内部号和文件其他描述信息)。

(1)假设某一目录文件共有 254 个文件控制块，试分别给出采用分解法前和分解法后，查找该目录文件的某一个文件控制块的平均访问磁盘次数。

(2)一般地，若目录文件分解前占用 n 个盘块，分解后改用 m 个盘块存放文件名和文件内部号部分，请给出访问磁盘次数减少的条件。(北京大学 1997 年研究生试题)

答：利用“文件控制块分解法”加快文件目录的检索速度，其原理是减少因查找文件而产生的访问磁盘次数。因为在进行查找文件的过程中，不再需要把文件控制块的所有内容读入内存。这样，在查找过程中所需读入的磁盘块减少(即减少了访问磁盘的次数)。但是，采用这种方法访问文件在查找到匹配的文件名后，还需要进行一次磁盘访问，才能读出全部的文件控制块信息。

(1)采用分解法前，查找该目录文件的某一文件控制块的平均访问磁盘次数为：

((64×254)/512)/2＝16 次

采用分解法后，查找该目录文件的某一文件控制块的平均访问磁盘次数为：

((10×254)/512)/2＋1＝4 次

(2)分解前平均访盘次数 $=1+2+3+\cdots+n=n\times(n+1)/2$

分解后平均访盘次数 $=2+3+4+\cdots+(m+1)=m\times(m+3)/2$

为了使分解后访盘次数减少，则应有：$m\times(m+3)/2<n\times(n+1)/2$

得到：$m\times(m+3)<n\times(n+1)$

由于 $m<n$，所以有 $m+3\leqslant n+1$，即访问磁盘次数减少的条件为：

$$m\leqslant n-2$$

也可考虑最坏情况：分解前需访盘 n 次，分解后需访盘 $m+1$ 次，即应有 $m+1<n$

得到：$m\leqslant n-2$

13. 有一磁盘组共有 10 个盘面，每个盘面上有 100 个磁道，每个磁道有 16 个扇区。假设分配以扇区为单位。

(1)若使用位示图管理磁盘空间，问位示图需要占用多少空间？

(2)若空白文件目录的每个表目占用 5 个字节，问什么时候空白文件目录大于位示图？

答：空白文件目录是管理磁盘空间的一种方法，该方法将文件存储设备上的每个连续空闲区看作一个空白文件，系统为所有空白文件单独建立一个目录，每个空白文件在这个目录中占一个表项；表项的内容至少包括第一个空白块的地址(物理块号)、空白块的数目。

(1)由题设所给条件可知，磁盘组扇区总数为 $16*100*10=16000$(个)

因此，使用位示图描述扇区状态需要的位数为 16000(位)/8(位/字节)=2000(字节)

(2)已知空白文件目录的每个表项占 5 个字节。而位示图需占 2000 字节，即 2000 字节可存放的表项数为 2000/5=400(个)。

当空白区数目大于 400 时，空白文件目录大于位示图。

14. 现有 3 名学生 S1、S2 和 S3 上机实习，程序和数据都存放在同一磁盘上。若 3 人编写的程序分别为 P1，P2 和 P3，要求这 3 个学生用自编的程序调用同一个数据文件 A 进行计算问：

(1)若文件 A 作为共享文件，系统应采用何种目录结构？画出示意图。

(2)若学生 S1，S2，S3 都将自己的程序名起为 P，则答案(1)中的目录结构能否满足要求？

(3)对于(2)，简要说明系统是如何使每个学生获取他的程序和数据的？

答：(1)系统采用二级目录结构即可满足需要，其示意图如图 4.1 所示：

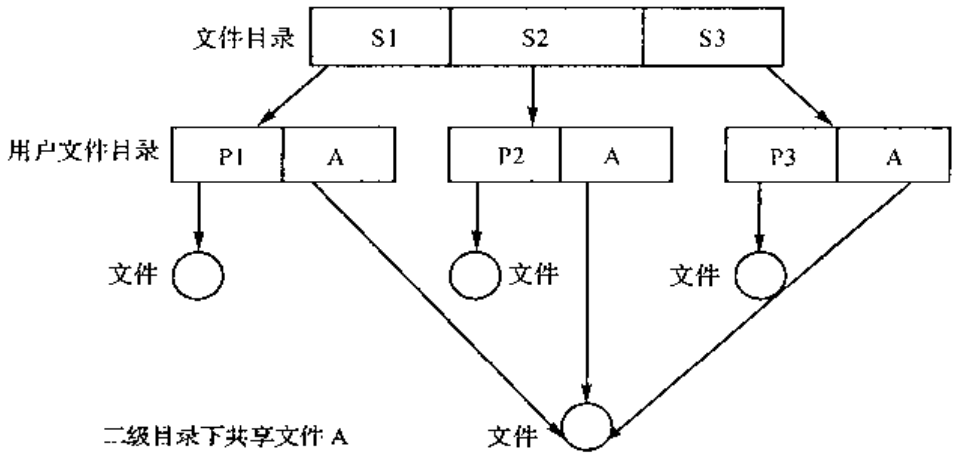


图 4.1

(2)如上图所示的二级目录结构能够满足要求。此时用户文件目录中的 P1，P2 和 P3 均改为 P 即可，由图 4.1 可看出，这三个 P 均指向各自不同的程序

(3)在学生存取程序和数据时,文件系统会先搜索主文件目录,找到该学生的用户目录后,即可在用户目录中查找指定的文件。比如对学生 S1,由路径/S1/P 找到的文件就是 S1 的程序文件,因为它与学生 S2 的程序文件/S2/P 不是同一个文件,所以不会引起冲突。文件/Sl/A 和/S2/A 是同一个文件,因此学生 S1 能够取到所需要的数据。当然,文件 A 可由三个学生同时打开执行读操作。

15. 信息在外存空间的排列方式也会影响存取等待时间。考虑几个逻辑记录 A,B,C,…,J,他们被存放于磁盘上,每个磁道存放 10 个记录,安排如表 4-2 所示:

表 4-2

物理块	1	2	3	4	5	6	7	8	9	10
逻辑记录	A	B	C	D	E	F	G	H	I	J

假定要经常顺序处理这些记录,磁盘旋转速度为 20ms/转,处理程序读出每个记录后花 4ms 进行处理,试问:

(1)处理的总时间为多少?

(2)考虑对信息的分布进行优化,如表 4-3 所示,信息分布优化后处理的总时间为多少?

表 4-3

物理块	1	2	3	4	5	6	7	8	9	10
逻辑记录	A	H	E	B	I	F	C	J	G	D

答:题中磁盘旋转速度为 20ms/转,每个磁道存放 10 个记录,因此读出一个记录的时间为:

$20/10=2(\text{ms})$。

(1)对于第一种记录分布情况,读出并处理记录 A 需要 6ms,则此时读写头已转到记录 D 的开始处,因此为了读出记录 B,必须再转一圈少两个记录(从记录 D 到记录 B)。后续 8 个记录的读取及处理与此相同,但最后一个记录的读取与处理只需 6ms。于是,处理 10 个记录的总时间为:$9*(2+4+16)+(2+4)=204(\text{ms})$

(2)对于第二种记录分布情况,读出并处理记录 A 后,读写头刚好转到记录 B 的开始处,因此立即就可读出并处理,后续记录的读取与处理情况相同。一共旋转二又十分之七圈。最后一个记录的读取与处理只需 6ms。于是,处理 10 个记录的总时间为:$20+20+20*7/10+6=60(\text{ms})$

16. 使用文件系统时,通常要显式地进行 OPEN、CLOSE 操作。

(1)这样做的目的是什么?

(2)能否取消显式的 OPEN、CLOSE 操作?应如何做?

(3)取消显式的 OPEN、CLOSE 操作有什么不利?

(北京大学 1992 年研究生试题)

答:(1)显式的 OPEN 操作完成文件的打开功能。它将待访问的文件的目录信息读入内存中,建立起用户进程与文件的联系。

显式的 CLOSE 操作完成文件的关闭操作。该命令撤消内存中有关该文件的目录信息,切断用户进程与文件的联系;若在文件打开期间,对该文件作过修改,则要将其写回外存。

(2)可以取消显式的 OPEN 和 CLOSE 操作。如果取消显式的 OPEN 和 CLOSE 操作,系统在

进行文件操作之前需要判断文件是否已打开，若文件未打开，则应自动完成文件的打开功能，以建立用户与文件间的联系。同时，在系统结束时，还应自动关闭所有已打开的文件。

(3)取消显式的 OPEN 和 CLOSE 操作，使得文件读写的系统开销增加。因为在每一次读写前都需要判断文件是否已被打开。系统结束时也要做一些额外的工作，以完成 CLOSE 命令的功能。当用户进程已使用完一个文件但尚未执行完时，因无显式的 CLOSE 命令也无法关闭文件，从而不利于系统资源的回收。

17. 文件系统要解决哪些问题？

答：文件系统的主要目标是提高存储空间的利用率，它要解决的主要问题有：完成文件存储空间的管理，实现文件名到物理空间的转换，实现文件和目录的操作，提供文件共享能力和安全措施，提供友好的用户接口。文件系统向用户提供了有关文件和目录操作的各种功能接口和系统调用。

18. 文件系统采用多重索引结构搜索文件内容。设块长为 512 字节，每个块号长 2 字节，如果不考虑逻辑块号在物理块中所占的位置，分别计算二级索引和三级索引时可寻址的文件最大长度。(清华大学 1999 年研究生试题)

答：由于块长为 512 字节，每个块号长 2 字节，因此，一个一级索引表可容纳 256 个磁盘块地址。同样，一个二级索引表可容纳 256 个磁盘块地址，一个三级索引表也可容纳 256 个磁盘块地址。

所以，采用二级索引时可寻址的文件最大长度为：256 * 512 + 256 * 256 * 512 = 128KB + 32GB.

采用上级索引时可寻址的文件最大长度为：256 * 512 + 256 * 256 * 512 + 256 * 256 * 256 * 512 = 128KB + 32GB + 8TB

19. 某个文件系统中，外存为硬盘，物理块大小为 512 字节，有文件 A 包含 598 个记录，每个记录占 255 个字节，每个物理块放 2 个记录。文件 A 所在的目录如图 4.2 所示

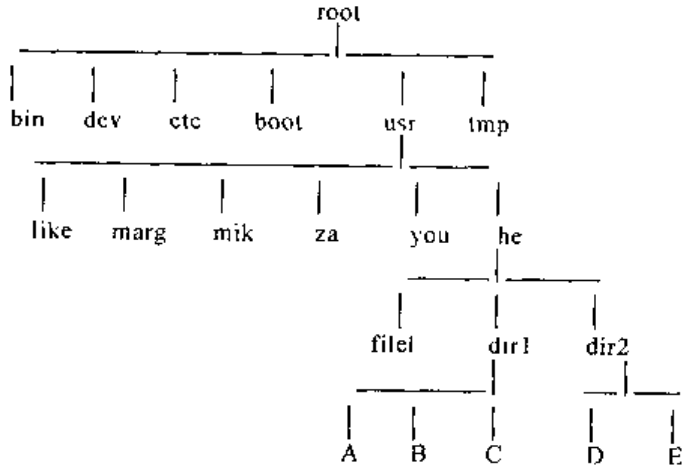


图 4.2

文件目录采用多级树型目录结构，由根目录结点，作为目录文件的中间结点和作为信息文件的树叶组成，每个目录项占 127 字节，每个物理块放 4 个目录项，根目录的第一块常驻内存。

(1)若文件的物理结构采用链式存储方式，链指针地址占 2 字节，那么要将文件 A 读入内存，至少需要存取几次硬盘？

(2)若文件为连续文件,那么要读文件 A 的第 487 个记录至少要存取几次硬盘?

(3)一般为减少读盘次数,可采取什么措施,此时可减少几次存取操作?

答:假设每次盘 I/O 的大小为一个盘块,即 512 字节。

(1)由于根目录的第一块常驻内存,根目录找到文件 A 需要 5 次读盘。由 255 * 2 + 2 = 512 可知,一个物理块在链式存储结构下可放 2 个记录及下一个物理块地址,而文件 A 共有 598 个记录,故读取 A 的所有记录需读盘次数为 598/2 = 299(次),所以将 A 读到内存至少需读盘 299 + 5 = 304(次)。

(2)当文件为连续文件时,同样需要 5 次读盘可找到 A,且知道 A 地址后通过计算只需一次读盘即可读出第 487 记录,所以至少需要 5 + 1 = 6(次)读盘。

(3)为减少因查找目录而读盘的次数可采用索引结点方法。如果一个目录项占 16 字节,则一个盘块可存放 512/16 = 32(个)目录项,与本题一个盘块仅能存放 4 个目录相比,可使因访问目录而读盘的次数减少 1/8。对查找文件的记录而言,可用一个或多个盘块来存放该文件的所有盘块号,即用链接索引方法;一个盘块可存放 512/2 - 1 = 255(个)盘块号,留下一个地址用来指向下一个存储盘块号(索引块)的磁盘块号。这样,就本题来说,查找目录时需启动 4 次磁盘。文件 A 共有 299 个盘块,则查找文件 A 某一记录时需两次取得所有盘块号,再需最多两次磁盘即可把 A 中任一记录读入内存,所以,查找一记录最多需要 8 次访盘,而原来的链接方法查找一个记录时,读盘的操作是在 6 次 - 304 次之间。

20. 文件系统中常用的外存分配策略有哪些?

答:常用的外存分配策略有:连续分配、链接分配、索引分配、混合分配等。

连续分配:是每一个文件都占据了一个完整且连续的磁盘区域。对于这样的文件,由于空间的连续性,当访问下一个磁盘块时,通常无需移动磁头。因此这种分配策略的磁头移动次数最少。对于这类文件,目录通常只需包括文件名、文件块的起始地址和文件长度。这种分配策略的优点是实现简单,存取速度快。缺点是文件长度不易动态增加。

链接分配:每一个文件都有一张相应的磁盘块的链表。这些磁盘块可以分散在磁盘的任何地方,除了最后一个磁盘块外,每一个磁盘块都有一个指针指向下一个磁盘块。这些指针对用户是透明的。对于采用链接空间分配的文件,目录项通常只需包括文件名、文件的开始块和结束块。这种分配策略的优点是没有外部碎片,每一个空闲块都可以用来分配。缺点是只有在顺序访问时,链接空间分配策略才是高效的。

索引分配:每一个文件都有一个索引块。这个索引块就是一个表,每一个表项存放文件所占有的单个磁盘块的地址。表的第 i 项就是指向文件的第 i 个磁盘块。对于这类文件,目录的每一项可包括文件名和文件索引块的地址。这种分配策略不但避免了连续空间分配存在的外部碎片问题和文件长度受限制的问题,而且还支持对任何一个文件块的直接访问。其缺点是由于索引块的分配增加了系统存储空间的开销。

混合分配是多种分配策略的组合。例如,在 UNIX 文件系统中,每个文件都有一个对应的 I 节点(I-node),其中可以有 15 个指针用于空间分配。前 12 个指针指向可以直接访问的磁盘块。剩下的 3 个指针指向间接块,即不包含文件数据的块。第一个指针指向一级间接块。一级间接块也是索引块,包含的不是数据,而是指向数据块的指针。类似地,第二个指针指向的二级间接块,其中包含指向一级间接块的指针。第三个指针指向的是三级间接块。这种分配策略的优点是不但具有以上多种分配策略的所有优点,而且非常灵活。

21. 删除文件时,存放文件的盘块常常返回到空闲盘块链中,有些系统同时清除盘块中的内容,

而另一些系统则不清除，请对这两种方式加以比较。

答：从性能上考虑，后一种方式在删除文件时减少了访问磁盘的次数，故其速度比前一种方式更快。

从安全性上考虑，把一个内容没有清除的盘块分配给下一个用户使用，则有可能使其获取盘块中的内容，故前一种方式比后一种方式更安全。

从方便性上考虑，如果盘块中的内容没被清除，则当用户因误操作而删除文件时，他有可能通过某种办法恢复被删除的文件，故后一种方式更为方便。

22. 设某文件为链接文件，由 5 个逻辑记录组成，每个逻辑记录的大小与磁盘块的大小相等，均为 512 字节，并依次存放在 50，121，75，80，63 号磁盘块上。若要存取文件的第 1569 逻辑字节处的信息，问要访问哪一个磁盘块？

答：因为：1569 = 512 * 3 + 33

所以要访问字节的逻辑记录号为 3，对应的物理磁盘块号为 80。故应访问第 80 号磁盘块。

23. 假定磁盘块的大小为 1K，对于 540M 的硬盘，其文件分配表 FAT 需要占用多少存储空间？当硬盘容量为 1.2G 时，FAT 需要占用多少空间？

答：由题目可知，硬盘大小为 540M，磁盘块的大小为 1K，所以该硬盘共有盘块：

540M/1K = 540K 个

又 512K < 540K < 1024K，故 540K 个盘块要用 20 位二进制表示，即文件分配表的每个表目为 2.5 个字节。

FAT 要占用的存储空间总数为：2.5 * 540K = 1350K

当硬盘大小为 1.2G 时，硬盘共有盘块：1.2G/1K = 1.2M（个）

又 1M < 1.2M < 2M，故 1.2M 个盘块要用 21 位二进制表示，为方便文件分配表的存取，即每个表目用 24 位，即文件分配表的每个表目为 3 个字节。

FAT 要占用的存储空间总数为：3 * 1.2M = 3.6M

24. 在磁盘上有一个文件系统，磁盘每块 512 字。假定每个文件在目录中占有一个目录项；该目录项给出了文件名、第一个索引块的地址、文件长度（块数）。在索引块中（包含第一个索引块）前面 511 个字指向文件块，即第 i 个索引项（i = 0，1，…，510）指向文件的 i 块，索引块中最后一个字指向下一个索引块，最后一个索引块中最后一个字为 nil。假定目录在存储器中，每个文件的逻辑块号均从 0 号开始标号，逻辑块长与物理块长相同，对这样的索引物理结构，该系统应如何将逻辑块号变换成物理块号？（国防科技大学 1999 年考研试题）

答：根据题意，首先将题设条件转化为磁盘存储结构如图 4.3 所示。设逻辑地址为 L，假设逻辑块号为 n

逻辑块号变换成物理块号的过程为：首先根据逻辑文件的文件名找到目录表中该文件对应的目录项，找出第一个索引块的地址 d1，若 n < 511，取出第一个索引块第 n 项的值，即为查找逻辑块号所对应的物理块号 w。如果 n > = 511 得到第二索引块号地址 d2，令 n = n - 511，若此时 n > = 511 则继续令 d2 + 511 得到第三个索引块地址 d3，……依次类推直到 n < 511 时，取出第 i 个索引块第 n 项的值，即为查找逻辑块号所对应的物理块号 w。

25. 一个树形结构的文件系统如图 4.4 所示：

该图中的框表示目录，圈表示文件。

（1）可否进行下列操作：

a. 在目录 D 中建立一个文件，取名为 A；

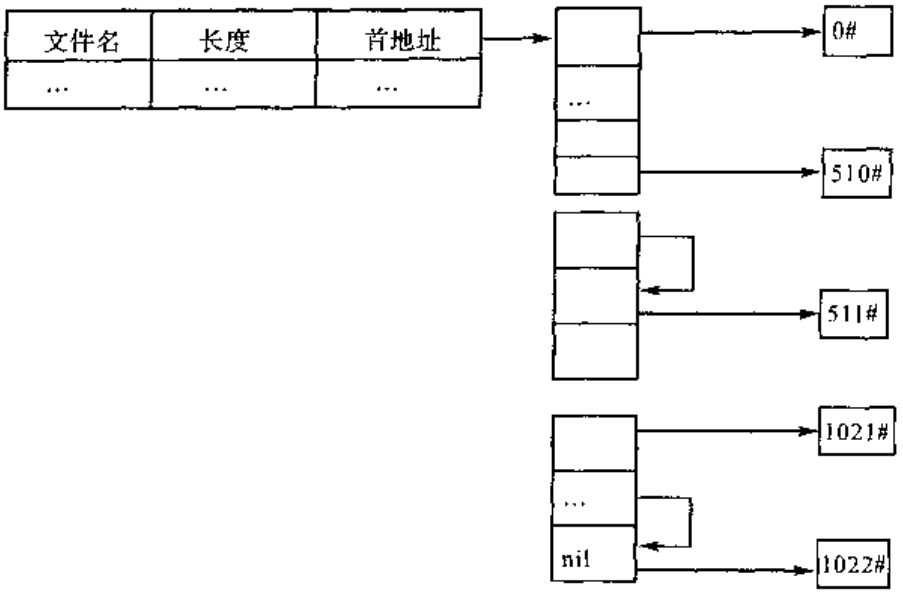


图 4.3

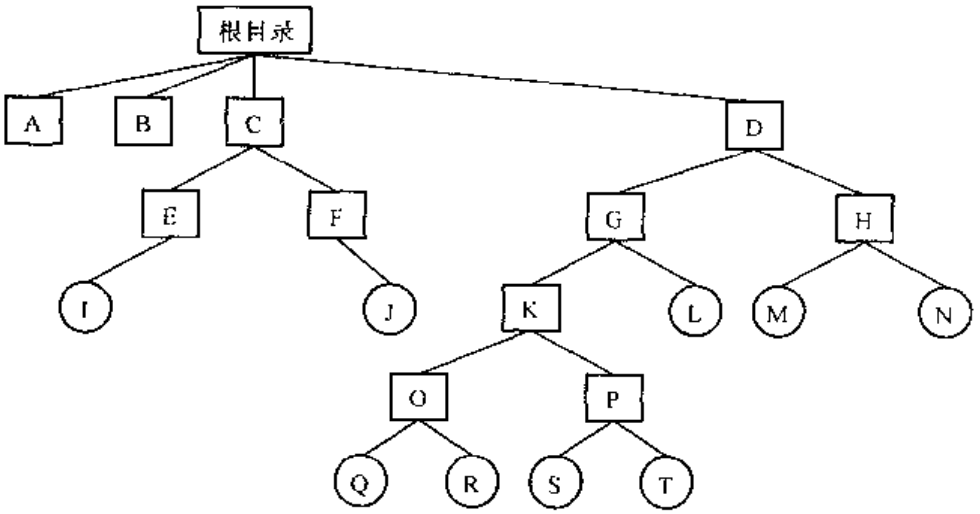


图 4.4

b. 将目录 C 改名为 A。

(2)若 E 和 G 分别为两个用户的目录:

a. 用户 E 欲共享文件 Q,应有什么条件,如何操作?

b. 在一段时间内,用户 G 主要使用文件 S 和 T。为简便操作和提高速度,应如何处理?

c. 用户 E 欲对文件 I 加以保护,不许别人使用,能否实现?如何实现?(北京大学 1990 年考研试题)

答:在本题中,文件系统采用了多级目录组织方式

(1)a. 由于目录 D 中没有已命名为 A 的文件,因此在目录 D 中,可以建立一个取名为 A 的文件。

b. 因为在文件系统的根目录下已存在一个取名为 A 的目录,所以根目录下的目录 C 不能改名为 A。

(2)a. 用户 E 欲共享文件 Q,需要用户 E 由访问文件 Q 的权限。在访问权限许可的情况下,用户 E 可通过相应路径来访问文件 Q,即用户 E 通过自己的主目录 E 找到其父目录 C,再访问到目录 C 的父目录(根目录),然后依次通过目录 D、目录 G、目录 K 和目录 O 访问

到文件 Q。若用户 E 当前目录为 E，则访问路径为：../../D/G/K/O/Q，其中符号“..”表示父目录，符号“/”用于分隔路径中的各目录名。

b. 用户 G 需要通过依次访问目录 K 和目录 P，才能访问到文件 S 和 T。为了提高文件访问速度，可以在目录 G 下建立两个链接文件，分别链接到文件 S 和 T 上。这样用户 G 就可以直接访问这两个文件了。

c. 用户 E 可以修改文件 I 的存取控制表来对文件 I 加以保护，不让别的用户使用。具体实现方法是：在文件 I 的存取控制表中，只留下用户 E 的访问权限，其他用户对该文件无操作权限，从而达到不让其他用户访问的目的。

26. 有一文件系统如图 4.5 所示。图中的框表示目录，圈表示普通文件。根目录常驻内存，目录文件组织成链接文件，不设文件控制块，普通文件组织成索引文件。目录表目指示下一级文件名及磁盘地址(各占 2 个字节，共 4 个字节)。若下级文件是目录文件，指示其第一个磁盘块地址。若下级文件是普通文件，指示其文件控制块的磁盘地址。每个目录文件磁盘块最后 4 个字节供拉链使用。下级文件在上级目录文件中的次序在图中为从左至右。每个磁盘块有 512 字节，与普通文件的一页等长。

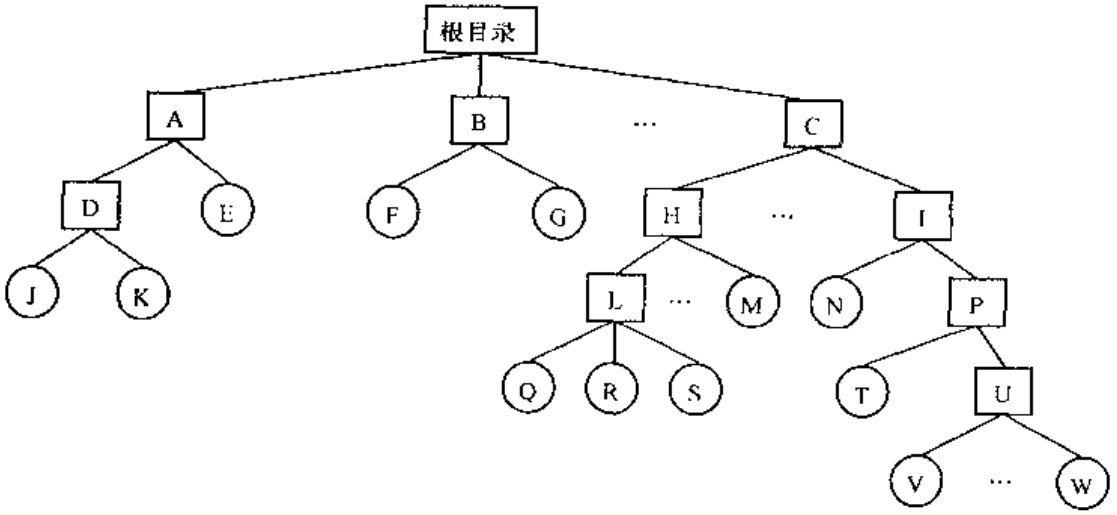


图 4.5

普通文件的文件控制块组织如图 4.6 所示。其中，每个磁盘地址占 2 个字节，前 10 个地址直接指示该文件前 10 页的地址。第 11 个地址指示一级索引表地址，一级索引表中每个磁盘地址指示一个文件页地址；第 12 个地址指示二级索引表地址，二级索引表中每个地址指示一个一级索引表地址；第 13 个地址指示三级索引表地址，三级索引表中每个地址指示一个二级索引表地址。问：

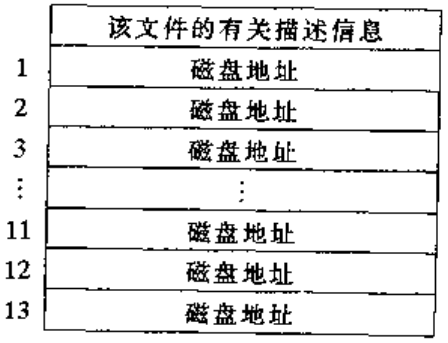


图 4.6

(1)普通文件最多可有多少个文件页?

(2)要读文件 J 中的某一页,最多启动磁盘多少次?

(3)若要读文件 W 中的某一页,最少启动磁盘多少次?

(4)就(3)而言,为最大限度减少启动磁盘的次数,可采用什么方法?此时,磁盘最多启动多少次?

(北京大学 1994 年研究生试题)

答:(1)由题目中所给的条件可知,磁盘块大小为 512 字节,每个磁盘地址占 2 个字节。因此,一个一级索引表可容纳 256 个磁盘地址。同样地,一个二级索引表可容纳 256 个一级索引表地址。一个三级索引表可容纳 256 个二级索引表地址。

一个普通文件最多可有页数为:

$$10 + 256 + 256 * 256 + 256 * 256 * 256 = 16843018$$

(2)从图中可以看出,目录文件 A 和目录文件 D 中,目录项都只有两个。因此这两个目录文件读不需要拉链。若要读文件 J 中的某一页,首先从内存的根目录中找到目录文件 A 的磁盘地址,将其读入内存(第 1 次访问磁盘)。然后再从目录 A 中找出文件 D 的磁盘地址,并将其读入内存(第 2 次访问磁盘)。从目录 D 中找出文件 J 的文件控制块地址,将文件 J 的文件控制块读入内存(第 3 次访问磁盘)。在最坏的情况下,要访问页的磁盘地址需通过三级索引才能找到,这时要三次访问磁盘才能将三级索引表读入内存(第 4、5、6 次访问磁盘)。最后读入文件 J 中的相应页(第 7 次访问磁盘)。

由此可知,若要读文件 J 中的某一页,最多启动磁盘 7 次。

(3)从图中可以看出,目录文件 C 和目录文件 U 中,目录项数目较多。若目录项数超过 127 (512/4 - 1 = 127),则目录文件的读入可能需要多次磁盘读(因为目录文件组织成链接文件)在最好的情况下,所找的目录都在目录文件的第一个磁盘块中。若要读文件 W 中的某一页,首先从内存的根目录中找到目录文件 C 的磁盘地址,将其读入内存(第 1 次访问磁盘)。在最好的情况下,能从目录 C 的第一个磁盘块中找出目录文件 I 的磁盘地址,并将其读入内存(第 2 次访问磁盘)。从目录 I 中找出文件 P 的磁盘地址,将其读入内存(第 3 次访问磁盘)。从目录 P 中找出文件 U 的磁盘地址,将其读入内存(第 4 次访问磁盘)。在最好的情况下,能从目录 U 的第一个磁盘块中找出文件 W 的文件控制块地址,将文件 W 的文件控制块读入内存(第 5 次访问磁盘)。在最好的情况下,要访问页的在前 10 页,这时可直接得到该页的磁盘地址。最后读入文件 W 中的相应页(第 6 次访问磁盘)。

由此可知,若要读文件 W 中的某一页,最少启动磁盘 6 次。

(4)由于通过文件控制块访问文件所需要的访问磁盘次数无法改变,要减少访问磁盘的次数,只有通过减少访问目录文件的次数来达到。为最大限度的减少启动磁盘的次数,可以将文件 W 直接链接在根目录的最左端(或其目录项在根目录的前 127 个项内)。这样,若要读文件 W 中的某页时,首先从内存的根目录中找到文件 W 的文件控制块地址,将文件 W 的文件控制块读入内存(第 1 次访问磁盘)。在最坏的情况下,要访问页的磁盘地址需通过三级索引才能找到,这时要三次访问磁盘才能将三级索引表读入内存(第 2、3、4 次访问磁盘)。最后读入文件 W 中的相应页(第 5 次访问磁盘)。

由此可知,若要将文件 W 直接链接在根目录的最左端,要读文件 J 中的某一页,最多启动磁盘 5 次。

27. 假定磁带记录密度为每英寸 800 字符,每一逻辑记录为 160 个字符,块间隙为 0.6 英寸。

今有 1500 个逻辑记录需要存储，试计算磁带利用率？若要使磁带空间利用率不少于 50%，至少应以多少个逻辑记录为一组？

答：(1)因磁带记录密度为每英寸 800 字符，则一个逻辑记录占的磁带长度为：

$$160/800=0.2 \text{ 英寸}$$

1500 个逻辑记录要占据的磁带长度为：

$$(0.2+0.6)*1500=1200 \text{ 英寸}$$

磁带利用率为：

$$0.2/(0.2+0.6)=25\%$$

(2)要使磁带利用率不少于 50%，即磁带利用率大于或等于 50%，则一组逻辑记录所占的磁带长度应与间隙长度相等，所以一组的逻辑记录数至少为：

$$0.6/0.2=3$$

28. 设磁盘组共有 n 个柱面，编号顺序为 0，1，2，…，n－1；共有 m 个磁头，编号顺序为 0，1，2，…，m－1；每个磁道内的 k 个信息块从 1 开始编号，依次为 1，2，…，k。现用 x 表示逻辑磁盘块号，用 a、b、c 分别表示任一逻辑磁盘块的柱面号、磁头号、磁道内块号，则 x 与 a，b，c 可通过如下公式进行转换：

$x=k\times m\times a+k\times b+c$

$a=(x-1)\text{DIV}(k\times m)$

$b=((x-1)\text{MOD}(k\times m))\text{DIV } k$

$c=((x-1)\text{MOD}(k\times m))\text{MOD } k+1$

若某磁盘组为 n＝200，m＝20，k＝10，问：

(1)柱面号为 185，磁头号为 12，道内块为 5 的磁盘块的逻辑磁盘块号为多少？

(2)逻辑磁盘块号为 1200，它所对应的柱面号、磁头号及磁道内块号为多少？

(3)若每一磁道内的信息块从 0 开始标号，依次为 0，1，…，k－1，其余均同题设，试写出 x 与 a，b，c 之间的转换公式。

答：(1)由上述公式可得，逻辑磁盘块号 x 为：

$$\begin{aligned}x&=k\times m\times a+k\times b+c\\&=10\times 20\times 185+10\times 12+5\\&=37125\end{aligned}$$

所以，柱面号为 185，磁带号为 12，磁道内块号为 5 的磁盘块的逻辑磁盘块号为 37125。

(3)由上述公式可得：

$$\begin{aligned}a&=(x-1)\text{ DIV }(k\times m)\\&=(1200-1)\text{ DIV }(10\times 20)\\&=1199\text{ DIV }200\\&=5\end{aligned}$$

$$\begin{aligned}b&=((x-1)\text{ MOD }(k\times m))\text{DIV } k\\&=((1200-1)\text{ MOD }(10\times 20))\text{ DIV }10\\&=(1199\text{ MOD }200)\text{ DIV }10\\&=19\end{aligned}$$

$$\begin{aligned}c&=((x-1)\text{ MOD }(k\times m))\text{ MOD } k+1\\&=((1200-1)\text{ MOD }(10\times 20))\text{ MOD }10+1\end{aligned}$$

=(1199 MOD 200) MOD 10+1

=9+1

=10

所以,逻辑磁盘块号为1200的盘块所对应的柱面号是5、磁头号是19、磁道内块号时10。

(4)对于磁盘组空间中的任一磁盘块,除了它的磁道内块号比原来小1以外,别的参数值均无变化,因此只要对转换公式中出现磁道内块号的公式作相应修改即可,x与a、b、c之间的转换公式如下:

x=k×m×a+k×b+c+1

a=(x-1) DIV (k×m)

b=((x-1) MOD (k×m)) DIV k

c=((x-1) MOD (k×m)) MOD k

29. 假设磁盘的每个磁道分成9个块,现有一文件有A,B,…,I共9个记录,每个记录的大小与块的大小相等,设磁盘转速为27ms/转,每读出一块后需要2ms的处理时间。若忽略其他辅助时间,试问:

(1)如果顺序存放这些记录顺序读取,处理该文件要多少时间?

(2)如果要顺序读取该文件,记录如何存放处理时间最短?

答:由题目所给条件可知,磁盘转速为每转27ms,每磁道存放9个记录,因此读出1个记录的时间是:

27/9=3 ms

(1)读出并处理记录A需要5ms,此时读写头已转到了记录B的中间,因此为了读出记录B,必须在转将近一圈(从记录B的中间到记录B)。后续8个记录的读取及处理与此相同,但最后一个记录的读取与处理只需要5ms,于是,处理9个记录的总时间为:

8*(27+3)+(3+2)=245ms

(2)由于读出并处理一个记录需要5ms,当读出并处理记录A时,不妨设记录A放在第1个盘块中,读写头已移到第2个盘块的中间,为了能顺序读到记录B,应将它放到第3个盘块中,即应将记录按表4-4顺序存放:

表4-4

盘块	1	2	3	4	5	6	7	8	9
记录	A	F	B	G	C	H	D	I	E

这样,处理一个记录并将磁头移到下一个记录的时间是:

3(读出)+2(处理)+1(等待)=6ms

所以,处理9个记录的总时间为:

6*8+5=53 ms

30. 某软盘有40个磁道,磁头从一个磁道移至另一磁道需要6ms。文件在磁盘上非连续存放,逻辑上相邻数据块的平均距离为13磁道,每块的旋转延迟时间及传输时间分别为100ms、25ms,问读取一个100块的文件需要多少时间?如果系统对磁盘进行了整理,让同一文件的磁盘块尽可能靠拢,从而使逻辑上相邻数据块的平均距离降为2磁道,这时读取一个100块的文件需要多少时间?

答:磁盘整理前,逻辑上相邻数据块的平均距离为13磁道,读一块数据需要的时间为:

$13 * 6 + 100 + 25 = 203$ ms

因此，读取一个 100 块的文件需要时间：

$203 * 100 = 20300$ms

磁盘整理后，逻辑上相邻数据块的平均距离为 2 磁道，读一块数据需要的时间为：

$2 * 6 + 100 + 25 = 137$ms

因此，读取一个 100 块的文件需要时间：

$137 * 100 = 13700$ms

31. 有文件系统如图 4.7 所示。图中的框表示目录，圈表示普通文件。（北京大学 1995 年研究生试题）

(1)可否建立 F 与 R 的连接？试加以说明。

(2)能否删除 R，为什么？

(3)能否删除 N，为什么？

(4)某用户指定 P 为当前目录，若想用 pr 命令打印 Q 及 S，请列出命令行。

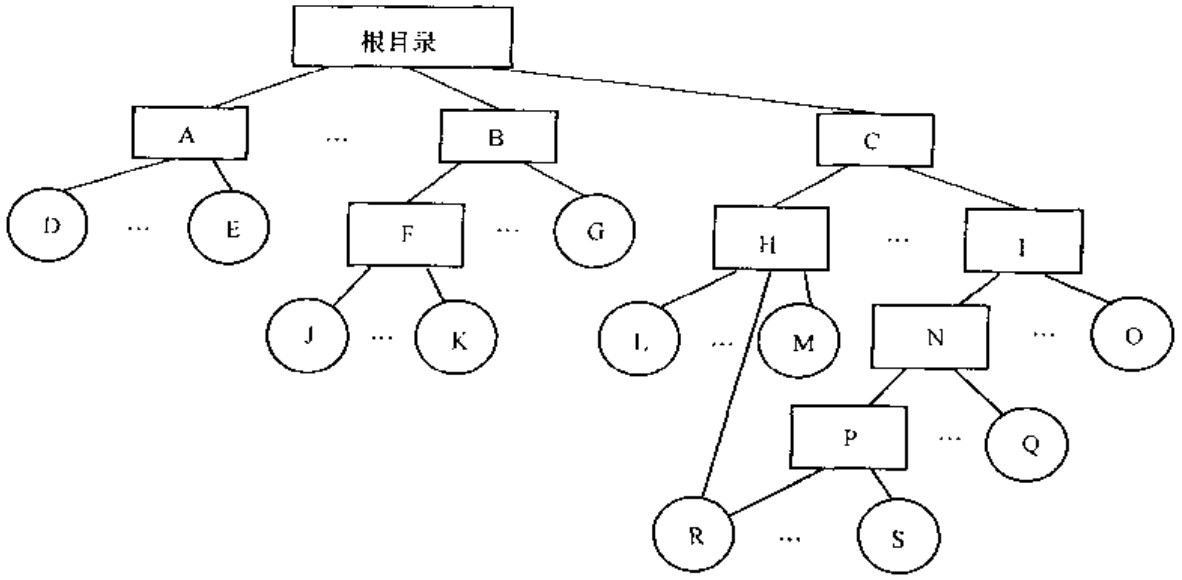


图 4.7　文件系统结构示意图

答：(1)不能建立连接。因为 F 是一个目录而 R 是一个文件，由于只能建立目录到目录或文件到文件的连接，所以不能把 F 和 R 连接起来。

(2)不一定能删除 R。由于 R 是被多个目录所共享，能否删除 R 取决于文件系统实现共享的方法。如果采用基于索引结点的共享方法，则因删除后存在指针悬空问题而不能删除 R 结点。如果采用基于符号共享的方法，则可以删除 R 结点。

(3)不一定能删除 N。由于 N 的于目录中存在共享文件 R，而 R 结点本身不一定能被删除。

(4)打印 Q 的命令行为：pr . . \ Q

打印 S 的命令行为：pr S

32. 有一个交叉存放信息的磁盘，信息在其上的存放方法如图 4.8 所示。每个磁道有 8 个扇区，每扇区 512 字节，旋转速度为 3000 转/分。假定磁头已在读取信息的磁道上，0 扇区转到磁头下需要 1/2 转，且设备对应的控制器不能同时进行输入/输出，在数据从控制器传送至内存的这段时间内，从磁头下通过的扇区数为 2，问依次读出一个磁道上的所有扇区需要多少时间？其数据传输速度为多少？

答:从图 4.8 中可知,信息块之间的间隔为 2 个扇区。由题目所给的条件可知,旋转速度为:

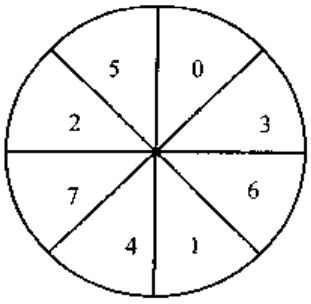


图 4.8 交叉存放方式

3000 转/分 = 50 转/秒,即 20ms/转。

读一个扇区需要时间:

20/8 = 2.5 ms

读一个扇区并将扇区数据送入内存需要时间:

2.5 * 3 = 7.5 ms

读出一个磁道上所有扇区需要时间:

20/2 + 8 * 7.5 = 70ms = 0.07s

每磁道数据量:

8 * 512 = 4KB

数据传输速度为:

4KB/0.07 = 57.1KB/秒

所以,依次读出一个磁道上的所有扇区需要 0.07 秒。其数据传输速度为 57.1KB/秒

33. 设某系统磁盘共有 500 块,块号从 0~499。

(1)若用位示图法管理这 500 块的盘空间,当字长为 32 位时,位示图需要多少个字?

(3)第 i 字第 j 位对应的块号是多少?

答:(1)位示图需要⌈500/32⌉ = 16 个字

(2)第 i 字第 j 位对应的块号是:32 × i + j

4.5 OSC 习题与解答

Chapter 11

11.1 考虑一个文件系统,某个文件被删除并且它的磁盘空间被收回时,指向该文件的链接却依然存在。如果在同一个磁盘空间建一个新的文件,将会出现什么问题?如何避免该问题?

答:假设 F1 是旧的文件,F2 是新的文件。当一个用户通过已存在的链接访问 F1,实际却是访问 F2。这里使用的是对文件 F1 的存取保护而不是与文件 F2 相关的存储保护。采用删除指向一个已删除文件的所有链接的方法避免该问题。可以通过几种方法实现:

(1)设置一个记录指向一个文件的所有链接的链表,当这个文件被删除时,删掉这些链接。

(2)保留这些链接,当试图访问一个已被删除的文件时,删掉这些链接。

(3)设置一个文件的指针链表(或计数器),当指向该文件的所有指针被删除时才真正删除这个文件。

11.2 某些文件系统,当一个用户退出系统或终止一个作业时,系统会自动地删除所有相关的文件,除非用户明确地要求保留它们;而另一些系统则保留所有的文件,除非用户明确要求删除它们。试讨论两种方法的优点。

答:删除所有文件,除非用户明确指定保存文件,这种方法有利于减少每个用户的不需要保存或没用文件所占用的空间。保存所有文件除非明确要求,这种方法对用户更加安

全，因为用户不会由于忘记保存文件而使文件丢失。

11.3 为什么有些系统文件类型有明确含义，另一些系统由用户自己确定或不实现多种文件类型？那种系统更好呢？

答：一些系统允许基于文件类型的不同操作（比如一个 ASCII 文件可以被当作流来读，而一个数据库文件通过一个指向一个块的索引来读）。而另一些系统把文件数据解释留给进程，而没有给出访问数据方法。哪一种方法更好取决于系统中的进程需要和用户对操作系统的要求。如果一个系统主要运行数据库应用程序，操作系统通过数据库类型文件和提供的相应操作比使每个程序完成同样的事情（可能是不同方法）更有效率。对于一般的系统，系统只实现基本文件类型，这样使操作系统更小，并且给系统中的进程更大的自由。

11.4 相似的，一些系统支持多种数据结构的文件类型，而有些系统只支持字节流文件。他们有优点和缺点是什么？

答：（参考上一题）

11.5 请说明在文件属性中记录创建该文件程序的名字的优点和缺点（Macintosh 操作系统实现这一功能）？

答：通过记录创建程序的名字，操作系统能够实现基于这个信息的一些功能（比如当访问文件时可以自动调用该创建程序）。然而在操作系统里实现这种功能需要文件描述的空间开销。

11.6 能否用一个可使用任意长度文件名单级目录结构来模拟一个多层级目录结构？如果能，那么请说明如何模拟，并把这个方案与多级目录结构机制相比较；如果不能，请说明理由。如果文件名限制为 7 个字符，那么你的方案应作如何修改？

答：这两个答案都是‘能够’。

给定一个多级目录结构 D1，那么每一级的信息被存储在某一地方，以便构造一个相等的单级目录结构 D2。如果允许使用任意长的名字，那么在 D2 中的所有文件都能象在 D1 中相应部分的绝对路径名一样被命名。从根节点到目标节点，在 D1 上的绝对路径名能够通过在路径上所有节点名字的方法而被构造，并用特制字符隔开如“/”。例如：‘root/Dir1/File1’，这表明 File1 是子目录 Dir1 中的文件，而 Dir1 是根目录 root 的文件。这样 D1 每一级的信息都被保留在 D2 中，并且关于 D1 所有的操作都可在 D2 上通过相应的操作来模拟。

这个方案的实现方法不同于普通的多级目录机制，例如：在多级目录中这些文件需要包含在这个目录中查找，而单级目录结构的所有文件都要进行扫描模拟，列出同一个目录的文件，或者把一个目录移动到另外一个地方。由于查找和修改都将存取根节点信息，单级目录结构的根节点也将变成一个瓶颈问题。

如果文件名长度限制在 7 个字符，那么上面的方案不能使用。一个变通的方法是通过把一个任意长的文件名字映射成为一个短文件名，放入一张表格中。象哈希表技术可以把映射时间减少到 O(1)。例如，如果使用 ASCII 字符集‘a-z’，那么，在这个单级目录结构的命名空间的文件名总字符数有 26 个，这足够全面模拟了。

11.7 请解释打开和关闭操作的作用

答：打开操作告知系统指定的文件将被激活。

关闭操作是告知系统，进行关闭操作的用户对指定文件不再处于使用状态。

11.8 有些系统在文件第一次被引用时会自动打开，而当作业终止时自动关闭该文件。在传统方法中用户必须明确地打开或关闭文件。通过与传统方法的比较，请给出这种方案的优缺点。

答：文件的自动打开和关闭减少了用户对这些功能的调用，这样对用户来说也变得更加方便；然而，它比直接打开和关闭文件需要更多的系统开销。

11.9 请给出文件中的数据必须按如下方法访问的应用例子：

a. 顺序访问

b. 随机访问

答：a. 打印文件内容

b. 数据库；打印第 i 条记录内容。这条记录通过哈希或索引技术找到。

11.10 有些系统通过保留文件的单个副本来提供文件共享；而另外一些系统则对每一共享的用户各保留一个副本的方式来提供文件共享。请讨论这两种方法的各自优缺点。

答：保留单个副本，有利于节省存储空间，但对同一文件的多次并发更新，可能导致用户得到不正确的信息，并且文件有可能在不正确的状态下退出。保留多个副本，则浪费一些存储空间且各个副本可能导致相互之间不一致的情况。

11.11 在一些系统中，一个授权用户可以像对普通文件那样对一个子目录进行读写操作。

(1)这样做可能产生什么保护问题？

(2)请提出一个处理这些问题方案。

答：(1)存在目录项中的信息之一是文件地址，如果用户能够修改这个地址，那么他能够访问别的文件，从而违反存取保护的规定；

(2)不允许用户直接对子目录写入信息，而应提供系统操作来完成这些事情。

11.12 考虑一个能支持 5000 个用户的系统。假如只允许这些用户中的 4990 个能够存取同一文件，那么(1)在 UNIX 中，如何实现这个保护方案？

(2)请你提出一个比 UNIX 方案更加有效的保护方案？

答：(1)有 2 种方法可以实现

i. 建立一张包含所有 4990 个用户名字的存取控制表。

ii. 把所有 4990 个用户放入一个组中，并设置这个组的存取权限。由于这些用户组受到系统限制，这个方案不一定总能实现。

(2)规定所有的用户都能存取文件信息，除非他们的名字出现在没有存取权限的存取控制表中。据此，可以把余下的 10 个用户的名字放入该存取控制表中，但是他们没有给予任何存取权限。

11.13 研究人员建议，我们应该用与每个用户相关的用户权限表(规定一个用户能存取哪些文件，并且如何存取)代替与每个文件相关的存取控制表(指定哪些用户能存取这个文件，并且如何存取)。试讨论这两个方案的优缺点。

答：用户权限表：当打开一个文件时，系统资源的开销更少。

文件(存取)控制表：由于存取控制信息被集中在某一地方，这样更加容易修改存取控制信息并且只需较少的空间。

11.14 考虑一个含有 100 块的文件。假如文件控制块(和索引块，当用索引分配时)已经在内存中。当使用连续、链接、单级索引分配策略时，各需要多少次磁盘 I/O 操作？假设在连续分配时，在开始部分没有扩张的空间，但在结尾部分有扩张空间，并且假设待添加

块的信息已在内存中：

(1)在开始增加一块。

(2)在中间增加一块。

(3)在末端增加一块。

(4)在开始删除一块。

(5)在中间删除一块。

(6)在末端删除一块。

答：各种策略相应的磁盘 I/O 操作次数如表

	连续	链接	索引
a.	201	1	1
b.	101	52	1
c.	1	102	1
d.	198	1	0
e.	98	52	0
f.	0	100	0

11.15 假如一个系统自由空间通过自由空间表管理。

(1)假设指向自由空间表的指针丢失了，系统还能重构自由空间表吗？为什么？

(2)请提出一个不因内存故障而丢失该指针的方案。

答：(1)能。为了重构自由空间表，必须实现“回收站”的功能。这需要查找全部的目录结构以判定哪些页面已分配给作业，而将那些未分配的页面重新链接成一个自由空间表。

(2)将自由空间表的指针存储在磁盘上，并且存在几个不同地方。

11.16 假如一个系统允许一个文件系统被同时安装在多个不同的地方，会发生什么问题？

答：对同一文件会有多条路径，用户很容易出差错。(因为当用一条路径删除一个文件时，也要删除其他所有路径上的这个文件)。

11.17 为什么用于文件分配的位示图要保存在外存储器上而不是内存储器上？

答：当系统崩溃(内存故障)时，自由空间表就不会被丢失，而如果位示图在内存中，空闲空间表有可能丢失。

11.18 考虑一个支持连续、链接和索引分配策略的系统，对一个给定的文件，决定采用哪个策略的条件是什么？

答：采用连续分配策略的条件是：对文件访问是顺序的，而且文件相对较小。

采用链接分配策略的条件是：如果文件较大并且通常是顺序访问的。

采用索引分配策略的条件是：如果文件较大并且通常是随机访问的。

11.19 考虑一个在磁盘上的文件系统，其中逻辑块和物理块大小为 512 字节。假定每个文件的信息已经在内存中，对于三种分配策略中的每一种(连续、链接、索引)，请回答下面这些问题。

(1)说明在这个系统中是如何实现从逻辑地址到物理地址映射的？(对于索引分配，假设文件的长度总是小于 512 块)。

(2)如果当前位于逻辑块 10(即最后一次访问的逻辑块是 10)，且希望访问逻辑块 4，必须从磁盘上读多少个物理块？

答:令 Z 是文件开始物理地址(块号)。

a.若使用连续分配策略时。用 512 去除逻辑地址,则 X 和 Y 分别表示得到的整数和余数。

(1)将 X 加上 Z 得到物理块号,Y 为块内的位移

(2)1

b.若使用链接分配策略。用 511 去除逻辑地址,则 X 和 Y 分别表示得到的整数和余数。

(1)查找链表到第 X+1 块,Y+1 位该块内的位移量。

(2)4

c.若使用索引分配策略。用 512 去除逻辑地址,则 X 和 Y 分别表示得到的整数和余数。

(1)把索引块读入内存中,则物理块地址存放在索引块在第 X 位置中,Y 为块内的位移量。

(2)2

11.20 与连续分配相关的一个问题是用户必须为每一个文件预分足够的空间,如果使用过程中文件变的大于预分配的空间,则必须采取相应的措施。解决这个问题的方法之一是先定义一个仅有一个(特殊大小)的初始连续区组成的文件结构,如果这个区域被填满了,则操作系统会自动定义一个和初始化连续区相链接的溢出区。如果这个溢出区也被填满了,则系统又分配另外一个溢出区,如此。试将这个方案与标准的连续分配和链接分配进行比较。

答:(略)

11.21 存储器上的碎片可以通过 compaction 来消除。典型的磁盘驱动器没有重新移动功能或初始化功能(像在内存拼接时所使用的),那么我们怎么样才能移动文件呢?给出三个理由来说明,为什么文件的拼接和移动常常可以避免?

答:在二级存储器中的文件移动要考虑开销,因为要将大量数据块读入到内存,然后又写回到它们的新位置。此外,重分配仅仅应用于连续文件,而许多磁盘上的文件是不连续的。由于同样的原因,许多新文件不需要连续的磁盘空间。

11.22 如何利用高速缓冲器改善性能?如果它们这么有用,为什么操作系统不用更多和更大的高速缓冲器呢?

答:高速缓冲器是临时存储来自慢速设备的数据或一个快速设备的数据,它允许不同速度设备之间更有效地通信。但同时,高速缓冲器比一般缓冲器的价格更高,因此增加高速缓冲器的数量或大小将大大增加系统成本。

11.23 在什么情况下,把内存作为一个 RAM 磁盘比它用作一个磁盘缓存器更有用?

答:当用户(或系统)明确地知道需要什么数据的时候。RAM 磁盘是用户直接控制的;而磁盘缓冲器是基于运算的,由操作系统控制的。

11.24 操作系统为用户动态分配内部表,为什么说是有利的?操作系统这样做,会出现什么样的后果?

答:在操作系统中使用动态表有更好的灵活性,它允许表格动态增长,避免了使用中的范围限制。但是,操作系统的核心结构和代码将会变得更加复杂,因此也会有更多的隐含错误。动态表和静态表相比,对一个资源的使用将会带走更多的系统资源。

11.25 考虑下面的备份方案:

第一天：从磁盘中拷贝所有文件到一个备份介质中；

第二天：拷贝所有更改过的文件到另一个备份介质中；

第三天：拷贝所有从第一天以来更改过的文件到另一个备份介质中。

对比通过拷贝所有随后的备份文件和拷贝自从第一次全备份后所有修改过的文件的计划表，试比较这个系统的优点是什么，缺点是什么？所有的存储操作变得更容易还是更复杂了？解释一下你的回答。

答：恢复将变得更加容易。因为你可以到最近的备份磁带介质中去找而不是到整个磁盘空间中找。也不需要再去读取中间磁盘介质，这将会有更多的磁盘空间能够用于更多的文件变动中。

第5章

I/O管理

5.1 概述

连接到计算机上的设备多种多样。某些设备一次传输一个字或一块字。某些设备只能顺序访问，而有些可以随机访问。某些设备同步传输数据，而有些是异步传输。有些是专用的，而有些则是共享的。有些设备只读而有些则是读写皆可。它们的速度差别很大。不过大部分情况下，它们都是整个计算机系统中最慢的部分。

既然设备之间存在很大差别，操作系统需要为应用程序提供大量功能以控制这些设备。操作系统的I/O子系统的关键目的就是为了提供尽可能简洁的接口。并且由于设备是整个计算机系统的一个瓶颈，所以I/O子系统的另一个关键是尽可能地进行优化。

5.2 I/O系统

I/O有关硬件的基本要素是总线、设备控制器和设备本身。设备与内存间的数据移动工作是由CPU按程序控制I/O来完成，或转交给DMA(Direct Memory Access，直接存储器访问)控制器。控制设备的内核模块称为设备驱动程序。为应用程序所用的系统调用设计为处理若干基本类型的硬件，包括块设备、字符设备、内存映射文件、网络socket、可编程间隔定时器。系统调用通常会使发布命令的进程阻塞，但是非阻塞和异步调用可以为内核自己所使用，也可为不需要等I/O完成的应用程序所使用。

内核I/O子系统提供了若干服务。其中有I/O调度、缓冲、假脱机、出错处理和设备预留。另一个服务是名称转换，即在硬件设备和应用程序所用和符号文件名之间建立连接。这包括多次映射，可以将字符文件名称映射到特定设备驱动程序和设备地址，然后到I/O端口和总线控制器的物理地址。这一映射可以包含在文件系统名称空间内，如同UNIX，也可以出现在独立的设备名称空间内，如MS-DOS。

STREAMS是使得设备驱动程序可以重用和更容易使用的一种实现方法。通过STREAM，驱动程序可以堆叠，数据可以按单向和双向来传输和处理。

由于物理设备和应用程序之间的多层软件，I/O系统调用所用的CPU周期较多。这些层意味着多种开销，如穿过内核保护边界的上下文切换，用于I/O设备的信号和中断处理，以及用于内核缓冲和应用程序空间之间的数据拷贝所需要的CPU和内存系统的负荷。

5.3 大容量存储器的结构

磁盘驱动器是绝大多数计算机的主要外存I/O设备。磁盘I/O的请求主要由文件系统和虚拟内存系统所产生。每个请求以逻辑块号的形式，指定所引用的磁盘的地址。

磁盘调度算法可改善有效带宽、平均响应时间、响应时间差异。许多算法如SSTF (Shortest-Seek Time-First，最短寻道时间优先)算法、SCAN算法、C-SCAN算法、LOOK算法和C-LOOK算法，通过磁盘队列的重排以改善这些指标。

性能可能因外部碎片而降低。有些系统提供工具，以扫描文件系统，进而确定碎片文件；它们可移动块以降低碎片。对一个严重变成碎片的文件系统进行碎片整理可显著地改善性能，但是在整理碎片时，系统性能也会受些影响。复杂文件系统如UNIX的FFS(Fast File System，快速文件系统)采用了许多措施以控制空间分配所引起的碎片，这样就不需要对磁盘进行重新组织。

操作系统管理磁盘块。首先，必须低级格式化磁盘从而在原来硬件上创建扇区，新磁盘通常已经低级格式化。接着，对磁盘进行分区，创建文件系统和分配启动块以存储系统的启动程序。最后，当一块损坏时，系统必须提供一种方法以便不再使用该坏块，或用另一备份块来逻辑替代它。

因为有效交换空间对于提高性能十分关键，系统通常绕过文件系统，而直接使用低级磁盘访问以进行调页。有的系统将整个磁盘分区用作交换空间，也有的系统使用文件系统内的一个文件作为交换空间。其他系统提供两种选择，以允许用户或系统管理员作出决定。

写前日志方案要求使用稳定存储。为了实现这种存储，我们需要在多个具有不同差错模式的非易失性存储设备(通常是磁盘)上重复所需信息。我们也需要按一定控制方式来更新信息以确保我们能在数据传输出错或恢复出错之后，能恢复数据。

由于大系统要求大量存储，所以经常通过RAID(Redundant Array of Independence Disks，冗余独立磁盘阵列)算法以使磁盘冗余。这些算法允许多个磁盘用于一个给定操作，即使在磁盘出错时也允许继续运行，甚至恢复数据。RAID算法分成不同级别，每个级别都有不同的可靠性和数据传输速度的组合。

磁盘可通过两种方式与计算机系统相连：(1)通过主机的本地I/O端口或(2)通过网络连接如存储域网络。

第三级存储包括磁盘和磁带驱动器，它们使用可移动媒介。可以使用许多不同技术，包括磁带、可移动磁盘、磁光盘、和光盘。

对于可移动磁盘，操作系统通常提供文件系统接口的全部功能，包括空间管理和请求队列调度。对于许多操作系统，可移动媒介上的文件名称由驱动器名称和该驱动器内的文件名称所组成。与采用一个名称以标识特定媒介相比，这种方法较简单但比较容易混淆。

对于磁带，操作系统通常只提供原始接口。对于光盘塔，许多操作系统都没有内置支持。光盘塔支持可通过驱动程序或专用于备份或HSM(Hierarchical Storage Management，层次存储管理)的应用程序来提供。

性能有三个重要衡量因素：带宽、延迟和可靠性。磁盘和磁带有各种不同带宽，但是磁带随机访问延迟与磁盘相比要慢很多。光盘塔的盘片切换也相对较慢。因为光盘塔与盘片相比，有较低的驱动器速率，所以从光盘塔中读入大量数据需要大量时间。光介质(通过透明涂

层保护敏感数据)要比磁介质(将磁材料暴露在外以至于可能更容易受到物理损坏)更耐用。

5.4 习题与解答

一、选择题

1. I/O 交通管制程序的主要功能是管理_____的状态信息。(陕西省 1998 年自考题)

A. 设备、控制器和通道　　B. 主存、控制器和通道

C. CPU、主存和通道　　D. 主存、辅存和通道

答:A

分析:对外围设备的控制常分为:设备、控制器和通道三个层次,所以 I/O 交通管制程序的主要功能是管理这三个层次的状态信息

2. 采用 SPOOLing 技术后,使得系统资源利用率_____(陕西省 1998 年自考题)

A. 提高了　　B. 有时提高有时降低了

C. 降低了　　D. 提高了,但出错的机会增加了

答:A

分析:采用 SPOOLing 技术后,可将高速的设备如磁盘虚拟化为多个"高速"的独占设备如打印机等,故可提高系统资源利用率。

3. 设备控制块是_____ (西安理工大学 2001 年研究生试题)

A. UCB　　B. JCB

C. PCB　　D. CCB

答:A

分析:JCB 是作业控制块,PCB 是进程控制块,

4. 在关于 SPOOLing 的叙述中,_____描述是不正确的。(西安电子科技大学 2000 年研究生试题)

A. SPOOLing 系统中不需要独占设备

B. SPOOLing 系统加快了作业执行的速度

C. SPOOLing 系统使独占设备变成共享设备

D. SPOOLing 系统利用了处理器与通道并行工作的能力

答:A

分析:SPOOLing 是操作系统中采用的一种将独占设备改造为共享设备的技术,其提高了处理器与通道,有效减少了进程等待读入/读出信息的时间,加快了作业的执行速度,并提高了处理器与通道并行工作的能力。故选 A。

5. 虚拟设备是靠_____技术来实现的。

A. 通道　　B. 缓冲　　C. SPOOLing　　D. 控制器

答:C

分析:SPOOLing 是操作系统中采用的一种将独占设备改造为共享设备的技术,通过这种技术处理后的设备通常可称为虚拟设备。

6. 磁盘设备的 I/O 控制主要是采取_____方式。

A. 位　　B. 字节　　C. 帧　　D. DMA

答:D

分析:DMA方式主要用于块设备,磁盘是典型的块设备。故选D。

7. 下面关于设备属性的论述中,正确的是_____。

A. 字符设备的基本特征是可寻址的,即能指定输入的源地址和输出的目标地址

B. 共享设备必须是可寻址的和可随机访问的设备

C. 共享设备是指同一时间内允许多个进程同时访问的设备

D. 在分配共享设备和独占设备时都可能引起进程死锁

答:B

分析:可寻址是块设备的基本特征,故A不正确,共享设备是指一段时间内允许多个进程同时访问的设备,在同一时间内,即对于某一时刻共享设备仍然允许一个进程访问,故C不正确。分配共享设备是不会引起进程死锁的,故D不正确。

8. 通道是一种特殊的_____

A. I/O设备　　B. I/O控制器　　C. 处理机　　D. 存储器

答:C

分析:通道又称为输入输出机(IOP),可以独立完成系统交付的输入输出任务,通过执行自身的输入输出专用程序进行主存和外设间的数据传输。

9. 缓冲技术的缓冲池在_____中。

A. 主存　　B. 外存　　C. ROM　　D. 寄存器

答:A

分析:由于CPU的速度比I/O的速度高得多,因此缓冲池通常是在内存/主存中设立的

10. 采用SPOOLing技术将磁盘的一部分作为公共缓冲区以代替打印机,用户对打印机的操作实际上是对磁盘的存储操作,用以代替打印机的部分是_____

A. 独占设备　　B. 共享设备　　C. 虚拟设备　　D. 一般物理设备

答:B

分析:SPOOLing是操作系统中采用的一种将独占设备改造为共享设备的技术,通过这种技术处理后的设备通常可称为虚拟设备。

11. 通道又称I/O处理机,用于完成_____之间的信息传输。(陕西省1998年自考题)

A. CPU与外设　　B. CPU与外存　　C 主存与外设　　D 主存与外存

答:C

分析:通道又称为输入输出机(IOP),可以独立完成系统交付的输入输出任务,通过执行自身的输入输出专用程序进行主存和外设间的数据传输。

12. 下列算法中用于磁盘移臂调度的是_____。(西安电子科技大学2002年研究生试题)

A. 时间片轮转法　　B. LRU算法

C. 最短寻找时间优先算法　　D. 优先级高者优先算法

答:C

分析:A、B、D均为作业调度算法,只有C是磁盘移臂调度算法。

13. 在采用SPOOLing技术的系统中,用户的打印数据首先被送到_____。(西北工业大学1999年研究生试题)

A. 磁盘固定区域　　B. 内存固定区域

C. 终端　　D. 打印机

答:A

分析:采用 SPOOLing 技术的系统中,用户的打印数据通常先出内存送至输出井暂存,待输出设备(打印机)空闲时再将输出井中的数据经过缓冲区送到输出设备(打印机)上。而输出井通常是在磁盘上开辟出来的固定存储区。故选 A。

14. 如果 I/O 所花费的时间比 CPU 的处理时间短得多,则缓冲区_____。(西北工业大学 1999 年研究生试题)

A. 最有效　　B. 几乎无效

C. 均衡　　D. 以上都不是

答:B

分析:缓冲区主要解决输入输出速度比 CPU 处理的速度慢而造成数据积压的矛盾,所以,如果 I/O 花费的时间比 CPU 处理时间短的多,则缓冲区没有必要设置。故选 B

15. 在 I/O 设备控制方式的发展过程中,最主要的推动力是_____。

A. 提高资源利用率

B. 提高系统吞吐量

C. 减少 CPU 对 I/O 控制的干预

D. 提高 CPU 和 I/O 设备并行操作的程序

答:C

分析:在 I/O 控制的发展过程中,始终贯穿着这样一个宗旨:尽量减少主机/CPU 对 I/O 控制的干预,把主机从繁杂的 I/O 控制事务中解脱出来,以更多地去完成其数据处理任务。故选 C。

16. 下面关于虚拟设备的正确叙述是_____。

A. 虚拟设备是指允许用户使用比系统中拥有的物理设备更多的设备

B. 虚拟设备是指允许用户以标准化方式来使用物理设备

C. 虚拟设备是指把一个物理设备变换成多个对应的逻辑设备

D. 虚拟设备是指允许用户程序不必全部装入内存就可以使用系统中的设备

答:C

分析:虚拟设备是指使用虚拟技术把独占设备改造成为多用户共享的设备,也就要把一台具体的物理设备变换为若干个逻辑设备,故 C 正确。

17. 设备独立性的说法正确的是_____。

A. 设备独立性是指 I/O 设备具有独立执行的 I/O 功能的一种特性。

B. 设备独立性是指用户程序独立于具体物理设备的一种特性。

C. 设备独立性是指能够实现设备共享的一种特性。

D. 设备独立性是指设备驱动程序独立于具体物理设备的一种特性。

答:B

分析:设备独立性的定义就是指用户程序独立于具体物理设备的一种特性,故选 B。A、C、D 与设备独立性无关。

18. SPOOLing 技术的主要目的是_____。

A. 提高 CPU 和设备交换信息的速度

B. 提高独占设备的利用率

C. 减轻用户编程负担

D. 提供主,辅存接口

答:B

分析:SPOOLing技术可将独占设备改造为共享设备,其主要目的是提高系统资源/独占设备的利用率。故选B。

19. 如果I/O设备与存储设备间的数据交换不经过CPU来完成,则这种数据交换方式是_____。

A. 程序查询方式　　B. 中断方式

C. DMA方式　　D. 无条件存取方式

答:C

分析:在DMA(直接内存存储)控制器控制下,外设直接与内存交换成批数据而不用CPU干预。故选C。

20. CPU输出数据的速度远远高于打印机的打印速度,为解决这一矛盾可采用_____。

A. 并行技术　　B. 通道技术

C. 缓冲技术　　D. 虚拟技术

答:C

分析:为解决设备间传送速率不匹配问题,通常采用缓冲技术。通道技术能最大地使CPU摆脱外设的速率制约,并行技术能有效地提高CPU与外设的效率,虚拟技术则能提高打印机的利用率。这些技术虽能有效地缓解CPU输出数据的速度远远高于打印机的打印速度的矛盾,但针对本题,最佳答案应该为C。

21. 在设备管理中,设备管理程序通常包括_____、_____、_____。(陕西省1998年自考题)

A. I/O交通控制程序　　B. I/O调度程序

C. I/O设备处理程序　　D. I/O循环测试程序

E I/O通道启动程序

答:A, B, C

分析:略

22. 驱动调度算法中,_____算法可能会随时改变移动臂的运动方向。(西安电子科技大学2002年研究生试题)

A. 电梯调度　　B. 最短寻找时间优先

C. 扫描　　D. 单向扫描

答:B

分析:最短寻找时间优先(SSTF)调度是查找距离磁头最短(也就是查找时间最短)的请求作为下一次服务对象。因此其可能会随时改变移动臂的运动方向。故选C。扫描算法可细分为电梯调度(SCAN)算法和单向扫描(CSCAN)算法。电梯调度(SCAN)算法是在磁头前进方向上最短寻找时间的服务,如果前进方向上没有请求(即处理完最高/低编号柱面请求后),则掉转方向。CSCAN算法很大程度上消除了SSTF算法的不公平性,但仍有利于中间磁道的请求。单向扫描(CSCAN)算法是对SCAN算法的改进,它总是按同一方向移动磁头,当处理完最高编号柱面请求后,不是掉转方向,而是把磁头移动最低编号的柱面请求处,然后按同一方向继续向上移动。这种算法彻底消除了对两端磁道请求的不公平。

23. 磁盘空间的地址是由组_____、_____、_____成的。

A. 柱面号(或磁道号)　　B. 块号　　C. 盘面号(或磁头号)　　D. 扇区号　　E. 索引

号 F.页号 G.段号

答:A,C,D

分析:磁盘空间的地址由:柱面号(或磁道号)、盘面号(或磁头号)、扇区号组成。块号可细分为:逻辑块号和物理块号是文件组织与结构中的术语,页号、段号是虚存管理中的术语。

24.磁盘存储器由_____、_____、_____组成的。

A 磁盘 B.通道 C.驱动器结构 D.控制器 E 缓冲区

答:A,C,D

分析:略

25.外部设备按照使用方式可分为_____、_____、_____。

A.系统设备 B.用户设备 C.输入设备 D.输出设备 E.输入输出设备 F.块设备 G.字符设备**答**:C,D,E

分析:设备的通常可按以下几种方法来分类:

按设备的从属关系可分为:系统设备和用户设备。

按使用方式可分为:输入设备、输出设备和输入输出设备。

按信息交换单位可分为:块设备和字符设备。

按资源分配角度可分为:独占设备、共享设备和虚拟设备。

按操作特性可分为:存储设备和 I/O 设备。

26.通道按传送数据的方式分为_____、_____、_____类型。

A.字节多路通道 B.数据选择通道 C.数组多路通道 D.DMA 控制器 E 寄存器

答:A,B,C

分析:通道按传送数据的方式分为:

字节多路通道:连接若干个低速设备。

数据选择通道:为一个高速设备服务。

数组多路通道:同时可为多个高速设备服务。

27.具有通道的 I/O 系统硬件结构通常不包括_____。

A.CPU B.通道 C.控制器 D.设备 E.中断

答:E

分析:具有通道装置的计算机系统,一般实施四级连接、三级控制的方式。其 I/O 系统硬件结构包括:CPU、通道、控制器和设备。

28.从资源分配角度来看,外部设备可分为_____、_____、_____。(陕西省 1998 年自考题)

A 独占设备 B 逻辑设备 C 物理设备 D 共享设备 E 虚拟设备 F 系统设备 G 用户设备

答:A,D,E

分析:设备的通常可按以下几种方法来分类:

按设备的从属关系可分为:系统设备和用户设备。

按使用方式可分为:输入设备、输出设备和输入输出设备。

按信息交换单位可分为:块设备和字符设备。

按资源分配角度可分为:独占设备、共享设备和虚拟设备。

按操作特性可分为:存储设备和 I/O 设备。

29. 下列算法中，用于磁盘驱动调度的算法是_____、_____、_____。（陕西省1996年自考题）

A. 先来先服务算法　B. 扫描算法　C. 优先级调度算法　D. 电梯调度算法

E. 响应比高者优先算法　F. 轮转法　G. 最优算法

答：A, B, D

分析：C、E属于作业调度算法 F常用于进程调度算法 G是虚存管理中的置换策略。

30. 能影响中断响应次序的技术是_____。（西安电子科技大学2002年研究生试题）

A. 时间片　B 中断　C 特权指令　D. 中断屏蔽

答：D

分析：通常通过中断优先级、中断屏蔽都可以影响中断响应次序，一般情况下中断的优先级是由硬件规定的。

31. 有关设备管理概念的下列叙述中，_____是不正确的。（西安电子科技大学2002年研究生试题）

A. 通道是处理输入，输出的软件

B. 所有外围设备的启动工作都由系统统一来做。

C. 来自通道的I/O中断事件由设备管理负责处理

D. 编制好的通道程序是存放在主存储器中的

答：A

分析：通道不是软件，而是硬件

32. 不属于主机和I/O设备的控制方式是_____。

A 循环测试方式　B. DMA 方式　C 中断方式　D 通道方式　E 控制器方式

答：E

分析：外部设备（I/O设备）和主机（内存/CPU）之间传递信息的控制方式有：循环测试方式（程序直接控制方式）、DMA方式、中断方式和通道方式

33. 通道管理没有涉及到的数据结构有_____。

A 设备控制块　B 控制器控制块　C 通道控制块　D 系统设备表　E 内存分配表

答：E

分析：为了实现对I/O设备的管理和控制、需要对每台设备、通道及控制器的情况进行登记。设备分配依据的主要数据结构如下：

系统设备表：记录系统中全部设备的情况，每个设备占用一个表目，包括设备类型、标识符、控制表、驱动程序入口等、设备控制表：系统为每个设备配置一张设备控制表，用于记录本设备的情况，如设备类型、标识号、状态、队列等。

控制器控制表：系统为每个控制器设置一张用于记录本控制器情况的控制器控制表，它反映控制器的使用状态及于通道的连接情况等。

通道控制表：用来记录通道的特性、状态、以及其他的管理信息。

34. 访问磁盘的时间由_____、_____、_____组成。

A. 寻道时间　B. 旋转等待时间　C. 读写时间　D. 估计运行时间　E. CPU 调度时间

答：C

分析：访问磁盘的时间由三部分组成，它们是寻道时间、旋转等待时间和读写时间（数据传

输时间)。其中寻道时间是决定因素。

35. 下面关于 SPOOLing 系统的说法中,正确的说法是_____。

A. 构成 SPOOLing 系统的基本条件是有外围输入机与外围输出机

B. 构成 SPOOLing 系统的基本条件是要有大容量,高速度的硬盘作为输入井和输出井

C. 当输出设备忙时,SPOOLing 系统中的用户程序暂停执行,待 I/O 空闲时在被唤醒执行输出操作

D. SPOOLing 系统中的用户程序可以随时将输出数据送到输出井中,待输出设备空闲时再由 SPOOLing 系统完成数据的输出操作

答:D

分析:构成 SPOOLing 系统的基本条件是要有大容量,高速度的外存作为输入井和输出井,因此 A、B 不对,同时利用 SPOOLing 技术,提高了系统和 I/O 设备的利用率用,进程不必等待 I/O 操作的完成,因此 C 也不对。

二、填空题

1. 缓冲区由_____和_____组成。(清华大学 1999 年研究生试题)

答:缓冲首部,缓冲体

2. 设备驱动程序是一种低级的系统例程,它通常分为_____和_____两个部分。(中科技大学 1998 年研究生试题)

答:驱动 I/O 设备工作程序,设备中断处理程序

3. 进行设备分配时所需的数据表格主要有_____,_____,_____和_____等。(中科院计算所 1999 年研究生试题)

答:设备控制表(DCT),控制器控制表(COCT),通道控制表(CHCT),系统设备表(SDT)

分析:为了实现对 I/O 设备的管理和控制、需要对每台设备、通道及控制器的情况进行登记。设备分配依据的主要数据结构如下:

系统设备表:记录系统中全部设备的情况,每个设备占用一个表目,包括设备类型、标识符、控制表、驱动程序入口等、设备控制表:系统为每个设备配置一张设备控制表,用于记录本设备的情况,如设备类型、标识号、状态、队列等。

控制器控制表:系统为每个控制器设置一张用于记录本控制器情况的控制器控制表,它反映控制器的使用状态及于通道的连接情况等。

通道控制表:用来记录通道的特性、状态、以及其他的管理信息。

4. 在进行分配设备的同时,还应分配相应的_____和_____,以保证在 I/O 设备和 CPU 之间有传输信息的通路。(陕西省 1998 年自考题)

答:控制器,通道

分析:具有通道装置的计算机系统,一般实施四级连接、三级控制的方式。因此,进行分配设备的同时,还应分配相应的控制器和通道。

5. 用通道命令编写的程序称为_____(陕西省 1996 年自考题)

答:通道程序

6. 在通道进行 I/O 操作期间,要访问两个内存固定的单元_____和_____(西安理工大学 2000 年研究生试题)

答:通道地址字(CAW),通道状态字(CSW)

分析:输入输出操作使用两固定的存储单元:通道地址字(CAW)和通道状态字(CSW),

CAW 是存放在主存固定单元的控制字，CSW 是通道向操作系统报告情况的汇集。

7. 为了记录设备的分配情况，操作系统应设置一张(系统设备表(SDT))和 3 个控制块：设备控制块，_____，_____。(西安电子科技大学 2000 年研究生试题)

答：控制器控制块，通道控制块

8. 磁盘的驱动调度先进行_____调度，再进行_____调度。(西安电子科技大学 2001 年研究生试题)

答：移臂或查找时间优化，旋转或等待时间优化

9. 为实现 CPU 与外部设备的并行工作，系统引入了_____硬件机制。(北京大学 1997 年研究生试题)

答：通道和中断

10. 盘地址空间是_____维的。

答：三

分析：磁盘空间的地址由：柱面号(或磁道号)、盘面号(或磁头号)、扇区号组成，因此是三维的。

11. 通道技术的引入，实现了_____与_____的并行；_____与_____的并行；_____与_____的并行。(中科院计算所 1998 年研究生试题)

答：CPU，设备，设备，设备，进程，进程

12. 逻辑设备表(LUT)的主要功能是_____和_____。(中国科技大学 1998 年研究生试题)

答：实现设备独立性，实现设备的灵活性

13. 设备管理中引入缓冲机制的主要原因是为了_____，_____和_____。(中科院软件所 1999 年研究生试题)

答：改善 CPU 和 I/O 之间速度不匹配的矛盾，实现 I/O 设备之间的并行性，有效减少 I/O 次数

14. 通常磁盘调度采用的算法至少有三种，它们是_____，_____，_____。(北京理工大学 1997 年研究生试题大学)

答：先来先服务(FCFS)，最短查找时间优先(SSTF)，电梯调度(SCAN)

15. 系统必须提供一种调度策略，使得能按最佳次序处理若干进程对磁盘提出的输入输出请求，这种调度称为_____。(陕西省 1998 年自考题)

答：磁盘驱动调度

16. SPOOLing 系统中，作业执行时从磁盘上的_____中读取信息，并把作业的执行结果暂时存放在磁盘上的_____中。(西安电子科技大学 2002 年研究生试题)

答：输入井，输出井

17. 中断优先级是由硬件规定的，若要调整中断的响应次序可通过_____。(北京大学 1997 年研究生试题)

答：中断屏蔽实现

18. 设备分配程序分配外部设备时，先分配_____，再分配_____，最后分配_____。(北京大学 1997 年研究生试题)

答：设备，控制器，通道

19. 磁盘输入输出，_____是磁头在移动臂带动下移动到指定柱面所花的时间，_____是

指定扇区旋转到磁头下所需的时间;它们与信息在_____有关。

答:寻道时间,旋转等待时间,磁盘上的位置

20. 实现虚拟设备必须要有一定的硬件和软件条件为基础,特别是硬件必须配置大容量的_____,要有中断装置和_____,具有_____。

答:磁盘,通道或 DMA,中央处理机与通道或 DMA 并行工作的能力

21. 虚拟设备是指采用某种 I/O 技术,将某个_____设备改进为多个用户可共享的设备。

答:独占

22. 采用 SPOOL 技术的计算机系统中,操作员只要启动_____程序工作,就可以把作业存放到_____中等待处理。

答:预输入,输入井

三、判断题

1. 计算进程需要数据时向设备发请求信号,设备数据准备完成时向计算进程或系统发出中断信号。这可以看作是设备和计算进程或系统之间的进程通信。(清华大学 1995 年研究生试题)

答:错,中断是由硬件设备发出的而不是由系统进程发出的,因此不能看作是进程间的通信。

2. 某一程序被中断后,转去执行中断处理程序,在中断处理程序结束后,一定返回到被中断的程序。(陕西省 1997 年自考题)

答:正确。也有例外,如果更改了中断返回地址,则可以跳转到更改后的目标地址。操作系统经常利用此特性。

3. CPU 和通道之间的关系是主从关系,CPU 是主设备,通道是从设备。(西安理工大学 2000 年研究生试题)

答:正确。

4. 磁盘上同一柱面上存储的信息是连续的。

答:正确。

5. SPOOLing 系统是外围设备同时脱机操作的意思。

答:错,SPOOLing 系统是外围设备同时联机操作。

6. 在具有通道结构的计算机系统中,一个通道只能连接一个控制器,一个控制器也只能连接一个设备。

答:错,是多对多的关系,一个控制器可以连多个通道,反之亦然。

7. 大多数低速设备都属于共享设备。

答:错,打印机等大多数低速设备都属于独占设备。

8. 虚拟设备技术是在一类物理设备上模拟另一类物理设备的技术,它可以将独占设备改造成为共享设备。(西安理工大学 2000 年研究生试题)

答:正确。

9. 只有引入通道后,CPU 计算与 I/O 操作才能并行执行。(电子科技大学 1996 年研究生试题)

答:错,引入通道只是进一步将 CPU 从 I/O 操作中解脱出来,而 CPU 计算与 I/O 操作的并行执行引入中断技术就能实现。

10. 移臂调度的目标是使磁盘旋转的周数最小。(陕西省 1997 年自考题)

答:错,其目标是使磁盘移臂的时间优化。

11. 通道又称 I/O 处理机，它实现主存和外设之间的信息传输，并与 CPU 并行工作。（陕西省 1997 年自考题）

答：正确。

12. 最短查找时间优先算法(SSTF)的调度原则，就是要求磁头的移动距离最小，该算法有产生"饿死"的可能。（西安理工大学 2001 年研究生试题）

答：正确。

13. I/O 交通管理程序的主要功能是管理主存，控制器和通道。（西安电子科技大学 2000 年研究生试题）

答：错，I/O 交通管理程序的功能不包括主存。

14. 缓冲技术是借用外存储器的一部分区域作为缓冲池。（西安交通大学 1999 年研究生试题）

答：错，设备管理的缓冲技术是借用内存储器的一部分区域作为缓冲池而不是外存储器的。当然，在网络系统等应用中，缓冲技术可能使用外存的一部分区域。

15. 固定头磁盘存储器的存取时间包括搜查定位和旋转延迟时间。（北京大学 1996 年研究生试题）

答：错，固定头磁盘存储器的存取时间不包括搜查定位。

16. 磁盘扇区的编号必须是连续的。

答：错，磁盘扇区的编号通常是交错的。

17. 引入缓冲的主要目的是提高 I/O 设备的利用率。

答：错，主要目的是解决 CPU 和 I/O 设备之间的速度不匹配问题。

18. 等待设备的进程队列有时不必以 FCFS 顺序排列。（电子科技大学 1996 年研究生试题）

答：正确。

四、问答题

1. 在一个 32 位 100MHz 的单总线计算机系统中(每 10ns 一个周期)，磁盘控制器使用 DMA 以每秒 40Mb 的速率从存储器中读出数据或者向存储器写入数据。假设计算机在没有被周期挪用的情况下，在每个循环周期中读取并执行一个 32 位的指令。这样做，磁盘控制器使指令的执行速度降低了多少？

答：在 32 位总线的系统中，磁盘控制器使用 DMA 传输数据的速率为每秒 40Mb，即每 100ns 传输 4 个字节(32 位)的数据。控制器每读取 10 个指令就挪用一次周期，因此，磁盘控制器使指令的执行速度降低了 10%。

2. 某个使用存储器映射 I/O 的计算机系统中，地址空间是 16 位。带有两个最高有效位的地址指向设备。在这样的系统中可以访问的最大存储器空间是多少？

答：从 20 到 214－1，即可以访问的 214 个存储单元。

3. 在一个虚拟存储器系统中，试给出一个功能，使这个功能可以提供给存储器映射系统的用户，而端口 I/O 系统则不能提供。

答：在存储器映射系统中，任何访问存储器的指令都用来访问 I/O 模块。当操作系统在用户进程地址空间中包含设备寄存器的地址时，进程可以直接访问 I/O；在端口 I/O 系统中，执行 I/O 操作的指令为操作系统执行的特权指令，只能由操作系统执行。

4. 在如下几种类型的系统中，采用忙等待 I/O 是否适合？

A. 专门用来控制单 I/O 设备的系统。

B. 运行一个单任务操作系统的个人计算机。

C.作为一个负载很大的网络服务器的工作站。

答:采用忙等待 I/O 方式,当 CPU 等待 I/O 操作完成时,进程不能继续执行。对于 A,B 这两种系统而言,执行 I/O 操作时,系统不需要处理其他的事务,因此忙等待 I/O 是合适的。而对于网络服务器而言,它需要处理网页的并发请求,它需要 CPU 有并行处理的能力,忙等待 I/O 不适合这种系统。

5. 一个输入设备以每秒 1000 字节的速度在它的 5120 字节的内部缓冲区中积累数据。当读数据时,操作系统使用轮询 I/O,在返回正常的处理之前将所有的缓冲区中的数据清空。如果目标是在缓冲区满之前就清空缓冲区,那么轮询的时间间隔应该是多少(假设清空缓冲区数据的轮询例程执行不需要时间)?

答:输入设备 5 秒钟即可把缓冲区装满,若在缓冲区满之前就清空缓冲区,则轮询的时间间隔为 5 秒。

6. 一个串行线能以最大每秒 50 000 字节的速度接收输入。数据平均输入速率是每秒 20 000 字节。如果用轮询来处理输入,不管是否有输入数据,轮询例程都需要 3μs 来执行。在下一个字节到达之前未从控制器中取走的字节将丢失。那么最大的安全的轮询时间间隔是多少?

答:串行线接收数据的最大速度为每秒 50000 字节,即每 20μs 1 个字节,而轮询例程需 3μs 来执行,因此,最大的安全的轮询时间间隔是 17μs。

7. 如果采用的系统和上题中所叙述的相同,假设对输入的处理改为中断 I/O,而且中断例程的执行需要 3μs,那么处理输入时,用中断的方法减少了多少处理时间?

答:采用中断 I/O 的方式时,每 50μs 发生一次中断,中断例程的执行时间为 3μs,轮询方式中每 20μs 轮询 3μs,中断 I/O 方式是轮询 I/O 方式开销的 40%。

8. 某磁盘文件的记录大小固定为 32 字节。物理 I/O 是以存储在一个操作系统缓冲区中的 512 字节大小的块为单位的。如果一个进程顺序地读一个文件的记录,那么多大比例的读请求会导致 I/O 操作?

答:一块包含 512/32 = 16 条记录。因此,读 16 条记录即产生一次 I/O 操作,即 1/16 = 6.25%

9. 阻塞 I/O 方式将阻止进程的继续执行,直到 I/O 完成为止。在非阻塞 I/O 的系统中,进程如何实现这种功能?这种实现有什么缺点?

答:在非阻塞 I/O 系统中,进程通过一个忙等待循环阻塞自己,直到 I/O 完成为止。这样做会降低 CPU 的利用率。

10. 某系统有连接到一个磁盘控制器上的两个磁盘驱动器,连接到一个并行端口上的一台行打印机和连接到由一个控制器控制的 10 个串行端口上的 10 个终端。磁盘控制器可以在一个磁盘正在处理一个命令时向另一个磁盘发出一个命令(例如,两个磁盘可以同时寻找),但是一次只能有一个磁盘在传输 I/O。串行控制器可以处理从所有它所连接的设备中同时传来的数据。所有的控制器都使用 DMA。设备的传输速率是:磁盘设备为 1000kb/s,打印机为 1kb/S,终端为 5kb/s。那么这个系统的最大 I/O 传输速率是多少?

答:所有的终端 I/O 都可以被控制器同时处理,一次只能有一个磁盘可以传输数据,所以最大 I/O 传输速率是 1000 + 1 + 5 * 10 = 1051kb/s。

11. 下面的任务在什么地方实现?在逻辑 I/O 层还是设备驱动程序层,还是两者都有?

(a)校验访问权限。

(b)调度 I/O 操作。

(c)检验请求信息是否在高速缓存中。

(d)处理一个设备中断。

答:(a)、(c)在逻辑 I/O 层实现。

(b)在设备驱动程序层实现。

(d)在逻辑 I/O 层或在设备驱动程序层实现。

12. 在使用磁盘高速缓存的系统中,平均访问时间取决于平均高速缓存的访问时间,平均磁盘访问时间和命中率。对于下面的几种情况,平均访问时间各是多少?

(a)高速缓存:1ms;磁盘:100ms;命中率:25%

(b)高速缓存:1ms;磁盘:100ms 命中率:50%

答:(a)1 * 0.25 + 100 * 0.75 = 75.25

(b)1 * 0.5 + 100 * 0.5 = 50.5

13. 在一个典型 PC 机上的设备中,需要延迟写高速缓存吗?对于一个设备来说,立即写高速缓冲机制是一个比较好的选择吗?

答:在一个 PC 机中,是否采用延迟写高速缓存应视不同的设备而定。例如,使用软盘时,由于用户可能随时会取走软盘,采用延迟写高速缓存不一定是一种好的方法。而对于硬盘而言,立即写高速缓冲机制使得在机器不正常关机时,丢失数据的可能性变小。

14. 在某个使用磁盘高速缓存的系统中,平均访问时间是 41.2ms,高速缓存的平均访问时间是 2ms,磁盘平均访问时间是 100ms,并且系统有 8Mb 的高速缓存内存。高速缓存的数量增大一倍时,非命中率将减半。那么应该增加多少高速缓存可以将平均访问时间减少到 20ms?假设高速缓存的数量只能以 2 倍的速度增长(8Mb,16Mb,32Mb,64Mb 等)。

答:设非命中率为 x,则 100 * x + 2 * (1 - x) = 41.2,得 x = 0.4

设高速缓存的数量增大 y 倍,则非命中率为 0.4/y,平均访问时间减少到 20ms,

有 100 * 0.4/y + 2 * (1 - 0.4/y) <= 20,得 y > 2.18,又高速缓存的数量只能以 2 倍的速度增长,所以应该增加 4 倍即 32Mb 高速缓存,可以将平均访问时间减少到 20ms。

15. 如果视频存储器的地址从 A000000(十六进制)开始。假设每象素需 16 位显示颜色,分辨率是 1024×768,那么视频存储器的末地址是多少?

答:1024 * 768 * 2 = 1572864(十进制) = 180000(十六进制),视频存储器的末地址是 A000000 + 180000 - 1 = A180000(十六进制) - 1 = A17FFFF。16. 在位映射显示器上显示视频的一个应用程序每秒刷新屏幕 50 次。如果操作系统为应用程序提供访问视频存储器的权限,应用程序就可以将屏幕上的图像直接存入视频存储器中。如果操作系统不提供访问存储器的权限,应用程序在它自己的缓冲区中生成屏幕图像,然后进行一个系统调用,将缓冲区的地址传给操作系统,然后操作系统必须将应用程序缓冲区中的图像拷贝到视频存储器中。屏幕分辨率是 800 X 600,每个像素用 8 位表示。所有的 CPU 指令的执行时间为 50ns。假设没有存储器—存储器的指令。但可以用 LOAD 和 STORE 指令在存储器和内部寄存器之间传送 8 字节的数据。如果应用程序不可以直接访问视频存储器,那么浪费了多大比例的 CPU 时间?

答:800 * 600 * 1 = 480000 字节,又每秒刷新 50 次,操作系统必须每秒传送 24 000000Byte,若每个指令传送 8Byte,则每秒须执行 3000000 个指令。而 CPU 指令的执行时间为 50ns,即每秒能执行 20000000 个指令,因此 CPU 浪费的时间比例为 15%。

17. 在操作系统中建立行编辑能力而不是由应用程序实现此功能有什么主要的好处?

答:在操作系统中建立行编辑能力可保持数据的一致性。使应用程序有相同的接口。

18. 可写的 CD-ROM 介质有 650Mb 和 700Mb 两种。除了价格外,700Mb 型号有什么主要的不足之处?

答:700Mb 的必须用高密度来存储,采用高密度存储方式没留空间处理错误。

19. 磁盘的几何状态如果超过了如下的最大限制,早期的 DOS 系统就无法处理了:

柱面 1024

读写头 16

每个磁道的扇区 63

这些系统能用的最大磁盘是多大?

答:1024 * 16 * 63 * 512 = 528(MB)

20. 一个磁盘的每个磁道有 8 个扇区,并且转速是 600rpm。从一个 I/O 操作结束到控制器发出下一个命令之前需要 10ms。那么用下面的交错系统读所有的 8 个扇区需要多少时间?

(a)非交错

(b)单扇区交错

(c)双扇区交错

答:磁盘转速为 600rpm,即每秒钟 10 转,转 1 周需 100ms.

(a)采用非交错系统,读取所有 8 个扇区需 8 * 100ms = 800ms

(b)采用单扇区交错系统,读取所有 8 个扇区需 2 * 100ms = 200ms

(c)采用双扇区交错系统,隔二个扇区读取一个扇区,读完 8 个扇区共需 2.75 周,因此读取所有 8 个扇区需 2.75 * 100ms = 275ms

21. 一个磁盘有 19.456 个柱面,16 个读写头,并且每个磁道有 63 个扇区。磁盘以 5400rpm 的速度旋转。

(1)在相邻的磁道之间移动的时间(即寻道时间)是 2ms。假设读写头已经在磁道 0 上,则读整个磁盘需要多长的时间?

(2)如果磁盘的平均寻道时间是 10ms,那么读一个扇区的平均时间是多少?

(3)在一个请求分页的系统中,若将该磁盘被用作交换设备,而且页的大小和扇区的大小相同。读入一个换出页的平均时间和上面计算的相同。假设如果一个页必须被换出,则寻找换入页的时间将只有 1ms。那么传输这两个页的平均时间是多少?

答:每个磁头连续读完一个柱面需要一周,即 60/5400 = 1/90s,16 个磁头需要 16/90s,因此 19456 个柱面共需要 19456 * 16/90s,读完整个磁盘需要 19456 * 16/90s + (19456 - 1) * 2ms = 3498s

(2)读一个扇区的平均等待时间为旋转半周的时间,即为(60/5400)/2 = 5.55ms,传输时间为 60/(63 * 5400) = 0.18ms,因此读一个扇区的平均时间是 5.55 + 0.18 + 10 = 15.73ms。

(3)换出页时间 15.73,换入页时间 1 + 5.55 + 0.18 = 6.73,传输这两个页的平均时间为 6.73 + 15.73 = 22.46ms。

22. 在一个磁盘上,有 1000 个柱面,从 0~999,用下面的算法计算为了满足磁盘队列中的所有请求,磁盘臂必须移过的磁道的数目。假设最后服务的请求是在磁道 345 上,并且读写头正在朝磁道 0 移动。在按 FIFO 顺序排列的队列中包含了如下磁道上的请求:

123,874,692,475,105,376。

(a)FIFO
(b)SSTF
(c)SCAN
(d)LOOK
(e)C_SCAN
(f)C_LOOK

答:(a)FIFO:移动磁道的顺序为 345,123,874,692,475,105,376。磁盘臂必须移过的磁道的数目为 222 + 751 + 182 + 217 + 370 + 271 = 2013。

(b)SSTF:移动磁道的顺序为 345,376,475,692,874,123,105。磁盘臂必须移过的磁道的数目为 31 + 99 + 217 + 182 + 751 + 18 = 1298。

(c)SCAN:移动磁道的顺序为 345,123,105,0,376,475,692,874。磁盘臂必须移过的磁道的数目为 222 + 18 + 105 + 376 + 99 + 217 + 182 = 1219。

(d)LOOK:移动磁道的顺序为 345,123,105,376,475,692,874。磁盘臂必须移过的磁道的数目为 222 + 18 + 271 + 99 + 217 + 182 = 1009。

(e)C_SCAN:移动磁道的顺序为 345,123,105,0,999,874,692,475,376。磁盘臂必须移过的磁道的数目为 222 + 18 + 105 + 999 + 125 + + 182 + 217 + 99 = 1967。

(f)C_LOOK:移动磁道的顺序为 345,123,105,874,692,475,376。磁盘臂必须移过的磁道的数目为 222 + 18 + 769 + 182 + 217 + 99 = 1507。

23. 哪一种磁盘调度算法将最好地优化 RAM 磁盘的性能。

答:RAM 磁盘是从主存中生成的虚拟块设备,主存是直接访问的,没有寻道和旋转延迟这种通常在磁盘设备中存在的现象。因此,所有的磁盘调度算法对于 RAM 磁盘的性能影响是相同的。

24. SSTF 优先访问磁盘中央的磁道,在一个使用 SSTF 的系统中,这将如何影响文件系统的设计?

答:文件系统设计时应将目录等会被频繁访问的块放在离磁盘中心较近的地方。

25. 因为访问存储器与大多数其他的 CPU 操作相比速度比较慢,一个加速指令执行的方法是要求所有的指令只访问 CPU 寄存器。只有两个指令行(LOAD 和 STORE)可以访问存储器,它们向寄存器拷贝入一个字或从寄存器拷贝出一个字。在这样的一个系统中,存储器映射 I/O 和端口 I/O,哪一个可能要用到?为什么?

答:端口 I/O 可能要用到。使用存储器映射 I/O 方式的优点在于任何访问存储器的指令都可用来访问 I/O 模块。

26. 轮询对于下列哪个设备来说是一个更好的选择:一个输出设备(一个打印机)还是一个输入设备(一个鼠标)为什么?

答:轮询 I/O 方式更适于一个输出设备。输入过程中引起的延迟可能引起输入数据的丢失。而输出的延迟只是打印速度变慢。

27. 对于一些应用程序来说,如果一个用户在提示的输入时间极限内没有输入任何内容,系统应该继续运行。在一个只有阻塞 I/O 的系统中,这是很难实现的。然而,假设系统有一个 alarm 系统调用,在规定的几秒之后向进程发出信号。接到一个信号之后,任何系统调用(包括一个 I/O)请求)立刻返回并且执行用户定义的函数 alarm_handler。写一个程序段和函数 alarm_handler,使程序每 5 秒就重新发出提示符直到收到输入为止。

答:int signal;

```
do{
signal = 0;
alarm(5);
prompt();
readinput(input);
}while ( signal = = 1 );
void alarm_handler()
{
signal = 1;
}
```

28. 下面的任务在什么地方实现？在逻辑 I/O 层，还是设备驱动程序，还是两者都有？

(a)将一个逻辑块号转换成磁盘的扇区、柱面和读写头。

(b)分配一个 I/O 缓冲器。

(c)检查设备的就绪状态。

(d)将一个从输入设备中接收到的回车字符转化成为通用的换行符。

答:(a)和(c)为设备驱动程序实现。(b)和(d)为逻辑 I/O 层实现。

29. 一个系统有 16Mb 视频存储器存储 64 位的位映射图像。它所能达到的最大的分辨率是如下的哪一个；640×480，800×600，1024×768，1152×864，1280×1024，1600×1200 和 1800×1440？

答：64=8*8，即每个象素需要 8 字节，所以 16MB 可以表达 2M 个象素。分辨率 1600×1200、640×480，800×600，1024×768，1152×864，1280×1024 等均小于 2M 个象素，而 1800*1440 = 2592000 大于 2M 个象素。因此最大的分辨率为 1600×1200。

30. 假设有一个系统使用位映射的图像，分辨率是 800×600，并且每个像素用 16 位表示。如果 CPU 用一个指令就可以传送 4 字节到视频存储器中，执行一个指令用 500ns，占用的时间不超过 CPU 时间的 25%，则每秒往视频存储器中回写的最大次数是多少？

答：图像需用 800*600*2=960000B 来存储。CPU 每秒最多执行 1s / 500ns=2000000 个指令，用于传送的指令不超过 2000000 * 25% = 500000 个，即传送 1000000 个象素。因此每秒往视频存储器中回写的最大次数为 1000000/(800*600)=2。

31. 一个磁盘的每个磁道有 16 个扇区，并且转速是 1200rpm。一个 I/O 操作结束到控制器发出下一个命令之前要 5ms。利用下面的交错的系统读所有的 16 个扇区需要多少时间？

(a)非交错

(b)单扇区交错

(c)双扇区交错

答：磁盘转速为 1200rpm，即每秒钟 20 转，转 1 周需 50ms。转 1 个扇区 50/16=3.1ms，不到 5ms。

(a)采用非交错系统，读取所有 16 个扇区需 16*50ms=800ms

(b)采用单扇区交错系统，读取所有 16 个扇区需 16*50ms=800ms

(c)采用双扇区交错系统，读取所有 16 个扇区需(2+0.125*7)*50ms=143.75ms

32. 在使用阻塞I/O的单处理机环境下哪一种磁盘调度算法更好？为什么？

答：在使用阻塞I/O的单处理机环境下，队列中只有一个请求。在这种情况下，应采用FCFS算法，它比较容易实现。

33. 设CPU和输入设备I，输出设备O并行执行，且输入设备I和输出设备O的启动受CPU指令的控制。另外，输出设备O的启动还受输出缓冲是否装满输出数据的限制。只有装满输出数据，输出设备才能被启动。试描述中断处理方式下的CPU动作过程。（清华大学1996年研究生试题）。

答：在中断处理方式下，CPU向相应的输入设备I和输出设备O的设备控制器发出I/O指令后，立即返回继续执行原来的任务；输入设备I的设备控制器接收到从CPU发来的读命令后，便开始接收从I来的数据，一旦数据输入完成，控制器便向CPU发出一中断信号，由CPU检查输入过程是否出错，若无错，则向控制器发出取走数据的信号，然后将数据写入指定的内存单元；对输出而言，当CPU发出I/O指令后，同时分配给该设备一缓冲，用来存放输出数据，并在该设备控制器的控制下，将输出数据装入缓冲，当缓冲满时，控制器向CPU发出中断信号，CPU便启动该设备输出。

34. 试给出两种I/O调度算法，并说明为什么I/O调度中不能采用时间片轮转法。（西北大学2000年研究生试题）。

答：先来先服务和优先级调度算法。

由于I/O设备的数据传输速率与CPU及系统总线相比非常低，而且I/O设备的启动和切换的时间开销很大；若采用时间片轮转算法，使得时间片的大小很难确定：时间片过小，会把时间都花费在设备的启动和切换上，导致每个设备都不能正常地运行；时间片过大，则演变为FCFS，失去时间片轮转的意义。对于像打印机这样的设备，轮转算法将导致文档交叉输出。所以在I/O调度中，一般不采用时间片轮转算法。

35. 以下工作分别在4个I/O软件层的哪一层完成？（上海交通大学2000年研究生试题）

a)为一个读操作计算磁道和扇区。

b)维护一个最近使用块的缓存。

c)在设备寄存器中设置命令。

答：a. 基本文件系统(物理I/O层)

b. 基本I/O管理程序(文件组织模块)

c. I/O控制层

36. 如果磁盘的每个盘面分成9段，现有一文件有A,B,…,I共9个记录(如图5.1)，每个记录的大小与段的长度相当，若磁盘转速为6000RPM，每读出一段后需要2.5ms的处理时间，若忽略其他辅助时间，试问：

1)顺序存放这些记录时，读出该文件需多少时间？

2)用一合适的磁盘调度算法，画出各记录的存放位置，计算该文件的读出时间，并与1)比较。（上海交通大学1995年研究生试题）

答：1)每分钟6000转，则旋转1周所需的时间为10ms，旋转1个记录需10/9ms。

A	B	C	D	E	F	G	H	I
1	2	3	4	5	6	7	8	9

图5.1

由于是顺序存放(如图5.1所示)，读完A记录后需2.5ms完成对数据的处理，此时磁头已

转到序号为4的记录块上，但第二次应读B记录，则磁盘应旋转到序号为2的记录块，则旋转需化时10ms－2.5ms＝7.5ms，同理，对C，D …I的读操作也需花费额外的旋转时间，故读取9个记录应花费的时间为：10 * 9 = 90ms

2)在(1)中，由于额外的旋转时间导致了读取记录的时间较长，为了减少额外的旋转时间，可以对记录块的存放顺序作修改，考虑到每读取一个记录后需2.5ms的数据处理时间，磁盘旋转3段所需的时间是3.33ms，因此可以每间隔3段存放相应的数据块，即1存放A，5存放B，9存放C，4存放D，8存放E，3存放F，7存放G，2存放H，6存放I，如图5.2所示：

A	H	F	D	B	I	G	E	C
1	2	3	4	5	6	7	8	9

图5.2

可采用FCFS算法，这所需读取时间为：10 * 3 + 1.11 * 6 = 36.66ms。

37. 有一计算机系统利用位示图来管理磁盘文件空间。假定该磁盘组共有100个柱面，每个柱面有20个磁道，每个磁道分成8个盘块(扇区)，每个盘块1k字节，位示图如图5.3所示：

```
j \ i   0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
  0     1 1 1 1 1 1 1 1 1 1 1  1  1  1  1  1
  1     1 1 1 1 1 1 1 1 1 1 1  1  1  1  1  1
  2     1 1 0 1 1 1 1 1 1 1 1  1  1  1  1  1
  3     1 1 1 1 1 1 0 1 1 1 1  1  0  0  0  0
  4     0 0 0 0 0 0 0 0 0 0 0  0  0  0  0  0
  ……
```

图5.3

1)试给出位示图中的位置(i，j)与对应盘块所在的物理位置(柱面号，磁头号，扇区号)之间的计算公式．假定柱面号，磁头号，扇区号都从1开始编号。

2)试说明分配和回收一盘块的过程。

答：1)根据位示图的位置(i，j)，得出盘块的序号 b＝i * 16 + j；用C表示柱面号，H表示磁头号，S表示扇区号。则

C＝b DIV (20 * 8)

H＝[b MOD (20 * 8)] DIV 8

S＝[b MOD (20 * 8)) MOD 8

2)分配：顺序扫描位示图，找出1个其值为"0"的二进制位("0"表示空闲)，利用上述公式将其转换成相应的序号b，并修改位示图，令(i，j)＝ 1；

回收：将回收盘块的盘块号换算成位示图中的i和j，转换公式为：

b＝C * 20 * 8 + H * 8 + S

i＝b DIV 16，j ＝ b MOD 16；

最后将计算得出的(i，j)在位示图中置"0"。

38. 简述UNIX系统V中块设备数据缓冲池的管理技术，给出缓冲池的结构和缓冲区的分配与释放操作。。(中科院计算机技术研究所1998年硕士生入学试题)

答：盘块缓冲区由两部分构成：一部分用于存放数据，另一部分为缓冲控制块；缓冲池有2组队列构成，分别是闲散队列和散列队列，散列队列是指为了加速对缓冲区的查找，系统把所有缓冲区按设备分类，并按其块号所计算的散列值不同，组织成多个散列队列。

分配:(1)从闲散队列中取得空闲缓冲区。检查空闲队列是否为空,是,等待,否则摘下队列中第一个缓冲区,并将标志置为忙 。

(2)从散列队列中取得空闲缓冲区。无论是读还是写,核心首先检查所需的数据是否在某个缓冲区中,若在,则不再读写,否则分 2 种情况:

a)缓冲区在散列队列上,则检查是否空闲,是,摘下缓冲区并置标志为忙;否则等待。

b)缓冲区不在散列队列上,则从空闲队列中找 1 个缓冲区,并调整该缓冲区所在的散列队列。

回收:首先唤醒等待空闲缓冲区的所有进程,接着将空闲缓冲区放入空闲队列,最后,在缓冲区首部(控制块)置空闲标志,空闲块缓冲区计数加 1,恢复处理机的执行后返回。

39. 某移动臂磁盘的柱面由外向里顺序编号,假定当前磁头停在 100 号柱面且移动臂方向是向里的,现有如下表 1 所示的请求序列在等待访问磁盘:

表 5-1 访问磁盘请求序列

请求次序	1	2	3	4	5	6	7	8	9	10
柱面号	190	10	160	80	90	125	30	20	140	25

回答下面的问题:

(1) 写出分别采用"最短查找时间优先算法"和"电梯调度算法"时,实际处理上述请求的次序。

(2) 针对本题比较上述两种算法,就移动臂所花的时间(忽略移动臂改向时间)而言,哪种算法更合适?简要说明之。(南京大学 1997 年研究生考试)

答:最短查找时间优先算法的次序为:90,80,125,140,160,190,30,25,20,10

电梯调度算法次序为:125,140,160,190,90,80,30,25,20,10

采用 SSTF,磁头移动的磁道数为:310;而采用 SCAN 则为:270,因此 SCAN 算法更合适。

40. 假定一磁盘有 200 个柱面,编号为 0 - 199,当前存取臂的位置在 110 号柱面上,并且刚刚完成了 125 号柱面上的服务请求,如果存在以下的请求序列:

86,146,89,178,94,150,102,174,130

磁盘移动采用电梯算法,请列出实际处理上述请求的顺序。

答:顺序为:102,94,89,86,130,146,150,174,178

41. 试述打印共享的基本原理。

答:实现打印共享的技术是 SPOOL 技术。SPOOL 系统有输入/输出井、输入/输出缓冲区,输入/输出管理进程三部分组成。当用户请求打印输出时,SPOOL 系统同意为他输出,但并不是真正将打印机分配给用户进程,而是做两件事:(1)由输出管理进程在输出井中为之申请一空闲盘块区,并将数据送入其中;(2)再为该用户申请一张请求打印表,将用户要求填入其中,并将该表挂在请求打印队列上。如果还有进程要求打印,系统仍可接受,也同样做上述两件事。当打印机空闲,输出管理进程从请求打印队列的队首取一打印表,并将数据从输出井送到缓冲区,由打印机输出,完成后再从队列中取下一张打印表,直至打印队列为空,才将自己阻塞,直到下次再有打印请求才被唤醒。

42. 什么是 DMA 方式?它与中断方式的主要区别是什么?(哈尔滨工业大学 2000 年研究生试题)

答:DMA 是 Direct Memory Access(直接存储器访问)的缩写。DMA 方式的特点是,数据传

输的基本单位是数据块，所传输的数据是从设备直接送入内存，或者相反；仅在传送一个或多个数据块的开始和结束时才需要 CPU 的干预，整块数据的传送是在 DMA 控制器的控制下完成的，与中断驱动 I/O 方式相比，DMA 方式大大减少了中断的次数，但增加了硬件开销。

43. I/O 控制可用那几种方式实现？各有何优缺点？（清华大学 2001 年研究生试题）

答：程序直接控制方式，中断控制方式，DMA 方式和通道方式。

程序直接控制方式的优点：实现方法简单。缺点：耗费大量的 CPU 时间和无法检测发现设备或其他硬件产生的错误，而且设备和 CPU、设备和设备只能用串行工作。

中断控制方式的优点：能够检测发现设备或其他硬件产生的错误，能够并行工作。缺点：中断次数多，CPU 仍需要花较多的时间处理中断，能够并行操作的设备台数也受到中断处理时间的限制，中断次数据增多导致数据丢失。

DMA 方式的优点：减少了 CPU 的工作负担因为采用了外设和内存直接交换数据的方式。缺点：一段数据传送结束时需中断处理，增加了硬件开销。

通道方式的优点：减轻了 CPU 的工作负担。缺点：增加了硬件开销，一段数据传送结束时需中断处理，要进行通道程序的设计。

44. 假如你是某操作系统的设计者，承担慢速字符设备管理任务．该操作系统要求：用户使用慢速字符设备和使用普通文件完全一样方便简捷．请问你在设计中至少要解决哪些问题？（东南大学年 2000 年研究生）

答：由于设备管理的主要操作是分配和回收，所以必须考虑下列情况：建立相应设备的一些数据结构，如：设备控制表，控制器控制表，通道控制表，系统设备表等，为增加设备的独立性，还应建立逻辑设备表；由于每类设备都有驱动程序，每一驱动程序都包含用于执行不同操作的多个函数，为使管理程序能方便地转向各函数，因此还必须为每类设备提供一个设备开关（包括每类设备各函数的入口地址），即要建立一张设备开关表。另外，还需要考虑缓冲区的管理。

45. 比较将 I/O 功能放在设备控制器内实现与放在内核中实现的三个优缺点。

答：优点：(1)引起操作系统出错的可能性小；(2)使用专用硬件及算法的性能提高；(3)内核对算法的处理操作简单。

缺点：(1)对于新的固件或者新的硬件来说，如果系统出错就更难修复；(2)改进的算法同样要求硬件更新，而不仅仅是内核或设备驱动程序的更新；(3)内置的算法可能和应用程序使用设备发生冲突，从而导致性能下降。

46. 考虑单用户 PC 机上的下列 I/O 操作：

a. 图形用户界面下使用鼠标

b. 在多任务操作系统下的磁带驱动器（假设没有设备预分配）

c. 包含用户文件的磁盘驱动器

d. 使用存储器映射 I/O，直接和总线相连的图形卡

在操作系统中使用缓冲技术，假脱机技术，Cache 技术，或者它们的组合来实现上述操作。实现时使用轮询 I/O 还是中断 I/O？为什么？

答：a. 在鼠标移动时，如果有高优先级的操作产生，为了记录鼠标活动的情况，必须使用缓冲技术，另外，假脱机技术和 Caching 技术不是很必要，而应采用中断驱动 I/O 方式。

b. 由于磁带驱动器和目标或源 I/O 设备间的吞吐量不同，必须采用缓冲技术；为了能对储

存在磁带上的数据进行快速访问，必须采用 Caching 技术；当有多个用户需要对磁带进行读或写的时候，假脱机技术也是必须采用的；为了取得最好的性能，应该采用中断驱动 I/O 方式。

c. 为了能使数据从用户作业空间传送到磁盘或从磁盘传送到用户作业空间，必须采用缓冲技术；同样道理，也必须采用 Caching 技术；由于磁盘是属于共享设备，故没必要采用假脱机技术；最好采用中断驱动 I/O 方式。

d. 为了使于多幅图形的存取及提高性能，缓冲技术是可以采用的，特别是在显示当前一幅图形时又要取得下一幅图形时，应采用双缓冲技术；基于存储器映射及直接和总线相连的图形卡是快速和共享设备，所以没必要采用假脱机技术和 Caching 技术；轮询 I/O 和中断 I/O 只对输入和 I/O 是否完成的检测有用，而对于采用存储器映射的设备不必用到上述两种 I/O 方式。

47. 分别列出使用阻塞 I/O 和使用非阻塞 I/O 的三个示例。为什么不只采用非阻塞 I/O 方式或只采用阻塞 I/O 方式？

答：一般来说，只有在进程等待某个特殊事件时，阻塞 I/O 方式才适用，例如：一个用户程序需要从磁盘，磁带或者键盘读取数据时。当从多个设备中读取数据或者执行 I/O 操作的顺序没有预先设定，就可以采用非阻塞 I/O 方式，例如：邮件收发程序检测到多个网络插件时，管理程序对鼠标和键盘的操作都会进行处理；I/O 管理程序，诸如在设备间执行一个备份操作；通过采用缓冲技术和非阻塞 I/O 方式，程序可以使设备满负荷地工作，从而优化性能。

由于一旦有 I/O 操作出现，有时需要异步方式操作，因此对程序设计者来说，非阻塞 I/O 方式的设计就变得很复杂；另外，忙等待 I/O 方式比中断驱动 I/O 方式的效率要低，只采用阻塞 I/O 方式会导致整个系统性能的下降；因此，系统中这两种方式一般都会采用。

48. 为什么系统中通常使用中断 I/O 方式管理串口，而使用轮询 I/O 方式管理前端处理器，例如一个终端集中器？

答：当 I/O 操作很频繁并且操作时间很短时，轮询方式比中断驱动 I/O 方式的执行效率要高。由于单个串口的 I/O 操作很少发生，因此应采用中断驱动 I/O 方式；但对于像终端集中器这样的一个串口集，其在执行时会产生相当多的执行时间很短的 I/O 操作，如果采用中断驱动 I/O 方式的话，会加重系统的负荷以及浪费许多系统的资源，而采用轮询方式则可以有效避免上述问题。

49. 轮询 I/O 方式会因为 CPU 循环等待 I/O 操作的完成而浪费大量 CPU 时间，但若 I/O 操作时设备已经就绪，则轮询方式的效率要高于中断处理方式。请设计一种综合轮询方式和中断方式的混合 I/O 策略。并分别对纯轮询方式，纯中断方式和混合方式，指出其优于其他两种方式的情形。

答：略

50. UNIX 利用共享内核中的数据结构来协同 I/O 组件的活动，而在 Windows NT 下内核 I/O 组件之间使用面向对象来传递消息。讨论这两种方法的利弊。

答：UNIX 方式的好处：效率高，代价低，数据传送量小；由于不需要其他的内核组件协同操作，因而执行快；总而言之，数据传送出错的可能性小。

不利之处：没有数据保护机制以及数据交换可能产生的副作用导致很难调试；由于需要新的数据结构而不是新的对象，因而很难执行新的 I/O 方式。

51. DMA 如何增强系统的并行性？为什么它会使得硬件的设计复杂化？

答：DMA 系统在通过系统总线和内存传输数据时，允许 CPU 执行其他任务，因而提高了系统的并行性。由于 DMA 控制器必须与系统结合并且 DMA 控制器必须成为系统的总线主控，因此硬件的设计就变得复杂；另外由于 CPU 和 DMA 控制器共享内存总线，必定产生周期挪用。

52. 当 CPU 速度提高时，为什么需要增加系统总线和相关设备速度？

答：假定一个系统中 I/O 操作和计算操作各占一半；如果将 CPU 的性能提高 1 倍，但整个系统的性能只提高了 50%；只有将系统中的所有组件的性能都提高 1 倍，系统的总性能才能提高 1 倍。因此只有解决系统的瓶颈，才能提高整个系统的性能，而不是盲目地提升单个组件的性能。

53. (OSC Ch14)除 FCFS 外的所有磁盘调度算法都不是真正公平的(例如，会出现“饥饿”现象)。

1)解释为什么这个论断是正确的。

2)提出一个类似于 SCAN 的确保公平的改进算法。

3)解释为什么在分时系统中公平性是一个重要指标？

4)给出三个以上在 I/O 请求服务中操作系统不公平情况的例子。

答：a. 因为理论上来说，一旦有新的访问磁道请求产生，磁头马上能从当前所在的磁道移动到新的需要被访问的磁道，或者说新的请求能被快速响应。

b. 所有比预定设置等待时间长的请求可以移动到等待队列的前部；例如：对于 SSTF 算法，可以对剩余队列中的请求时间先后重新排列。

c. 为了防止有非常长的响应时间的出现。

d. 分页系统和内存交换是根据用户请求的优先级而进行的；像对文件系统数据元进行写操作这样的内核级 I/O 操作，就要比用户级 I/O 操作具有较高的优先级；在实时系统中，进程是采用优先级调度算法的，具有高优先级的进程的 I/O 操作相应地得到优先调度的权力。

54. 假设一个磁盘驱动器有 5000 个柱面，从 0 到 4999，当前处理的请求在磁道 143 上，上一个完成的请求在磁道 125 上，按 FIFO 顺序排列的未处理的请求队列如下：86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130。为了满足所有的磁盘队列中的请求，从当前位置开始，对下列各种磁盘调度算法计算磁盘臂必须移动的磁道数目。

a. fcfs　b. sstf　c. scan　d. look　e. _ scan　f. c_look

答：a. FCFS：143, 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130. 移动的磁道数目为：7081

b. SSTF：143, 130, 86, 913, 948, 1022, 1470, 1509, 1750, 1774. 移动的磁道数目为：1745

c. SCAN：143, 913, 948, 1022, 1470, 1509, 1750, 1774, 4999, 130, 86. 移动的磁道数目为：9769

d. LOOK：143, 913, 948, 1022, 1470, 1509, 1750, 1774, 130, 86. 移动的磁道数目为：3319

e. C－SCAN：143, 913, 948, 1022, 1470, 1509, 1750, 1774, 4999, 0, 86, 130. 移动的磁道数目为：9985

f. C-LOOK :143, 913, 948, 1022, 1470, 1509, 1750, 1774, 86, 130. 移动的磁道数目为:3363

55. 物理学指出,当物体以常量 a 加速时,距离 d 和时间 t 之间的关系为 $d=at^2/2$。假设,在寻道过程中,上题的磁盘在前半寻道时间以一个常速加速磁盘臂,在后半寻道时间以相同的加速度减速磁盘臂,假设磁盘在相邻的柱面寻道时间为 1ms,从 0 柱面到 4999 柱面的寻道时间为 18ms,

A. 寻道距离是磁头在柱面上移动磁道的数目,解释为什么寻道时间和寻道距离的平方根成比例?

B. 写出寻道时间和寻道距离间的计算函数,公式的形式为 t = x + y,t 的单位是 ms,L 是柱面的寻道距离。

C. 计算练习 56 中每种调度算法的总寻道时间。确定哪个调度算法是最快的(有最小的总寻道时间)。

D. 加速比是指节省时间除以原始时间的比例,相对 FCFS 而言,最快的调度算法加速比是多少?

答:a. 由 $d=1/2at^2$ 可知:$t=\sqrt{\frac{2d}{a}}$

b. 将 t=1,L=1 和 t=18,L=4999 代入公式:$t=x+y\sqrt{L}$即可求得 x,y 的值,

$$t=0.7561+0.2439\sqrt{L}$$

c. 各算法的寻道时间分别是:FCFS 65.20; SSTF 31.52; SCAN 62.02; LOOK 40.29; C-SCAN62.10; C-LOOK 40.42;因此 SSTF 算法最快。

d. (65.20 — 31.52)/65.20 = 0.52,则相对 FCFS 而言,SSTF 的加速比是 52%;如果考虑到旋转延时和数据传输的时间,加速比还会低一些。

56. 假设练习 57 中的磁盘转速是 7200RPM

A. 磁盘驱动器的平均旋转延迟是多少?

B. 在 A 中所给出的时间内,能覆盖多少寻道距离?

答:a. 7200 rpm 可得出 120 转/秒,则旋转一周所需时间是 8.33ms,所以平均旋转延时(即旋转半周所需时间)是:4.167ms.

b. 根据公式:$t=0.7561+0.2439\sqrt{L}$将 t=4.167 代入可求得 L=195.58,即能寻道 195,约 4%的磁盘。

57. 练习 57 中描述的是典型的硬盘驱动器加速寻道过程,而软盘(以及 80 年代中期前制造的许多硬盘)通常以一种固定的速率寻道。假设练习 57 中是常速度寻道而不是常量加速度寻道,则寻道时间公式为 t=x+yL,t 是时间,单位是 ms;L 是寻道长度,假设相邻柱面的寻道时间是 1ms,对于每个附加的柱面时间为 0.5ms

A. 写出寻道时间和寻道距离间的计算函数。

B. 使用 A 中的寻道时间函数,为练习 56 中的每个调度算法,计算总寻道时间。它们和练习 57 中 C 的答案一样吗?为什么?

C. 在这种情况下,相对 FCFS 而言,最快的调度算法加速比是多少?

答:

a. t=0.95+0.05L

b. FCFS 362.60; SSTF 95.80; SCAN 497.95; LOOK 174.50; C-SCAN 500.15; C-LOOK

176.70.

SSTF 最快

c.(362.60_95.80)/362.60 = 0.74，则相对 FCFS 而言，SSTF 的加速比是 74%；如果考虑到旋转延时和数据传输的时间，加速比还会低一些。

58. 对于服从均匀分布的请求，请从平均响应时间(请求到达至请求完成经历的时间)，响应时间的方差，和有效带宽的角度考虑，比较 C-SCAN 和 SCAN 算法的性能。寻道时间和旋转延迟的相对大小对性能有何影响？

答：对于比较 SCAN 和 C-SCAN 的性能，最好能做个小模拟试验，这样答案就能很容易得出。一般情况下，C-SCAN 的相应时间比 SCAN 要长一些，但对于一次访问包含较多个数据块的寻道操作，C-SCAN 相应时间的影响就较小；可以直观地观察到 SCAN 有利于对磁盘中间部位柱面的请求，而 C-SCAN 是循环寻道，即从磁盘的一端移动到磁盘需要请求的另一端，移动期间是不进行寻道服务的，虽然与一次中等距离的寻道相比，从一端移到另一端所需时间并不是很长，但这会导致性能的略微下降。

由于 SCAN 和 C-SCAN 并没有考虑旋转延时，因此，相对于旋转延时，随着寻道时间的降低，这两种算法的性能差异也会变小。

59. 在单用户环境下，除了 FCFS 调度外，其他的磁盘调度算法还有用吗？为什么？

答：在单用户环境下，I/O 请求队列通常是空的；一般来说，单用户进程总是请求 1 个记录块或是一连串连续的记录块，因此 FCFS 调度算法是最经济的。但当有多个进程在并发执行 I/O 操作时，例如当浏览器在后台处理数据时，同时在前台操作系统在进行分页处理并且有另外的用户程序也正在工作，此时 LOOK 算法具有非常好的性能，而且该算法也较容易设计编写。

60. 解释为什么 SSTF 调度算法有利于磁盘中间部位柱面的使用。

答：由于磁盘中部到磁盘其他磁道的距离最短，因此根据 SSTF，磁头就会从磁盘的边缘磁道往中间移动。

61. 通常请求不是均匀分布的，例如，包含文件 FAT 或 INODE 的柱面比仅包含文件的柱面访问要频繁。假定 50%的请求是访问一些小的，固定的柱面，则 A. 有没有一种磁盘调度算法特别适用于这种情况？

B. 通过利用这种访问“热点”，给出一种性能较好的磁盘调度算法。

C. 文件系统通过间接表寻找文件数据块，如 DOS 中 FAT 表和 UNIX 中的 INODE，针对这种间接法，请给出一种或几种提高磁盘性能的方案。

答：a. 采用 SSTF 算法可取得最好的性能效果。假如请求的柱面分布很分散，采用 FCFS 算法的话，可能会使磁头产生不必要的移动。

b. 可把需要访问的“热点”置于磁盘的中部，采用 SSTF 算法，但为了避免“饥饿现象”必须对算法进行修改，在算法中增加这样一条措施：当磁盘没有请求的时间超过一定量，比如 50ms，操作系统就对“热点”进行寻道操作；因为下一次的请求很有可能出现在“热点”。

c. 可把这些数据元放在主存，或者把文件的数据放置在性能上最接近于主存的磁盘区域。

62. 为什么在磁盘调度算法中通常不考虑旋转延迟时间？若考虑延迟时间优化，如何改进 SSTF，SCAN 和 C-SCAN 算法？

答：多数磁盘都不将旋转定位的信息传输给主机系统。即使调度程序接收到此类信息，该信息相关的时间参数也是不精确的，并且调度程序用来处理的时间是变化的，因此最终导

致旋转定位信息的不准确。另外，用户的磁盘请求操作一般是以请求逻辑数据块的方式给出，因此逻辑块与数据所在的物理地址间必须进行映射，用户才能真正访问该数据块，而这种操作是相当复杂的。所以，在磁盘调度算法中通常不考虑旋转延迟时间。

63. 使用 RAM 磁盘对选择磁盘调度算法会有产生什么样的影响？其中要考虑哪些因素？对于一个硬盘调度算法，若把文件系统中最近使用过的块存储在主存中开辟的一个高速缓存缓冲区中，以上因素还需考虑吗？

答：磁盘调度算法是用来减少由于磁头定位产生的时间开销的。由于 RAM 磁盘有统一的存取周期，因此就不必采用任何调度算法。

另外系统只是对缓冲失配的情况进行处理，对缓冲命中的情形是不预处理的，因此像 RAM 磁盘一样，系统采用主存高速缓冲的，不必用任何磁盘调度算法来进行调度的。

64. 在多任务环境下，为什么说将文件系统的 I/O 操作均匀地分散到磁盘和磁盘控制器中去接受处理很重要？

答：系统运行的速度快慢取决于系统各组件中最慢那个组件的速度，这就是瓶颈。在现代计算机系统中，磁盘和磁盘控制器的速度大大慢于 CPU 和系统总线的速度，因此成为整个系统运行的瓶颈。将文件系统的 I/O 操作均匀地分散到磁盘和磁盘控制器中去接受处理，可以最大限度地避免瓶颈现象的发生。

65. 从文件系统中重读代码页与在对换区中存储代码页，这两种方法间可以作哪些权衡？

答：由于对换区比文件区有较高的性能，如果把代码页面储存在对换区，它们就能较快地被传输到主存。对换区的启动需要时间，像进程启动时所需的一些在对换区的页面传输就需要花费对换区的启动时间。同样，有更多的对换区需求时，比如进行运行需要较多的代码页面和数据页面，这也要花费对换区的启动时间。

66. 有没有方法可以实现真正的稳定存储？为什么？

答：真正的稳定存储可以保证不丢失数据。实现真正的稳定存储的策略是使数据具有的多个备份，这样，如果一个备份损坏的话，还有另外的备份可用。但万一所有的备份都被损坏了呢？因此没有真正意义上的稳定存储。

67. MTBF(平均无故障时间)是硬盘可靠性的一个指标。虽然这个指标被称作"时间"，但实际上 MTBF 通常是以设备的正常工作小时数度量的。

A. 如果一个系统包含 1000 个磁盘驱动器，每个驱动器的 MTBF 是 750000 小时，下面的描述中哪一个最符合该系统发生一次磁盘故障的时间：每 1000 年，每世纪，每十年，每个月，每个星期，每天，每小时，每分钟，每秒钟？

B. 统计表明，一个 20 到 21 岁的美国公民平均死亡率为千分之一，由此推论 20 岁的 MTBF 时间(单位由小时转换为年)，对于一个 20 岁的人来说，MTBF 给出期望的寿命是多大？

C. 某类磁盘驱动器，生产商保证的 MTBF 为 100 万小时 你能推算出它们的保质期是多少年吗？

答：a. 750000 / 1000 = 750(小时)约等于 31 天，每个月发生一次磁盘故障。

b. 1 年是 8760 小时，8760 小时 / 0.001 = 8760000 小时(1000 年)也就是说对于一个 20 岁的人来说，MTBF 给出期望的寿命是 1000 年，这没有任何实际意义。

c. 从上一小题可看出，MTBF 给出期望的寿命没有任何实际意义。一般来说，磁盘驱动器设计的寿命是 5 年，假如真的有一个 MTBF 为 1 百万小时的磁盘，那么在其期望的寿命内是不可能有故障的。

68. 术语快速宽带 SCSI-II(fast wide SCSI-II)表示 SCSI 总线在主机和设备间传递数据时传输率为每秒 20MBytes。假设一个快速宽带 SCSI-II 磁盘的转速为 7200RPM，每扇区 512 字节，每磁道 160 个扇区。

A. 计算该驱动器连续的传输率为每秒多少兆？

B. 假设驱动器有 7000 个柱面，每个柱面有 20 个磁道，磁头交换时间为 0.5ms，相邻磁道间的寻道时间为 2ms，根据这些信息，请计算出在连续传输大量数据时的传输速率。

C. 假设一个磁盘的平均寻道时间为 8ms，计算随机存取一个分散在磁盘上的单个扇区时的单位时间内的 I/O 次数和有效传输速率。

D. 若 I/O 数据的大小为 4KB，8KB，和 64KB，计算随机访问时单位时间内的 I/O 次数和传输速率。

E. 如果在队列中有多个请求，类似 SCAN 之类的磁盘调度算法可以减少平均寻道距离。假设随机存取 8K 的页面，平均队列长度是 10，调度算法减少平均寻道距离到 3ms，计算磁盘每秒的 I/O 次数和有效的传输率。

答：a. 磁盘每秒旋转 120 周，旋转 1 周传输的数据量是：512×160＝80 KB，该驱动器连续的传输率为每秒 9600KB，9.6M。

b. 假设大量数据是 100 个柱面的大小，则数据的大小是：100 cyl * 20 trk/cyl * 80 KB/trk，即 160,000 KB。所需传输的时间：磁头切换时间 19 trk * 100 cyl * 0.5 ms = 950 ms，柱面切换时间 99 * 2 ms = 198 ms，及旋转时间 (20 * 100) trks / 120 trks per sec = 16.667s；共计 16.667 + 0.950 + 0.198 = 17.815 (s)。这样传输率为 160,000KB / 17.815 s = 8981.2 KB/s。(这里忽略了寻道启动所需的时间和旋转延时)c. 磁盘旋转 1 周所需时间是 1 / 120＝ 8.3ms，故平均旋转延时是 4.167 ms；读/写 1 个扇区的时间是 1 / (120 转/秒×160) ＝0.052 (ms)，合计为：8 + 4.167 + 0.052 = 12.219 (ms)，也就是说对 1 个扇区进行数据操作的时间是 0.012219s，则每秒可进行 1/0.012219＝81.8 次 I/O 操作，有效传输速率为 0.5KB/扇区×81.8 扇区/s = 40.9 KB/s

d. 为了方便计算，我们忽略交叉访问磁道和柱面带来的开销。类似于 C 小题解题思路，访问 4 KB 的时间是 8 + 4.167 + 1 / (120 转/秒×20) = 0.012583s，访问 8 KB 的时间是 8 + 4.167 + 1 / (120 转/秒×10) = 0.012999s，访问 64KB 的时间是 8 + 4.167 + 1 / (120 转/秒×(160/128)) = 0.018833s。则每秒可进行相应的 I/O 操作分别是：1/(0.0126)，1/(0.013)，和 1/(0.019)，即 79.4 次，76.9 次和 52.6 次，相应的传输速率为：4KB×79.4，8KB×76.9，64KB×52.6，即 318 KB/s，615 KB/s 和 3366 KB/s。

e. 同样根据 C 小题，每秒可进行 的 I/O 操作是：1/(0.001 * 3 + 0.001 * 4.167 + 1 / (120 转/秒×(160/16)))＝ 125(次)，传输速率为：8KB×125 = 1000(KB)。

69. 可以有多个磁盘和总线相连，事实上，快速宽带 SCSI-II 至少可以和 15 个磁盘驱动器相连。若总线的带宽为每秒 20MB，在任何时候，主机和磁盘的内部高速缓存之间只有一个数据包在总线上传递。然而，当其他磁盘在总线上传递包时磁盘可以移动磁盘臂。针对 70 题中各种情况下计算出的传输速率，讨论在一条快速宽带 SCSI-II 总线上连接多少个磁盘最有效。

答：根据 70 题，对于 1 个随机访问磁盘数据大小为 8KB 的 I/O 操作，随机访问 1 块数据的时间约为 13ms，有效传输率为 615KB/s，就算 15 个磁盘全部工作，其集合数据传输率也不会超过 10MB/s，大大低于总线的带宽每秒 20MB；对于 1 个随机访问磁盘数据大小为

64KB 的 I/O 操作，其传输率约为 3.4MB/s，超过 5 个，就会超出总线的带宽的要求，因此连接 5 个最有效；对于随机存取 8K 的页面，且有 1 个很长的请求队列，采用调度算法减少平均寻道距离到 3ms，则其输率约为 1MB/s，总线带宽足够和 15 个磁盘驱动器相连。

70. 用扇区闲置(sector sparing)或扇区滑动(sector slipping)重新映射坏块会影响系统性能。假设在习题 70 中驱动器在随机的位置有 100 个坏扇区，每个坏扇区映射到位于相同柱面的不同磁道的备份上，假设一个随机访问工作量包含 8K 的读操作，队列长度是 1，估算磁盘每秒的 I/O 次数和有效的传输率。坏扇区对于性能有何影响？

答：根据 70 题，磁盘拥有 22400000 个扇区，因而一次请求碰到 100 个坏扇区中的 1 个的可能性很小很小。这里假设一个极端的例子：读取 8KB 页面的数据，其中有 1 坏扇区，要重新映射到别的磁道中也有 1 坏扇区。在这种情况下，为了保证数据正确，必须重新寻道，就需要多花费 1 次寻道时间，2 次磁头切换时间和 2 次旋转延时；但一般来说，现代控制器在把数据重新映射到磁道前，能保证从原始磁道读取的都是好的扇区，这样的话，为保证数据正确传输，只要额外花费的是 1 次磁头切换时间和 1 次旋转延时，因此传输 8KB 数据所需的时间为：8ms(寻道时间) + 4.17ms(平均旋转延时) + 0.83ms(8KB 数据传输时间) + 0.5ms(磁头切换时间) + 8.3ms(旋转延时)，合计为 21.8ms，则每秒可进行 1000/21.8 = 45.9 次 I/O 操作，有效的传输率为 367 KB/s。这对于一个有严格时间限定的应用程序来说，可能很关键，但从 100 个坏扇区和 22.4M 个好扇区的加权平均值来分析，对整个系统的影响几乎没有。

71. 讨论扇区闲置和扇区滑动的优缺点。

答：扇区闲置会引起多花费 1 次寻道时间，从而需要额外花费 1 次磁头切换时间和 1 次旋转延时。虽然扇区滑动在以后的读数据操作中影响较小，但在重新映射时，它可能会将需要进行读和写的磁道内的数据移动到坏的扇区中。

72. 操作系统通常把可移动磁盘当作共享文件系统看待，而把磁带驱动器只分配给一个应用程序。解释这种对磁盘和磁带不同处理方式的三个原因。若要以共享文件系统的形式访问磁带，操作系统需作哪些支持？应用程序对共享方式访问磁带需有哪些特性？是否可以象驻留在磁盘中的文件一样使用它们？为什么？

答：a. 磁盘有快速随机存取周期，因此如果交叉存取数据的话就有很好的性能。相反，磁带定位时间很长，如果 2 个用户共享一个磁带，磁带驱动器要花费的磁带切换和定位的时间就很长，相应的数据传输时间就会很短，这就像在虚拟存储器中由于物理内存不足，导致页面不断换进换出产生“抖动”现象一样。

b. 磁带是可移动的，即保存数据后可方便地带离计算机所在的场所，万一该场所发生意外(如发生火灾)，还可以有数据备份。

c. 磁带通常用来发送大批量的数据，因此磁带可为一些特殊数据的传输作为一个备用，一般不作为一个多用途的共享储存空间来使用。

若要以共享文件系统的形式访问磁带，操作系统需下列支持：对磁带组的文件系统命名进行管理，能对磁带空间进行分配的操作，能对磁带 I/O 操作进行调度。另外，需要对常驻磁带的文件进行存取的应用程序必须能容忍长时间的延时。为了提高性能，应用程序必须提供所需 I/O 操作的次数，这样磁带调度算法就能对这些磁带 I/O 操作进行有效的调度。

73. 当打开的文件数目大大多于磁盘机中驱动器的数目时，对磁盘机有什么影响？

答：可能会产生两个后果，第一个后果是“饥饿”，这是由于请求磁带 I/O 操作的应用程序

得不到满足而长时间在阻塞队列等待，从而导致“饥饿”；第二个后果是“抖动”，由于每次磁带 I/O 操作完成后，都要进行磁带的换进换出，从而导致“抖动”。

74. 若磁带存储和磁盘存储有相同的面密度，对价格和性能有何影响？

答：若磁带存储和磁盘存储有相同的面密度，那磁带存储比磁盘存储的价格要便宜得多；而且单个磁带的容量可以达到 1000G，按现在的技术，一个磁带就可以替换磁带机的一组磁带，从而进一步降低成本；虽然面密度与数据传输率没有直接关系，但单个超大容量的磁带可以减少由于磁带切换带来的开销。

75. 若磁带的单位存储价格和磁盘一样，磁带还需要吗？为什么？

答：磁带可方便地拆卸和移动，即保存数据后可方便地带离计算机所在的场所，万一该场所发生意外（如发生火灾），还可以有数据备份。另外磁带还可储存大批量的数据，因此磁带还是很有用处的。而硬盘只是一种不可移动的储存介质。

76. 请简单地比较完全使用磁盘的和带有一体化的三级存储的千兆级存储系统性能和价格。假设磁盘每 10G 花费 1000 美元，每秒传输 5M，平均访问延迟 15ms。磁带每 G 花费 10 美元，每秒传输 10M，平均访问延迟 20s。对一个纯磁盘系统，计算总成本，最大总数据率，及平均等待时间。假设 5% 的数据经常使用，他们必须常驻磁盘，另 95% 存放在磁带库中，假设磁盘系统处理 95% 的请求，磁带库处理 5% 的请求，则这个层次存储系统的总成本，最大的总数据率和平均等待时间分别是多少？

答：对于纯磁盘系统：由于 1T = 1024GB，为了计算方便，假设 1T = 1000GB。这样 1T 容量的系统需要 100 个容量为 10GB 的驱动器，其成本为 $100,000，另外还要算上连接电缆，电源等一些附件，约占驱动器成本的 20% 为 $20,000，合计成本是 $120,000；最大总数据率为：100 × 5MB/s = 500MB/s；平均等待时间与访问的数据量大小有关，假设访问的数据较小，为 8KB，并随机分布在各个驱动器中，它的访问延时是 15ms，另需 2ms 的数据传输时间，合计 17ms；如果访问的数据量很大，那么数据传输时间还会增加，但基本上与请求访问队列的长度成正比。

对于分等级的储存系统，5% 的容量是使用磁盘，5% × 1T = 50GB，需要 5 个磁盘，其余的 950GB 用磁带，则总的成本为：5 × $1000 + 5 × $1000 × 20% + 950 × $10 = $15,500；最大的数据传输率与磁带驱动器的个数有关，这里假设只有 1 个磁带驱动器，则最大的数据传输率为：5 × 5MB/s + 10 MB/s = 35MB/s；如果是一个访问数据量不大的操作，其平均等待时间为：95% × 0.017 s + 5% × 20s，约等于 1s，从前面的式子可以看到，虽然只有 5% 的操作，但磁带的延时几乎等于总延时；同样，如果访问的数据量很大，平均等待时间还会增加，基本上与请求访问磁带的队列长度成正比。

综上所述，分等级的储存系统比纯磁盘系统要便宜得多，而且由于 95% 的数据访问集中在磁盘操作，所以性能和纯磁盘系统相差无几；但最大数据传输率要低得多，并且平均等待时间也大得多。

77. 通常认为磁带是顺序存取介质，而磁盘是一种随机存取介质。事实上，一种存储设备是否适于随机存取，取决于其传输大小，流传输速率表示在不考虑访问延迟影响下的传输过程中的数据速率。相对的，有效的传输率是数据总和除以时间总和的比率，包括了内部开销（如访问延迟）。

假设在计算机中，二级高速缓存的访问延迟是 8ns，流传输速率为每秒 800M，主存的访问延迟是 60ns，流传输速率为每秒 80MB，磁盘的访问延迟是 15ms，流传输速率为每秒

5MB，磁带的访问延迟是60s，流传输速率为每秒2MB，则

A. 随机存取降低了设备的有效传输速率，因为在访问时间中没有数据传输。对于磁盘而言，以平均存取流传输512B，8KB，1MB，16MB数据，有效传输速率是多少？

B. 设备的利用率是有效传输速率和流传输速率的比率。分别计算随机存取磁盘驱动器实现问题A中4个数据的利用率。

C. 假设25%或更高的利用率是可接受的，利用性能给出的数字，计算最小的传输长度。

D. 完成下面的句子：磁盘是传输大小大于( )字节的随机存取设备，小于这个传输大小的设备是一个顺序存取设备。

E. 对于高速缓存，主存，和磁带所给出的可接受的利用率，计算最小的传输大小。

F. 磁带在什么时候是一个随机存取设备？什么时候是一个顺序存取设备？

答：略

78. 假设一个全息存储驱动器的价格是1万美元，平均存取时间为40ms，若一张CD的大小使用一个100美元的软片，一张软片存放40000个图像，每个图像是分辨率为6000 * 6000的正方形黑白图片(每个象素存储1bit)，若这个驱动器能在1ms读或写一个图像。

A. 这种设备有什么好的应用？

B. 这种设备对计算机系统的I/O性能有什么影响？

C. 有哪些其他类型的存储设备？当全息的存储驱动器发明后，这些设备还需要吗？

答：略

79. 假设一个单面5寸光磁盘片的面密度是每平方英寸1Gbit，一个磁带的面密度是每平方英寸20Mbit，1800英尺长，1/2英寸宽，估算这两种存储介质的存储能力。假设有一个大小和磁带相同，存储密度和光盘相同的光带存在，则光磁带能存储多大的数据容量？若磁带的价格是25美元，则光磁带的市场价是多少？

答：5.25英寸的光盘面积是19.625平方英寸，假设光盘中间的转轴的直径是1.5英寸，转轴的面积为1.77平方英寸，则可用来储存数据的磁盘面积为17.86平方英寸，光盘的储存容量是：$17.86 \times 1\text{Gbit}/8 = 2.2\text{GB}$；磁带的储存面积是$1800 \times 12 \times 0.5 = 10800$平方英寸，容量为$10800 \times 20\text{Mbit} / 8 = 27\text{GB}$；

如果有大小和磁带相同，存储密度和光盘相同的光带存在，则光带能存储的数据容量为：$10800 \times 1\text{Gbit} / 8 = 1350\text{GB}$，因为光带的容量是普通磁带的50倍，所以其价格为$50 \times 25 = \$1250$。

80. 若1K = 1024B，1M = 1024 * 1024B，1G = 1024 * 1024 * 1024B，级数延续到T，P，E(1024^6)，在未来的十年中，一些新提议的科学项目计划能记录和存储一些E级的数据。做一些合理的假设，回答下列问题：

A. 容纳4E的数据需要多少磁盘驱动器？

B. 容纳4E的数据需要多少磁带驱动器？

C. 容纳4E的数据需要多少光带驱动器？(参考81题)

D. 容纳4E的数据需要多少全息存储片？(参考80题)

E. 上述每个选项需要多少立方英尺的存储空间？

答：假设磁盘的容量是10GB，4EB的数据需要400 M个磁盘；如果磁带的容量是40GB，4EB的数据需要100 M个磁带；光带的容量是磁带的50倍，4EB的数据需要2M个光磁带；全息光盘的容量是180GB，4EB的数据需要22.2 M个全息光盘；

3.5″的磁盘驱动器长，宽，高分别是 6″，4″，1″，即 1/2 英尺，1/3 英尺，1/12 英尺，体积为 1/72 立方英尺，400 M 个磁盘的容积是 5.6 英尺；0.5″的磁带大约 1″高，4.5 平方英寸见方，其体积约 1/85 立方英尺，100 M 个磁带的容积是 1.2M 立方英尺；2M 个光磁带的容积是 23400 立方英尺；一个 CD-ROM 直径为 4.75″，厚度为 1/16″，而存放一张全息光盘需要 5 平方英寸见方，1/8″的高度，则 22.2 M 个全息光盘所需的容积是 40000 立方英尺。

81. 在一个磁带文件系统中，讨论操作系统是如何维护空闲区表的？假定磁带是一种“只能附加”(append only)技术的设备，通过 EOT 标记、定位、间隔、读位置操作来使用。

答：由于磁带采用了 append only 技术，因此空闲区肯定在磁带的最后，这样就可以用 EOT 标记、定位、间隔、读位置等操作进行空闲区的定位，空闲区的位置信息就不必保存；在 EOT 标志位后面要有一个可提供可用空闲区大小的确切参数；由于采用了“只能附加”技术，当一些磁带中的文件被删除后，这些空间还没有回收，所以还要提供被删除文件所占空间有多少大小这样一个参数，以便决定什么时候把没删除的文件储存到新的磁带中，回收旧的磁带以重新使用；另外可将这些参数在磁带末尾的最后一个数据块中留有备份，当将磁带中的空闲区再分配给新的储存时，可对最后一个数据块的内容进行修改。

82. 按资源分配管理技术，输入输出设备类型可分为哪三类？

答：按资源分配管理的特点，输入输出设备可分为独享设备、共享设备和虚拟设备三类。

独享设备：即不能共享的设备，一段时间只能由一个作业独占。如打印机、读卡机、磁带机等。所有字符型输入输出设备原则上都应是独享设备。

共享设备：可由若干作业同时共享的设备，如磁盘机等。共享分配技术保证多个进程可以同时方便地直接存取一台共享设备。共享提高了设备的利用率。块设备都是共享设备。

虚拟设备：利用某种技术把独享设备改造成多台同类型独享设备或共享设备。虚拟分配技术就是利用独享设备去模拟共享设备，从而使独占设备成为可共享的、快速 I/O 的设备。实现虚拟分配的最有名的技术是 SPOOLing 技术，即假脱机技术。

83. 设备管理的目标和功能是什么？

答：(1)目标：

①按用户需求提出的要求接入外部设备，系统按一定算法分配和管理控制，而用户不必关心设备的实际地址和控制指令。

②尽量提高输入输出设备的利用率，例如，发挥主机与外设以及外设与外设之间的真正并行工作能力。

(2)功能：

①分配设备：按设备的不同类型和操作系统选设备的算法分配。包括分配相应的通道、设备控制器以及对未分配到的任务或作业进行排队等。

②控制和实现真正的输入输出操作。包括通道程序控制、启动设备、及时响应及处理中断讯号等。

③对输入输出缓冲区进行管理。例如，逻辑名的管理，多个缓冲区的分时以及串并行操作，同类多个外部设备的工作，避免“忙的忙”和“闲的闲”。

④在一些较大系统中实现虚拟设备技术。

84. 什么是通道？试画出通道控制方式时的 CPU、通道和设备的工作流程图。

答：通道是一个独立于 CPU 的专管输入/输出控制的处理机，它控制设备与内存直接进行数据交换。它有自己的通道指令，这些通道指令受 CPU 启动，并在操作结束时向 CPU 发

中断信号。

85. 什么叫中断？中断处理？中断响应？关中断？开中断？中断屏蔽？陷入？软中断？试述中断、陷入和软中断之间异同。

答：中断　指 CPU 对系统发生的某个事件作出的一种反应：CPU 暂停正在执行的程序，保留现场后自动地转去执行相应的处理程序，处理完该事件后再返回断点，继续执行被“打断”的程序。

中断处理　中断响应后，就由软件（中断处理程序）进行相应处理。大致分为四阶段：保存被中断程序的现场，分析中断原因，转入相应处理程序进行处理，恢复被中断程序现场（即中断返回）

中断响应　对中断请求的整个处理过程是由硬件和软件结合起来而形成的一套中断机构实施的。发生中断时，CPU 暂停执行当前的程序，而转去处理中断。这个由硬件对中断请求作出反应的过程，称为中断响应。

(1)中止当前程序的执行；

(2)保存原程序的断点信息；

(3)转到相应的处理程序。

关中断　为保证在中断周期中，指令操作的执行不受外部干扰，将允许中断标志位清 0。即表明现行程序的优先级比所有请求的优先级都高，任何请求都不予响应。

开中断　当允许中断标志位为 1，表明现行程序的优先级低于所有中断请求的优先级，因此一旦出现中断请求，CPU 便能响应。

中断屏蔽　中断请求能否参加判优，需根据屏蔽字的状态决定，若某屏蔽位为 1，其对应的请求无效，不可参加判优。

陷入　系统调用引发的事件往往称做陷入

软中断　用软中断指令来设置程序断点的方式称为软中断。

86. 什么是缓冲，为什么要引入缓冲？

答：缓冲就是暂存数据。I/O 的外部设备一般因含有机电部件，特别是机械部件，速度较慢，而主机的 CPU 和内存是纯电子部件，速度较快，两者传输工作极不协调。在不同时刻，系统中各部件的并行程度仍不能得到充分发挥。为了减少中断次数和中断处理时间，在设备管理中引入缓冲技术。

87. 用于设备分配的数据结构有哪些？它们之间的关系是什么？

答：设备 I/O 请求队列、设备表、设备开关表。

按照设备表中的设备名、设备号信息去到设备开关表读该设备的驱动程序的入口地址，设备 I/O 请求队列是由对该设备有 I/O 要求的缓冲区链接而成，链接指针的信息存在设备表中。

88. 设计一个设备分配的安全检查程序，以保证把某台设备分配给某进程时不会出现死锁。

答：参照开锁与关锁原语或 P、V 原语实现互斥，程序略。

89. 什么是 I/O 控制？它的主要任务是什么？

答：I/O 控制包括：对用户进行 I/O 请求命令的处理，进行设备分配、缓冲区分配、启动通道指令程序或驱动程序进行真正 I/O 操作，以及分析中断响应中断等。

任务：完成对 I/O 设备各种复杂动作的控制

90. 试述对缓冲池队列 em, in 和 out 采用最近最少使用算法对改善 I/O 操作性能有什么好处？

答：减少 I/O 操作中断次数。

91. 驱动程序是什么？为什么要有设备驱动程序？用户进程怎样使用驱动程序？

答：用户进程与设备控制器之间的通信程序称为设备驱动程序。

设备驱动程序是控制设备动作的核心模块，如设备的打开、关闭、读、写等，用来控制设备上数据的传输。它直接与硬件密切相关，处理用户进程发出的 I/O 请求。

用户进程使用设备驱动程序时，设备驱动程序的处理过程为：将用户进程抽象的 I/O 要求转换为具体的要求、检查 I/O 请求的合法性，读出和检查设备的状态，传送必要的参数，设置设备工作方式，启动 I/O 设备。

92. UNIX 系统将设备分为块设备和字符设备，它们各有什么特点？

答：字符设备是以"字符"为单位进行输入、输出的设备，即这类设备每输入或输出一个字符就要中断一次主机的 CPU，请求处理，故称为慢速设备。

块设备是以"字符块"为单位进行输入输出的设备，在不同的系统或系统的不同版本中，块的大小定义不同。但在一个具体的系统中，所有的块一旦选定都是一样大小，便于管理和控制，传送效率较高。

93. 为什么在一些操作系统中可以将设备和普通文件进行统一处理？

答：因为实际操作内容是与设备驱动程序紧密相连的，操作系统会把对特殊文件的操作转换为相应的设备操作，真正的设备驱动程序不包含在这特殊文件中，而是在操作系统核心中。

94. 什么叫通道技术？通道的作用是什么？

答：通道是一个独立于 CPU 的专管输入/输出控制的处理机，它控制设备与内存直接进行数据交换。它有自己的通道指令，这些通道指令受 CPU 启动，并在操作结束时向 CPU 发中断信号。

通道方式进一步减轻了 CPU 的工作负担，增加了计算机系统的并行工作程度。

5.5 OSC 题与解答

Chapter 12

12.1 一个 DMA 控制器有四个通道，控制器的请求速度为每 100ns 一个字(32 位)，其响应速度也一样是 100nsec 一个字。那么为了避免堵塞，总线的传输速度应为多少？

答：总线的每次操作都有一个请求和一个响应组成，故为匹配本题的控制器速度，每次总线操作需要 200ns，即总线的操作频率是每秒 5,000,000 次。如果总线宽度是 32 位，则总线的处理速度应为 20MB/s。注意：题目中一个 DMA 控制器有四个通道的条件在本题中是不需要的，对总线而言，不关心请求来自哪个通道，只关心控制器发出请求、接受响应的频率。

12.2. 假设一台计算机读/写一个存储字需花 10ns，每处理一个中断时，需要把所有的 32 个 CPU 寄存器，以及程序计数器和 PSW 压入到栈中，请问：这台机器每秒最多能处理多少个中断？

答：一个中断出现时需要压入 34 个字到栈中，处理完后又要从栈中取出这 34 个字，这样，系统开销是：$34 \times 2 \times 10\text{ns} = 680\text{ns}$，所以每秒最多能处理的中断个数是：$1 \div 0.68 \times 106$ 次 $= 1.47 \times 106$ 次。(这里假设每次中断系统不做任何工作)

12.3 一个典型的文本打印页面有 50 行，每行 80 个字符，假定一台标准的打印机每分钟能打印 6 页，向打印机的输出寄存器中写一个字符的时间很短，可忽略不计。如果每打印一个字符都需要花费 50us 的中断处理时间(包括所有服务)，那么使用中断驱动 I/O 方式运行这台打印机是否有意义？

答：这台打印机每分钟打印 $50\times80\times6=24000$(个)字符，即每秒打印 400 个字符。每个字符打印中断需要占用 CPU 时间 50μs，所以在每秒用于中断的系统开销为 $400\times50\mu s=20ms$。如果使用中断驱动 I/O，那么 CPU 剩余的 980ms 可用于其他处理。换而言之，中断的系统开销仅占 CPU 的 2%，这几乎不会对程序的运行产生任何影响。因此，使用中断驱动 I/O 方式运行这台打印机是有意义的。

12.4 什么叫设备无关性？

答：设备无关性是指用户程序独立于所使用的具体物理设备。它体现在两个方面：

(1)从程序设计的角度看，对于文件的访问，以及对于各种设备的访问，所体现的接口都是一致的。

(2)操作系统对所有的设备操作都采用统一的管理方式。

12.5 以下各项工作是在 4 个 I/O 软件层的哪一层完成的？

(a)为一个磁盘进行读操作，计算磁道、扇区、磁头。

(b)向设备寄存器写命令。

(c)检查用户是否有权使用设备。

(d)将二进制整数转换成 ASCII 码以便打印。

答：通常，将 I/O 软件组织成四个层次，1)中断处理程序(底层)；2)设备驱动程序；3)与设备无关的操作系统软件；4)用户层软件(高层)。因此：

(a)设备驱动程序层。

(b)设备驱动程序层。

(c)设备无关软件层。

(d)用户软件层。

12.6 一个局域网以如下方式工作：用户发出一个请求将数据包写到网上的系统调用，操作系统便先将数据拷贝到内核缓冲区，再将数据拷贝到网络控制器上。当所有数据都安全存放在控制器中后，它们就从网上以 10Mbit/s 速率发送出去。1μs 后，接收端的网络控制器开始收到报文，并将它们保存在控制器中。当报文的最后一位到达时，控制器便中断其 CPU，于是内核将新到的数据包拷贝到内核缓冲区中并进行检查。识别出该数据包的接收用户后，内核就将报文拷贝到该用户进程空间。假设每次中断及其相关处理过程需花费 1ms，每个数据包有 1024 字节(忽略报文的头部)，拷贝一个字节花费 1μs，则将数据从一个进程转储到另一个进程的最大速率是多少？假设发送者以停等方式进行发送，即发送一个报文后，直到接收到对方对报文的确认后再发送下一报文。为了简单起见，假设得到确认的时间是很小的，可忽略不计。

答：在这个过程中数据包被复制了四次，需要：$4\times1024\times1us=4.1ms$；有两次中断，需 2ms；每个字节的发送时间是：$8*(1/10M)=0.80\mu s$；传播延时是 1$\mu$s；接收端接收第 1 个字节的时延为 1.8$\mu$sec，以后每 0.8$\mu$s 接收一个字节，因此总传输时间为 $1023\times0.8+1.8\mu s=0.82ms$。所以这个过程处理 1024 字节总共需要：$(4.1+2+0.82)ms=6.92ms$。故最大的数据速率是：1024 字节 ÷ 0.00692 秒 = 147977 字节/秒，大约是名义

10Mbit/s 网络容量的 12%。(如果包括网络协议,还达不到这个速率)

12.7 为什么打印机的输出文件在打印前通常都假脱机输出在磁盘上?

答:如果一开始需要输出数据时就分配打印机,由于 CPU 的处理速度远远高于打印机的速度,所以,打印几个字后,进程就有可能在大部分时间里都被阻塞,等待打印机的输出。

12.8 对一个具有 1ms 寻道时间,7200 转/分的磁盘,其柱面偏移(cylinder skew)为多少?假设每个磁道上有 200 个扇区,每个扇区 512 字节。

答:磁盘每转需要时间为 1÷(7200÷60)s=1000/120ms,转一个扇区的时间为:(1000/120)÷200 ms=1/24ms,因此在 1ms 内有 24 个扇区通过磁头,故柱面偏移(cylinder skew)为 24。

12.9 计算第 104 题中磁盘的最大数据速率,以 MB/s 表示。

答:因为转一个扇区的时间为:1/24ms,所以磁盘每秒钟能读 24000 个扇区。又因为每个扇区的容量是 512 字节,故其最大的数据速率可达:24000×512=12288000 字节/秒,相当于 $12288000 \div 2^{20}$ MB/s=11.7MB/s

12.10 RAID3 通过一个独立的奇偶驱动器(校验盘)能纠正一位错误,那么为什么还要使用 RAID2,因为 RAID2 也只能纠正一位,但需要更多的驱动器?

答:RAID2 级不仅能修复损坏的磁盘,而且还能校验数据传输的错误。例如:一个驱动器发送了一个错误位,那么 RAID2 能纠正它,而 RAID3 则不能。

12.11 在一个短的时间间隔内,如果有两个或两个以上的驱动器损坏,将导致 RAID 无法正常工作。假设一个驱动器在某一小时内损坏的概率为 p,那么由 k 个驱动器组成的 RAID 在一小时内失效的概率是多少?

答:k 个驱动器在给定时间内全部正常的概率为:$(1-p)^k$,k 个驱动器在给定时间内有一个损坏的概率为:$kp(1-p)^{k-1}$,所以由 k 个驱动器组成的 RAID 在给定时间内失效的概率为:1 $(1-p)^k$ $kp(1-p)^{k-1}$。

12.12 如果一个磁盘控制器没有内部缓冲区,一旦从磁盘上接收到字节就将它们写到内存中,请问这时交叉存取有用吗?请讨论。

答:如果大多数文件存储在逻辑上连续的扇区上,那么进行交叉编址,给进程一些时间处理刚接收到的数据,以致当下次请求出现时,磁盘刚好在正确的位置,这是值得的。交叉存取是否值得取决于运行程序的种类及它们的实际行为。

12.13 一个双交叉编址的磁盘,在进行寻道时为了避免错过数据它还需要柱面偏移(cylinder skew)吗?请讨论。

答:可能需要,也可能不需要。双交叉编址是一个有效的两扇区的柱面偏移,如果磁头寻道时间少于旋转两个扇区所耗费的时间,那么柱面偏移是不需要的,否则,柱面偏移是需要的。

12.14 一个磁盘制造商有两个 5.25 英寸的磁盘,每一种有 10000 个柱面,新磁盘的线性记录密度两倍于老磁盘。在新老两种磁盘的特性中,哪些特性是新的优于老的?哪些特性是一样的?

答:新磁盘驱动器的容量和数据传输速率是老的两倍。而查找时间和平均旋转等待时间是相同的。

12.15 一个计算机制造商决定重新设计奔腾硬盘分区表,可提供多于四个的分区。这种改变将产生怎样的后果?

答：一个相当明显的后果是：当前所有的操作系统都将不能工作，因为它们都是按四个分区的格式来确定分区的位置。要使当前所有的操作系统还能正常工作，惟一的解决方案是在改变分区表的同时改变所有操作系统，使之使用新的格式。

12.16 磁盘请求以 10，22，20，2，40，6，38 柱面的次序到达磁盘驱动器。寻道时，移动一个柱面需要 6ms。假设，磁臂起始定位于柱面 20，请分别计算下列三种算法的寻道时间？

(a)先来先服务

(b)下一个最邻居柱面(最短寻道时间优先)

(c)电梯算法（初始时，磁臂向上移动）

答：(a)$[(20-10)+(22-10)+(22-20)+(20-2)+(40-2)+(40-6)+(38-6)]\times 6 = 876$ msec.

(b)$[(20-20)+(22-20)+(22-10)+(10-6)+(6-2)+(38-2)+(40-38)]\times 6 = 360$ msec.

(c)$[(20-20)+(22-20)+(38-22)+(40-38)+(40-10)+(10-6)+(6-2)]\times 6 = 348$ msec.

12.17 一个 PC 机销售商在访问阿姆诗特丹西南部的一所大学时，极力推销他的产品，声称他的公司投入了很大的力量使它们的 UNIX 版本运行速度非常快。例如：他们的磁盘驱动程序使用电梯调度法，并将一个柱面内的多个请求按扇区次序排队。学生 Harry Hacker 买了一台，回家后他编写了一个程序随机地读取分布在磁盘上的 10000 个块，令他惊讶的是，他的测试结果与先到先服务方式的结构一样。是销售商在骗人吗？

答：并不是。一个读 10000 块的 UNIX 程序(属于一个进程)是按顺序依次发出磁盘请求的，在一个请求未处理完成前，它将被阻塞，即后面的请求必须等前面的请求完成后才能发出。这样，磁盘驱动器在同一时刻只有一个请求。他只能按到达的顺序进行处理，没有机会使用其电梯调度算法。Harry 应该同时运行多个进程，在这种情况下看看电梯调度算法是否起作用。

12.18 某个计算机的时间中断处理程序每滴答一下需要 2ms(包括进程切换的开销)。时钟以每 60Hz 的频率运行，那么 CPU 用于时钟处理的时间是多少？

答：时钟以每 60Hz 的频率运行，所以，每秒时钟滴答 60 下，故每秒中 CPU 用于时钟处理的时间为：$60\times 2\text{ms} = 120\text{ms}$，占整个处理时间的 12%。

12.19 许多 UNIX 版本使用一个 32 位的无符号整数从原始时间以秒为单位记录时间。系统何时将回到起点？你认为实际上会发生吗？

答：一年共有：$365.25\times 24\times 3600 = 31557600$(秒)。一个 32 位的无符合整数出现计数循环要在 2^{32} 秒后才会出现。而 $2^{32}/31,557,600 = 136.1$(年)，也就是在使用 136.1 年后会出现循环，所以在实际使用过程中时间循环是不会发生的。

12.20 一些计算机需要大量的 RS-232 串行线，例如，服务器或因特网服务商。为此，有一种集成多条 RS-232 串行口的插件卡。假如这样一块卡包含一个处理器，以 8 倍于传输波特率的采样速率对每条线采样。假定一次采样需 $1\mu s$。对于运行在 3200 波特的 28800bps 线，一个处理器可以支持多少条这样的串行线？提示：一条线的波特率是信号每秒改变的次数。如果使用不同的振幅、频率和相位，每次信号间隔编码 9 个位，一条 3200 波特的线能支持 28800bps。另外，56K 的调制解调器并不使用 RS-232，所以并不适合 RS-232 定时。

答:每条线每秒需要采样 $3200 \times 8 = 25600$(次),每次采样需 $1\mu s$,故每条线每秒要占用处理器的时间为:$25600 \times 1\mu s = 25.6ms$,$1000 \div 25.6 = 39.06$,所以这样的一个处理器可以支持 39 条 3200 波特的线。

12.21 为什么 RS232 终端是中断驱动的,而存储映像终端不是中断驱动的?

答:对于 RS232 终端写入一个字符后,打印出来需要延迟相对较长的时间,因此在这期间引入中断是有必要的。而对于存储映像终端,字符的接受是即时的,所以没有必要引入中断。

12.22 考虑 56Kps 调制调解器的性能,驱动程序输出一个字符后就阻塞,当一个字符打印完毕后,产生一个中断通知阻塞的驱动程序,输出下一个字符,然后再阻塞。如果发消息、输出一个字符和阻塞的时间总和为 $100\mu s$,那么由于处理调制调解器而占用的 CPU 时间是多少?假定每个字符有一个开始位和一个结束位,共占 10 位。

答:因为一个字符占 10 位,因此在 56 kbps 的速率下,每秒传送:$56000 \div 10 = 5600$(个)字符,即产生 5600 次中断。每次中断需 $100\mu s$,故处理调制调解器占用 CPU 的时间总共为:$5600 \times 100\mu s = 560ms$,占 56% 的 CPU 时间。

附 录

研究生入学考试操作系统试卷选编

1997(浙江大学)

(说明:试题1至试题3为选择题,分别从供选择的答中选出一个正确的,填入"_____"。)

试题1(6分):支持多道程序设计的操作系统在运行过程中,不断地选择新进程运行,来实现CPU的共享。但其中_____,不是引起操作系统选择新进程的直接原因。

A.运行进程的时间片用完; B.运行进程出错;

C.运行进程要等待某一事件发生; D.有新进程进入就绪状态。

试题2(8分):在下述四段描述中,_____是错误的。

A.若进程A和进程B在临界段上互斥,那么当进程A处于该临界段时,它不能被进程B打断。

B.虚拟存储管理中采用对换(swapping)策略后,用户进程可使用的存储空间似乎增加了。

C.虚拟存储管理中的抖动(thrashing)现象是指页面置换(page replacement)时用于换页的时间远多于执行程序的时间。

D.进程可以由程序、数据和进程控制块(PCB)描述。

试题3(3分):存放在磁盘上的文件,_____。

A.既可随机访问,又可顺序访问。 B.只能随机访问。

C.只能顺序访问。 D.必须通过操作系统访问。

试题4(8分):下述流程是解决两进程互斥访问临界区问题的一种方法。试从"互斥"(mutual exclusion)、"空闲让进"(progress)、"有限等待"(boundedwaiting)等三方面讨论它的正确性。如果它是正确的,则证明之;如果它不正确,请说明理由。

```
program attemp;
    var c1,c2:integer;
procedure p1; ( * 对第一个进程 p1 * )
    begin
      repeat
        Remain Section 1;
        repeat
```

```
            c1:=1-c2
          until c2〈〉0;
          Critical Section;(* 临界区 *)
          c1:=1
            until false
      end;
procedure p2;(* 对另一个进程 p2 *)
      begin
        repeat
          Remain Section 2;
          repeat
            c2:=1-c1
          until c1〈〉0;
          Critical Section;(* 临界区 *)
          c2:=1
            until false
      end;
      begin(* 主程序 *)
        c1:=1;
        c2:=1;
        cobegin
          p1;p2(* 两进程 p1, p2 开始执行 *)
        coend
      end.
```

试题 5(15 分):

(1)假设某计算机系统 NONAME 共有 4 页物理地址空间(4 frames),其操作系统的虚拟地址管理采用"最近最少使用"页面置换算法(LRU)。当一进程依次访问下列虚拟地址空间的页面时(又称引用串,reference string),请计算其缺页(page fault)次数:

1,2,3,4,5,3,4,1,6,7,8,7,8,9,7,8,9,5,4,5,4,2

(2)LRU 需要一定的硬件支持(如计数器、堆栈等)。假设 NONAME 为每页内存配备了一个标志位(dirty bit),请设计一种变形的 LRU 算法,并且说明:

(a)标志位的初始值

(b)何时修改标志位

(c)如何选择应换出的页面

(d)利用此变形的 LRU 算法,计算对上述引用串的缺页次数。

试题 6(10 分):试描述避免死锁的银行家算法。若系统运行中出现下述资源分配情况:

进程	ALLOCATION	NEED	AVAILABLE
	A B C D	A B C D	A B C D
P0	0 0 3 2	0 0 1 2	1 6 2 2
P1	1 0 0 0	1 7 5 0	
P2	1 3 5 4	2 3 5 6	
P3	0 3 3 2	0 6 5 2	
P4	0 0 1 4	0 6 5 6	

该系统是否安全?如果进程 P2 此时提出资源申请(1,2,2,2),系统能否将资源分配给它?为什么?

1998(浙江大学)

(**说明**:试题 1 至试题 4 为选择题,分别从供选择的答中选出一个惟一正确的,填入"_____"。)

试题 1(3 分):下列选择中,_____不是操作系统关心的主要问题。

A. 管理计算机裸机　　B. 设计、提供用户程序与计算机硬件系统的界面

C. 管理计算机系统资源　　D. 高级程序设计语言的编译器

试题 2(5 分):采用_____不会产生内部碎片。

A. 分页式存储管理　　B. 分段式存储管理

C. 固定分区式存储管理　　D. 段页式存储管理

试题 3(5 分):下列几种关于进程的叙述,_____最不符合操作系统对进程的理解。

A. 进程是在多程序并行环境中的完整的程序

B. 进程可以由程序、数据和进程控制块描述

C. 线程(THREAD)是一种特殊的进程

D. 进程是程序在一个数据集合上运行的过程,它是系统进行资源分配和调度的一个独立单位

试题 4(5 分):关于临界区问题的一个算法(假设只有进程 P0 和 P1 可能会进入该临界区)如下(i 为 0 或 1),该算法_____。

A. 不能保证进程互斥进入临界区,且会出现"饥饿"(Starvation)

B. 不能保证进程互斥进入临界区,但不会出现"饥饿"

C. 保证进程互斥进入临界区,但会出现"饥饿"

D. 保证进程互斥进入临界区,不会出现"饥饿"

```
repeat
retry:if (turn -1 ) turn := i;
      if (turn i ) go to retry;
      turn := -1;
Critical Section(临界区)
```

```
    turn := 0;
    remainder Section(其他区域)
until false;
```

试题 5(10 分):磁盘系统调度中,采用 SCAN("扫描")调度算法为任务队列 67,65,124,14,122,37,183,98 服务。试计算服务结束时,磁头总共移动了几个磁道。假设磁头总在第 0 道至第 199 道之间移动;开始服务时,磁头刚从 60 移到 67。

试题 6(10 分):某操作系统(假设为"LINUX")设定一个定时器,每间隔 10ms 产生一次"时间到"中断,该中断的服务程序采用下述算法:

```
struct task_struct * p = current; /* 指针 p 指向当前运行进程的进程控制块 PCB */
        /* ticks 保存最近两次中断服务期间的中断次数,通常是 1 */
    p->counter = p->counter - ticks; /* counter 记录尚可运行的时间段 */
    if (p->counter < 0) { /* 如果 PCB 的 counter 变量小于 0 */
        p->counter = 0;
        need_resched = 1; } /* 全程量 need_resched 置位表示应进行新一轮调度 */
```

"LINUX"若发现 need_resched 置位,则调用如下的 schedule(),进行新一轮调度:

```
void schedule(void)
{   need_resched = 0;
    prev = current; /* 指针 prev 指向当前运行进程的 PCB */
    cli();      /* move an exhausted process to be last.. */
    if (! prev->counter && prev->policy == SCHED_RR) {
        prev->counter = prev->priority; /* priority 表示进程的优先权 */
        move_last_runqueue(prev); } /* 将当前进程搬到就绪进程队列末尾 */
    p = init_task.next_run; /* 指针 p 指向就绪进程队列中第一个 PCB */
    sti(); c = -1000; next = idle_task;
    while (p ! = &init_task) { /* 遍历就绪进程队列中所有 PCB */
        int weight = p->priority; /* priority 表示进程的优先权 */
        if (weight > c) c = weight, next = p;
        p = p->next_run; /* 下一个进程 */
    }
    switch_to(prev,next); } /* CPU 从 prev 进程切换到 next 进程 */
```

试运用操作系统进程调度有关原理,分析"LINUX"的进程调度策略。

试题 7(12 分):临界区(Critical Section)问题要求设计若干进程间的协调策略,以保证进程互斥访问共享数据。例如,如下所示之面包房算法(Bakery Algorithm)就是关于 N 个进程的临界区问题的著名算法。试论述该算法采用 choose 数组的必要性,并举例说明。

```
repeat
```

```
choosing[i] := true;
number[i] := MAX(number[0], number[1], ……, number[N-1]) + 1;
choosing[i] := false;
for j = 0 to N-1 do
begin
  while choosing[j] do no-operation ;
  while number[j] 0 AND (number[j], j ) < (number[i], i) do no-operation;
end ;
```

```
Critical Section(临界区)
number[i] := 0;
remainder Section(其他区域)
until false;
```

1999(浙江大学)

(说明:试题1至试题3为选择题,分别从供选择的答案中选出一个惟一正确的,写到答题纸上。)

试题1(5分):以下关于UNIX操作系统的叙述中,_____是错误的。

A. UNIX对实时系统是不合适的,因为进程在核心态运行时不可抢占。

B. UNIX终究会在市场上消失。

C. UNIX是目前最流行的操作系统之一。

D. UNIX比较适用于高档计算机系统和网络环境,它不能用于普通的INTEL微机。

试题2(5分):测得某个采用按需调页(Demand-paging)策略的计算机系统部分状态数据为:CPU利用率20%,用于对换空间的硬盘的利用率97.7%,其他设备的利用率5%。由此断定系统出现异常。此种情况下,_____能提高CPU利用率。

A. 安装一个更快的硬盘

B. 通过扩大硬盘容量,增加对换空间

C. 增加运行进程数

D. 加内存条,增加物理空间容量

试题3(8分):文件系统中,设立打开文件(open)系统功能调用的基本操作是__(1)__,关闭文件(close)系统功能调用的基本操作是__(2)__。

(1)A. 把文件信息从辅存读到内存

B. 把文件的控制管理信息从辅存读到内存

C. 把文件的FAT表信息从辅存读到内存

D. 把磁盘的超级块从辅存读到内存

(2)A. 把文件的最新信息从内存写入磁盘

B. 把文件当前的控制管理信息从内存写入磁盘

C. 把位示图从内存写回磁盘

D. 把超级块的当前信息从内存写回磁盘

试题 4（10 分）：以下是生产者—消费者问题（Producer-Consumer Problem）的一种解法。

1. 写出 PRODUCER 进程和 CONSUMER 进程各自的临界区。
2. 给出解决临界区互斥问题的一种方法。




```
VAR counter:0..n;
    buffer:array[0..n-1] of item;
```

PRODUCER

```
repeat
    ....    .....
  produce an item in nextp
    .....
    while counter=n do no-op;
    buffer[in]:=nextp;
    in:=in+1 mod n;
    counter:=counter+1;
until false;
```

CONSUMER

```
repeat
  wrhile counter=0 do no-op;
  nextc:=buffer[out];
  out:=out+1mod n;
  counter:=counter-1;
    .....
  consume the item in nextc
    .....
  until false;
```

试题 5（7 分）：假设计算机系统可供用户使用的内存共 150MB，目前分配给 3 个进程的数量如下表所示。这时，第 4 个进程产生，它最终需要内存 60MB，目前的申请数 25MB。

进程	最大需要内存数	已经得到内存数
1	70MB	45MB
2	60MB	40MB
3	60MB	15MB

应用关于死锁问题的银行家算法（Banker's Algorithm），回答是否可以分配给第 4 个进程 25MB 内存，为什么？

试题 6（15 分）：某系统采用页式（paging）存储管理策略，拥有逻辑空间 32 页，每页 2KB；拥有物理空间 1MB。

1. 写出逻辑地址的格式；
2. 若不考虑访问权限位等，进程的页表有多少项（entry）？每项至少多少位（bit）？
3. 如果物理空间减少一半，页表结构应相应作怎样的改变？

2000（浙江大学）

试题 1（5 分）：现有的操作系统对进程状态的定义不尽相同，有的还引入了挂起（suspend）状态。试简要分析挂起状态的意义。

试题 2（10 分）：下述关于双进程临界区问题的算法（对编号为 id 的进程）是否正确：

```
do{
    blocked[id] = true;
    while (turn ! = id)
    {
      while (blocked[1 - id]) ;
      turn = id;
    }

    编号为 id 的进程的临界区

    blocked[id] = false;

    编号为 id 的进程的非临界区

} while (true);
```

其中,布尔型数组 blocked[2]初始值为{false, false},整型 turn 初始值为 0, id 代表进程编号(0 或 1)。请说明它的正确性,或指出错误所在。

试题 3 (10 分):信号量如果只能取 0 或 1 为值,就变成了二元信号量。二元信号量更容易实现。而且,信号量可以由二元信号量替换。以下所列函数试图用二元信号量操作 waitB()和 signalB()替换信号量 wait()、signal():

```
wait(semaphore s)
{
  waitB(mutex);
  s = s - 1 ;
  if (s < 0)
  {
    signalB(mutex);
    waitB(delay);
  }
  else
    signalB(mutex);
  }

signalB(semaphore s)
{
  waitB(mutex);
  s = s + 1 ;
  if (s <= 0)
    signalB(delay);
```

```
    signalB(mutex);
  }
```

其中,用于互斥的二元信号量 mutex 初始化为 1,用于进程挂起的二元信号量 delay 初始化为 0。请指出该替换算法的错误所在。

试题 4(5 分):已知某系统页面长 4K 字节,页表项 4 字节,采用多层分页策略映射 64 位虚拟地址空间。若限定最高层页表占 1 页,问它可以采用几层分页策略。

试题 5(6 分):一进程已分配到 4 个页帧(page frame),如下表(所有数字都为 10 进制数,且以 0 开始)。

虚拟页号	页帧	装入时间	最近访问时间	访问位	修改位
2	0	60	161	0	1
1	1	130	160	0	0
0	2	26	162	1	0
3	3	20	163	1	1

当进程访问第 4 页时,产生缺页中断。请分别用 FIFO(先进先出)、LRU(最近最少使用)、NRU(最近不用)算法,决定缺页中断服务程序选择换出的页面。

试题 6(14 分):现代操作系统必须支持多种文件系统类型(如 CD-ROM 的 ISO 9660、DOS 的 FAT 等)。若要求用如下所示的 file_system_type 结构描述文件系统类型,用 vfsmount 结构描述一个已安装(mount)的文件系统:

```
struct file_system_type {
  struct super_block *(*read_super)(struct super_block *, void *,
int);
  /* read_super 所指的函数用于读出该文件系统在外存的超级块 */
  const char *name; /* 所描述文件系统的类型名,如 FAT */
  int requires_dev; /* 支持文件系统的设备 */
  struct file_system_type * next; /* 指向另一种文件系统类型 */
};

struct vfsmount
{
  kdev_t mnt_dev;              /* 文件系统所在设备的主设备号、次设备号
*/
  char *mnt_devname;                  /* 设备名,如/dev/hda1 */
  char *mnt_dirname;                  /* 安装目录名称 */
  unsigned int mnt_flags;             /* 设备标志,如 ro */
  struct semaphore mnt_sem;           /* 对设备 I/O 操作时的信号量 */
  struct super_block *mnt_sb;         /* 指向超级块 */
  struct file *mnt_quotas[MAXQUOTAS]; /* 指向配额文件的指针 */
  time_t mnt_iexp[MAXQUOTAS];         /* inode 有效期 */
```

```
    time_t mnt_bexp[MAXQUOTAS];          /* 数据块有效期 */
    struct vfsmount *mnt_next;           /* 指向另一个已注册的文件系统
*/
};
```

试设计一套数据结构,以描述一操作系统已经安装的文件系统及其类型(答题时,如已知条件不够,请作必要补充)。

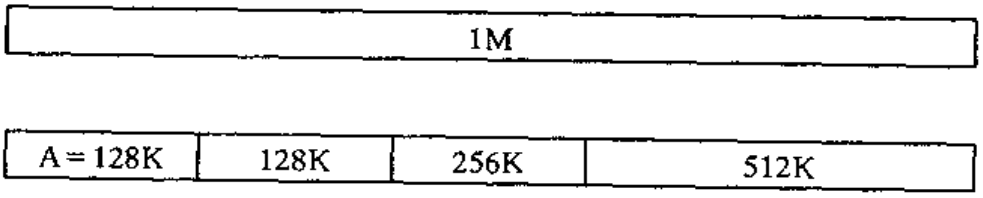
2001(浙江大学)

试题 1 (6 分):虚拟存储管理利用了 swap area(交换区)、内存以及 cache(高速缓存)。假设:从 cache 读取一个字节长的数据需 A 纳秒;如果该数据不在 cache,却在内存,则从内存读至 cache 需 B 纳秒,然后还需从 cache 得到;如果该数据既不在 cache,又不在内存,则从 swap area 读入内存需 C 纳秒,然后还需传至 cache,才能读取。已知 cache 的命中率(hit ratio)是(n-1)/n,内存的命中率是(m-1)/m。求平均访问时间。

试题 2 (6 分):用伙伴系统(buddy system)分配 1M 长度的内存块。试根据如下所示的申请/释放序列,画出该内存块的使用状况图:

申请 70K(分配结果记作 A),申请 35K(分配结果记作 B),申请 80K(分配结果记作 C),释放 A 块,申请 60K(分配结果记作 D),释放 B 块,释放 D 块,释放 C 块。

提示:分配 A 块前后的内存使用状况如下两图



试题 3 (6 分):分时系统里,在条件相同的情况下,通常 KLT(内核级线程)比 ULT(用户级线程)得到更多的 CPU 时间,请简要解释之。

试题 4 (12 分):就文件系统的设计,回答如下三个问题:

1. 子目录可以当作特殊的文件,也可以当作一般数据文件看待。请分析其优缺点。
2. 列举 5 种以上流行的文件系统,并指出它们分别在什么操作系统中使用。
3. 用路径、文件名可以惟一地确定一个文件,但是操作效率不是很理想(或者说,通过路径找文件比较费时)。采取什么方法可以改进之。

试题 5 (8 分):假设缓冲区 buf1 和缓冲区 buf2 都无限大,进程 p1 向 buf1 写数据,进程 p2 向 buf2 写数据。现要求 buf1 数据个数与 buf2 数据个数的差保持在[m,n]之间(m 小于 n,皆为正整数)。请用信号量描述此同步关系。

试题 6 (12 分):一个 SPOOLING 系统由输入进程 I、用户进程 P、输出进程 O、输入缓冲区、输出缓冲区组成。进程 I 通过输入缓冲区为进程 P 输入数据,进程 P 的处理结果通过输出缓冲区交给进程 O 输出。进程间数据交换以等长度的数据块为单位。这些数据块均存储在同一个磁盘上。因此,SPOOLING 系统的数据块通信原语保证始终满足:

$$i + o \leqslant max \tag{1}$$

其中,max 为磁盘容量(以该数据块为单位),i 为磁盘上输入数据块总数,o 为磁盘上输出数据块总数。

该 SPOOLING 系统运行时：

● 只要有输入数据，进程 I 终究会将它放入输入缓冲区；

● 只要输入缓冲区有数据块，进程 P 终究会读入、处理、并产生结果数据写到输出缓冲区；

● 只要输出缓冲区有数据块，进程 O 终究会输出它。

请说明该 SPOOLING 系统在什么情况下死锁。请说明如何修正约束条件(1)避免死锁，同时仍允许输入数据块和输出数据块均存储在同一个磁盘上。

2002(浙江大学)

试题一(8 分)：针对以下 4 个问题，从供选择的答案中选出一个正确的，写到_____里。

(　)1. 在任何时候，系统里总有若干个任务在运行，虽然可能处于不同的执行阶段。

(　)2. 一种商用操作系统，具有强大的安全保护、网络互连功能。

(　)3. 内存中一些保留区域，专门用于输入/输出操作(　)4. 永久存储在 ROM(或其他类似介质)中，在计算机开机后首先执行的系统。

A. autoexeC. bat 文件

B. BIOS

C. 缓冲区(buffer)

D. "控制面板"

E. 文件分配表 FAT

F. 多任务系统(multitasking)

G. "回收站"

H. Windows NT

I. Windows Explorer

试题 2 (12 分)：针对以下 4 个问题，从各自供选择的答案中选出一个正确的，写到(　)里。

(　)1. _____是帮助管理计算机资源的一整套程序。

A. 应用程序　　B. 备份程序　　C. 诊断程序　　D. 操作系统程序

(　)2. _____不是一个操作系统环境。

A. 赛扬(celeron)　B. Windows CE　　C. LINUX　　D. Solaris

(　)3. _____系统具有同时管理和运行多个应用程序的能力。

A. GUI　　B. Windows Applications

C. Multitasking　　D. Networking

(　)4. _____是一个 Microsoft 公司竭力反对的操作系统。

A. OS/2　　B. Windows NT　　C. LINUX　　D. Macintosh

试题 3 (6 分)：假设一个计算机系统具有如下性能特征：

●处理一次中断，平均耗用 1ms。

●一次进程调度，平均需要 2ms。

●将 CPU 分配给选中的进程，又需要平均 1ms。

再假设其定时器芯片每秒产生 100 次中断。请回答：

1. 操作系统将百分之几的 CPU 时间用于时钟中断处理。

2. 如果操作系统采用轮转法(round robin)调度，10 个时钟中断为 1 个时间片。那么，操

作系统将百分之几的 CPU 时间用于进程调度(包括调度、分配 CPU 和引起调度的时钟中断处理时间)。

试题 4(12 分):按需调页(demand paging)的虚拟存储管理系统是如何工作的?分页机制和交换机制各起什么作用?请解释在一个存储管理系统中是否需要同时提供这两种机制。

试题 5(12 分):操作系统中为什么要引入缓冲区(buffer)?为什么缓冲区的头部(header)往往要和存放缓冲区内容的内存分开?

2003(浙江大学)

试题一(各 2 分,共 24 分):以下 12 小题为选择题,各从供选择的答案中选出一个正确的,写到_____里。

1. 下列选择中,_____不是操作系统关心的主要问题。

A. 管理计算机裸机　　B. 设计、提供用户程序与计算机硬件系统的界面

C. 管理计算机系统资源　　D. 高级程序设计语言的编译器

2. 采用_____不会产生内部碎片(internal fragmentation)。

A. 分页式存储管理　　B. 分段式存储管理

C. 固定分区式存储管理　　D. 段页式存储管理

3. 下列几种关于进程的叙述,_____最不符合操作系统对进程的理解。

A. 进程是在多程序并行环境中的完整的程序

B. 进程可以由程序、数据和进程控制块描述

C. 线程(THREAD)是一种特殊的进程

D. 进程是程序在一个数据集合上运行的过程,是系统进行资源分配和调度的一个独立单位

4. 关于临界区问题(critical section problem)的一个算法(假设只有进程 P0 和 P1 可能会进入该临界区)如下(i 为 0 或 1),该算法_____。

A. 不能保证进程互斥进入临界区,且会出现"饥饿"(Starvation)

B. 不能保证进程互斥进入临界区,但不会出现"饥饿"

C. 保证进程互斥进入临界区,但会出现"饥饿"

D. 保证进程互斥进入临界区,不会出现"饥饿"

```
repeat
    retry:if (turn - 1 ) turn := i;
          if (turn i ) go to retry;
          turn := - 1;
        Critical Section(临界区)
    turn := 0;
    remainder Section(其他区域)
until false;
```

5. 按照计算机系统层次结构的一般原则,从用户的角度将依次(①、②、③)看到_____。

A.① C语言编译程序,② 用C语言写的某单位人事管理程序,③ 操作系统

B.① 用C语言写的某单位人事管理程序,② C语言编译程序,③ 操作系统

C.① 操作系统,② 用C语言写的某单位人事管理程序,③ C语言编译程序

D.① 用C语言写的某单位人事管理程序,② 操作系统,③ C语言编译程序

6.系统功能调用是_____。

A.用户编写的一个子程序　　B.高级语言中的库程序

C.操作系统中的一条命令　　D.操作系统向用户程序提供的接口

7.磁盘的读写单位是_____。

A.块　　B.扇区　　C.簇　　D.字节

8.文件系统中设立打开文件(open)系统调用的主要目的是_____。

A.把文件从辅存读到内存

B.把文件的控制管理信息从辅存读到内存

C.把文件的FAT表信息从辅存读到内存

D.把磁盘文件系统的控制管理信息从辅存读到内存

9.在操作系统中,P－V操作(或wait操作和signal操作)是一种_____。

A.机器指令　　B.系统调用命令

C.作业控制命令　　D.低级进程通讯原语

10.通常对文件系统来说,文件名及其属性可以集中在_____中以便查找。

A.目录　　B.索引　　C.字典　　D.作业控制块

11.拿内存加上外存容量之和与虚拟存储空间相比,其大小关系是_____。

A.前者比后者大　　B.前者比后者小

C.二者相等　　D.不一定

12.在_____的情况下,系统出现死锁。

A.计算机系统发生重大故障

B.有多个封锁的进程同时存在

C.若干进程因竞争资源而无休止地相互等待对方释放已占有资源

D.资源数大大小于进程数或进程同时申请的资源数大大超过资源总数

试题二(7分):如图1(297页),相交的四条单行线不幸塞车。

(1)请根据死锁4条件分析,判断其是否死锁现象。

(2)请添加新的规则,以保证不再出现死锁,并简要说明之。

试题三(9分):"按需调页"式虚拟存储管理(demand paging)需要交换空间(swapping space)的支持,如专用的磁盘分区。

(1)已知硬盘上有一交换区(swapping area),试为它设计交换区的内部组织结构,说明如何访问其中的页面内容。

(2)物理内存的页面交换往往采用LRU算法。在交换区里,LRU算法还有多大价值?试分析之。

(3)如果以磁盘作为交换区,磁盘与内存在存取速度上的差异必然影响操作系统的性能。请提出一种改善性能的方案,并简要说明。

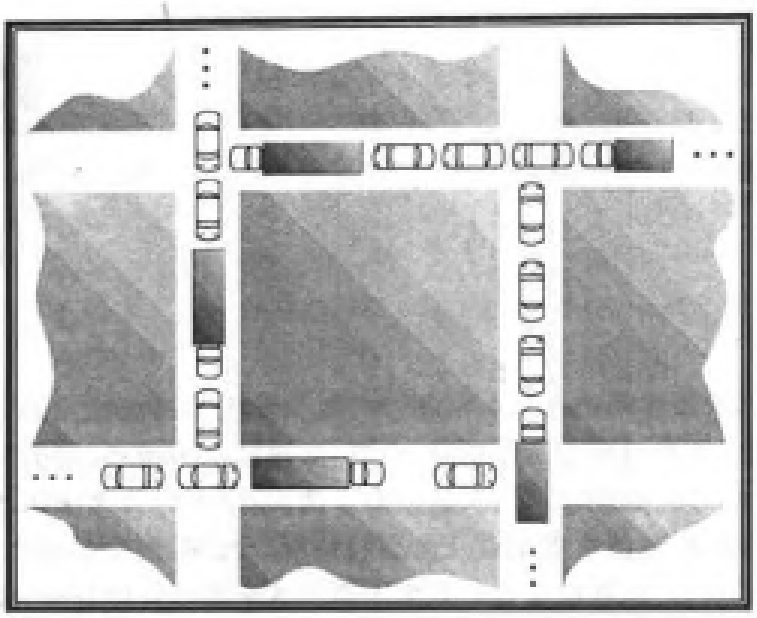


图 1

参考文献

[1] 曾平，李春葆编著.操作系统习题与解答.北京：清华大学出版社，2001

[2] 曾平，曾慧.编著.考点精要与解题指导.北京：人民邮电出版社，2002

[3] 何炎祥主编.操作系统原理学习与解题指南。武汉：华中科技大学出版社，2001

[4] 陆丽娜，柯丽芳.操作系统重点难点及典型题精解。西安：西安交通大学出版社，2002

[5] 前沿考试研究室编著.计算机专业研究生入学考试全真题解(3)——操作系统与编译原理分册.北京：人民邮电出版社，2001

[6] 胡元义，余健明等编著.操作系统课程辅导与习题解析.北京：人民邮电出版社，2002

[7] (美)Harris J A 著.操作系统习题与解答.须德等译.北京：机械工业出版社，2003

[8] Silberschatz A, Galvin P B and Gagne G. Operating System Concepts (6th Edition). New York: John Wiley&Sons, 2001

[9] J Arther Harris. Operating Systems. New York: McGraw－Hill, 2002

[10] Tanenbaum A S. Modern Operating Systems (2nd Edition). New York: Prentice Hall, 2001